Supplementary Information

A flexible modelling framework for estimating thermal sensitivity across life

Daniel W.A. Noble*[1], Pieter A. Arnold[1], Patrice Pottier[2]

[1] *Division of Ecology and Evolution, Research School of Biology, Australian National University, Canberra, Australia*

[2] *Department of Biological and Environmental Sciences, University of Gothenburg, Gothenburg, Sweden*

2026-05-25

Table of contents

*Corresponding author: daniel.noble@anu.edu.au

```
# Setup chunk must NOT be cached - caching skips the package-loading side
# effects on subsequent renders, so library() calls in pacman::p_load() never
# run and downstream chunks fail with "could not find function ...".
pacman::p_load(here, dplyr, tidyr, tibble, purrr, ggplot2, brms, posterior, tidybayes, tinytabl

# Force tinytable to register its LaTeX preamble dependencies (tabularray,
# siunitx, etc.) on every render. Tinytable injects these via
# knit_meta_add() when tt() is called, but if all tt() chunks are cached,
# the side-effect doesn't fire and PDF builds fail with "Environment tblr
# undefined". Calling tt() here (in this cache: false chunk) guarantees
# the dependencies are always registered.
invisible(tinytable::tt(data.frame(x = 1)))

# With `cache: true` set globally, knitr otherwise serves a display chunk's
# stale cached output even after an *upstream* chunk it depends on has changed
# (e.g. a table that prints `et_leaf` kept showing the old CTmax after the
# `et_leaf` computation was fixed). autodep tracks cross-chunk object
# dependencies so downstream caches are invalidated when upstream ones change.
knitr::opts_chunk$set(autodep = TRUE)
knitr::dep_auto()

set.seed(123)

# Cache directory for brms fits (per CLAUDE.md: save fits manually as .rds).
models_dir <- here::here("output", "models")
if (!dir.exists(models_dir)) dir.create(models_dir, recursive = TRUE)
```

1 Introduction

This supplement demonstrates the joint Bayesian 4-parameter logistic (4PL) framework from the main manuscript using the **bayesTLS** R package included with this project. The package fits the

4PL, extracts classical TLS quantities (z, $CT_{max_{1hr}}$, and, for lethal endpoints, T_{crit}) with posterior uncertainty, and predicts heat-injury accumulation and survival under arbitrary temperature time series, optionally incorporating a Sharpe-Schoolfield repair kernel.

Install once from GitHub, then load:

```
# install.packages("remotes")   # if not already installed
remotes::install_github("daniel1noble/bayesTLS")
```

```
library(bayesTLS)
```

The functions chain together as a single pipeline:

1. **Standardise** raw experimental data to a common column schema (`temp`, `duration`, `n_total`, `n_surv`, …).
2. **Fit** the joint Bayesian 4PL with the built-in `beta_binomial(link = "identity")` family and asymptote-bounded reparameterisation.
3. **Extract** z, $CT_{max_{1hr}}$, and, for lethal endpoints, T_{crit}, all with posterior uncertainty.
4. **Predict** survival and heat-injury accumulation under fluctuating field temperature traces, with optional repair.
5. **Plot** each step with shared theming so figures across the workflow read consistently.

Every exported function is documented with roxygen2 (`?fit_4pl`, etc.), and the package includes `testthat` unit tests plus gated `brms` integration tests for recovery of known simulation truths. The supplement first demonstrates the pipeline in controlled simulations and then applies it to brown shrimp, zebrafish, snow gum leaf PSII, and mountain hickory seed datasets.

1.1 *What each function does*

Table S1 lists every exported function in the library, grouped by pipeline stage. Each function is single-purpose; the full roxygen2 documentation (with runnable `@examples`) lives in the corresponding R file.

2 A Tutorial with Simulations

This tutorial uses simulated data to show how the Bayesian hierarchical 4PL recovers thermal sensitivity (z) and $CT_{max_{1hr}}$ in both binomial and beta-binomial (overdispersed) settings. The joint model propagates uncertainty through downstream quantities and accommodates extensions that are awkward in a two-stage analysis. The subsequent case studies apply the same workflow to data presented in the main manuscript.

Table S1: Functions exported by the TDT analysis library, grouped by pipeline stage. Each function is documented with roxygen2 in its R file; runnable examples accompany every function.

Stage	Function	Purpose
Data	standardize_data()	Rename raw columns to project schema; attach metada
Model spec	make_4pl_priors()	Weakly informative priors for the disjoint-bounds 4PL.
Model spec	make_4pl_formula()	brms non-linear formula; identity link, all four sub-para
Fit	fit_4pl()	Joint Bayesian 4PL fit; returns a 'bayes_tls' object (us
Inspection	print.bayes_tls()	Compact one-screen header: data shape, T_bar, asymp
Inspection	summary.bayes_tls()	Delegates to 'summary()' on the underlying 'brmsfit'; r
Inspection	plot.bayes_tls()	Delegates to 'plot()' on the underlying 'brmsfit'; return
Inspection	has_fit()	Predicate: is a 'bayes_tls' workflow fitted (TRUE) or s
Inspection	extract_4pl_pars()	Posterior draws of the four natural-scale 4PL paramete
Accessors	get_z_draws()	Per-draw posterior of z from an 'extract_tdt()' output
Accessors	get_ctmax_draws()	Per-draw posterior of CTmax_1hr from an 'extract_to
Accessors	get_tcrit_draws()	Per-draw posterior of T_crit from an 'extract_tdt(leth
Accessors	get_surv_draws()	Per-draw posterior of survival probability from 'predict
Accessors	get_hi_draws()	Per-draw posterior of heat-injury integrals from 'predic
Diagnostics	diagnose_tdt_fit()	Rhat / ESS / divergences / treedepth, with pass-fail fla
Diagnostics	tdt_parameter_table()	Posterior summary on the natural scale (low, up, k, mi
Predictions	predict_survival_curves()	Survival curves on a temp x duration grid with credibl
Predictions	derive_tdt_landscape()	Dense 2-D survival surface for a heatmap.
TDT quantities	derive_tdt_curve()	Time to the threshold survival across temperature. De
TDT quantities	derive_temperature_for_duration()	Temperature at threshold survival for a fixed exposure
TDT quantities	derive_z()	Thermal sensitivity z read directly from the posterior (
TDT quantities	extract_tdt()	Bundled wrapper: z and CTmax always; T_crit opt-in
Heat injury	make_temperature_scenarios()	Three reference traces (flat / single spike / multi-spike)
Heat injury	planted_dose_from_trace()	Analytical HI under known z, CTmax, T_c (validation
Heat injury	predict_heat_injury()	Posterior HI and survival under a temperature trace.
Heat injury	repair_rate_schoolfield()	Sharpe-Schoolfield repair TPC (Kelvin internally).
Plotting	plot_survival_curves()	Posterior curves with observed proportion overlay.
Plotting	plot_tdt_curve()	LT_x curve on a log-time axis (classical TDT view).
Plotting	plot_tdt_landscape()	Survival heatmap with contour overlay.
Plotting	plot_temperature_density()	Posterior density for CTmax / T_crit with a 95-percen
Plotting	plot_temperature_scenarios()	Three traces stacked vertically.
Plotting	plot_heat_injury()	HI and predicted survival trajectories with CrI bands.
Plotting	plot_repair_rate()	Sharpe-Schoolfield TPC curve.

2.0.1 *Simulating Data*

2.0.1.1 Step 1 — Setting up parameters

We choose values that might be typical of a thermal mortality assay. The four 4PL parameters are the lower and upper asymptotes (ℓ, u), the slope on the $\log_{10}$-time axis (k), and a midpoint that varies linearly with temperature: $\text{mid}(T) = \beta_0 + \beta_1(T - \bar{T})$. Each value below has a clear biological meaning.

```
true_params <- list(
  ell      = 0.03,   # 3% of individuals survive even at the longest exposures
  u        = 0.97,   # 97% are alive at very short exposures
  k        = 8,      # steepness of the survival drop on the log10(t) axis
  m_beta0  = 1.5,    # midpoint at the grand-mean temperature: ~31.6 min
  m_beta1  = -0.15,  # midpoint drops 0.15 log10-min per °C; z = -1/beta1   6.7 °C
  T_bar    = 34      # grand-mean temperature
)
```

The implied thermal sensitivity is $z = -1/\beta_1 \approx 6.7$ °C: a temperature change of this magnitude shifts LT50 by a factor of 10. The same parameters determine the uncentred TDT intercept $\alpha = \beta_0 + \frac{1}{k}\log\frac{u-0.5}{0.5-\ell} - \beta_1\bar{T}$ and the 1-hour critical thermal limit $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$. We use these known values to assess recovery below.

We first calculate α, z, and $CT_{max_{1hr}}$ from the generating parameters to provide reference values for the fitted models.

```
true_z      <- -1 / true_params$m_beta1
true_alpha  <- true_params$m_beta0 +
  (1 / true_params$k) *
  log((true_params$u - 0.5) / (0.5 - true_params$ell)) -
  true_params$m_beta1 * true_params$T_bar
true_CTmax  <- (log10(60) - true_alpha) / true_params$m_beta1

# T_crit under the rate-multiplier definition (see §[Deriving z, CTmax, and
# T_crit from the posterior]). For each posterior draw, T_crit is computed as
# CTmax_1hr + z * log10(r*/100) where r* is sampled uniformly on log10
# across the default range c(0.1, 1) % HI per hour. The MEDIAN of that
# distribution lies at log10(r*/100) = -2.5 (the geometric mean of the
# range), so:
true_T_crit <- true_CTmax - 2.5 * true_z
```

2.0.1.2 Step 2 — Simulated study design

We mimic a well-replicated TDT experiment: five assay temperatures crossed with six log-spaced exposure durations, with 30 replicates of 30 individuals per cell. The resulting 900 trials and 27,000 individuals make parameter recovery clear; the case studies below illustrate less information-rich designs.

```
temps     <- c(30, 32, 34, 36, 38)             # 5 assay temperatures (°C)
durations <- c(1, 5, 15, 45, 135, 405)         # 6 durations (minutes, log-spaced)
n_rep     <- 30                                # 30 replicates per cell
n_per_rep <- 30                                # 30 individuals per replicate

design <- tidyr::expand_grid(
  T   = temps,
  t   = durations,
  rep = seq_len(n_rep)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params$T_bar,                      # mean-centred T
    mid     = true_params$m_beta0 + true_params$m_beta1 * T_c,  # mid(T)
    p_true  = true_params$ell +
      (true_params$u - true_params$ell) /
      (1 + exp(true_params$k * (log10_t - mid))),
    n = n_per_rep
  )
```

2.0.1.3 Step 3 — Look at the truth before simulating

Before generating noisy data, let's plot the *true* survival surface (Figure S1) so we know what the model is being asked to recover. Each line is the 4PL at one assay temperature, plotted against $\log_{10}$ exposure time.

```r
truth_grid <- tidyr::expand_grid(
  T = temps,
  t = 10 ^ seq(log10(0.5), log10(1000), length.out = 200)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params$T_bar,
    mid     = true_params$m_beta0 + true_params$m_beta1 * T_c,
    p_true  = true_params$ell +
      (true_params$u - true_params$ell) /
      (1 + exp(true_params$k * (log10_t - mid)))
  )

ggplot2::ggplot(truth_grid,
                ggplot2::aes(log10_t, p_true, colour = factor(T))) +
  ggplot2::geom_hline(yintercept = 0.5, linetype = "dashed", colour = "grey50") +
  ggplot2::geom_line(linewidth = 0.8) +
  ggplot2::labs(
    x = expression(log[10]~exposure~time~(min)),
    y = "True survival probability",
    colour = "Temperature (°C)"
  ) +
  ggplot2::theme_minimal()
```

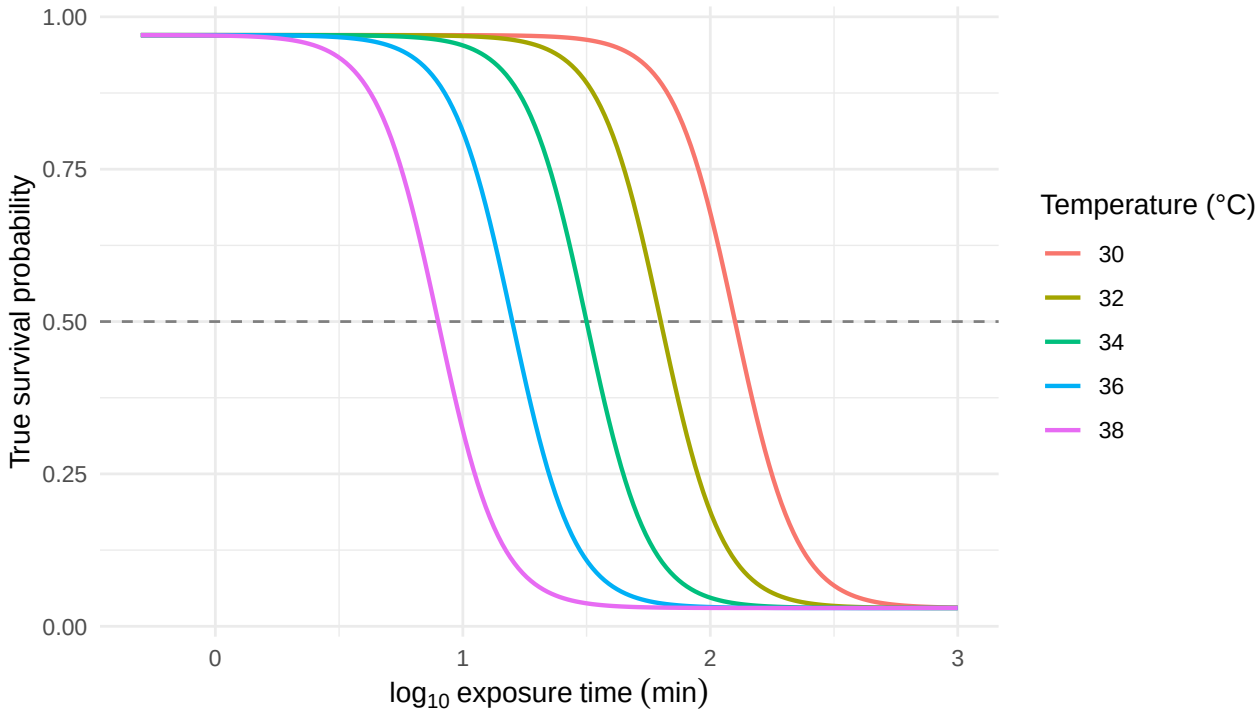


Figure S1: The known 4PL dose-response surface used to generate the simulated data, evaluated on a dense time grid at each of the five assay temperatures. The horizontal line marks 50% survival; where each curve crosses it gives the true LT50 at that temperature.

Notice the curves shift left as temperature rises: hotter temperatures kill faster, so the LT50 is reached at shorter durations. That horizontal shift in midpoints is exactly what β_1 encodes.

2.0.1.4 Step 4 — Generate noisy data: binomial and beta-binomial

Survival counts may exhibit two forms of variation. Under **binomial** sampling, each replicate's survival count is drawn from Binomial(n, p), where p is given by the 4PL surface. **Beta-binomial** sampling adds *overdispersion*: replicate-to-replicate variation in p, for example from cohort effects or unmodelled heterogeneity. When TDT data show this extra variation, the beta-binomial likelihood is more appropriate.

```
# (a) Binomial: y ~ Binomial(n, p_true)
sim_bin <- design |>
  dplyr::mutate(y = rbinom(dplyr::n(), size = n, prob = p_true))

# (b) Beta-binomial: replicate-level p drawn from Beta(p*phi, (1-p)*phi)
phi <- 5  # smaller phi = more overdispersion
sim_bb <- design |>
  dplyr::mutate(
    p_draw = rbeta(dplyr::n(), p_true * phi, (1 - p_true) * phi),
    y      = rbinom(dplyr::n(), size = n, prob = p_draw)
  )
```

2.0.2 *The classical two-stage TDT pipeline*

We first run the simulated data through the **classical two-stage TDT pipeline** (Ørsted *et al.* 2022; Rezende *et al.* 2014), which is dominant in the thermal-tolerance literature.

1. **Stage 1.** At each assay temperature, fit a logistic dose-response curve to the count data and read off a point estimate of $\log_{10} \mathrm{LT50}_T$.
2. **Stage 2.** Regress those Stage-1 point estimates on assay temperature with ordinary least squares, then derive $z = -1/\beta_1$ and $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ from the fitted line.

2.0.2.1 Stage 1 — Dose-response curves at each temperature

We fit a binomial GLM with a logit link separately at each assay temperature. This is the most-common Stage-1 specification in the TDT literature: a two-parameter logistic with asymptotes constrained to 0% and 100%. The midpoint $\log_{10} \mathrm{LT50}_T = -\beta_0/\beta_1$ falls out of the GLM's two coefficients.

```
fit_stage1 <- function(sim_data) {
  sim_data |>
    dplyr::group_by(T) |>
    dplyr::group_modify(function(d, key) {
      f  <- glm(cbind(y, n - y) ~ log10_t, data = d, family = binomial())
      co <- coef(f)
      tibble::tibble(log10_lt50 = as.numeric(-co[1] / co[2]))
```

Table S2: Per-temperature Stage-1 estimates of $\log_{10}$ LT50 from a binomial GLM with logit link, fit separately at each of the five assay temperatures. Truth values are the simulated midpoints (which coincide with $\log_{10}$ LT50 here because the simulation asymptotes are near 0 and 1).

T (°C)	Truth log10(LT50)	Binomial log10(LT50)	Beta-binomial log10(LT50)
30	2.1	2.067	2.092
32	1.8	1.794	1.791
34	1.5	1.491	1.498
36	1.2	1.193	1.197
38	0.9	0.921	0.939

```
    }) |>
    dplyr::ungroup()
}

stage1_bin <- fit_stage1(sim_bin)
stage1_bb  <- fit_stage1(sim_bb)
```

```
stage1_table <- dplyr::full_join(
  stage1_bin |> dplyr::rename(bin_log10_lt50 = log10_lt50),
  stage1_bb  |> dplyr::rename(bb_log10_lt50  = log10_lt50),
  by = "T"
) |>
  dplyr::mutate(
    truth_log10_lt50 = true_params$m_beta0 +
      true_params$m_beta1 * (T - true_params$T_bar)
  ) |>
  dplyr::select(T, truth_log10_lt50, bin_log10_lt50, bb_log10_lt50) |>
  dplyr::rename(
    `T (°C)`             = T,
    `Truth log10(LT50)`  = truth_log10_lt50,
    `Binomial log10(LT50)` = bin_log10_lt50,
    `Beta-binomial log10(LT50)` = bb_log10_lt50
  )

tinytable::tt(stage1_table, digits = 3, escape = TRUE)
```

With the simulation's true asymptotes near 0 and 1, the 2PL's forced-asymptote constraint introduces negligible bias here, so each Stage-1 estimate sits close to the simulation truth.

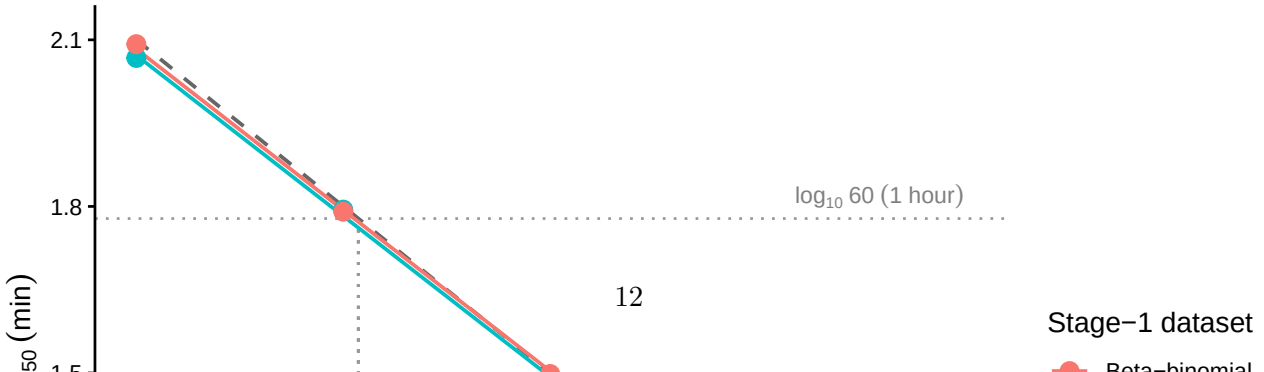
2.0.2.2 Stage 2 — log-linear TDT regression

Stage 2 regresses Stage-1 $\log_{10}$ LT50 on assay temperature with ordinary least squares. The classical TDT quantities — $z = -1/\beta_1$ and $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ — fall out of the fitted line.

```r
fit_stage2 <- function(stage1) {
  fit2 <- lm(log10_lt50 ~ T, data = stage1)
  co   <- coef(fit2)

  tibble::tibble(
    quantity = c("z (°C)", "CTmax at 1 hr (°C)"),
    est      = c(as.numeric(-1 / co["T"]),
                 as.numeric((log10(60) - co["(Intercept)"]) / co["T"]))
  )
}

ts_bin <- fit_stage2(stage1_bin)
ts_bb  <- fit_stage2(stage1_bb)
```

Figure S2 visualises the Stage-2 fit. Each point is one Stage-1 $\log_{10}$ LT50 estimate from Table S2; the solid line is the OLS regression through them. The slope is β_1, from which $z = -1/\beta_1$ — i.e., the temperature increase that causes LT50 to change by a factor of 10. The line's height at $\log_{10} 60 \approx 1.78$ (1 hour) is at the temperature equal to $CT_{max_{1hr}}$ — the vertical dotted line drops from this intersection straight to the temperature axis to mark it. The dashed grey line is the simulation truth.

```r
stage1_combined <- dplyr::bind_rows(
  stage1_bin |> dplyr::mutate(dataset = "Binomial"),
  stage1_bb  |> dplyr::mutate(dataset = "Beta-binomial")
)

truth_line <- tibble::tibble(
  T = seq(min(temps), max(temps), length.out = 100)
) |>
  dplyr::mutate(
    log10_lt50 = true_params$m_beta0 +
      true_params$m_beta1 * (T - true_params$T_bar)
  )

ggplot2::ggplot(stage1_combined,
                ggplot2::aes(x = T, y = log10_lt50, colour = dataset)) +
  ggplot2::geom_hline(yintercept = log10(60),
                      linetype = "dotted", colour = "grey60") +
  ggplot2::geom_segment(
    ggplot2::aes(x = true_CTmax, xend = true_CTmax,
                 y = log10(60), yend = -Inf),
    inherit.aes = FALSE,
    linetype = "dotted", colour = "grey60"
  ) +
  ggplot2::geom_line(data = truth_line,
                     ggplot2::aes(x = T, y = log10_lt50),
                     inherit.aes = FALSE,
                     linetype = "dashed", colour = "grey40",
                     linewidth = 0.7) +
  ggplot2::geom_smooth(method = "lm", se = FALSE, linewidth = 0.7) +
  ggplot2::geom_point(size = 3) +
  ggplot2::annotate("text", x = max(temps), y = log10(60),
                    label = "log[10]~60~(1*' hour')", parse = TRUE,
                    hjust = 1, vjust = -0.4, colour = "grey50", size = 3) +
  ggplot2::annotate("text", x = true_CTmax, y = -Inf,
                    label = "CT[max[1*hr]]", parse = TRUE,
                    hjust = -0.15, vjust = -0.7, colour = "grey50", size = 3) +
  ggplot2::labs(
    x = "Assay temperature (°C)",
    y = expression(log[10]~LT[50]~(min)),
    colour = "Stage-1 dataset"
  ) +
  ggplot2::theme_classic()
```



This completes the two-stage workflow. We carry its point estimates of z and $CT_{max_{1hr}}$ forward (`ts_bin` and `ts_bb`) for comparison with the joint Bayesian results.

> **!** Important
>
> Classical two-stage analyses often treat Stage-1 LT50 estimates as known when fitting Stage 2. Delta-method or bootstrap procedures can propagate uncertainty, but they require an additional step and must account for uncertainty from both stages.

2.0.3 *A joint Bayesian 4PL*

Instead of fitting separate logistic curves and then regressing their LT50 estimates, the joint fit estimates ℓ, u, k, β_0, and β_1 simultaneously from the raw counts. We use the same simulated data to derive z and $CT_{max_{1hr}}$ directly from this posterior.

2.0.3.1 Specifying the 4PL in `brms`

The model has two pieces. The first is the 4PL itself — survival probability as a function of $\log_{10}$ exposure time:

$$p_{ij} \;=\; \ell \;+\; \frac{u-\ell}{1+\exp\!\big(k\,(\log_{10} t_{ij} - \mathrm{mid}_{ij})\big)}, \tag{S1}$$

where ℓ and u are the lower and upper asymptotes, k is the slope on the $\log_{10} t$ axis, and mid_{ij} is the value of $\log_{10} t$ at which the curve crosses the midpoint between ℓ and u. The second piece lets the midpoint vary linearly with mean-centred temperature:

$$\mathrm{mid}_{ij} \;=\; \beta_0 + \beta_1\,(T_{ij} - \bar{T}), \tag{S2}$$

so that β_1 encodes how fast the dose-response curve shifts on the time axis as assay temperature changes; $\bar{T}$ is the grand-mean temperature, included so that β_0 is interpretable as the midpoint at the average assay temperature.

The function library exposes this model as a single call: `fit_4pl()` constructs the brms `bf()` formula, the default weakly-informative priors, and the sampling controls, and returns a workflow object containing the fit. Internally the formula uses the disjoint-bounds reparameterisation described in §[Loading the function library] — `low = 0.001 + \text{inv\_logit}(\text{lowraw}) \cdot 0.498` and `up = 0.501 + \text{inv\_logit}(\text{upraw}) \cdot 0.498` — so the 4PL output is guaranteed to lie in $(0, 1)$ regardless of where MCMC takes the unconstrained `lowraw`, `upraw`, and `logk` parameters. The default also puts a `temp_c` slope on every 4PL sub-parameter; when temperature has no real effect on a given parameter, the slope shrinks toward zero under the prior. We inspect the brms formula `fit_4pl()` builds with:

```
make_4pl_formula()
```

```
n_surv | trials(n_total) ~ (0.001 + inv_logit(lowraw) * 0.498) + ((0.501 + inv_logit(upraw) * (
lowraw ~ temp_c
upraw ~ temp_c
logk ~ temp_c
mid ~ temp_c
```

> **!** Important
>
> The default model puts a `temp_c` slope on all four 4PL sub-parameters and includes no random effects. Random intercepts on `mid` can be added by passing `random_effects = c("Date", "Tank", ...)` to `fit_4pl()`. The simulation here has no batch structure so we leave it off; the shrimp case study below uses `random_effects = c("Date", "Tank")`.

2.0.3.2 Standardising and fitting

The first step is to pass the simulated data through `standardize_data()` so the columns match the schema the rest of the library expects (`temp`, `duration`, `logd`, `temp_c`, `n_total`, `n_surv`).

```r
std_bin <- standardize_data(sim_bin,
                            temp = "T", duration = "t",
                            n_total = "n", n_surv = "y",
                            duration_unit = "minutes")
std_bb  <- standardize_data(sim_bb,
                            temp = "T", duration = "t",
                            n_total = "n", n_surv = "y",
                            duration_unit = "minutes")
```

We now call `fit_4pl()` twice — once with the built-in `binomial(link = "identity")` family for the non-overdispersed simulation, and once with the default `beta_binomial(link = "identity")` for the overdispersed case. Caching is handled by brms's `file = ...` mechanism, so subsequent renders reload the cached fits unless the formula or data changes.

```r
# file_refit = "never": once the .rds exists, always reuse it. brms's
# default ("on_change") over-triggers refits on these simulated-data fits
# because it picks up subtle differences in tibble attributes between
# renders. Tutorial sims are stable - to force a refit, delete the .rds.
wf_bin <- fit_4pl(std_bin,
                  family = binomial(link = "identity"),
                  chains = 4, iter = 2000, cores = 4, seed = 123,
                  file       = file.path(models_dir, "sim_4pl_binomial_v2"),
                  file_refit = "never")

wf_bb  <- fit_4pl(std_bb,
                  chains = 4, iter = 2000, cores = 4, seed = 123,
                  file       = file.path(models_dir, "sim_4pl_betabinomial_v2"),
                  file_refit = "never")
```

```
# `print()` on a bayes_tls workflow gives the one-screen header view: data
# shape, centred-temperature mean, asymptote bounds, draws count.
print(wf_bin)
```

```
<bayes_tls>
  Data:     900 rows; 5 temperatures; 6 durations
  T_bar:    34
  Bounds:   response in (0.000, 1.000); low in (0.001, 0.499); up in (0.501, 0.999)

  Status:   fitted (4000 draws)
```

```
print(wf_bb)
```

```
<bayes_tls>
  Data:     900 rows; 5 temperatures; 6 durations
  T_bar:    34
  Bounds:   response in (0.000, 1.000); low in (0.001, 0.499); up in (0.501, 0.999)
  Status:   fitted (4000 draws)
```

```
# Convenience: extract the underlying brmsfit object for downstream brms
# helpers. `get_brmsfit()` is the package accessor - it pulls `wf$fit` and
# errors clearly if the workflow was never fitted (pairs with `has_fit()`).
fit_bin <- get_brmsfit(wf_bin)
fit_bb  <- get_brmsfit(wf_bb)
```

We also summarise the proportion of outcome variation explained by each fitted model using Bayesian R^2.

```
# Obtain the Bayes R2 for each model
brms::bayes_R2(fit_bin)
```

```
    Estimate    Est.Error     Q2.5     Q97.5
R2 0.9875041 0.0001303429 0.987185 0.9877035
```

```
brms::bayes_R2(fit_bb)
```

```
    Estimate   Est.Error      Q2.5    Q97.5
R2 0.9263435 0.001128046 0.9238269 0.928245
```

2.0.3.3 Checking parameter recovery

We assess recovery by comparing posterior medians and 95% credible intervals with the known generating values (Table S3). Given the large simulated sample, the posterior should concentrate near those values.

```r
post_summary <- function(fit) {
  d <- posterior::as_draws_df(fit)
  # Transform the raw posterior columns back to the natural 4PL scale.
  # See compute_4pl_bounds(0, 1): low  (0.001, 0.499), up  (0.501, 0.999).
  draws <- tibble::tibble(
    ell               = 0.001 + plogis(d$b_lowraw_Intercept) * 0.498,
    u                 = 0.501 + plogis(d$b_upraw_Intercept)  * 0.498,
    k                 = exp(d$b_logk_Intercept),
    `mid (intercept)` = d$b_mid_Intercept,
    `mid (slope)`     = d$b_mid_temp_c
  )
  purrr::map_dfr(names(draws), \(p) tibble::tibble(
    param  = p,
    median = stats::median(draws[[p]]),
    lower  = stats::quantile(draws[[p]], 0.025, names = FALSE),
    upper  = stats::quantile(draws[[p]], 0.975, names = FALSE)
  ))
}

truth <- tibble::tibble(
  param = c("ell", "u", "k", "mid (intercept)", "mid (slope)"),
  truth = c(true_params$ell, true_params$u, true_params$k,
            true_params$m_beta0, true_params$m_beta1)
)

recovery <- truth |>
  dplyr::left_join(post_summary(fit_bin) |>
                     dplyr::rename_with(~ paste0("bin_", .x), -param),
                   by = "param") |>
  dplyr::left_join(post_summary(fit_bb) |>
                     dplyr::rename_with(~ paste0("bb_", .x), -param),
                   by = "param") |>
  dplyr::rename(
    Parameter        = param,
    Truth            = truth,
    `Bin. median`    = bin_median,
    `Bin. lower`     = bin_lower,
    `Bin. upper`     = bin_upper,
    `BB median`      = bb_median,
    `BB lower`       = bb_lower,
    `BB upper`       = bb_upper
  )

tinytable::tt(recovery, digits = 3, escape = TRUE)
```

We can also overlay the posterior fitted curves on the simulated data (Figure S3). If the model has recovered the true surface, the fitted lines should track the simulated proportions at every assay

Table S3: Posterior medians and 95% credible intervals for the four 4PL parameters and the temperature slope, compared to the known truth used to generate the simulated data. Recovery is judged successful when the credible interval covers the truth.

Parameter	Truth	Bin. median	Bin. lower	Bin. upper	BB median	BB lower	BB upper
ell	0.03	0.029	0.0245	0.0335	0.0239	0.0141	0.0346
u	0.97	0.967	0.9633	0.9709	0.9701	0.962	0.9772
k	8	8.316	7.8925	8.7709	7.3271	6.5893	8.176
mid (intercept)	1.5	1.501	1.4927	1.5099	1.5176	1.4969	1.5378
mid (slope)	-0.15	-0.15	-0.1525	-0.1464	-0.1496	-0.157	-0.1423

temperature.

```r
# Use the library's predict_survival_curves() and plot_survival_curves().
psc_bin <- predict_survival_curves(
  wf_bin, temps = temps,
  durations = 10 ^ seq(log10(0.5), log10(800), length.out = 100),
  ndraws = 200
)

plot_survival_curves(psc_bin, observed = std_bin) +
  ggplot2::scale_x_log10() +
  ggplot2::labs(x = expression(exposure~time~(min)))
```

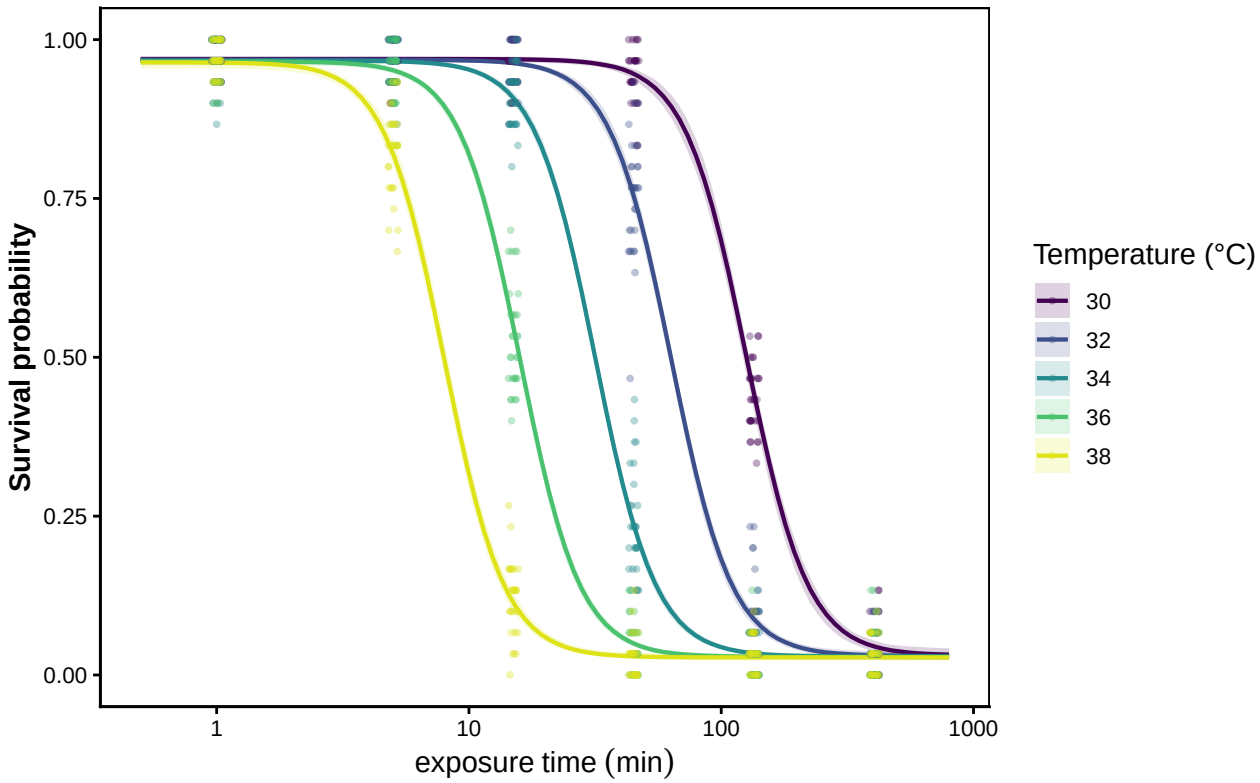


Figure S3: Posterior fitted 4PL curves (solid lines, with shaded 95% credible bands) overlaid on the simulated binomial data (points). One panel-coloured line per assay temperature. Where the fitted curves track the simulated proportions across all five temperatures, the model has recovered the underlying dose-response surface.

2.0.4 *Deriving z, $CT_{max_{1hr}}$, and T_{crit} from the posterior*

The classical TLS quantities are transformations of the fitted joint posterior. `extract_tdt()` always returns the first two quantities below and returns the third when `lethal = TRUE`:

- z — thermal sensitivity, read directly from the posterior as $z = -1/(\mathrm{d}\log_{10}\mathrm{LT}_{\mathrm{threshold}}/\mathrm{d}T)$ per draw (Equation 7 of the manuscript). Under the relative threshold $\log_{10}\mathrm{LT}(T) = \mathrm{mid}(T)$ is linear, so $z = -1/\beta_1$ exactly; no regression is involved.

- $CT_{max_{1hr}}$ — temperature at which the threshold is reached after 1 h of exposure, obtained from the fitted 4PL at the reference time, per posterior draw.
- T_{crit} *(only when* `lethal = TRUE`*)* — the temperature below which damage accumulates more slowly than a chosen rate floor. The damage rate at temperature T in the TDT framework is $r(T) = 100 \cdot 10^{(T - CT_{max,1hr})/z}$ % LT50-dose per hour, so inverting for a chosen rate floor r^* gives

$$T_{crit}(r^*) \;=\; CT_{max,1hr} \;+\; z \cdot \log_{10}(r^*/100). \tag{S3}$$

The threshold is set by `target_surv`. The default `"relative"` evaluates at (low + up)/2 per draw, which equals classical absolute 50% survival when asymptotes are anchored at 0 and 1 but accommodates sub-unit upper asymptotes (for example, background mortality unrelated to heat stress; see Section 5.5). Passing `target_surv = "absolute"` or any numeric value in $(0, 1)$ uses an absolute survival threshold. Because the simulated asymptotes are near 0 and 1, both definitions give essentially the same results here.

Different r^* values imply different T_{crit} thresholds, and the literature does not converge on one value. Empirical breakpoint analyses across taxa including *Drosophila suzukii* and *Lemna gibba* place r^* in the range $[0.1, 1]$ % HI per hour, corresponding to approximately 100 to 1000 hours (about 4 to 42 days) of continuous exposure to reach LT_{50} (Arnold *et al.* 2025; Faber *et al.* 2026; Jørgensen *et al.* 2021). We therefore integrate over a uniform prior on $\log_{10} r^*$ in that range. The resulting posterior for T_{crit} includes parameter uncertainty in $CT_{max,1hr}$ and z as well as operational uncertainty in r^*. Its median is $CT_{max,1hr} - 2.5 \cdot z$, corresponding to $r^* \approx 0.3$ % HI per hour.

> ⚠ T_{crit} is meaningful only for lethal-endpoint data
>
> The rate-multiplier T_{crit} in Equation Equation S3 interprets z as the temperature sensitivity of *damage accumulation*, which is appropriate only for lethal-endpoint dose-response data (or irreversible knock-out endpoints). For **sublethal** endpoints such as knockdown time or photosystem-II decline, fitted z instead describes performance loss. Substituting that value into Equation Equation S3 can shift T_{crit} substantially and even reverse biological ordering outside the observed assay window. In the shrimp example below, the smaller sublethal z produces a T_{crit} about 3.5 °C higher than the lethal estimate, despite similar $CT_{max_{1hr}}$. Interpreting T_{crit} as an intersection between thermal-performance and thermal-death-time curves requires both lethal and sublethal data (Faber *et al.* 2026; Jørgensen *et al.* 2021; Ørsted *et al.* 2022).
> `extract_tdt()` reflects this by making T_{crit} **opt-in**: the default `lethal = FALSE` returns only z and $CT_{max_{1hr}}$, and users with lethal-endpoint data set `lethal = TRUE` to additionally derive T_{crit}. When `lethal = TRUE`, a one-line console message reiterates the lethal-only validity of the quantity.

The two always-returned quantities inherit the same posterior, so every credible interval reflects joint uncertainty in $(\ell, u, k, \beta_0, \beta_1)$. The opt-in T_{crit} pools that same posterior with the additional uniform prior on $\log_{10} r^*$.

> **Note**
>
> The default `fit_4pl()` puts a `temp_c` slope on every 4PL sub-parameter, so ℓ, u and k can each carry a linear effect of temperature in addition to the linear-mid term — when those slopes shrink to zero under the prior the model reduces cleanly to the constant-shape case. When the data carry real temperature signal on any of the shape parameters, the asymmetry correction $\frac{1}{k(T)} \log \frac{u(T)-0.5}{0.5-\ell(T)}$ varies with T and the resulting $\log_{10} \mathrm{LT50}(T)$ curve is bent rather than straight:
>
> $$\log_{10} \mathrm{LT50}(T) \;=\; \beta_0 + \beta_1 (T - \bar{T}) \;+\; \tfrac{1}{k(T)} \log \tfrac{u(T)-0.5}{0.5-\ell(T)}. \tag{S4}$$
>
> `extract_tdt()` requires no special handling in either regime, because both quantities are read **directly from the joint posterior** rather than by fitting a line. It extracts the population-level 4PL coefficient draws once, subsamples once, and computes z and $CT_{max_{1hr}}$ from that single set of draws — so the two quantities share draws and their correlation (and the joint pairing used for T_{crit}) is preserved. $CT_{max_{1hr}}$ is the temperature at which $\log_{10} \mathrm{LT}(T)$ crosses t_{ref}, per draw (closed-form inversion of $\mathrm{mid}(T)$ under the relative threshold; interpolated crossing of the closed-form LT curve under an absolute threshold). For z, under the relative threshold $\log_{10} \mathrm{LT}(T) = \mathrm{mid}(T)$ is exactly linear, so $z = -1/\beta_1$ per draw. Under an *absolute* threshold the asymmetry-correction term $\frac{1}{k(T)} \log \frac{u(T)-0.5}{0.5-\ell(T)}$ bends the curve when u, ℓ or k carry T effects; then a per-draw **local** $z(T) = -1/(\mathrm{d} \log_{10} \mathrm{LT}_{0.5}/\mathrm{d}T)$ is computed at each assay temperature (slope by central finite difference of the closed-form LT curve, validated against the analytic derivative), and the reported z is the per-draw mean of those local values over the assay range — with the full per-temperature $z(T)$ available via `z_local = TRUE`. The worked example in Section 2.0.6 demonstrates this with a simulation where u and k carry strong T effects.

```
# Data are in minutes (standardize_data was called with duration_unit = "minutes"),
# so set time_multiplier = 1 to keep the model's time axis unchanged on output,
# and t_ref = 60 to read CTmax at the 1-hour reference. The simulated data are
# binomial / beta-binomial *lethal* survival counts, so we opt into T_crit via
# the rate-multiplier integration (lethal = TRUE; default TC_rate_range =
# c(0.1, 1) % HI per hour). See the next paragraph for why T_crit is opt-in.
et_bin <- extract_tdt(wf_bin, t_ref = 60, time_multiplier = 1,
                      ndraws = 1000, lethal = TRUE)
et_bb  <- extract_tdt(wf_bb,  t_ref = 60, time_multiplier = 1,
                      ndraws = 1000, lethal = TRUE)
```

```
# Helper to pull (median, lower, upper) out of either a $z or temperature summary.
row_from_summary <- function(s, kind = c("z", "temp")) {
  kind <- match.arg(kind)
  if (kind == "z") {
    c(s$z_median, s$z_lower, s$z_upper)
  } else {
    c(s$temp_median, s$temp_lower, s$temp_upper)
  }
}
```

Table S4: Posterior medians and 95% credible intervals for thermal sensitivity (z), the critical thermal limit at a 1-hour reference duration ($CT_{max_{1hr}}$), and the rate-multiplier critical temperature (T_{crit}, integrated over $r^* \in [0.1, 1]$ % HI per hour). All three derived via `extract_tdt()` from a single posterior. Each row's truth value is computed analytically from `true_params`.

Quantity	Truth	Bin. median	Bin. lower	Bin. upper	BB median	BB lower	BB upper
z (°C)	6.67	6.69	6.57	6.83	6.69	6.37	7.06
CTmax at 1 hr (°C)	32.15	32.15	32.08	32.22	32.26	32.1	32.4
T_crit (°C)	15.48	15.46	12.18	18.59	15.51	12.04	18.81

```
tls_recovery <- tibble::tibble(
  Quantity = c("z (°C)", "CTmax at 1 hr (°C)", "T_crit (°C)"),
  Truth    = c(true_z, true_CTmax, true_T_crit),
  `Bin. median` = c(row_from_summary(et_bin$z$summary,      "z")[1],
                    row_from_summary(et_bin$CTmax$summary,  "temp")[1],
                    row_from_summary(et_bin$T_crit$summary, "temp")[1]),
  `Bin. lower`  = c(row_from_summary(et_bin$z$summary,      "z")[2],
                    row_from_summary(et_bin$CTmax$summary,  "temp")[2],
                    row_from_summary(et_bin$T_crit$summary, "temp")[2]),
  `Bin. upper`  = c(row_from_summary(et_bin$z$summary,      "z")[3],
                    row_from_summary(et_bin$CTmax$summary,  "temp")[3],
                    row_from_summary(et_bin$T_crit$summary, "temp")[3]),
  `BB median`   = c(row_from_summary(et_bb$z$summary,      "z")[1],
                    row_from_summary(et_bb$CTmax$summary,  "temp")[1],
                    row_from_summary(et_bb$T_crit$summary, "temp")[1]),
  `BB lower`    = c(row_from_summary(et_bb$z$summary,      "z")[2],
                    row_from_summary(et_bb$CTmax$summary,  "temp")[2],
                    row_from_summary(et_bb$T_crit$summary, "temp")[2]),
  `BB upper`    = c(row_from_summary(et_bb$z$summary,      "z")[3],
                    row_from_summary(et_bb$CTmax$summary,  "temp")[3],
                    row_from_summary(et_bb$T_crit$summary, "temp")[3])
)

tinytable::tt(tls_recovery, digits = 3, escape = TRUE)
```

```
# Per-draw posteriors for the three quantities, both likelihoods. The
# accessors return one-column tibbles keyed on `.draw`, so we join on
# `.draw` to keep posterior draws paired correctly.
draws_bin_d <- get_z_draws(et_bin) |>
  dplyr::inner_join(get_ctmax_draws(et_bin), by = ".draw") |>
  dplyr::inner_join(get_tcrit_draws(et_bin), by = ".draw") |>
  dplyr::transmute(z, CTmax_1hr = CTmax, T_crit, model = "Binomial")
draws_bb_d <- get_z_draws(et_bb) |>
  dplyr::inner_join(get_ctmax_draws(et_bb),  by = ".draw") |>
  dplyr::inner_join(get_tcrit_draws(et_bb),  by = ".draw") |>
  dplyr::transmute(z, CTmax_1hr = CTmax, T_crit, model = "Beta-binomial")

post_long <- dplyr::bind_rows(draws_bin_d, draws_bb_d) |>
  tidyr::pivot_longer(c(z, CTmax_1hr, T_crit),
                      names_to = "quantity", values_to = "value") |>
  dplyr::mutate(quantity = factor(quantity,
                                  levels = c("z", "CTmax_1hr", "T_crit"),
                                  labels = c("z (°C)",
                                             "CTmax_1hr (°C)",
                                             "T_crit (°C)")))

truth_long <- tibble::tibble(
  quantity = factor(c("z", "CTmax_1hr", "T_crit"),
                    levels = c("z", "CTmax_1hr", "T_crit"),
                    labels = levels(post_long$quantity)),
  truth    = c(true_z, true_CTmax, true_T_crit)
)

ggplot2::ggplot(post_long, ggplot2::aes(value, fill = model)) +
  ggplot2::geom_density(alpha = 0.5, colour = NA) +
  ggplot2::geom_vline(data = truth_long,
                      ggplot2::aes(xintercept = truth),
                      linetype = "dashed", colour = "black") +
  ggplot2::facet_wrap(~ quantity, scales = "free") +
  ggplot2::labs(x = NULL, y = "Posterior density", fill = "Model") +
  ggplot2::theme_classic()
```

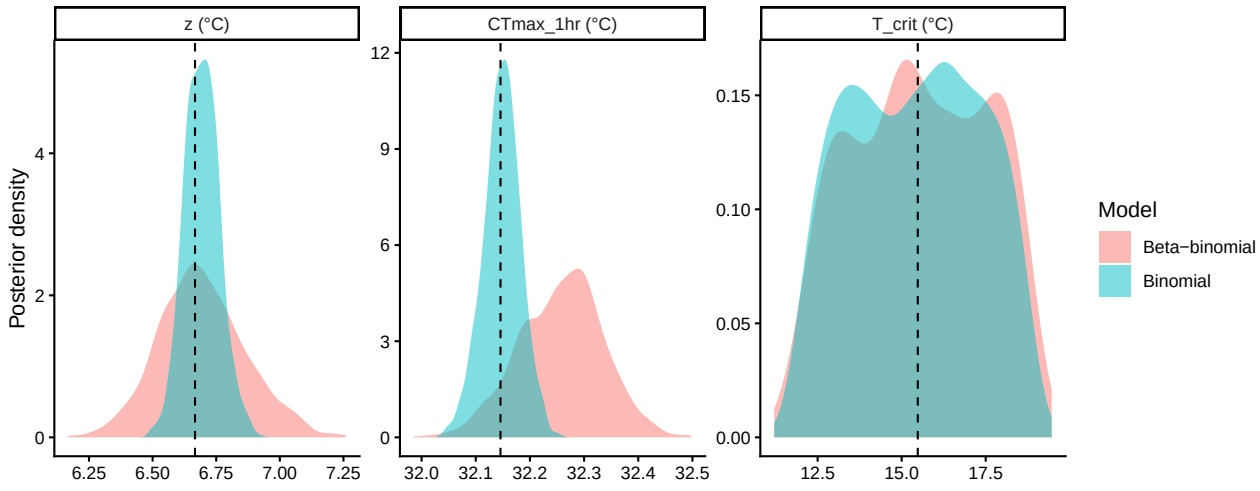


Figure S4: Posterior densities of z, $CT_{max_{1hr}}$, and T_{crit} from the binomial (blue) and beta-binomial (orange) joint fits. Vertical dashed lines mark the simulation truth. All three posteriors concentrate around their respective truths, illustrating that a single call to

Both fits recover all three simulation truths (Table S4, Figure S4), with credible intervals reflecting joint uncertainty in the 4PL parameters. In the joint approach, every classical TLS quantity is derived from the same fitted posterior.

Posterior summary statistics — mean, SD, median, and 95% credible interval — are also readily available from the `$draws` slot of `extract_tdt()`'s output:

```
summarise_draws <- function(x) {
  q <- stats::quantile(x, c(0.025, 0.5, 0.975), names = FALSE, na.rm = TRUE)
  tibble::tibble(mean = mean(x, na.rm = TRUE),
                 sd   = stats::sd(x, na.rm = TRUE),
                 median = q[2], lower = q[1], upper = q[3])
}

uncert_summary <- dplyr::bind_rows(
  summarise_draws(get_z_draws(et_bin)$z)          |> dplyr::mutate(quantity = "z (°C)",
  summarise_draws(get_ctmax_draws(et_bin)$CTmax)  |> dplyr::mutate(quantity = "CTmax at 1 hr (
  summarise_draws(get_tcrit_draws(et_bin)$T_crit) |> dplyr::mutate(quantity = "T_crit (°C)",
  summarise_draws(get_z_draws(et_bb)$z)           |> dplyr::mutate(quantity = "z (°C)",
  summarise_draws(get_ctmax_draws(et_bb)$CTmax)   |> dplyr::mutate(quantity = "CTmax at 1 hr (
  summarise_draws(get_tcrit_draws(et_bb)$T_crit)  |> dplyr::mutate(quantity = "T_crit (°C)",
) |>
  dplyr::select(quantity, likelihood, mean, sd, median, lower, upper)

# Scalars for inline reporting in the next paragraph.
sd_z_bin  <- round(stats::sd(get_z_draws(et_bin)$z),         2)
sd_z_bb   <- round(stats::sd(get_z_draws(et_bb)$z),          2)
sd_ct_bin <- round(stats::sd(get_ctmax_draws(et_bin)$CTmax), 2)
sd_ct_bb  <- round(stats::sd(get_ctmax_draws(et_bb)$CTmax),  2)
ci_z_bin  <- round(diff(stats::quantile(get_z_draws(et_bin)$z,         c(0.025, 0.975), names =
ci_z_bb   <- round(diff(stats::quantile(get_z_draws(et_bb)$z,          c(0.025, 0.975), names =
ci_ct_bin <- round(diff(stats::quantile(get_ctmax_draws(et_bin)$CTmax, c(0.025, 0.975), names =
ci_ct_bb  <- round(diff(stats::quantile(get_ctmax_draws(et_bb)$CTmax,  c(0.025, 0.975), names =

uncert_summary |>
  dplyr::rename(
    Quantity   = quantity,
    Likelihood = likelihood,
    Mean       = mean,
    SD         = sd,
    Median     = median,
    Lower      = lower,
    Upper      = upper
  ) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

Both z and $CT_{max_{1hr}}$ are estimated precisely, and their credible intervals come directly from the joint posterior. A frequentist joint fit could similarly quantify transformation uncertainty using the

Table S5: Posterior summaries of z, $CT_{max_{1hr}}$, and T_{crit} from the joint Bayesian 4PL: posterior mean, standard deviation, median, and 95% credible interval, reported separately for the binomial and beta-binomial fits.

Quantity	Likelihood	Mean	SD	Median	Lower	Upper
z (°C)	Binomial	6.69	0.0689	6.69	6.57	6.83
CTmax at 1 hr (°C)	Binomial	32.15	0.0344	32.15	32.08	32.22
T_crit (°C)	Binomial	15.4	1.916	15.46	12.18	18.59
z (°C)	Beta-binomial	6.69	0.1708	6.69	6.37	7.06
CTmax at 1 hr (°C)	Beta-binomial	32.26	0.0772	32.26	32.1	32.4
T_crit (°C)	Beta-binomial	15.53	1.9997	15.51	12.04	18.81

delta method or a bootstrap.

Under the binomial likelihood, the posterior SD for z is 0.07 °C and its 95% credible interval is 0.27 °C wide; $CT_{max_{1hr}}$ has SD 0.03 °C and interval width 0.14 °C.

The beta-binomial fit is slightly wider for both quantities (SD for z: 0.17 °C; for $CT_{max_{1hr}}$: 0.08 °C) because ϕ captures extra-binomial variation and propagates it into the derived quantities. A two-stage pipeline requires an additional bootstrap or delta-method calculation to achieve comparable propagation.

> **!** Important
>
> For real data, compare binomial and beta-binomial fits rather than assuming either likelihood is adequate. When extra dispersion is present, estimating it can materially affect uncertainty and inference.

2.0.5 *Comparing the two pipelines*

2.0.5.1 Point-estimate agreement on the same data

Table S6 puts the two-stage point estimates next to the joint Bayesian posterior medians for both simulated datasets.

```
compare_table <- tibble::tibble(
  quantity   = c("z (°C)", "CTmax at 1 hr (°C)"),
  truth      = c(true_z, true_CTmax),
  ts_bin     = ts_bin$est,
  joint_bin  = c(median(draws_bin_d$z),
                 median(draws_bin_d$CTmax_1hr)),
  ts_bb      = ts_bb$est,
  joint_bb   = c(median(draws_bb_d$z),
                 median(draws_bb_d$CTmax_1hr))
)
```

Table S6: Point estimates (two-stage) and posterior medians (joint Bayesian) for thermal sensitivity (z) and $CT_{max_{1hr}}$, recovered by each of four estimator–dataset combinations from the same simulated datasets. Truth values are the simulation parameters.

Quantity	Truth	Two-stage (binomial)	Joint Bayesian (binomial)	Two-stage (beta-binomial)
z (°C)	6.67	6.91	6.69	6.9
CTmax at 1 hr (°C)	32.15	32.03	32.15	32.1

```
compare_table |>
  dplyr::rename(
    Quantity                          = quantity,
    Truth                             = truth,
    `Two-stage (binomial)`            = ts_bin,
    `Joint Bayesian (binomial)`       = joint_bin,
    `Two-stage (beta-binomial)`       = ts_bb,
    `Joint Bayesian (beta-binomial)`  = joint_bb
  ) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

All four estimators give similar point estimates for z and $CT_{max_{1hr}}$, close to the simulation truth. Under this simple generating scenario, the joint Bayesian 4PL reproduces the classical pipeline's central estimates while retaining a coherent posterior for downstream inference.

2.0.6 *A worked example: temperature effects on every shape parameter*

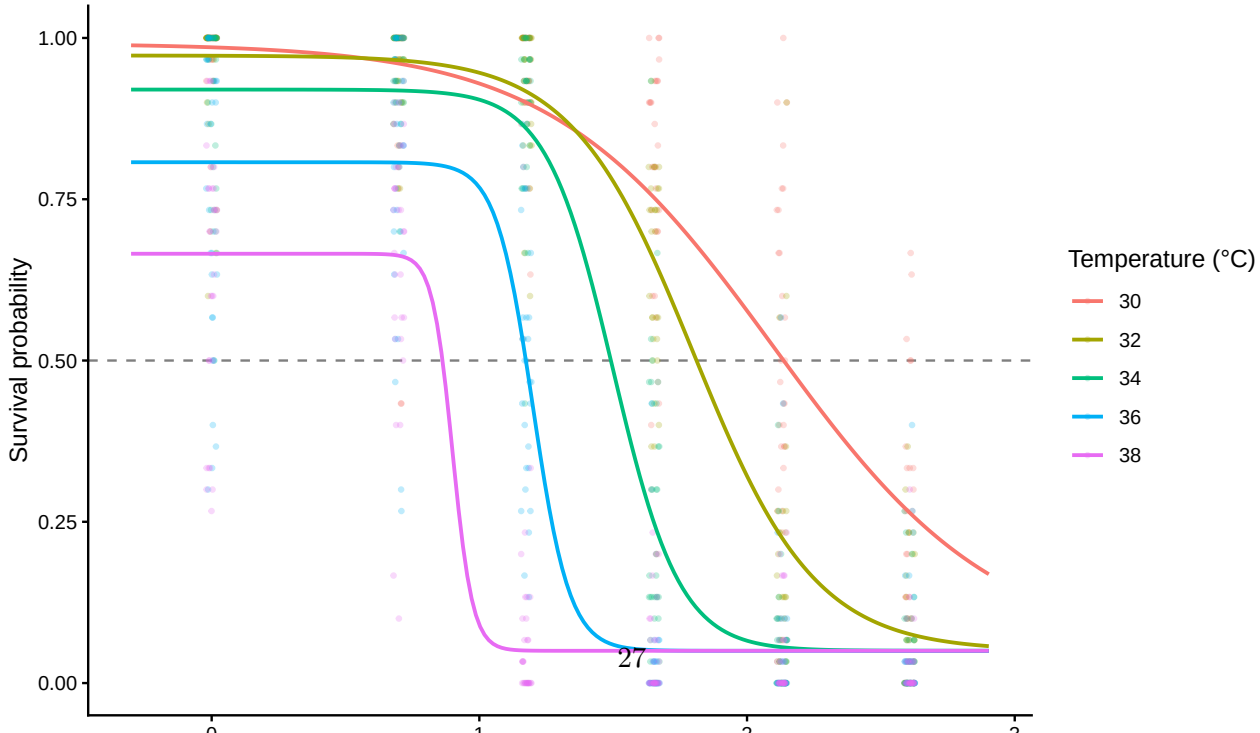
The default `fit_4pl()` places a `temp_c` slope on ℓ, u, and k, estimating temperature effects on curve shape alongside the linear midpoint. To test recovery when the constant-shape assumption fails, we re-simulate beta-binomial data on the design in Section 2.0.1.2 with u declining and k increasing sharply with T. We then apply the same `fit_4pl()` and `extract_tdt()` calls used above, without a specialised formula or priors.

```
# Same factorial design as @sec-sim-step2; re-declared here so this chunk is
# self-contained.
temps     <- c(30, 32, 34, 36, 38)
durations <- c(1, 5, 15, 45, 135, 405)
n_rep     <- 30
n_per_rep <- 30
phi       <- 5

# Truth on the same raw scale fit_4pl() uses internally (so we can compare the
# posterior coefficients to truth directly):
#   ell = 0.49 * inv_logit(ell_raw)                    (held constant in T)
#   u   = 0.51 + 0.49 * inv_logit(u_raw_0 + u_raw_T*T_c)  (declines in T)
#   k   = exp(log_k_0 + log_k_T*T_c)                      (rises in T)
```

```r
true_params_ext <- list(
  ell_raw  = qlogis(0.05 / 0.49),                # ell = 0.05
  u_raw_0  = qlogis((0.92 - 0.51) / 0.49),       # u = 0.92 at T_bar
  u_raw_T  = -0.60,                               # strong decline in u with T
  log_k_0  = log(8),                              # k = 8 at T_bar
  log_k_T  = 0.30,                                # strong rise in k with T
  m_beta0  = 1.5,
  m_beta1  = -0.15,
  T_bar    = 34
)

design <- tidyr::expand_grid(
  T   = temps,
  t   = durations,
  rep = seq_len(n_rep)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params_ext$T_bar,
    n       = n_per_rep
  )

sim_bb_ext <- design |>
  dplyr::mutate(
    ell    = 0.49 * plogis(true_params_ext$ell_raw),
    u      = 0.51 + 0.49 * plogis(true_params_ext$u_raw_0 +
                                  true_params_ext$u_raw_T * T_c),
    k      = exp(true_params_ext$log_k_0 + true_params_ext$log_k_T * T_c),
    mid    = true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c,
    p_true = ell + (u - ell) / (1 + exp(k * (log10_t - mid))),
    p_draw = rbeta(dplyr::n(), p_true * phi, (1 - p_true) * phi),
    y      = rbinom(dplyr::n(), size = n, prob = p_draw)
  )
```

Here the lower asymptote remains at $\ell = 0.05$, while u falls from 0.99 at $T = 30$ to 0.67 at $T = 38$, and k rises from 2.4 to 26.6. The extreme-temperature curves therefore differ in shape, not merely position (Figure S5).

```
truth_grid_ext <- tidyr::expand_grid(
  T = temps,
  t = 10 ^ seq(log10(0.5), log10(800), length.out = 200)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params_ext$T_bar,
    ell     = 0.49 * plogis(true_params_ext$ell_raw),
    u       = 0.51 + 0.49 * plogis(true_params_ext$u_raw_0 +
                                   true_params_ext$u_raw_T * T_c),
    k       = exp(true_params_ext$log_k_0 + true_params_ext$log_k_T * T_c),
    mid     = true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c,
    p_true  = ell + (u - ell) / (1 + exp(k * (log10_t - mid)))
  )

ggplot2::ggplot() +
  ggplot2::geom_hline(yintercept = 0.5, linetype = "dashed",
                      colour = "grey50") +
  ggplot2::geom_jitter(
    data = sim_bb_ext,
    ggplot2::aes(x = log10_t, y = y / n, colour = factor(T)),
    width = 0.02, height = 0, alpha = 0.25, size = 0.7
  ) +
  ggplot2::geom_line(
    data = truth_grid_ext,
    ggplot2::aes(x = log10_t, y = p_true, colour = factor(T)),
    linewidth = 0.8
  ) +
  ggplot2::labs(x = expression(log[10]~exposure~time~(min)),
                y = "Survival probability",
                colour = "Temperature (°C)") +
  ggplot2::theme_classic()
```



Before fitting, we compute the analytical targets: $\log_{10}\mathrm{LT50}(T)$, local thermal sensitivity $z(T)$ (negative reciprocal of the local slope of $\log_{10}\mathrm{LT50}$ on T), and $CT_{max_{1hr}}$ (the temperature at which $\log_{10}\mathrm{LT50}$ crosses $\log_{10} 60$).

```
# True log10(LT50) at any T from the T-varying 4PL: solve p(log10_t) = 0.5
# for log10_t. With the 4PL,
#   log10_t = mid + (1/k) * log((u - 0.5) / (0.5 - ell))
true_log10_LT50 <- function(T_value) {
  T_c <- T_value - true_params_ext$T_bar
  ell <- 0.49 * plogis(true_params_ext$ell_raw)
  u   <- 0.51 + 0.49 * plogis(true_params_ext$u_raw_0 +
                              true_params_ext$u_raw_T * T_c)
  k   <- exp(true_params_ext$log_k_0 + true_params_ext$log_k_T * T_c)
  mid <- true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c
  mid + (1 / k) * log((u - 0.5) / (0.5 - ell))
}

# Local z(T) via central finite difference of true log10(LT50)(T).
true_local_z <- function(T_at, half_step = 0.5) {
  -1 / ((true_log10_LT50(T_at + half_step) -
         true_log10_LT50(T_at - half_step)) / (2 * half_step))
}

# True CTmax_1hr: search the fine grid for the T at which log10(LT50)
# crosses log10(60).
T_grid_truth   <- seq(20, 45, by = 0.001)
ll_grid_truth  <- vapply(T_grid_truth, true_log10_LT50, numeric(1))
true_CTmax_1hr <- T_grid_truth[which.min(abs(ll_grid_truth - log10(60)))]

truth_ext <- tibble::tibble(
  T               = temps,
  log10_LT50_true = vapply(temps, true_log10_LT50, numeric(1)),
  LT50_min_true   = 10 ^ log10_LT50_true,
  local_z_true    = vapply(temps, true_local_z, numeric(1))
)

true_mean_local_z <- mean(truth_ext$local_z_true)
```

Across the assay range the true $\log_{10}\mathrm{LT50}(T)$ curve is bent: the implied local z varies from 6.05 °C at $T = 30$ to 6.53 °C at $T = 38$ (truth mean $z = 6.28$ °C). The truth implies $CT_{max_{1hr}} = 32.21$ °C. With straight-line $\log_{10}\mathrm{LT50}(T)$ — the constant-shape case — local z would not vary across temperature at all.

We now fit with exactly the same `fit_4pl()` call as before. The default formula already puts a `temp_c` slope on each of the four 4PL sub-parameters, so the model has the capacity to recover the truth's temperature signal on u and k without any modification.

```
std_ext <- standardize_data(sim_bb_ext,
                            temp = "T", duration = "t",
                            n_total = "n", n_surv = "y",
                            duration_unit = "minutes")

wf_ext <- fit_4pl(std_ext,
                  chains = 4, iter = 2000, cores = 4, seed = 123,
                  file       = file.path(models_dir,
                                         "sim_4pl_ext_betabin_DEFAULT_strong_v1"),
                  file_refit = "never")     # see fit-4pl-tutorial note above
```

Table S7 reports recovery of the four 4PL sub-parameters on their natural scale at each assay temperature. The posterior keeps ℓ near 0.05, while recovering the decline in u and increase in k across the assay range.

```
ext_post  <- posterior::as_draws_df(get_brmsfit(wf_ext)) |> as.data.frame()
ext_meta  <- wf_ext$meta
ext_bnds  <- ext_meta$bounds         # compute_4pl_bounds(0, 1)
ext_temp_mean <- ext_meta$temp_mean  # what the fit centred on

ext_param_long <- purrr::map_dfr(temps, function(T_value) {
  T_c_fit  <- T_value - ext_temp_mean
  T_c_true <- T_value - true_params_ext$T_bar

  draws_at_T <- tibble::tibble(
    ell = ext_bnds$low_min +
          stats::plogis(ext_post$b_lowraw_Intercept +
                        ext_post$b_lowraw_temp_c * T_c_fit) * ext_bnds$low_w,
    u   = ext_bnds$up_min  +
          stats::plogis(ext_post$b_upraw_Intercept +
                        ext_post$b_upraw_temp_c * T_c_fit)  * ext_bnds$up_w,
    k   = exp(ext_post$b_logk_Intercept + ext_post$b_logk_temp_c * T_c_fit),
    mid = ext_post$b_mid_Intercept + ext_post$b_mid_temp_c * T_c_fit
  )

  truth_at_T <- tibble::tibble(
    ell = 0.49 * plogis(true_params_ext$ell_raw),
    u   = 0.51 + 0.49 * plogis(true_params_ext$u_raw_0 +
                               true_params_ext$u_raw_T * T_c_true),
    k   = exp(true_params_ext$log_k_0 +
              true_params_ext$log_k_T * T_c_true),
    mid = true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c_true
  )

  purrr::map_dfr(c("ell", "u", "k", "mid"), function(p) {
    q <- stats::quantile(draws_at_T[[p]], c(0.025, 0.5, 0.975),
                         na.rm = TRUE, names = FALSE)
```

```
    tibble::tibble(
      Parameter         = p,
      `T (°C)`         = T_value,
      Truth             = truth_at_T[[p]],
      Median            = q[2],
      Lower             = q[1],
      Upper             = q[3]
    )
  })
})

ext_param_long |>
  tinytable::tt(digits = 3, escape = TRUE)
```

Now we apply the same `extract_tdt()` call as in Section 2.0.4 — there is no separate "T-varying" workflow because `extract_tdt()` already inverts the fitted surface numerically per draw:

```
et_ext <- extract_tdt(wf_ext, t_ref = 60, time_multiplier = 1,
                      ndraws = 1000, lethal = TRUE)
```

Table S8 compares the three classical TDT quantities from `extract_tdt()` to the analytical truth derived above.

```
tibble::tibble(
  Quantity     = c("z (°C)",
                   "CTmax at 1 hr (°C)",
                   "T_crit (°C)"),
  Truth        = c(round(true_mean_local_z, 3),
                   round(true_CTmax_1hr, 3),
                   NA_real_),  # T_crit truth involves r* integration; see below
  Median       = c(et_ext$z$summary$z_median,
                   et_ext$CTmax$summary$temp_median,
                   et_ext$T_crit$summary$temp_median),
  Lower        = c(et_ext$z$summary$z_lower,
                   et_ext$CTmax$summary$temp_lower,
                   et_ext$T_crit$summary$temp_lower),
  Upper        = c(et_ext$z$summary$z_upper,
                   et_ext$CTmax$summary$temp_upper,
                   et_ext$T_crit$summary$temp_upper)
) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

Both posterior intervals cover their analytical truths. The single-summary z read from the posterior ($-1/\beta_1$ per draw under the relative threshold) is close to the average local sensitivity of the bent curve, and its credible interval covers the truth (truth = 6.28 °C; posterior median 6.65 °C, 95% CrI [6.14, 7.3] °C). The bent absolute-threshold local $z(T)$ itself is available with `target_surv`

Table S7: Posterior recovery of the four 4PL sub-parameters on the natural scale at each assay temperature. The fit uses the disjoint-bounds reparameterisation (ℓ, u via inv_logit, k via exp); raw posterior columns are transformed back to the biological scale before summarising. Posterior medians and 95% credible intervals are reported alongside the analytical truth.

Parameter	T (°C)	Truth	Median	Lower	Upper
ell	30	0.05	0.0625	0.0293	0.1084
u	30	0.991	0.9935	0.989	0.9962
k	30	2.41	2.4043	2.1161	2.7386
mid	30	2.1	2.1065	2.0379	2.1701
ell	32	0.05	0.0536	0.0311	0.0797
u	32	0.973	0.9791	0.9697	0.9863
k	32	4.39	4.5723	4.1102	5.1149
mid	32	1.8	1.8061	1.7625	1.847
ell	34	0.05	0.0457	0.0328	0.0596
u	34	0.92	0.932	0.9166	0.9463
k	34	8	8.7122	7.1951	10.6985
mid	34	1.5	1.5056	1.4818	1.5292
ell	36	0.05	0.0391	0.0317	0.0475
u	36	0.807	0.8168	0.7885	0.842
k	36	14.577	16.5544	12.261	22.8179
mid	36	1.2	1.2049	1.1814	1.2328
ell	38	0.05	0.0335	0.0248	0.0444
u	38	0.666	0.6592	0.6201	0.6995
k	38	26.561	31.5023	20.6197	48.8663
mid	38	0.9	0.9041	0.8624	0.9533

Table S8: Classical TDT quantities from `extract_tdt()` on the T-varying fit, compared against analytical truth. The single-summary z is read directly from the posterior (under the relative threshold $z = -1/\beta_1$ per draw — the slope of mid(T), with no regression); the truth row reports the mean of the analytical local $z(T)$ of the bent absolute-0.5 $\log_{10}$ LT50(T) curve at the assay temperatures. $CT_{max_{1hr}}$ comes from direct inversion of the fitted 4PL at 60 min per draw.

Quantity	Truth	Median	Lower	Upper
z (°C)	6.28	6.65	6.14	7.3
CTmax at 1 hr (°C)	32.21	32.18	31.89	32.4
T_crit (°C)	NA	15.44	11.95	19.1

= "absolute", z_local = TRUE. Direct 4PL inversion at the reference duration also recovers $CT_{max_{1hr}}$ (truth 32.21 °C; posterior median 32.18 °C, 95% CrI [31.89, 32.43] °C), without assuming a constant asymmetry correction in T.

The bent $\log_{10}$ LT50(T) curve itself is the intermediate object both quantities are built from; it lives in `et_ext$lt50_curve` (the output of `derive_tdt_curve()`) and can be plotted directly (Figure S6).

```
truth_overlay <- {
  Ts <- seq(min(temps) - 2, max(temps) + 2, by = 0.05)
  tibble::tibble(temp = Ts,
                 duration_median = 10 ^ vapply(Ts, true_log10_LT50,
                                               numeric(1)))
}

plot_tdt_curve(et_ext$lt50_curve) +
  ggplot2::geom_line(data = truth_overlay,
                     ggplot2::aes(x = temp, y = duration_median),
                     linetype = "dashed", colour = "black",
                     inherit.aes = FALSE)
```

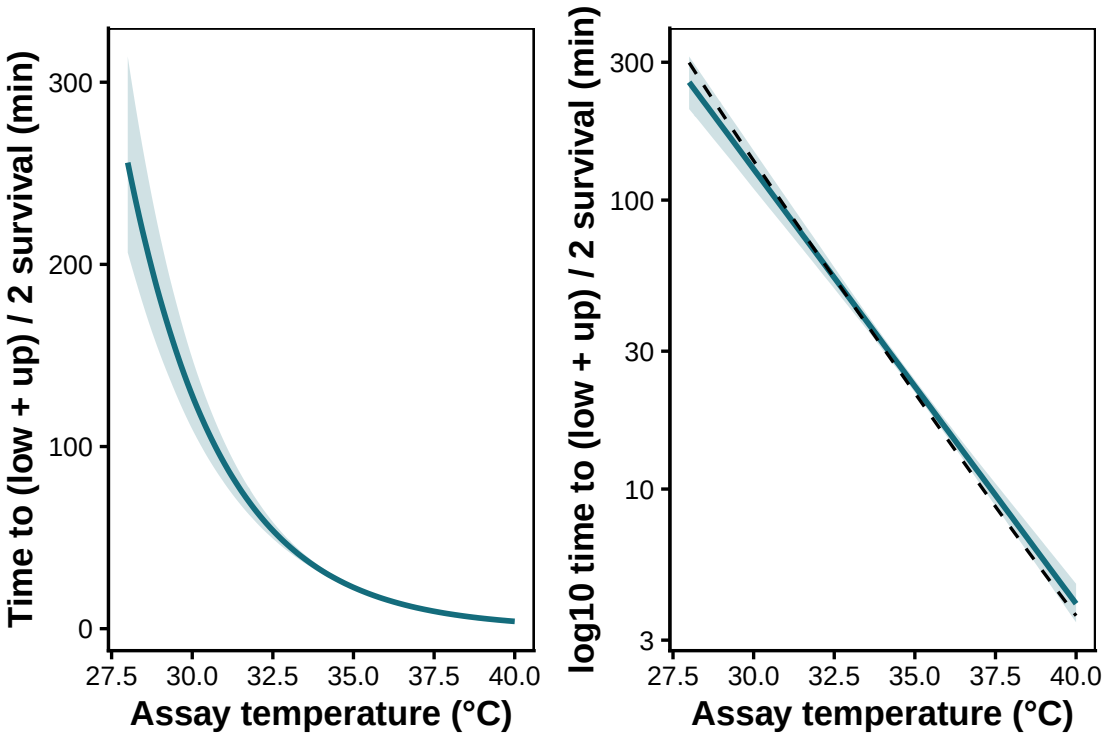


Figure S6: Posterior LT50(T) curve (median + 95% CrI, blue) recovered by `derive_tdt_curve()` from the T-varying fit, with the analytical truth overlaid (dashed black). The curve is visibly bent rather than straight in T, and the posterior tracks the truth.

We can also drop down to the per-draw level and compute a *local* $z(T)$ at each assay temperature from the same `et_ext$lt50_curve` posterior — a finite difference of $\log_{10}$ LT50 around T per draw,

summarised across draws (Figure S7).

```
# Finite-difference local z at each assay T, per posterior draw, derived
# from et_ext$lt50_curve (the same per-draw LT50(T) curve extract_tdt
# already computed for the OLS).
half_step <- 0.5

local_z_summary <- purrr::map_dfr(temps, function(T_at) {
  z_per_draw <- et_ext$lt50_curve$draws |>
    dplyr::group_by(.draw) |>
    dplyr::summarise(
      z_local = -1 / ((stats::approx(temp, log10(duration_out),
                                     xout = T_at + half_step)$y -
                       stats::approx(temp, log10(duration_out),
                                     xout = T_at - half_step)$y) /
                      (2 * half_step)),
      .groups = "drop"
    )
  q <- stats::quantile(z_per_draw$z_local, c(0.025, 0.5, 0.975),
                       na.rm = TRUE, names = FALSE)
  tibble::tibble(T_at           = T_at,
                 z_local_median = q[2],
                 z_local_lower  = q[1],
                 z_local_upper  = q[3])
})

local_z_summary |>
  dplyr::left_join(
    tibble::tibble(T_at = temps, z_local_truth = truth_ext$local_z_true),
    by = "T_at"
  ) |>
  dplyr::select(T_at, z_local_truth, z_local_median,
                z_local_lower, z_local_upper) |>
  dplyr::rename(
    `T (°C)`              = T_at,
    `z local truth (°C)`  = z_local_truth,
    Median                = z_local_median,
    Lower                 = z_local_lower,
    Upper                 = z_local_upper
  ) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

T (°C)	z local truth (°C)	Median	Lower	Upper
30	6.05	6.65	6.14	7.3
32	6.22	6.65	6.14	7.3
34	6.27	6.65	6.14	7.3
36	6.34	6.65	6.14	7.3
38	6.53	6.65	6.14	7.3

```
truth_curve_z <- {
  Ts <- seq(min(temps), max(temps), by = 0.1)
  tibble::tibble(T_at = Ts,
                 z_local_truth = vapply(Ts, true_local_z, numeric(1)))
}

ggplot2::ggplot(local_z_summary,
                ggplot2::aes(x = T_at, y = z_local_median)) +
  ggplot2::geom_ribbon(ggplot2::aes(ymin = z_local_lower,
                                    ymax = z_local_upper),
                       fill = "#146C7C", alpha = 0.2) +
  ggplot2::geom_line(colour = "#146C7C", linewidth = 1) +
  ggplot2::geom_line(data = truth_curve_z,
                     ggplot2::aes(x = T_at, y = z_local_truth),
                     linetype = "dashed", colour = "black",
                     inherit.aes = FALSE) +
  ggplot2::labs(x = "Assay temperature (°C)",
                y = expression(local~italic(z)~"(°C per 10-fold change in time)")) +
  ggplot2::theme_classic()
```

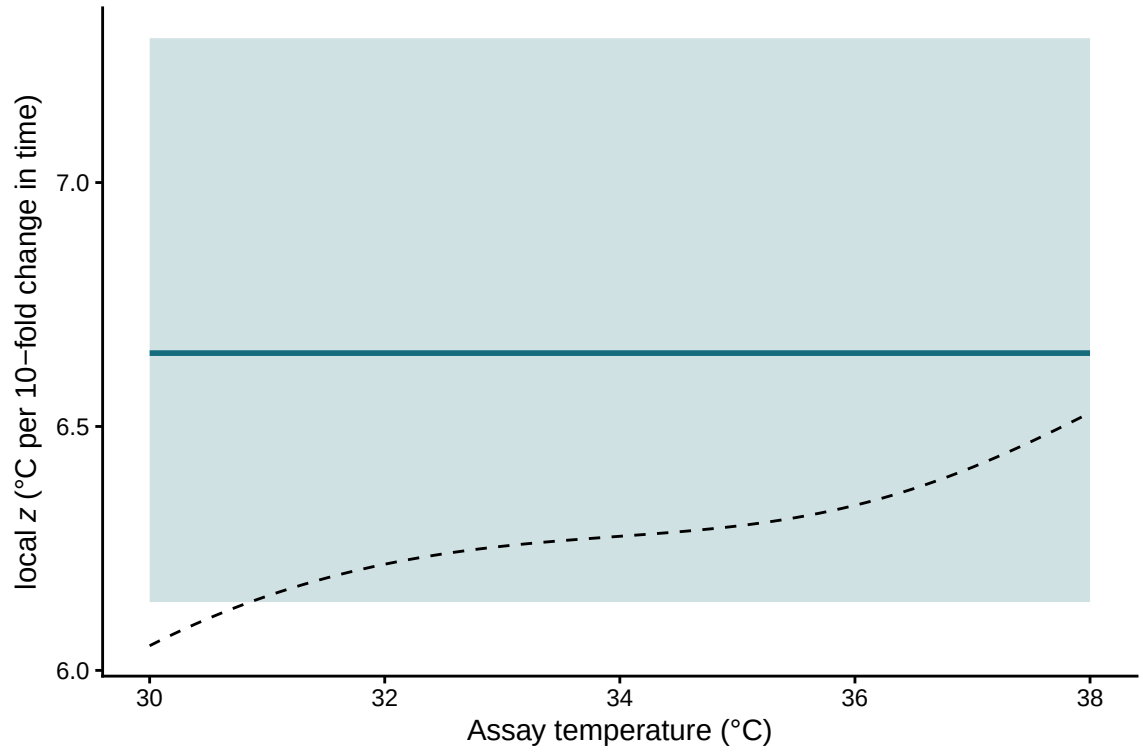


Figure S7: Local thermal sensitivity $z(T)$ recovered from the per-draw $\log_{10}\mathrm{LT50}(T)$ curve (blue median + 95% CrI), with the analytical truth overlaid (dashed black). Where the constant-shape simple-case fit would give the same z at every assay temperature, here

The same `fit_4pl()` and `extract_tdt()` calls recover the bent $\log_{10}\mathrm{LT50}(T)$ curve, average z across the assay grid, and per-draw $CT_{max_{1hr}}$. Under the constant-shape truth in Section 2.0.4, near-zero slopes on u, ℓ, and k reduce this calculation to the closed-form case.

2.0.7 Heat injury accumulation and survival under a fluctuating trace

TLS models can translate dynamic field temperatures into predicted heat-injury (HI) accumulation toward a threshold such as LT_{50} (Arnold *et al.* 2025; Jørgensen *et al.* 2021; Noble *et al.* 2026; Ørsted *et al.* 2022; Rezende *et al.* 2020). The joint 4PL posterior propagates uncertainty in z and $CT_{max_{1hr}}$ through the HI integral and uses the fitted surface to predict survival fraction $S_{\mathrm{pred}}(t)$ along the same trajectory (Equation 9 of the manuscript). `predict_heat_injury()` returns both trajectories in one call.

2.0.7.1 A simulated temperature profile across 5 days

To mimic conditions for a moderately heat-tolerant ectotherm, we generate 5 days of hourly temperatures with a sinusoidal diurnal cycle, ranging from 12 °C overnight to a midday peak of 24 °C. We set the net-accumulation threshold T_c to 20 °C, below which HI does not increase (Jørgensen *et al.* 2021; Ørsted *et al.* 2022). The simulated organism has $CT_{max_{1hr}} \approx 32.1$ °C and $z \approx 6.7$ °C, so the daily peak remains below the 1-hour critical limit and generates gradual sublethal accumulation.

```
hi_trace <- tibble::tibble(
  time_h = 0:(24 * 5 - 1),
  temp   = 18 + 6 * sin(2 * pi * (0:(24 * 5 - 1) - 6) / 24)   # range 12-24 °C
)
hi_T_c <- 20    # damage threshold (°C)
```

```r
ggplot2::ggplot(hi_trace, ggplot2::aes(time_h, temp)) +
  ggplot2::geom_line(colour = "darkred", linewidth = 0.6) +
  ggplot2::geom_hline(yintercept = hi_T_c, linetype = "dashed", colour = "grey40") +
  ggplot2::geom_hline(yintercept = true_CTmax, linetype = "dotted", colour = "grey40") +
  ggplot2::annotate("text", x = max(hi_trace$time_h), y = hi_T_c, label = "T_c",
                    hjust = 1, vjust = -0.4, size = 3, colour = "grey40") +
  ggplot2::annotate("text", x = max(hi_trace$time_h), y = true_CTmax,
                    label = "CTmax_1hr",
                    hjust = 1, vjust = -0.4, size = 3, colour = "grey40") +
  ggplot2::ylim(10, 45) +
  ggplot2::labs(x = "Hour", y = "Temperature (°C)") +
  ggplot2::theme_classic()
```

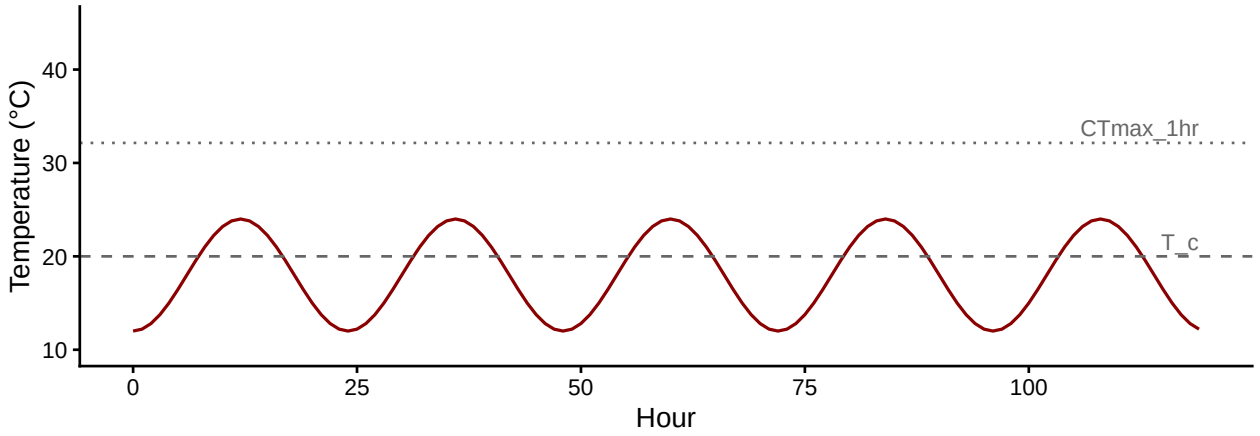


Figure S8: 5-day diurnal temperature trace. The dashed line is the damage-accumulation threshold $T_c = 20$ °C; the dotted line is the simulated organism's $CT_{max_{1hr}} \approx 32.1$°C — well above the daily peaks, so accumulation is sub-lethal and gradual.

2.0.7.2 Posterior HI and survival via `predict_heat_injury()`

`predict_heat_injury()` takes a temperature trace plus a fitted workflow, propagates the full posterior of $(\ell, u, k, \beta_0, \beta_1)$ through the HI integral, and returns the median and 95% credible band of cumulative HI and predicted survival at every time point. We supply `T_c` to enforce zero damage accumulation below the threshold (matching Equation 7 of the manuscript). The model time units here are minutes (because `standardize_data()` was called with `duration_unit = "minutes"`), so the trace's `time_h` is in hours and damage rates are computed accordingly internally.

```r
# Convert hourly trace to minutes to match the model's time units.
hi_trace_min <- dplyr::mutate(hi_trace, time_h = time_h * 60)

hi <- predict_heat_injury(
  trace        = hi_trace_min,
  workflow     = wf_bb,
  target_surv  = 0.5,
  T_c          = hi_T_c,
```

```
  ndraws        = 500
)
```

```r
# Convert back to hours for plotting.
hi_plot <- dplyr::mutate(hi$summary, time_h = time_h / 60)

p_hi <- ggplot2::ggplot(hi_plot, ggplot2::aes(time_h, hi_median)) +
  ggplot2::geom_ribbon(ggplot2::aes(ymin = hi_lower, ymax = hi_upper),
                       alpha = 0.2, fill = "darkred") +
  ggplot2::geom_line(colour = "darkred", linewidth = 0.8) +
  ggplot2::geom_hline(yintercept = 100, linetype = "dashed", colour = "grey40") +
  ggplot2::labs(x = NULL, y = "Cumulative HI (%)") +
  ggplot2::theme_classic()

p_surv <- ggplot2::ggplot(hi_plot, ggplot2::aes(time_h, surv_median)) +
  ggplot2::geom_ribbon(ggplot2::aes(ymin = surv_lower, ymax = surv_upper),
                       alpha = 0.2, fill = "steelblue") +
  ggplot2::geom_line(colour = "steelblue", linewidth = 0.8) +
  ggplot2::geom_hline(yintercept = 0.5, linetype = "dashed", colour = "grey40") +
  ggplot2::scale_y_continuous(limits = c(0, 1)) +
  ggplot2::labs(x = "Hour", y = "Predicted survival") +
  ggplot2::theme_classic()

patchwork::wrap_plots(p_hi, p_surv, ncol = 1)
```

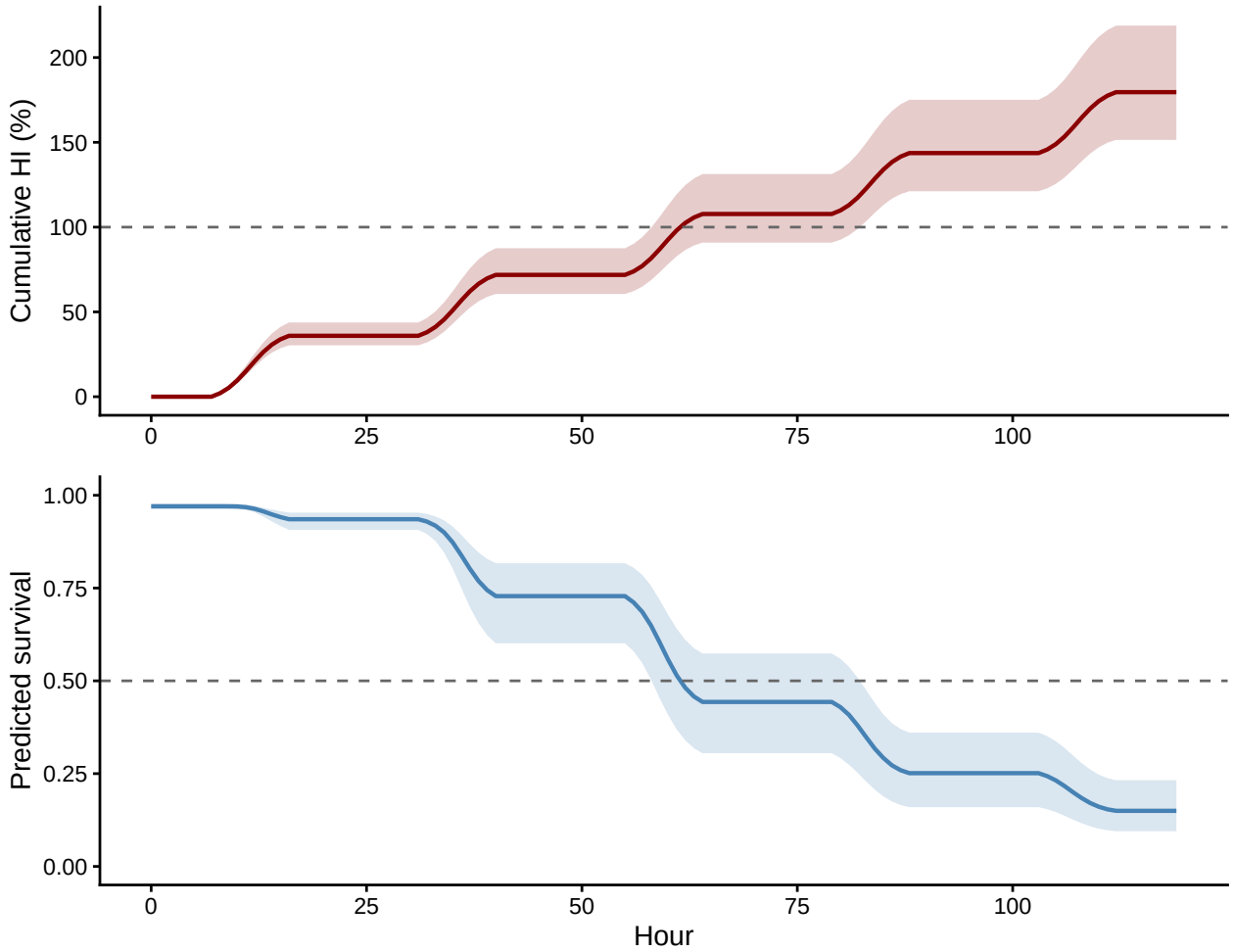


Figure S9: Posterior heat-injury accumulation (top, red) and predicted survival fraction (bottom, blue) under the 5-day field trace. Solid lines are posterior medians across 500 draws from `wf_bb`; shaded bands are 95% credible intervals. The dashed lines mark the LT50 threshold (HI = 100%) and 50% survival respectively. The two trajectories tell the same

By the end of day 5 the posterior median HI is 180% (95% CI: 151% to 219%), and median predicted survival is 0.15 (95% CI: 0.09 to 0.23).

2.0.7.3 Why are the credible bands so wide?

The posterior bands are wide relative to the precision in Table S4 because the rate multiplier, $10^{(T-CT_{max,1hr})/z}$, is exponential. A 1 °C shift in $CT_{max_{1hr}}$ multiplies per-hour HI by $10^{1/z}$, which is approximately 1.41-fold for the simulated $z \approx 6.7$ °C. Across a 5-day integral, such differences accumulate. A two-stage projection based only on point estimates hides this uncertainty; the joint posterior makes it visible.

2.0.8 Validation: recovering known planted doses

Before trusting `predict_heat_injury()` on real field data, we validate it against analytical truth on three reference traces with known dosing structure. `make_temperature_scenarios()` builds the traces; `planted_dose_from_trace()` implements the classical HI integral (Equation 7) directly given known `z` and `CTmax_1hr`. Posterior medians from `predict_heat_injury()` should sit near the analytical truth, and the 95% credible intervals should cover it.

```
# Three traces: flat baseline (zero HI), single hourly spike, three spikes.
val_scens <- make_temperature_scenarios(
  baseline    = 20,
  spike_temp  = 28,
  n_hours     = 96,
  spike_times_single = 24,
  spike_times_multi  = c(24, 48, 72)
)
val_T_c <- 24

# Analytical truth, given the simulation parameters (true_z, true_CTmax).
planted <- purrr::map(val_scens, planted_dose_from_trace,
                      z = true_z, CTmax_1hr = true_CTmax, T_c = val_T_c)

# Posterior predictions from the fitted beta-binomial workflow. The model
# expects minutes; convert each trace's time axis.
predicted <- purrr::map(val_scens, function(sc) {
  predict_heat_injury(
    trace       = dplyr::mutate(sc, time_h = time_h * 60),
    workflow    = wf_bb,
    target_surv = 0.5,
    T_c         = val_T_c,
    ndraws      = 300
  )
})

# Final HI per scenario: analytical truth vs posterior median + CrI.
val_table <- tibble::tibble(
```

Scenario	Planted HI (%)	Posterior median (%)	Lower (%)	Upper (%)	Truth in CrI?
Flat	0	0	0	0	TRUE
Single spike	24	23	21	26	TRUE
Multi spike	72	70	62	78	TRUE

```
  Scenario     = c("Flat", "Single spike", "Multi spike"),
  `Planted HI (%)` = c(tail(planted$flat$hi_cumulative,         1),
                       tail(planted$single_spike$hi_cumulative, 1),
                       tail(planted$multi_spike$hi_cumulative,  1)),
  `Posterior median (%)` = c(tail(predicted$flat$summary$hi_median,        1),
                             tail(predicted$single_spike$summary$hi_median, 1),
                             tail(predicted$multi_spike$summary$hi_median,  1)),
  `Lower (%)` = c(tail(predicted$flat$summary$hi_lower,        1),
                  tail(predicted$single_spike$summary$hi_lower, 1),
                  tail(predicted$multi_spike$summary$hi_lower,  1)),
  `Upper (%)` = c(tail(predicted$flat$summary$hi_upper,        1),
                  tail(predicted$single_spike$summary$hi_upper, 1),
                  tail(predicted$multi_spike$summary$hi_upper,  1))
)
val_table$`Truth in CrI?` <- with(val_table,
                                  `Lower (%)` <= `Planted HI (%)` &
                                  `Planted HI (%)` <= `Upper (%)`)
tinytable::tt(val_table, digits = 2, escape = TRUE)
```

The flat trace stays below T_c for its entire duration and so accrues zero HI, matching the analytical truth exactly. The single- and multi-spike traces deliver known doses analytically; `predict_heat_injury()` recovers each with the truth lying inside its credible interval. This sanity-check pattern — three traces with known doses, plus the recovery check — is what each new dataset's heat-injury prediction should pass before being trusted.

3 Extended Simulation Results

This section provides more detail on the simulation results from the various scenarios we explored to understand bias and coverage of z and CT_{max}. The main manuscript figures summarize the two shape-perturbing scenarios (drifting upper asymptote `u`, varying slope `k`) and the bias results from the two partial-mortality sweeps.

Here we document the remaining scenarios we explored: the strict-equivalence baseline, likelihood misspecification alone, symmetric asymptote compression, and the design-impact — plus the coverage panels for the two sweeps reported as bias-only in the main text.

3.1 *Strict equivalence baseline*

This scenario is the strict-equivalence baseline. The data-generating curve spans the full survival range ($u = 0.999$, low $= 0.001$), has a shared slope ($k = 8$), and has a midpoint that declines linearly with temperature (slope -0.15) under a binomial likelihood with no overdispersion. We crossed this scenario with two replicate budgets, $n_{\text{reps}} \in \{3, 5\}$, with $N_{\text{sim}} = 1000$ datasets per cell.

We included this scenario to check whether the estimators agree when the classical two-stage assumptions are satisfied. As expected, all methods recovered z and $CT_{max,1h}$ with little bias. The main difference was interval calibration: two-stage intervals based on Normal quantiles undercovered, whereas the t-quantile delta-method intervals were close to nominal 95% coverage.

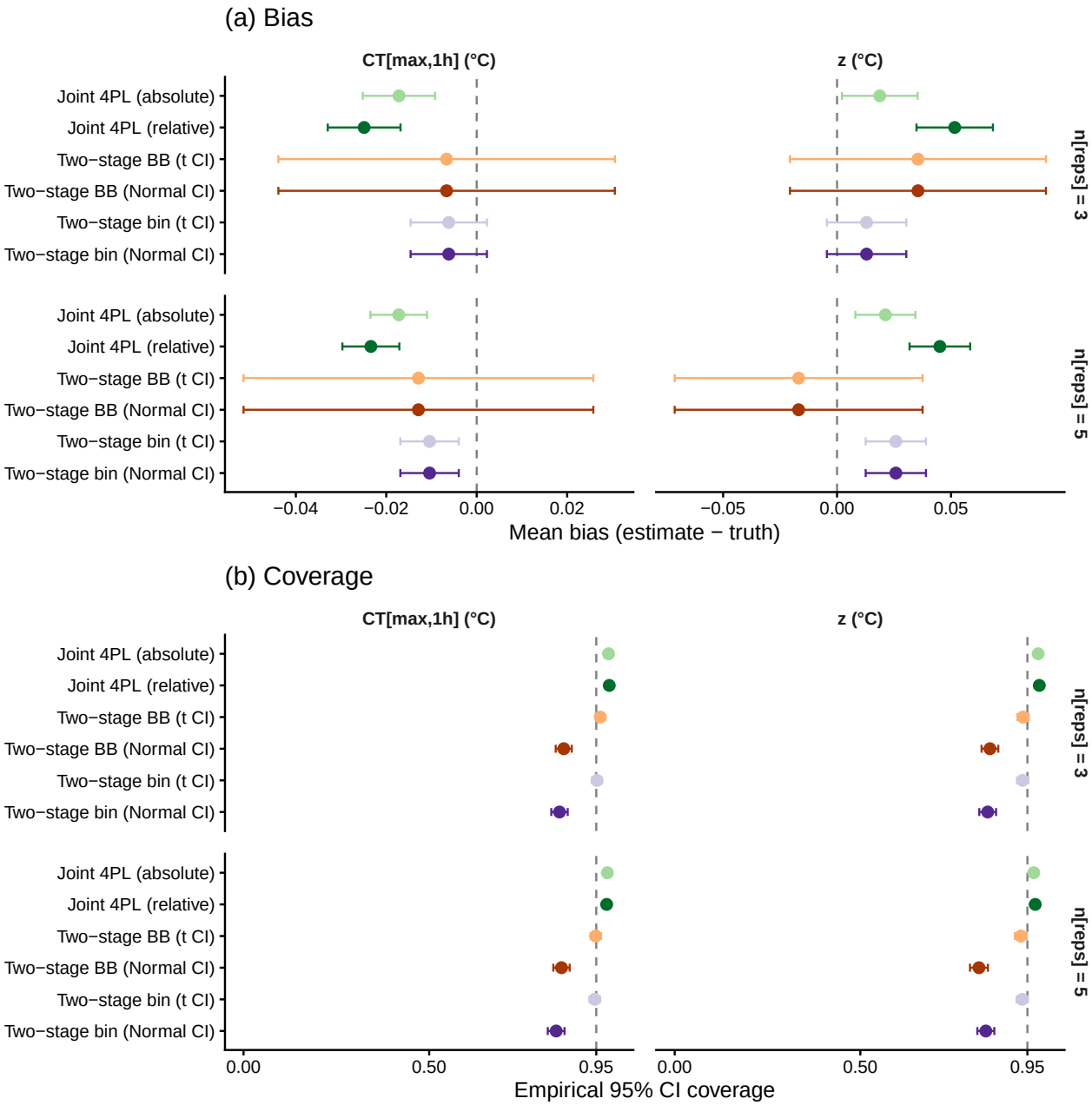


Figure S10: Bias and coverage at the strict-equivalence baseline (binomial DGP, asymptotes near 0/1, single slope k). Rows: $n_{reps} = 3$ (top) and $n_{reps} = 5$ (bottom). Columns: z (left) and $CT_{max,1h}$ (right). Dashed reference lines at zero bias and nominal 0.95 coverage.

3.2 *Likelihood misspecification alone*

This scenario keeps the same mean dose-response shape as the strict-equivalence baseline, but generates counts from a beta-binomial likelihood with overdispersion $\phi = 5$. We again used two replicate budgets, $n_{\text{reps}} \in \{3, 5\}$, with $N_{\text{sim}} = 1000$ datasets per cell.

We included this scenario to isolate the cost of likelihood misspecification: the mean structure

is correct for all estimators, but the binomial two-stage variant ignores overdispersion at Stage 1. Point estimates remained largely unbiased, showing that overdispersion alone did not shift the target estimands. The main effect was on uncertainty, with likelihood-aware beta-binomial and joint 4PL fits giving better-calibrated intervals than binomial two-stage fits.

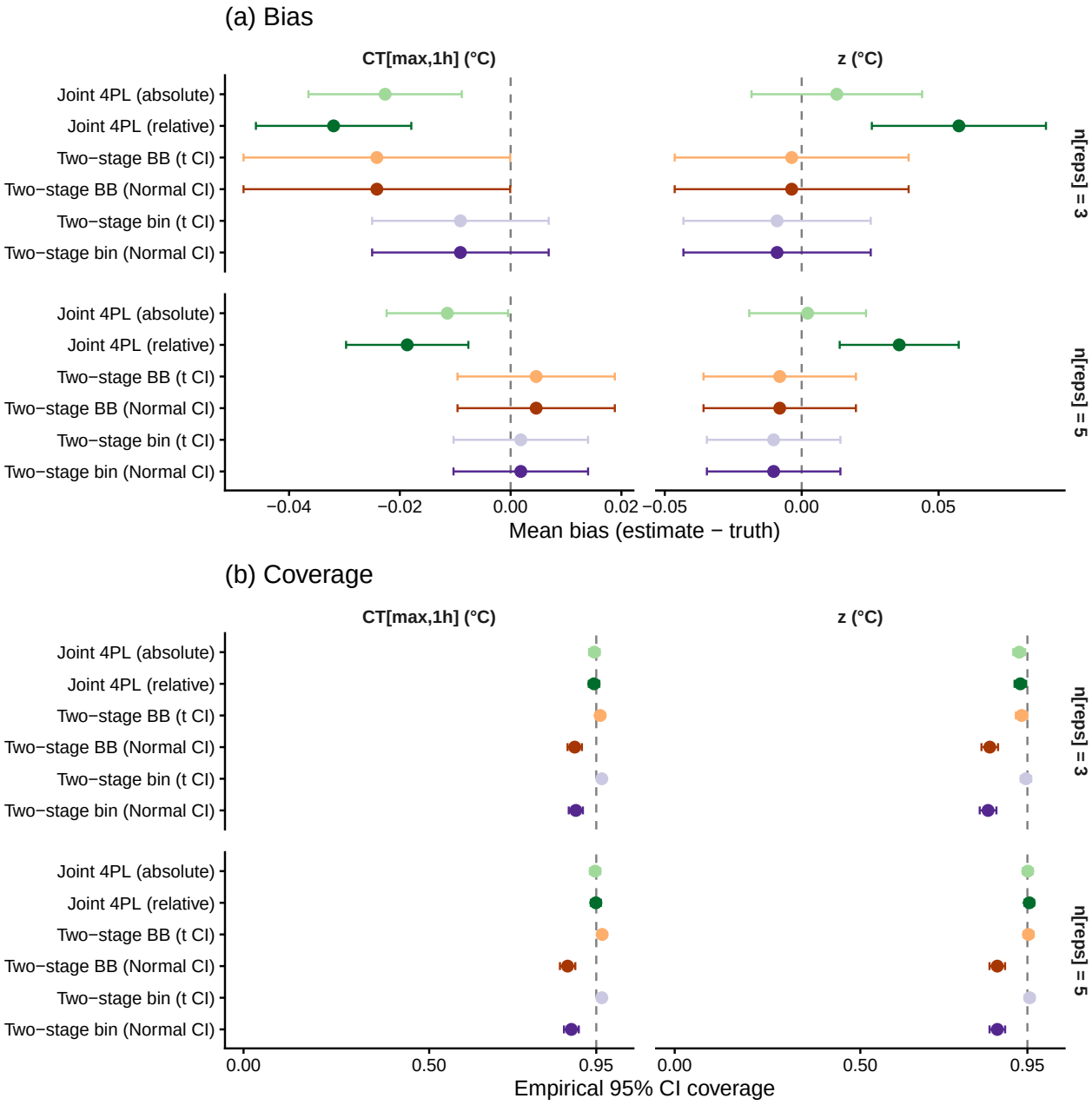


Figure S11: Bias and coverage with likelihood misspecification only (beta-binomial DGP with $\phi = 5$; curve shape as in the strict-equivalence baseline). Rows: $n_{reps} = 3$ (top) and $n_{reps} = 5$ (bottom).

3.3 *Asymptotes compress symmetrically*

In this scenario, both asymptotes move toward each other as temperature increases: u declines by 0.01 per °C from 0.92, while low increases by 0.01 per °C from 0.05. This preserves the approximate logit-difference but compresses the response range. Although this symmetric compression is not biologically realistic, it is useful as a diagnostic test: it changes the dose-response range while largely preserving the midpoint-temperature relationship. We used two replicate budgets, $n_{\text{reps}} \in \{3, 5\}$, with $N_{\text{sim}} = 1000$ datasets per cell.

Bias was smaller than in the asymmetric upper-asymptote scenario, but the two-stage fits still showed poorer calibration than the joint 4PL because the classical model cannot represent changing asymptotes directly.

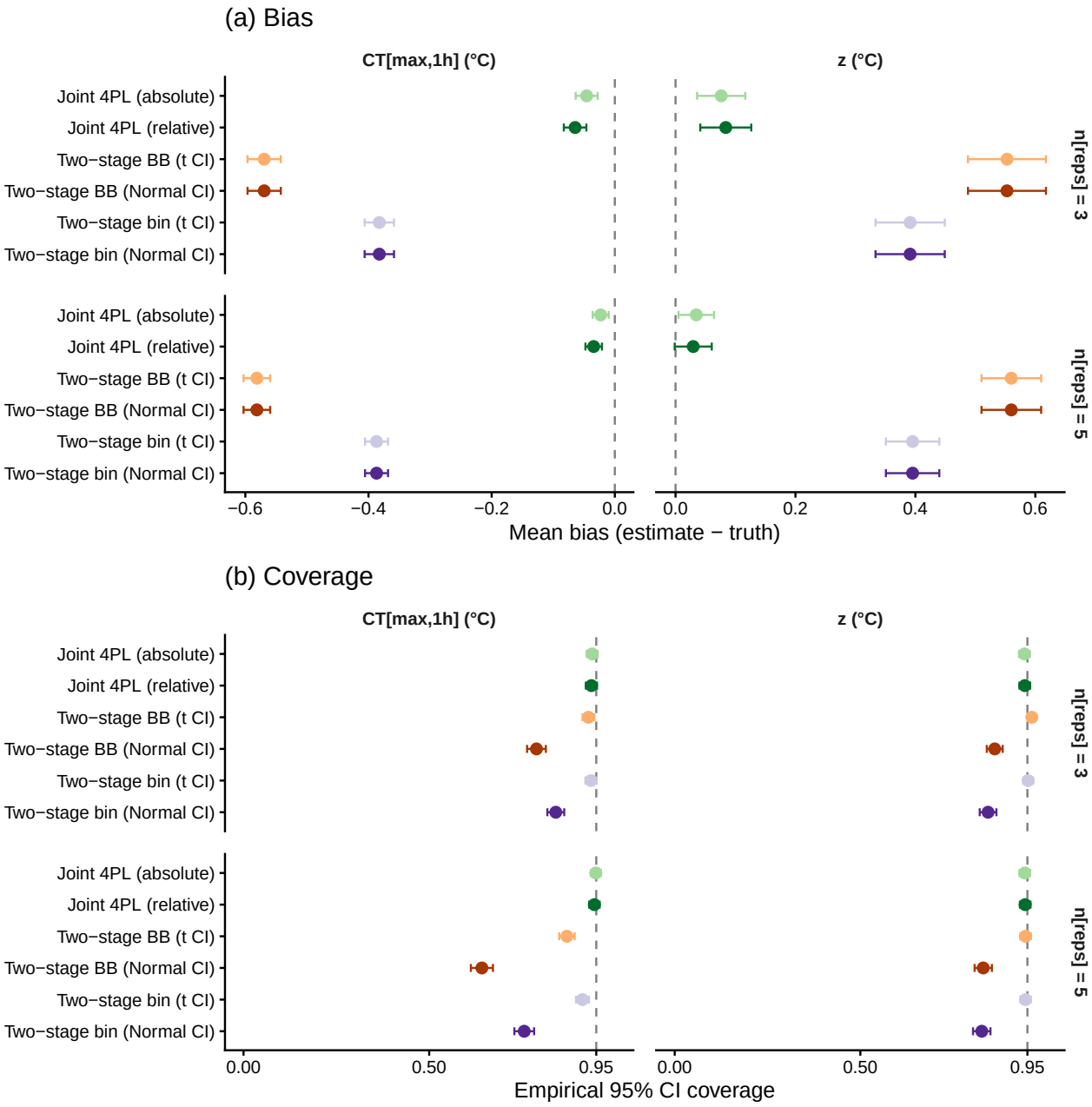


Figure S12: Bias and coverage when both asymptotes compress symmetrically with temperature. Rows: $n_{reps} = 3$ (top) and $n_{reps} = 5$ (bottom).

3.4 *Coverage for the drifting-u across temperatures (β_u)*

This complements the main-text β_u bias sweep with the matching coverage panels.

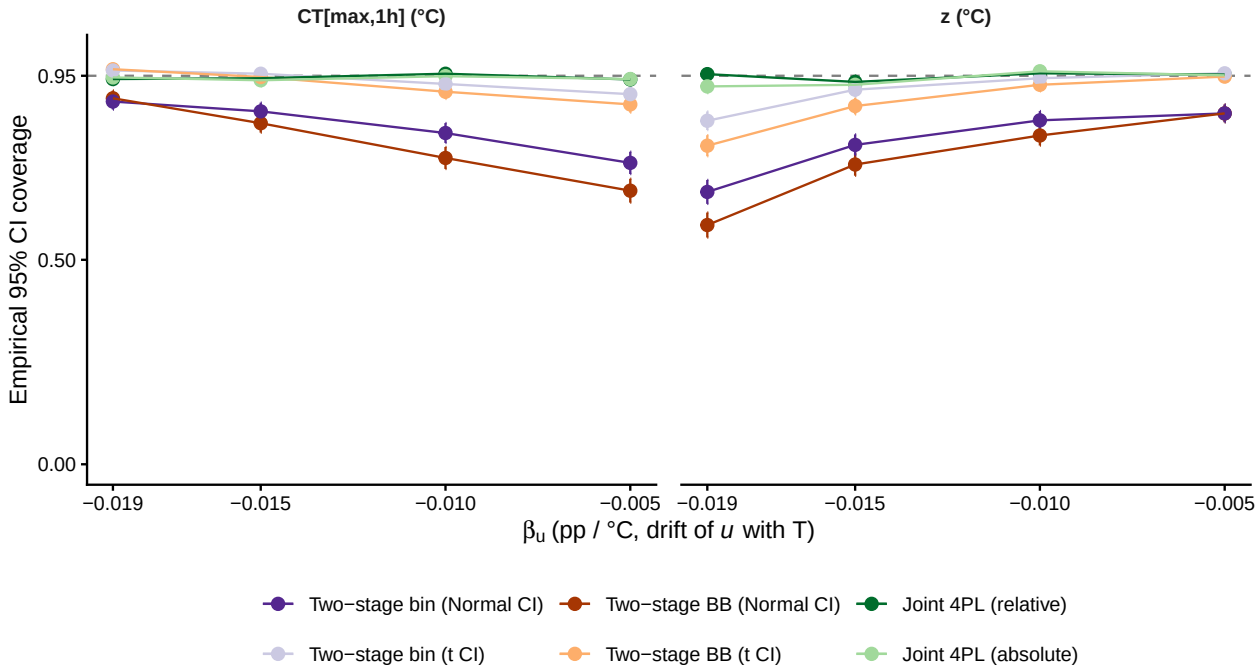


Figure S13: Empirical 95% CI coverage across the β_u sweep (Scen 6). Left: z. Right: $CT_{max,1h}$. Dashed line at nominal 0.95. Same DGP and method set as the main-text β_u bias sweep, with coverage shown for all six methods. Normal- and t-quantile two-stage CIs differ in coverage even though their point estimates are identical.

3.5 *Coverage for the constant-but-reduced-u (u_0)*

This complements the main-text u_0 bias sweep. With a low u_0, the two-stage Stage-1 GLMs miscalibrate per temperature, the Stage-2 OLS inherits that bias, and CI coverage on $CT_{max,1h}$ collapses well below nominal even after the t-quantile small-sample correction.

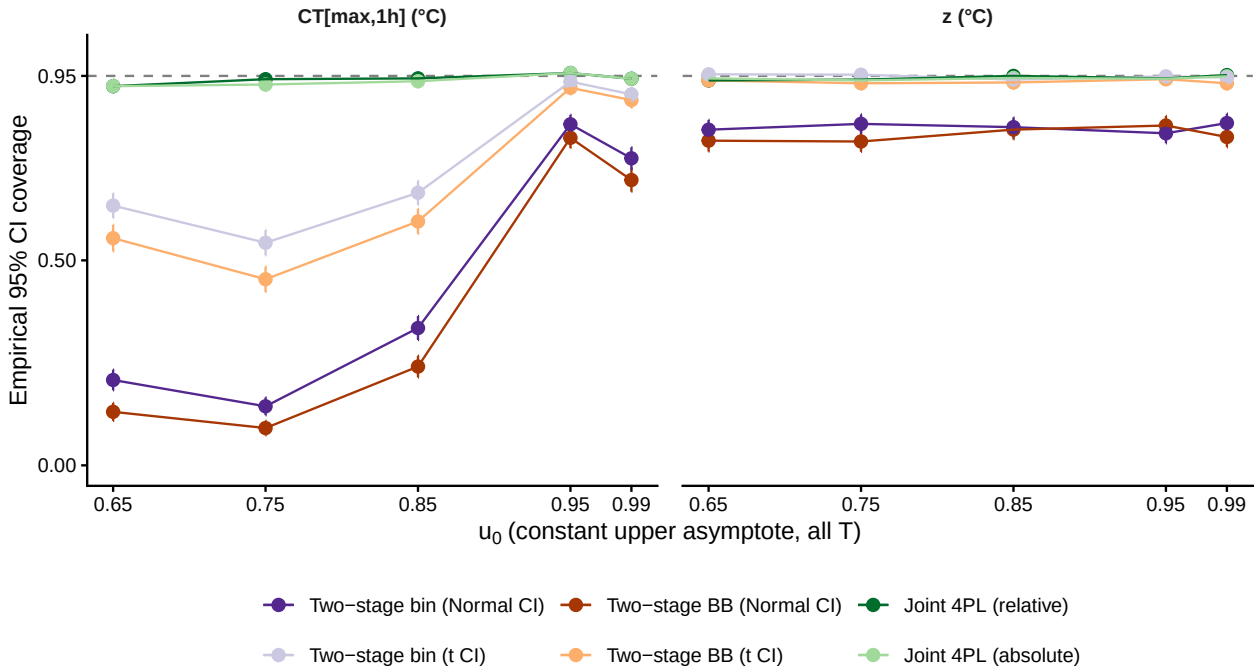


Figure S14: Empirical 95% CI coverage across the u_0 sweep (Scen 7). Same DGP and method set as the main-text u_0 bias sweep. Two-stage CTmax coverage collapses as u_0 falls; the t-correction widens CIs without recovering nominal calibration. Joint 4PL coverage holds at 92–96% throughout.

3.6 *Design variation impacts*

This scenario varies experimental design while holding the data-generating process fixed. We crossed two temperature-duration grids (full: 5 temperatures × 6 durations; sparse: 3 temperatures × 4 durations) with three levels of replication, $n_{\text{reps}} \in \{1, 3, 5\}$. Here, n_{reps} is the number of replicate cups per temperature-by-duration cell; each cup contained $N \sim \text{Uniform}\{10, \ldots, 20\}$ organisms. The DGP used the baseline beta-binomial shape ($u = 0.92$, low $= 0.05$, $k = 8$, midpoint linear in T, $\phi = 5$); the sparsest design had only 12 'cups' total.

We included this scenario to test how much information each estimator needs before the two-stage reduction becomes unstable. Performance improved with denser designs and more replication, but the sparse one-replicate design was especially problematic for the two-stage pipeline. The joint 4PL was more stable across the design gradient because it shares information across temperatures and durations rather than estimating each temperature-specific curve separately.

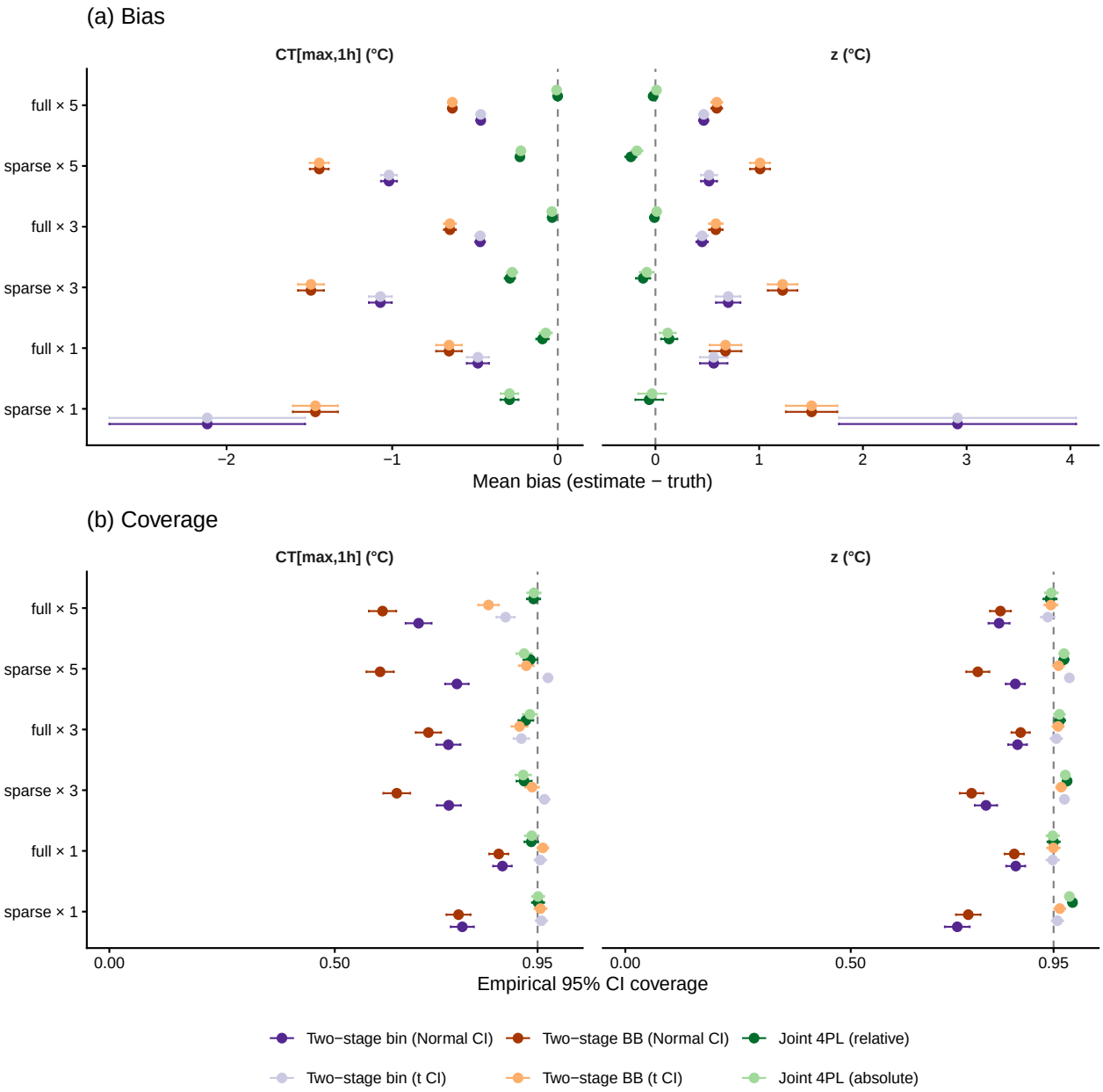


Figure S15: Bias and coverage across six design × replication cells (Scen 8). Two-stage at sparse × 1 produces large, unstable estimates with implausibly wide t-corrected CIs; joint 4PL returns coherent intervals of appropriate width across all six cells.

4 Case Study 1: Shrimp data

We now apply the full workflow described above — `standardize_data()` → `fit_4pl()` → `extract_tdt()` → `predict_heat_injury()` — to lethal-TDT data from brown shrimp (*Crangon crangon*). The data are 148 replicate trials spanning 30–33 °C and 5 min to 6 hours of exposure, with each replicate exposing a tank of individuals to a fixed temperature × duration combination on a specific date. The two grouping factors `Date` and `Tank` enter the model as random intercepts

Table S9: Summary of the shrimp lethal-TDT dataset after standardisation. Temperature centre is the grand mean used to mean-centre the temperature covariate.

n trials	Temperature range (°C)	Duration range (hr)	Median individuals/trial	Temperature centre (°C)
148	30–33	0.08–6	10	32

on `mid`, allowing day-to-day batch variation and tank-level cohort variation to be partitioned out of the dose-response surface.

4.1 Data and standardisation

```
# Bundled with the package (see ?shrimp_lethal); raw CSV in inst/extdata/.
shrimp_raw <- shrimp_lethal

shrimp_std <- standardize_data(
  data           = shrimp_raw,
  temp           = "Temperature_assay",
  duration       = "Duration_exposure_hours",
  n_total        = "N_individuals_after_trial",
  mortality      = "Mortality_after_trial",
  random_effects = c("Date", "Tank"),
  duration_unit  = "hours"
)
```

```
shrimp_summary <- tibble::tibble(
  `n trials`              = nrow(shrimp_std),
  `Temperature range (°C)` = paste(range(shrimp_std$temp), collapse = "-"),
  `Duration range (hr)`   = paste(round(range(shrimp_std$duration), 2),
                                  collapse = "-"),
  `Median individuals/trial` = stats::median(shrimp_std$n_total),
  `Temperature centre (°C)`  = attr(shrimp_std, "tdt_meta")$temp_mean,
  `n Dates`               = length(unique(shrimp_std$Date)),
  `n Tanks`               = length(unique(shrimp_std$Tank))
)
tinytable::tt(shrimp_summary, digits = 2, escape = TRUE)
```

4.2 Classical two-stage TDT fits

Before fitting the joint 4PL, we also run the classical two-stage pipeline on the shrimp counts. Stage 1 fits a separate logistic dose-response curve at each assay temperature. Stage 2 regresses the resulting $\log_{10}$ LT50 estimates on assay temperature. We repeat Stage 1 with a binomial likelihood and a beta-binomial likelihood; the latter is a small maximum-likelihood fit with an estimated beta-binomial precision parameter ϕ.

```r
fit_stage1_binomial_case <- function(dat) {
  validate_stage1_case <- function(log10_lt50, slope, d) {
    is.finite(log10_lt50) &&
      is.finite(slope) &&
      slope < 0 &&
      abs(slope) > 1e-4 &&
      log10_lt50 >= min(d$logd, na.rm = TRUE) - 0.5 &&
      log10_lt50 <= max(d$logd, na.rm = TRUE) + 0.5
  }

  dat |>
    dplyr::group_by(temp) |>
    dplyr::group_modify(function(d, key) {
      if (dplyr::n_distinct(d$logd) < 3) {
        return(tibble::tibble(
          method       = "Two-step binomial",
          log10_lt50   = NA_real_,
          stage1_slope = NA_real_,
          phi          = NA_real_,
          stage1_ok    = FALSE
        ))
      }
      fit <- stats::glm(cbind(n_surv, n_total - n_surv) ~ logd,
                        data = d, family = stats::binomial())
      co <- stats::coef(fit)
      lt <- as.numeric(-co[1] / co[2])
      sl <- as.numeric(co[2])
      tibble::tibble(
        method       = "Two-step binomial",
        log10_lt50   = lt,
        stage1_slope = sl,
        phi          = NA_real_,
        stage1_ok    = validate_stage1_case(lt, sl, d)
      )
    }) |>
    dplyr::ungroup()
}

fit_stage1_betabinom_case <- function(dat) {
  validate_stage1_case <- function(log10_lt50, slope, d) {
    is.finite(log10_lt50) &&
      is.finite(slope) &&
      slope < 0 &&
      abs(slope) > 1e-4 &&
      log10_lt50 >= min(d$logd, na.rm = TRUE) - 0.5 &&
      log10_lt50 <= max(d$logd, na.rm = TRUE) + 0.5
  }
```

```r
bb_nll <- function(par, y, n, x) {
  mu  <- stats::plogis(par[1] + par[2] * x)
  mu  <- pmin(pmax(mu, 1e-8), 1 - 1e-8)
  phi <- exp(par[3])
  -sum(
    lchoose(n, y) +
      lbeta(y + mu * phi, n - y + (1 - mu) * phi) -
      lbeta(mu * phi, (1 - mu) * phi)
  )
}

dat |>
  dplyr::group_by(temp) |>
  dplyr::group_modify(function(d, key) {
    if (dplyr::n_distinct(d$logd) < 3) {
      return(tibble::tibble(
        method       = "Two-step beta-binomial",
        log10_lt50   = NA_real_,
        stage1_slope = NA_real_,
        phi          = NA_real_,
        stage1_ok    = FALSE
      ))
    }
    fit_glm <- suppressWarnings(
      stats::glm(cbind(n_surv, n_total - n_surv) ~ logd,
                 data = d, family = stats::binomial())
    )
    start <- c(stats::coef(fit_glm), log(50))
    if (any(!is.finite(start))) {
      p0 <- pmin(pmax(mean(d$n_surv / d$n_total), 1e-4), 1 - 1e-4)
      start <- c(stats::qlogis(p0), -1, log(50))
    }

    fit <- stats::optim(
      par     = start,
      fn      = bb_nll,
      y       = d$n_surv,
      n       = d$n_total,
      x       = d$logd,
      method  = "BFGS",
      control = list(maxit = 1000)
    )
    co <- fit$par
    lt <- as.numeric(-co[1] / co[2])
    sl <- as.numeric(co[2])
    tibble::tibble(
      method       = "Two-step beta-binomial",
```

```
        log10_lt50   = lt,
        stage1_slope = sl,
        phi          = exp(co[3]),
        stage1_ok    = validate_stage1_case(lt, sl, d)
      )
    }) |>
    dplyr::ungroup()
}

fit_stage2_case <- function(stage1, time_multiplier = 60, t_ref = 60,
                            TC_rate_range = c(0.1, 1)) {
  stage1_all <- stage1
  stage1 <- stage1 |>
    dplyr::filter(stage1_ok, is.finite(log10_lt50))
  if (nrow(stage1) < 3) {
    return(list(
      fit = NULL,
      summary = tibble::tibble(
        method       = unique(stage1_all$method),
        intercept    = NA_real_,
        slope_T      = NA_real_,
        z            = NA_real_,
        CTmax_1hr    = NA_real_,
        T_crit       = NA_real_,
        r_squared    = NA_real_,
        n_stage1     = nrow(stage1),
        n_excluded   = nrow(stage1_all) - nrow(stage1)
      )
    ))
  }
  fit <- stats::lm(log10_lt50 ~ temp, data = stage1)
  co  <- stats::coef(fit)
  intercept_out <- as.numeric(co[1] + log10(time_multiplier))
  slope         <- as.numeric(co[2])
  z             <- -1 / slope
  ctmax         <- (log10(t_ref) - intercept_out) / slope
  log_rate_mid  <- mean(log10(TC_rate_range / 100))

  list(
    fit = fit,
    summary = tibble::tibble(
      method       = unique(stage1$method),
      intercept    = intercept_out,
      slope_T      = slope,
      z            = z,
      CTmax_1hr    = ctmax,
      T_crit       = ctmax + z * log_rate_mid,
```

```
      r_squared    = summary(fit)$r.squared,
      n_stage1     = nrow(stage1),
      n_excluded   = nrow(stage1_all) - nrow(stage1)
    )
  )
}

make_twostep_curve_case <- function(stage2, temp_grid,
                                    time_multiplier = 60,
                                    output_time_unit = "min") {
  if (is.null(stage2$fit)) {
    return(tibble::tibble(
      temp = temp_grid,
      duration_median = NA_real_,
      method = stage2$summary$method,
      output_time_unit = output_time_unit
    ))
  }
  co <- stats::coef(stage2$fit)
  tibble::tibble(
    temp              = temp_grid,
    duration_median   = 10 ^ (as.numeric(co[1]) +
                              as.numeric(co[2]) * temp_grid) * time_multiplier,
    method            = stage2$summary$method,
    output_time_unit  = output_time_unit
  )
}

# Standard two-step uncertainty: treat each Stage-1 LT50 as a known point
# and propagate only the Stage-2 lm uncertainty.
#  - z (= -1/slope), CTmax_1hr and T_crit: quantiles over 1000 MVN(coef, vcov)
#    draws of the Stage-2 coefficients. (Using one consistent draw-based method
#    avoids the reversed/invalid bounds the closed-form -1/confint(slope) gives
#    when the Stage-2 slope is imprecise.)
#  - Line band: `predict.lm(..., interval = "confidence")`, exponentiated.
propagate_twostep_case <- function(stage2_fit, temp_grid,
                                   n_sim = 1000,
                                   time_multiplier = 60,
                                   t_ref = 60,
                                   TC_rate_range = c(0.1, 1),
                                   seed = 123) {
  if (is.null(stage2_fit)) {
    return(list(
      summary_ci = tibble::tibble(
        z_lower = NA_real_, z_upper = NA_real_,
        CTmax_lower = NA_real_, CTmax_upper = NA_real_,
        Tcrit_lower = NA_real_, Tcrit_upper = NA_real_
```

```
    ),
    curve_ci = tibble::tibble(
      temp = temp_grid,
      duration_lower = NA_real_,
      duration_upper = NA_real_
    )
  ))
}
set.seed(seed)
log_rate_mid <- mean(log10(TC_rate_range / 100))

# z CI: invert the Stage-2 slope's 95% CI. z = -1/slope is monotonically
# increasing in slope on (-Inf, 0), so when the slope CI is entirely
# negative the inversion yields a clean, point-containing asymmetric CI for
# z. If the slope CI crosses zero (slope poorly identified, as happens when
# Stage 1 yields few valid LT50s), z is unidentified - report NA rather
# than fabricate finite bounds via MVN simulation, which would give heavy
# right-tailed draws that don't contain the point estimate.
beta_ci  <- stats::confint(stage2_fit, "temp", level = 0.95)
slope_lo <- as.numeric(beta_ci[1, 1])
slope_hi <- as.numeric(beta_ci[1, 2])
if (is.finite(slope_lo) && is.finite(slope_hi) && slope_hi < 0) {
  z_lower <- -1 / slope_lo
  z_upper <- -1 / slope_hi
} else {
  z_lower <- NA_real_
  z_upper <- NA_real_
}

# CTmax and T_crit CIs come from MVN simulation: both are ratios of
# correlated normals (intercept and slope), so joint propagation is needed.
draws <- MASS::mvrnorm(n_sim,
                       mu    = stats::coef(stage2_fit),
                       Sigma = stats::vcov(stage2_fit))
alpha <- draws[, "(Intercept)"]
beta  <- draws[, "temp"]
ctmax_draws <- (log10(t_ref) -
                (alpha + log10(time_multiplier))) / beta
tcrit_draws <- ctmax_draws + (-1 / beta) * log_rate_mid

qfun <- function(x, p) stats::quantile(x, p, na.rm = TRUE, names = FALSE)
summary_ci <- tibble::tibble(
  z_lower     = z_lower,
  z_upper     = z_upper,
  CTmax_lower = qfun(ctmax_draws, 0.025),
  CTmax_upper = qfun(ctmax_draws, 0.975),
  Tcrit_lower = qfun(tcrit_draws, 0.025),
```

```
    Tcrit_upper = qfun(tcrit_draws, 0.975)
  )

  pred <- stats::predict(stage2_fit,
                         newdata  = data.frame(temp = temp_grid),
                         interval = "confidence", level = 0.95)
  curve_ci <- tibble::tibble(
    temp           = temp_grid,
    duration_lower = 10 ^ as.numeric(pred[, "lwr"]) * time_multiplier,
    duration_upper = 10 ^ as.numeric(pred[, "upr"]) * time_multiplier
  )
  list(summary_ci = summary_ci, curve_ci = curve_ci)
}
```

```
shrimp_stage1_twostep <- dplyr::bind_rows(
  fit_stage1_binomial_case(shrimp_std),
  fit_stage1_betabinom_case(shrimp_std)
)

shrimp_stage2_twostep <- lapply(
  split(shrimp_stage1_twostep, shrimp_stage1_twostep$method),
  fit_stage2_case,
  time_multiplier = 60,
  t_ref = 60,
  TC_rate_range = c(0.1, 1)
)

shrimp_twostep_tdt <- dplyr::bind_rows(
  lapply(shrimp_stage2_twostep, `[[`, "summary")
)

shrimp_twostep_grid <- seq(min(shrimp_std$temp) - 1.5,
                           max(shrimp_std$temp) + 1.5, by = 0.05)

shrimp_twostep_unc <- list(
  "Two-step binomial" = propagate_twostep_case(
    shrimp_stage2_twostep[["Two-step binomial"]]$fit,
    temp_grid = shrimp_twostep_grid,
    n_sim = 1000, time_multiplier = 60, t_ref = 60,
    TC_rate_range = c(0.1, 1), seed = 123
  ),
  "Two-step beta-binomial" = propagate_twostep_case(
    shrimp_stage2_twostep[["Two-step beta-binomial"]]$fit,
    temp_grid = shrimp_twostep_grid,
    n_sim = 1000, time_multiplier = 60, t_ref = 60,
    TC_rate_range = c(0.1, 1), seed = 124
  )
```

Table S10: Classical two-stage TDT quantities for brown shrimp. Point estimates are from the Stage-2 regression. The 95% CIs for z, $CT_{max_{1hr}}$ and T_{crit} all come from 1000 parametric simulations from $\text{MVN}(\hat{\theta}, \widehat{\text{Cov}}(\hat{\theta}))$ of the Stage-2 coefficients. Each Stage-1 $\log_{10}\text{LT50}$ is treated as a known point, as is standard practice. T_{crit} uses the geometric midpoint of the same $r^* \in [0.1, 1]$ % HI per hour range used for the joint 4PL extraction.

Stage-1 likelihood	z (C)	CTmax_1hr (C)	T_crit (C)	Stage-2 R2	Stage-1 est
Two-step binomial	2.32 [1.66, 3.84]	32.64 [32.36, 33.06]	26.85 [24.8, 27.95]	0.924	6
Two-step beta-binomial	2.32 [1.66, 3.84]	32.64 [32.34, 33.08]	26.85 [25.01, 27.97]	0.925	6

```
)
```

```
shrimp_twostep_tdt_ci <- dplyr::bind_rows(lapply(
  names(shrimp_twostep_unc),
  function(m) {
    pt <- dplyr::filter(shrimp_twostep_tdt, method == m)
    ci <- shrimp_twostep_unc[[m]]$summary_ci
    tibble::tibble(
      `Stage-1 likelihood` = m,
      `z (C)`         = format_interval(pt$z,         ci$z_lower,     ci$z_upper,     digits =
      `CTmax_1hr (C)` = format_interval(pt$CTmax_1hr, ci$CTmax_lower, ci$CTmax_upper, digits =
      `T_crit (C)`    = format_interval(pt$T_crit,    ci$Tcrit_lower, ci$Tcrit_upper, digits =
      `Stage-2 R2`                 = round(pt$r_squared, 3),
      `Stage-1 estimates used`     = pt$n_stage1,
      `Stage-1 estimates excluded` = pt$n_excluded
    )
  }
))

tinytable::tt(shrimp_twostep_tdt_ci, escape = TRUE)
```

```
shrimp_twostep_line_min <- dplyr::bind_rows(lapply(
  shrimp_stage2_twostep,
  make_twostep_curve_case,
  temp_grid = shrimp_twostep_grid,
  time_multiplier = 60,
  output_time_unit = "min"
))

shrimp_twostep_band_min <- dplyr::bind_rows(lapply(
  names(shrimp_twostep_unc),
  function(m) {
    shrimp_twostep_unc[[m]]$curve_ci |>
      dplyr::mutate(method = m)
  }
```

```
))

ggplot2::ggplot() +
  ggplot2::geom_ribbon(
    data = shrimp_twostep_band_min,
    ggplot2::aes(x = temp,
                 ymin = log10(duration_lower),
                 ymax = log10(duration_upper),
                 fill = method),
    alpha = 0.18, colour = NA
  ) +
  ggplot2::geom_point(
    data = dplyr::filter(shrimp_stage1_twostep, stage1_ok),
    ggplot2::aes(x = temp,
                 y = log10_lt50 + log10(60),
                 colour = method),
    size = 2.5
  ) +
  ggplot2::geom_line(
    data = shrimp_twostep_line_min,
    ggplot2::aes(x = temp, y = log10(duration_median),
                 colour = method),
    linewidth = 1
  ) +
  ggplot2::labs(
    x = "Assay temperature (C)",
    y = expression(log[10]~LT[50]~"(min)"),
    colour = "Two-stage fit",
    fill   = "Two-stage fit"
  ) +
  ggplot2::theme_classic()
```

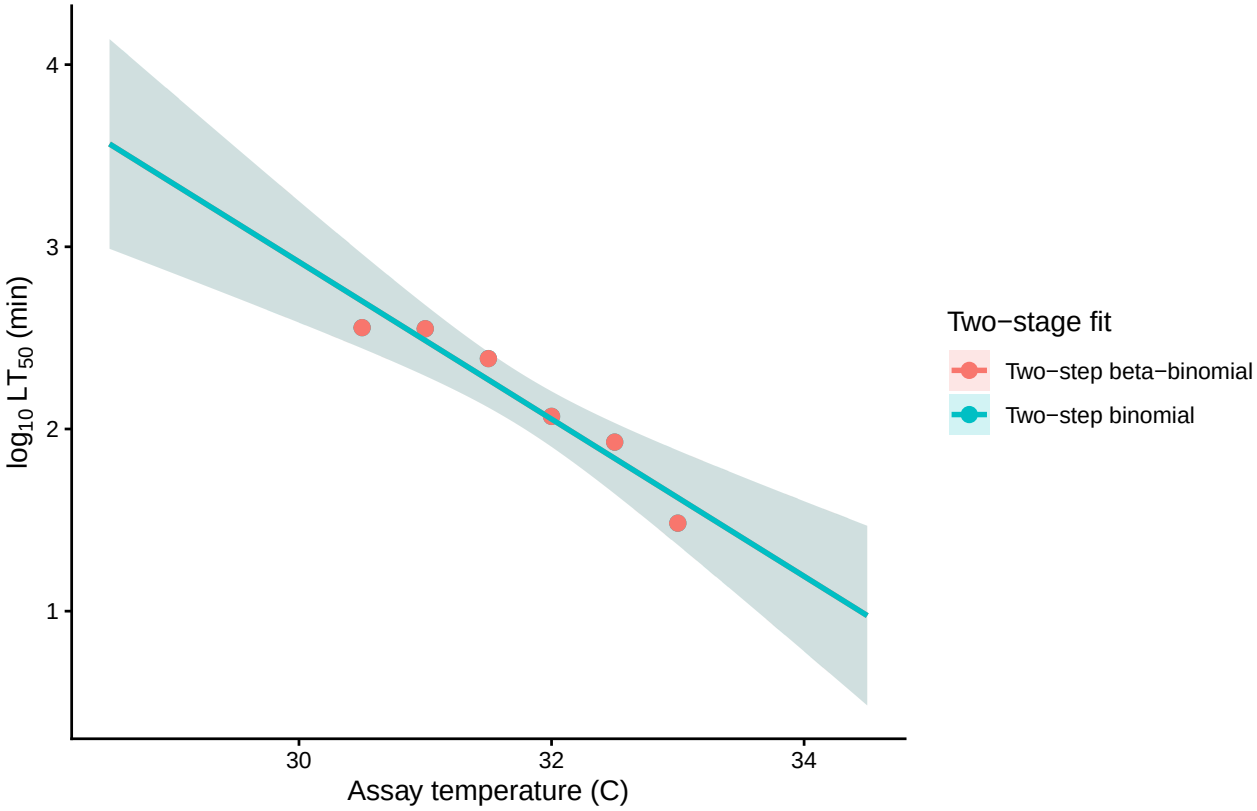


Figure S16: Classical two-stage TDT lines for brown shrimp. Points are Stage-1 $\log_{10}$ LT50 estimates; lines are the Stage-2 regressions; shaded bands are pointwise 95% confidence intervals from `predict.lm()` on the Stage-2 regression (Stage-1 LT50s treated as known points, as in standard practice). The beta-binomial Stage-1 fit allows overdispersion within each assay temperature, but both approaches reduce to a single point estimate per temperature for Stage 2.

4.3 Fitting the joint Bayesian 4PL

`fit_4pl()` is the package's high-level fitting wrapper. It takes the standardised data and configures the joint 4PL with sensible defaults: a beta-binomial likelihood, the disjoint-bounds reparameterisation that keeps the asymptotes identifiable, and a `temp_c` slope on each 4PL sub-parameter. We add random intercepts on `mid` for `Date` and `Tank` to absorb the day-to-day and tank-to-tank variation that would otherwise contaminate the dose-response surface.

```
wf_shrimp <- fit_4pl(
  shrimp_std,
  random_effects = c("Date", "Tank"),
  chains  = 4, iter = 8000, warmup = 4000, cores = 4, seed = 123,
  control = list(adapt_delta = 0.99, max_treedepth = 12),
  file    = file.path(models_dir, "fit_shrimp_lethal_4pl")
)
```

Table S11: Sampling diagnostics for the shrimp 4PL fit. Healthy targets: Rhat < 1.01, ESS > 400, zero divergences.

rhat_max	ess_bulk_min	ess_tail_min	divergences	treedepth_hits	bfmi_min	rhat_pass	ess_pass
1	2812	4788	0	2	0.699	TRUE	TRUE

```
# Workflow header: data shape, T_bar, asymptote bounds, random effects, draws.
print(wf_shrimp)
```

```
<bayes_tls>
  Data:     148 rows; 7 temperatures; 8 durations
  T_bar:    31.5
  Bounds:   response in (0.000, 1.000); low in (0.001, 0.499); up in (0.501, 0.999)
  RE:       Date, Tank
  Status:   fitted (16000 draws)
```

4.3.1 Sampling diagnostics

```
tinytable::tt(diagnose_tdt_fit(wf_shrimp), digits = 3, escape = TRUE)
```

The diagnostics table is a numerical pass/fail summary. The canonical visual check is the chain trace ("fuzzy caterpillar"): healthy chains overlap and explore the same posterior support; chains that drift, diverge, or hang in different regions signal a problem with the fit.

```
brms::mcmc_plot(
  get_brmsfit(wf_shrimp),
  type     = "trace",
  variable = c("b_lowraw_Intercept", "b_upraw_Intercept",
               "b_logk_Intercept",   "b_mid_Intercept",
               "b_mid_temp_c",       "phi")
) +
  ggplot2::theme_classic(base_size = 11) +
  ggplot2::theme(legend.position = "bottom")
```

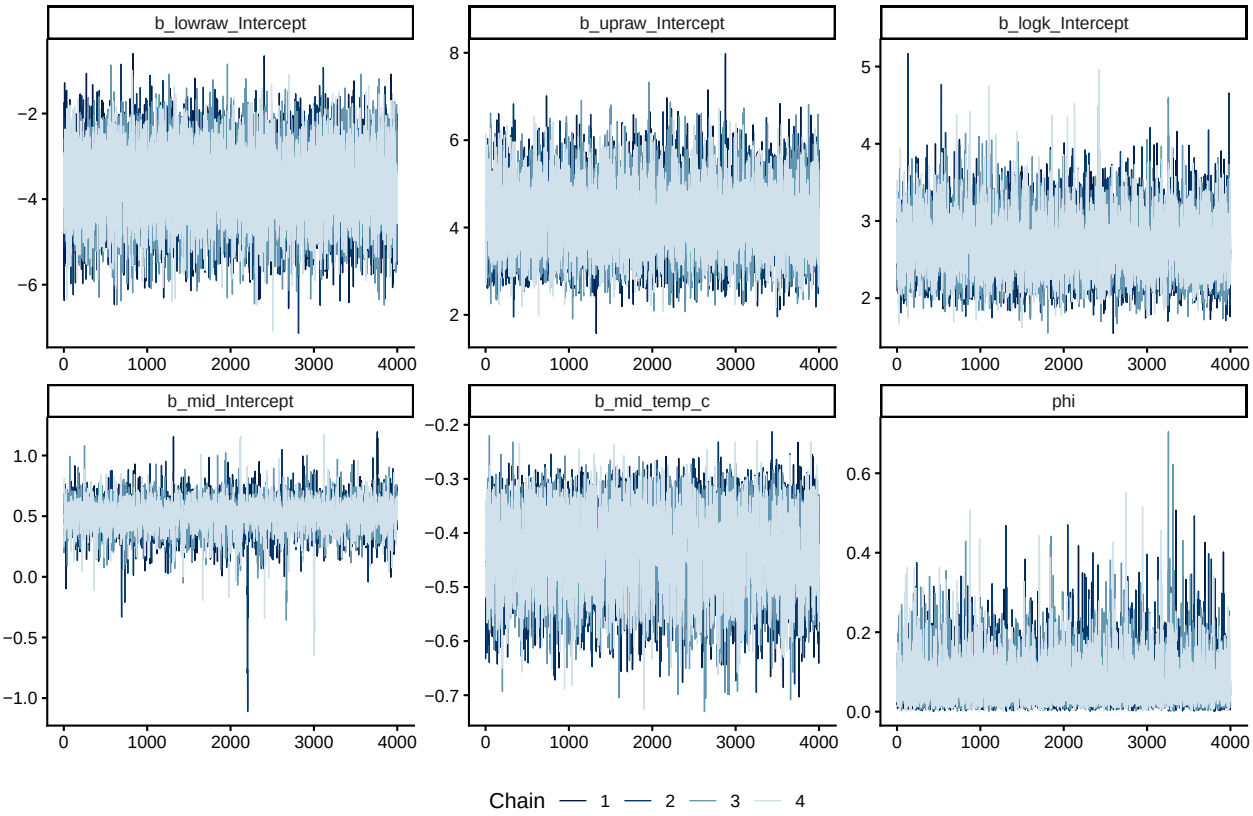


Figure S17: Posterior chain traces for the shrimp 4PL fixed-effect parameters and the beta-binomial dispersion ϕ. Each panel overlays four chains; well-mixed chains overlap throughout.

4.3.2 Posterior parameters on the natural scale

```
tinytable::tt(tdt_parameter_table(wf_shrimp), digits = 3, escape = TRUE)
```

4.4 Survival curves with observed overlay

`predict_survival_curves()` evaluates the fitted 4PL on a grid of assay temperatures and exposure durations and returns posterior survival probabilities. Overlaying the per-replicate observed proportions gives a visual check that the fitted curves track the data at every temperature.

```
psc_shrimp <- predict_survival_curves(
  wf_shrimp,
  temps     = sort(unique(shrimp_std$temp)),
  ndraws    = 500
)
plot_survival_curves(psc_shrimp, observed = shrimp_std) +
  ggplot2::labs(x = "Exposure duration (hours)") +
  ggplot2::coord_cartesian(xlim = c(0, 10))
```

Table S12: Posterior medians and 95% credible intervals for the shrimp 4PL fit, transformed back to the natural scale. `low` and `up` are the bottom and top asymptotes of the survival curve at the grand-mean temperature; `k` is the slope on $\log_{10}$ time; `mid` intercept is $\log_{10}$ LT50 in hours at the grand-mean temperature; `mid temp_c slope` is β_1, from which $z = -1/\beta_1$.

parameter	median	lower	upper
low (lower asymptote)	0.0139	0.00331	0.0585
up (upper asymptote)	0.9915	0.97161	0.9975
k (slope)	14.479	7.76483	33.426
mid intercept (at T_bar)	0.5082	0.24413	0.731
mid temp_c slope	-0.4305	-0.5834	-0.3077
z (°C)	2.3229	1.7141	3.2503

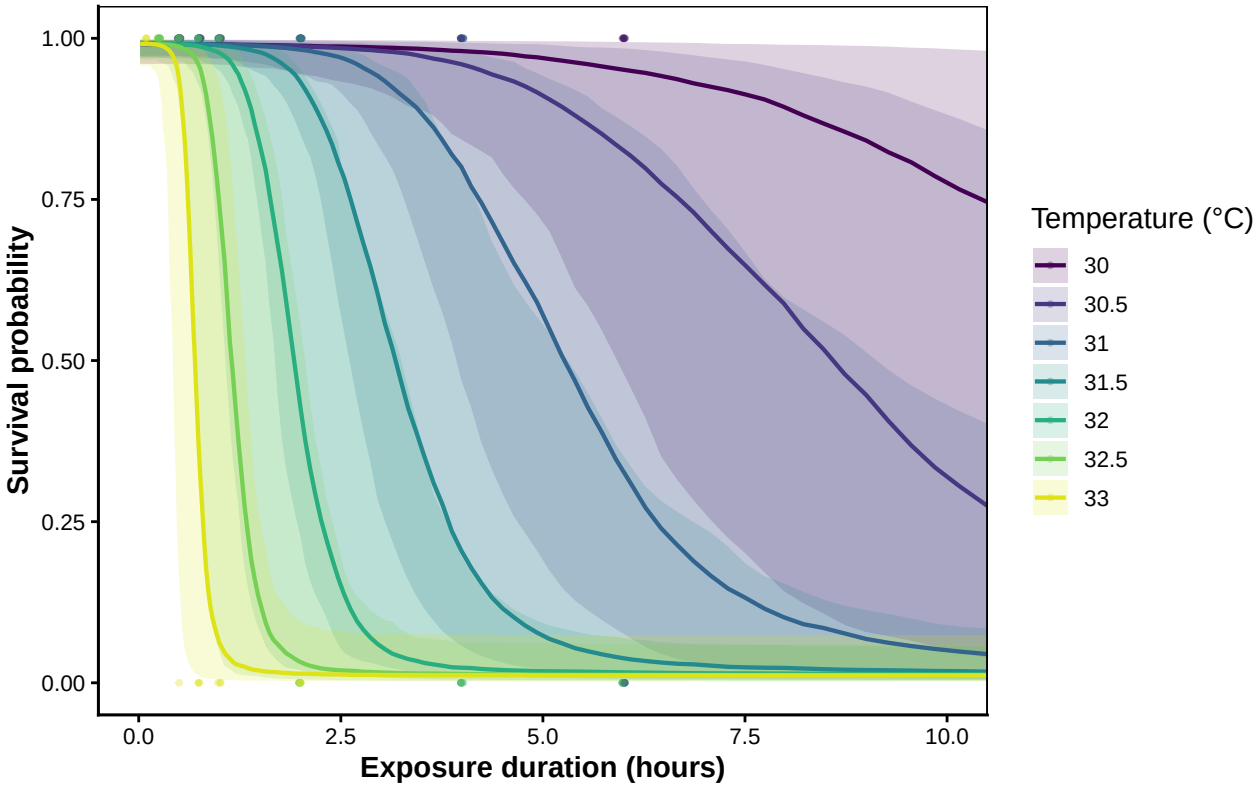


Figure S18: Posterior survival curves for brown shrimp at each assayed temperature, with 95% credible bands. Coloured points are observed per-replicate survival proportions.

4.5 TDT landscape

`derive_tdt_landscape()` evaluates the fitted 4PL on a dense (temperature, duration) grid and `plot_tdt_landscape()` renders it as a heatmap with overlaid contours at 25, 50, and 75% survival.

This is the two-dimensional view of the dose-response surface that the TDT line summarises with one number per temperature.

```
shrimp_dur_grid <- seq(min(shrimp_std$duration), max(shrimp_std$duration),
                       length.out = 120)
lsp_shrimp <- derive_tdt_landscape(wf_shrimp,
                                   duration_grid = shrimp_dur_grid,
                                   ndraws        = 500)
plot_tdt_landscape(lsp_shrimp, observed = shrimp_std)
```

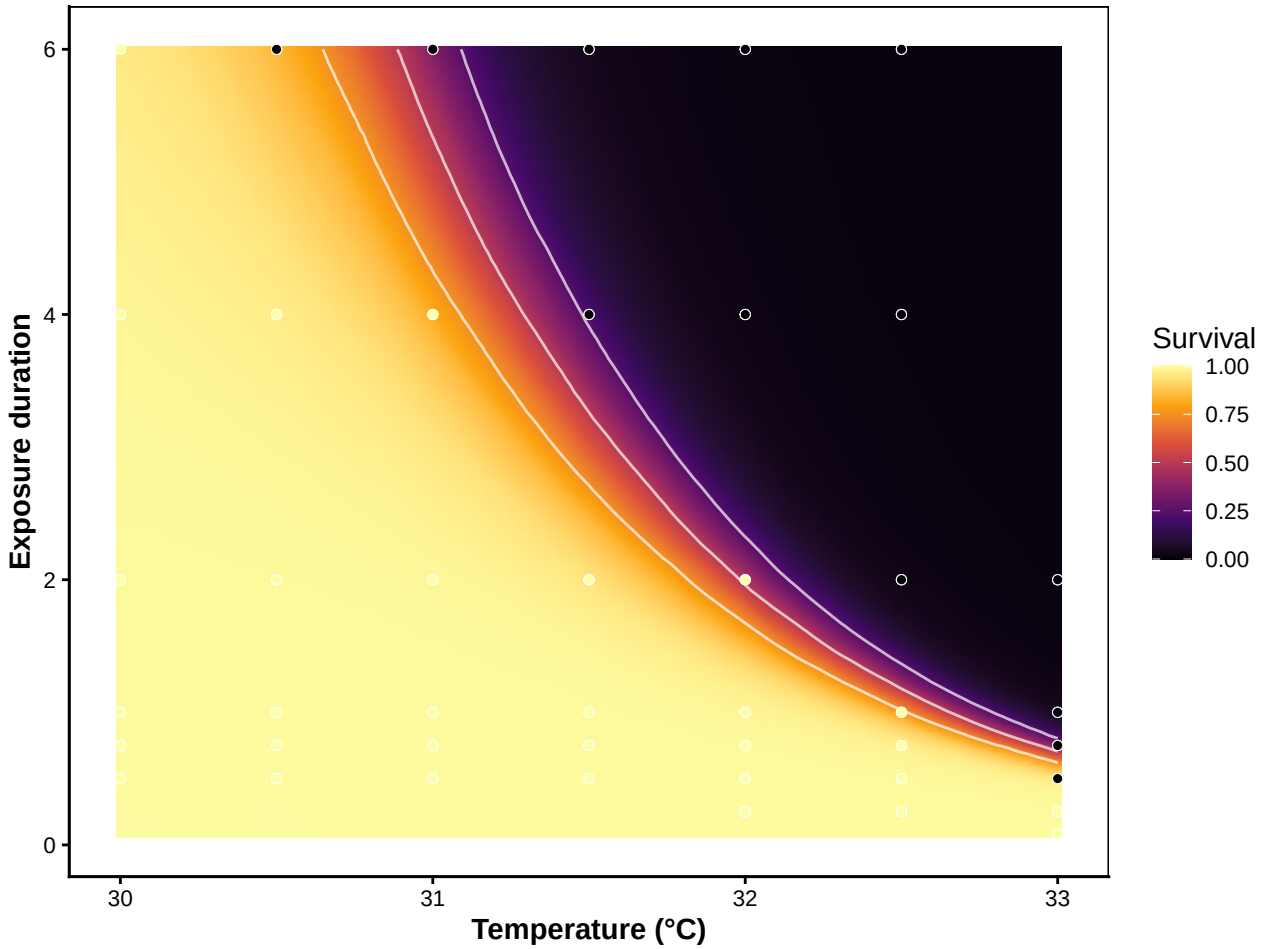


Figure S19: TDT landscape for brown shrimp: posterior median predicted survival across a 120 × 120 grid of assay temperatures × durations. White contours mark 25%, 50%, and 75% survival isolines. Overlaid points are raw observed proportions.

4.6 TDT line: time to the threshold vs temperature

The one-dimensional **TDT line** summarises the dose-response surface as the predicted time to the survival threshold at each temperature. `derive_tdt_curve()` builds it from the joint fit, and `plot_tdt_curve()` shows it on linear and $\log_{10}$ time axes side-by-side: the linear panel reads as

"hours of exposure half the animals tolerate" for ecological interpretation; the $\log_{10}$ panel linearises the relationship into the canonical TDT straight line, whose slope is $-1/z$. The default threshold is mid (the midpoint between the fitted asymptotes); pass `target_surv = "absolute"` for the classical absolute-t_{50} definition. For the shrimp data the two coincide because the asymptotes are anchored near 0 and 1.

```
ltx_shrimp <- derive_tdt_curve(
  wf_shrimp,
  temp_grid        = seq(min(shrimp_std$temp) - 1.5,
                         max(shrimp_std$temp) + 1.5, by = 0.05),
  ndraws           = 500,
  time_multiplier  = 1,
  output_time_unit = "h"
)
```

```
plot_tdt_curve(ltx_shrimp, panels = "both", colour = "#b2182b")
```

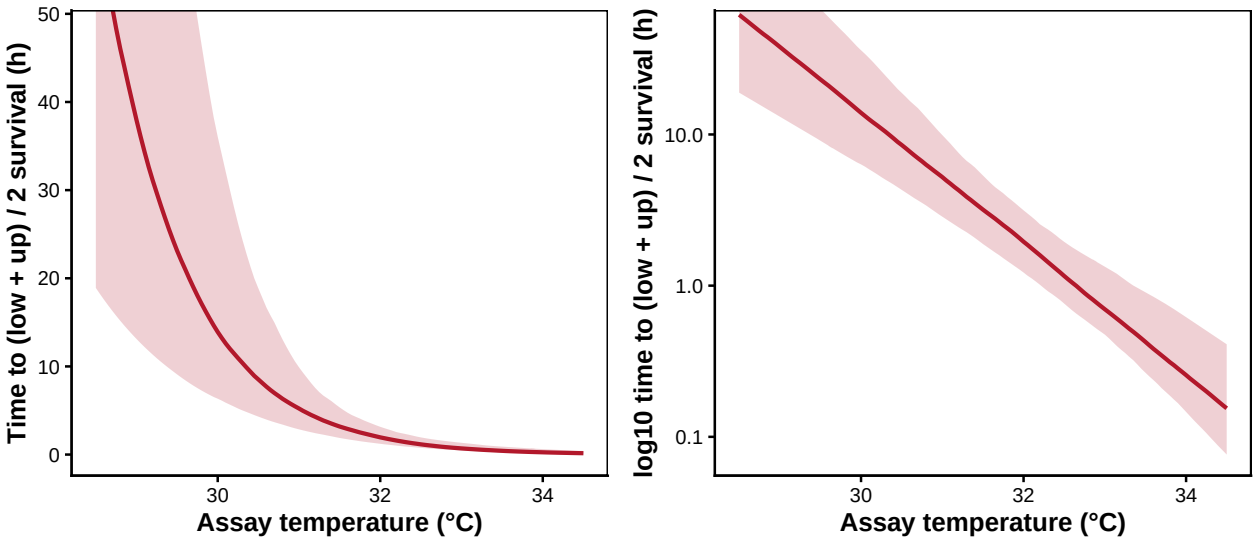


Figure S20: Brown shrimp posterior TDT line: predicted time to threshold vs assay temperature, with 95 % credible band. Linear-time panel on the left, $\log_{10}$-time panel on the right (slope is $-1/z$). Both panels share the same posterior.

4.7 Classical TDT quantities: z, $CT_{max_{1hr}}$, T_{crit}

`extract_tdt()` derives the classical thermal-tolerance summaries from a fitted 4PL workflow. It always returns z (thermal sensitivity read directly from the posterior — at the relative threshold $z = -1/\beta_1$ per draw, the slope of mid(T)) and $CT_{max_{1hr}}$ (the temperature at which the threshold is reached after 1 hour of exposure, obtained by per-draw 4PL inversion). Passing `lethal = TRUE` additionally returns T_{crit} under the rate-multiplier definition; this only makes sense when the endpoint is mortality (see Equation S3), so we opt in for the shrimp data.

Table S13: Classical TDT summaries for brown shrimp, with full posterior uncertainty. z is read directly from the posterior (the temperature slope on mid; $z = -1/\beta_1$ at the relative threshold). $CT_{max_{1hr}}$ is the temperature at 50% survival after 1-hour exposure (4PL inversion per draw). T_{crit} is the temperature at which the TDT damage rate falls below r^* % HI per hour, with r^* sampled uniformly on $\log_{10}$ across $[0.1, 1]$ — the pooled posterior carries both parameter uncertainty and operational uncertainty in r^*.

Quantity	Median	Lower	Upper
z (°C)	2.3	1.7	3.2
CTmax_1hr (°C)	32.7	32.2	33.6
T_crit (°C, r* in [0.1, 1] per hour)	26.9	23.9	29.1

```
et_shrimp <- extract_tdt(
  wf_shrimp,
  t_ref         = 60,
  TC_rate_range = c(0.1, 1),
  ndraws        = 500,
  lethal        = TRUE
)
```

```
shrimp_extract <- tibble::tribble(
  ~Quantity,                          ~Median, ~Lower, ~Upper,
  "z (°C)",                               et_shrimp$z$summary$z_median,
                                      et_shrimp$z$summary$z_lower,
                                      et_shrimp$z$summary$z_upper,
  "CTmax_1hr (°C)",                   et_shrimp$CTmax$summary$temp_median,
                                      et_shrimp$CTmax$summary$temp_lower,
                                      et_shrimp$CTmax$summary$temp_upper,
  "T_crit (°C, r* in [0.1, 1] per hour)",
                                      et_shrimp$T_crit$summary$temp_median,
                                      et_shrimp$T_crit$summary$temp_lower,
                                      et_shrimp$T_crit$summary$temp_upper
)
tinytable::tt(shrimp_extract, digits = 2, escape = TRUE)
```

```
z_draws_shrimp   <- get_z_draws(et_shrimp) |>
  dplyr::transmute(temp = z)
z_q <- stats::quantile(z_draws_shrimp$temp,
                       c(0.025, 0.5, 0.975), names = FALSE, na.rm = TRUE)
z_post_shrimp <- list(
  draws   = z_draws_shrimp,
  summary = tibble::tibble(temp_lower  = z_q[1],
                           temp_median = z_q[2],
                           temp_upper  = z_q[3])
)
```

```
patchwork::wrap_plots(
  plot_temperature_density(z_post_shrimp,
                           x_label = expression(italic(z)~"(°C per 10-fold change in time)")),
  plot_temperature_density(et_shrimp$CTmax,
                           x_label = expression(CT[max[1*hr]]~("°C"))),
  plot_temperature_density(et_shrimp$T_crit,
                           x_label = expression(T[crit]~("°C"))),
  ncol = 3
)
```

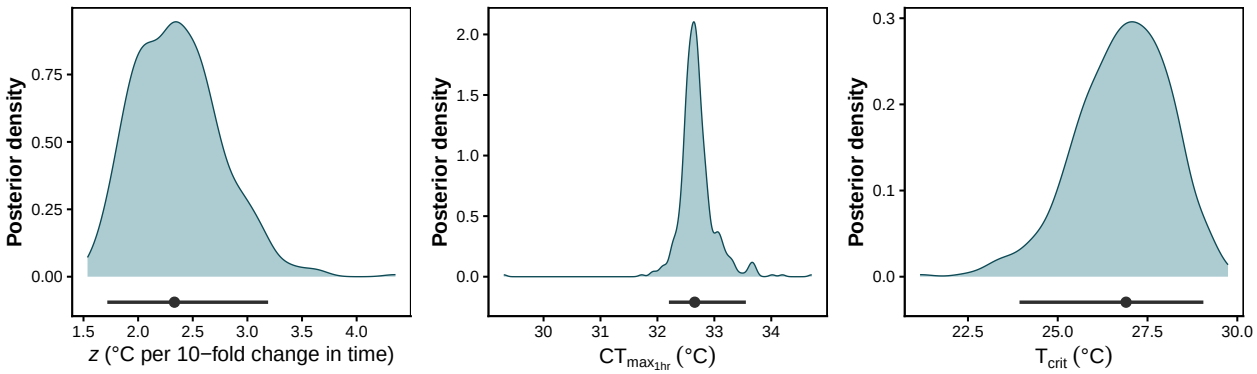


Figure S21: Posterior densities of z (left), $CT_{max_{1hr}}$ (middle), and T_{crit} (right) for brown shrimp. The horizontal bar below each density is the 95% credible interval; the point marks the posterior median. The ~5 °C gap between $CT_{max_{1hr}}$ and T_{crit} is the window between the onset of detectable damage accumulation and the LT50 temperature at 1 h.

4.8 Comparison between approaches

```
shrimp_compare_approaches <- dplyr::bind_rows(
  dplyr::bind_rows(lapply(
    names(shrimp_twostep_unc),
    function(m) {
      pt <- dplyr::filter(shrimp_twostep_tdt, method == m)
      ci <- shrimp_twostep_unc[[m]]$summary_ci
      tibble::tibble(
        Approach        = m,
        `z (C)`         = format_interval(pt$z,         ci$z_lower,     ci$z_upper,     digits
        `CTmax_1hr (C)` = format_interval(pt$CTmax_1hr, ci$CTmax_lower, ci$CTmax_upper, digits
        `T_crit (C)`    = format_interval(pt$T_crit,    ci$Tcrit_lower, ci$Tcrit_upper, digits
      )
    }
  )),
  tibble::tibble(
    Approach = "Joint Bayesian 4PL",
```

Table S14: Brown shrimp TDT quantities from the classical two-stage pipeline and the joint Bayesian 4PL. Two-stage rows report point estimates with 95% intervals derived from the Stage-2 linear model only — z, $CT_{max_{1hr}}$ and T_{crit} from 1000 parametric simulations of the Stage-2 MVN coefficient distribution. Stage-1 LT50 uncertainty is not propagated, mirroring how classical two-step TDT estimates are reported in the literature. The 4PL row reports posterior medians with 95% credible intervals.

Approach	z (C)	CTmax_1hr (C)	T_crit (C)
Two-step binomial	2.32 [1.66, 3.84]	32.64 [32.36, 33.06]	26.85 [24.8, 27.95]
Two-step beta-binomial	2.32 [1.66, 3.84]	32.64 [32.34, 33.08]	26.85 [25.01, 27.97]
Joint Bayesian 4PL	2.33 [1.72, 3.19]	32.66 [32.2, 33.55]	26.9 [23.94, 29.06]

```
    `z (C)` = format_interval(et_shrimp$z$summary$z_median,
                              et_shrimp$z$summary$z_lower,
                              et_shrimp$z$summary$z_upper,
                              digits = 2),
    `CTmax_1hr (C)` = format_interval(et_shrimp$CTmax$summary$temp_median,
                                      et_shrimp$CTmax$summary$temp_lower,
                                      et_shrimp$CTmax$summary$temp_upper,
                                      digits = 2),
    `T_crit (C)` = format_interval(et_shrimp$T_crit$summary$temp_median,
                                   et_shrimp$T_crit$summary$temp_lower,
                                   et_shrimp$T_crit$summary$temp_upper,
                                   digits = 2)
  )
)

tinytable::tt(shrimp_compare_approaches, escape = TRUE)
```

```
shrimp_twostep_line_h <- dplyr::bind_rows(lapply(
  shrimp_stage2_twostep,
  make_twostep_curve_case,
  temp_grid = shrimp_twostep_grid,
  time_multiplier = 1,
  output_time_unit = "h"
))

shrimp_twostep_band_h <- dplyr::bind_rows(lapply(
  names(shrimp_twostep_unc),
  function(m) {
    shrimp_twostep_unc[[m]]$curve_ci |>
      dplyr::transmute(
        temp,
        duration_lower = duration_lower / 60,
        duration_upper = duration_upper / 60,
```

```
        method = m
      )
  }
))

shrimp_4pl_line_h <- ltx_shrimp$summary |>
  dplyr::transmute(temp,
                   duration_median,
                   duration_lower,
                   duration_upper,
                   method = "Joint Bayesian 4PL")

shrimp_method_cols <- c(
  "Joint Bayesian 4PL" = "#b2182b",
  "Two-step binomial" = "#2166ac",
  "Two-step beta-binomial" = "#1b9e77"
)

p_shrimp_cmp_lin <- ggplot2::ggplot() +
  ggplot2::geom_ribbon(
    data = shrimp_4pl_line_h,
    ggplot2::aes(x = temp, ymin = duration_lower, ymax = duration_upper,
                 fill = method),
    alpha = 0.20, colour = NA
  ) +
  ggplot2::geom_ribbon(
    data = shrimp_twostep_band_h,
    ggplot2::aes(x = temp, ymin = duration_lower, ymax = duration_upper,
                 fill = method),
    alpha = 0.15, colour = NA
  ) +
  ggplot2::geom_line(
    data = shrimp_4pl_line_h,
    ggplot2::aes(x = temp, y = duration_median, colour = method),
    linewidth = 1
  ) +
  ggplot2::geom_line(
    data = shrimp_twostep_line_h,
    ggplot2::aes(x = temp, y = duration_median, colour = method),
    linewidth = 0.9
  ) +
  ggplot2::scale_colour_manual(values = shrimp_method_cols) +
  ggplot2::scale_fill_manual(values = shrimp_method_cols) +
  ggplot2::labs(x = "Assay temperature (C)",
                y = "Time to 50% survival (hours)",
                colour = "Approach", fill = "Approach") +
  ggplot2::theme_classic()
```

```
p_shrimp_cmp_log <- p_shrimp_cmp_lin +
  ggplot2::scale_y_log10() +
  ggplot2::labs(y = expression(log[10]~"time to 50% survival (hours)")) +
  ggplot2::theme(legend.position = "none")

patchwork::wrap_plots(p_shrimp_cmp_lin, p_shrimp_cmp_log, ncol = 2) +
  patchwork::plot_layout(guides = "collect") &
  ggplot2::theme(legend.position = "bottom")
```

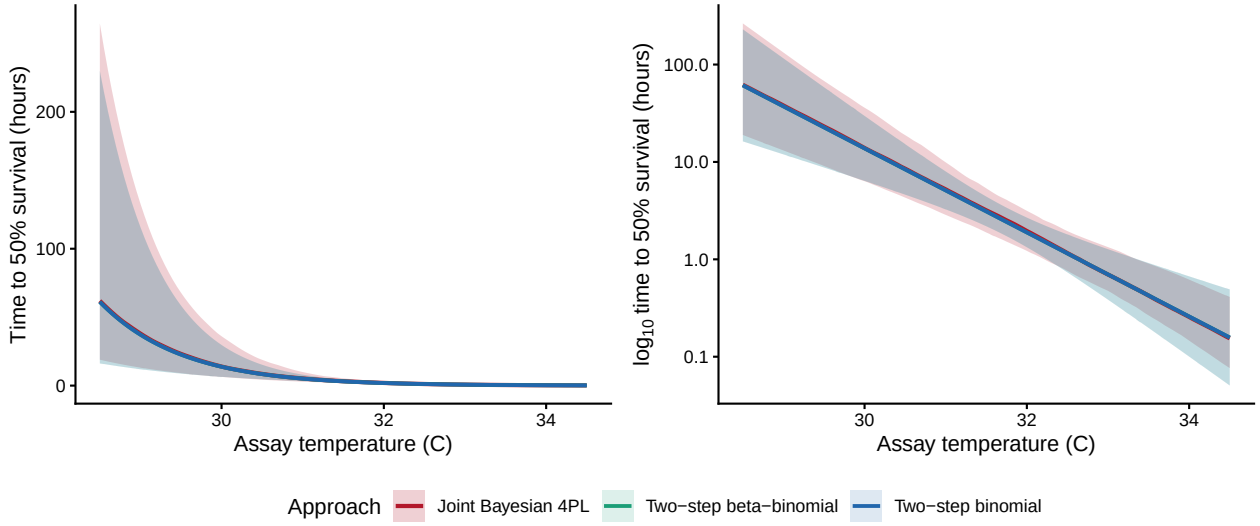


Figure S22: Brown shrimp TDT-line comparison. The joint 4PL posterior median and 95% credible band are repeated from Figure S20; the two classical two-stage lines are overlaid with their pointwise 95% confidence bands from `predict.lm()` on the Stage-2 regression (Stage-1 LT50s treated as known points).

4.9 Heat injury under reference traces

`predict_heat_injury()` projects the fitted dose-response surface through a temperature trace and returns cumulative heat injury (HI) and the resulting survival fraction at each time step. The damage threshold T_c below which injury does not accumulate is taken from the posterior median of T_{crit} extracted above, so the threshold is model-derived rather than hand-picked. We illustrate the function on four reference traces built by `make_temperature_scenarios()`: a flat baseline at the shrimp's natural holding temperature, a single one-hour spike, three one-hour spikes, and a four-day diurnal cycle that mimics a clustered coastal heatwave. The single-spike trace is calibrated as an analytical sanity check — its spike temperature is set to the posterior median of $CT_{max_{1hr}}$, so by definition the trace delivers approximately one LT50 dose by the end of the spike hour.

```
shrimp_T_c       <- et_shrimp$T_crit$summary$temp_median
shrimp_CTmax_med <- et_shrimp$CTmax$summary$temp_median

shrimp_scens <- make_temperature_scenarios(
```

```
  baseline   = 17,
  spike_temp = shrimp_CTmax_med,
  n_hours    = 96,
  spike_times_single = 24,
  spike_times_multi  = c(24, 48, 72),
  diurnal_n_days     = 4,
  diurnal_day_peaks  = c(22.0, 27.0, 30.0, 30.5),
  diurnal_night_temp = c(18.0, 19.5, 21.5, 22.0),
  diurnal_peak_fwhm  = 5,
  diurnal_noise_sd   = 0.3,
  diurnal_seed       = 1L
)

shrimp_hi <- purrr::map(shrimp_scens, function(sc) {
  predict_heat_injury(trace = sc, workflow = wf_shrimp,
                      T_c = shrimp_T_c, ndraws = 500)
})
```

Setting `repair = TRUE` adds a temperature-dependent Sharpe-Schoolfield repair kernel so injury accumulated at hot temperatures can be partially cleared during cooler intervals. Here we re-run only the two scenarios that accrue meaningful HI without repair — the multi-spike and the diurnal traces.

```
shrimp_repair_pars <- list(
  TA = 14065, TAL = 50000, TAH = 120000,
  TL = 10.5 + 273.15, TH = 22.5 + 273.15, TREF = 17 + 273.15,
  r_ref = 0.05
)

shrimp_hi_multi_rep <- predict_heat_injury(
  shrimp_scens$multi_spike, wf_shrimp, T_c = shrimp_T_c, ndraws = 500,
  repair = TRUE, repair_pars = shrimp_repair_pars
)
shrimp_hi_diurnal_rep <- predict_heat_injury(
  shrimp_scens$diurnal, wf_shrimp, T_c = shrimp_T_c, ndraws = 500,
  repair = TRUE, repair_pars = shrimp_repair_pars
)
```

```
plot_temperature_scenarios(shrimp_scens, T_c = shrimp_T_c)
```

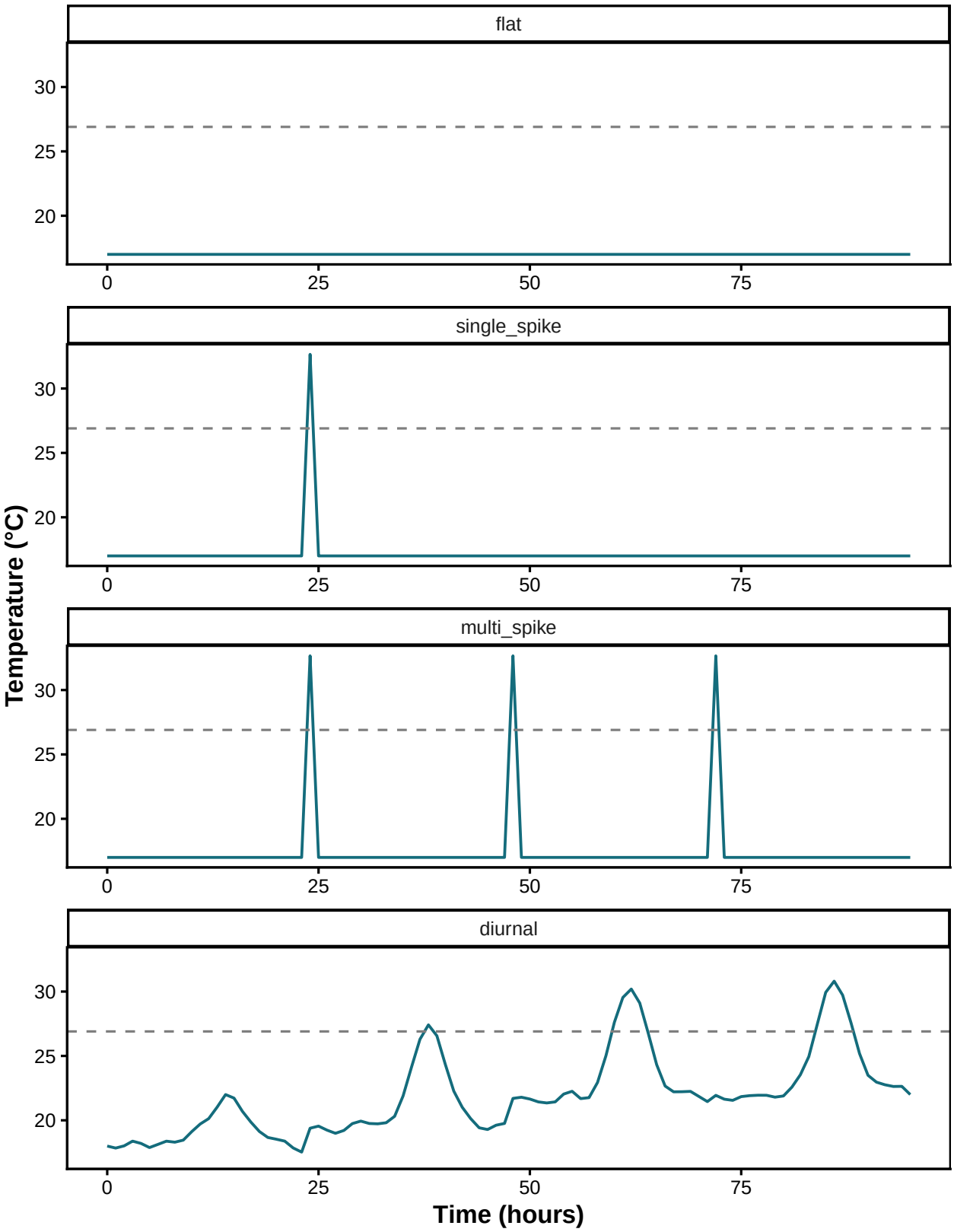


Figure S23: The four reference temperature traces used for the shrimp heat-injury prediction. Baseline 17 °C is the natural holding temperature; the 1-hour spikes are set to the posterior median $CT_{max_{1hr}}$ so each spike delivers approximately one LT50 dose. The diurnal scenario is a clustered 4-day heatwave: a cool day below T_{crit} followed by three progressively hotter days. The dashed line marks the damage threshold $T_c = T_{crit}$.

```
plot_heat_injury(shrimp_hi$single_spike)
```

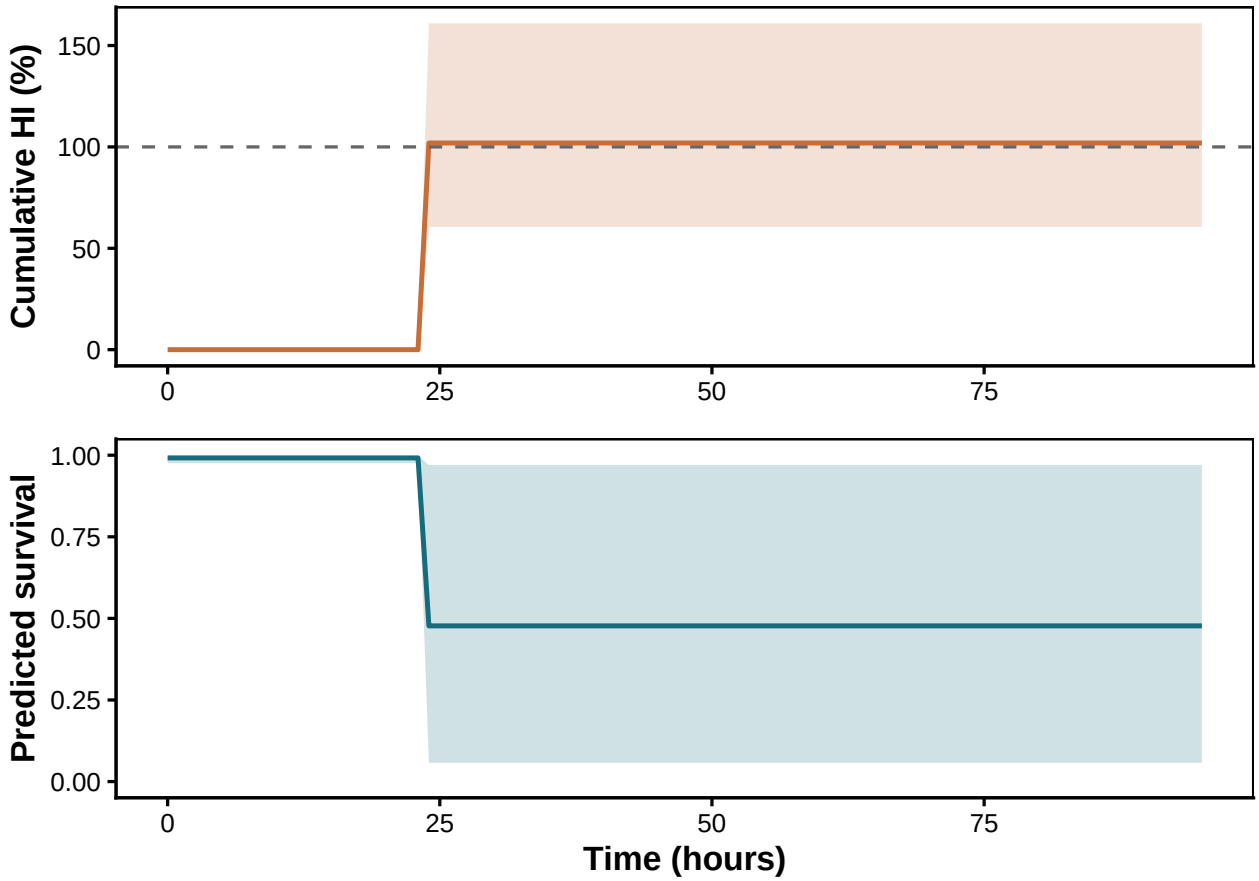


Figure S24: Cumulative heat injury (top) and predicted survival (bottom) under the **single-spike** trace, **no repair**. Because the spike sits at the posterior median $CT_{max_{1hr}}$, ~100% LT50-dose accrues by the end of the spike hour, producing ~50% mortality.

```
plot_heat_injury(shrimp_hi$multi_spike)
```

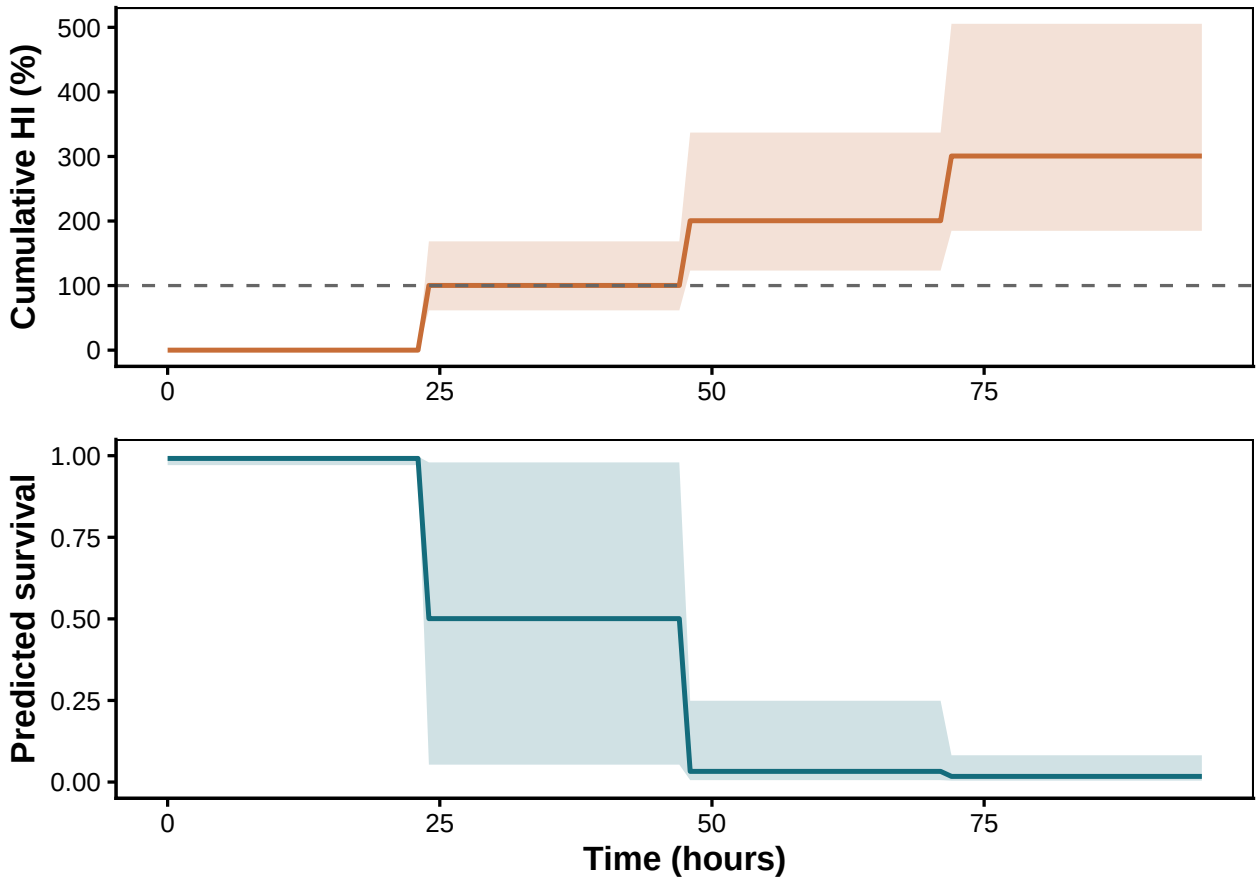


Figure S25: **Multi-spike** trace, **no repair**. Three 1-hour spikes at $CT_{max_{1hr}}$ deliver ~3 × LT50-dose, driving cumulative HI well past 100% and survival to near zero.

```
plot_heat_injury(shrimp_hi_multi_rep)
```

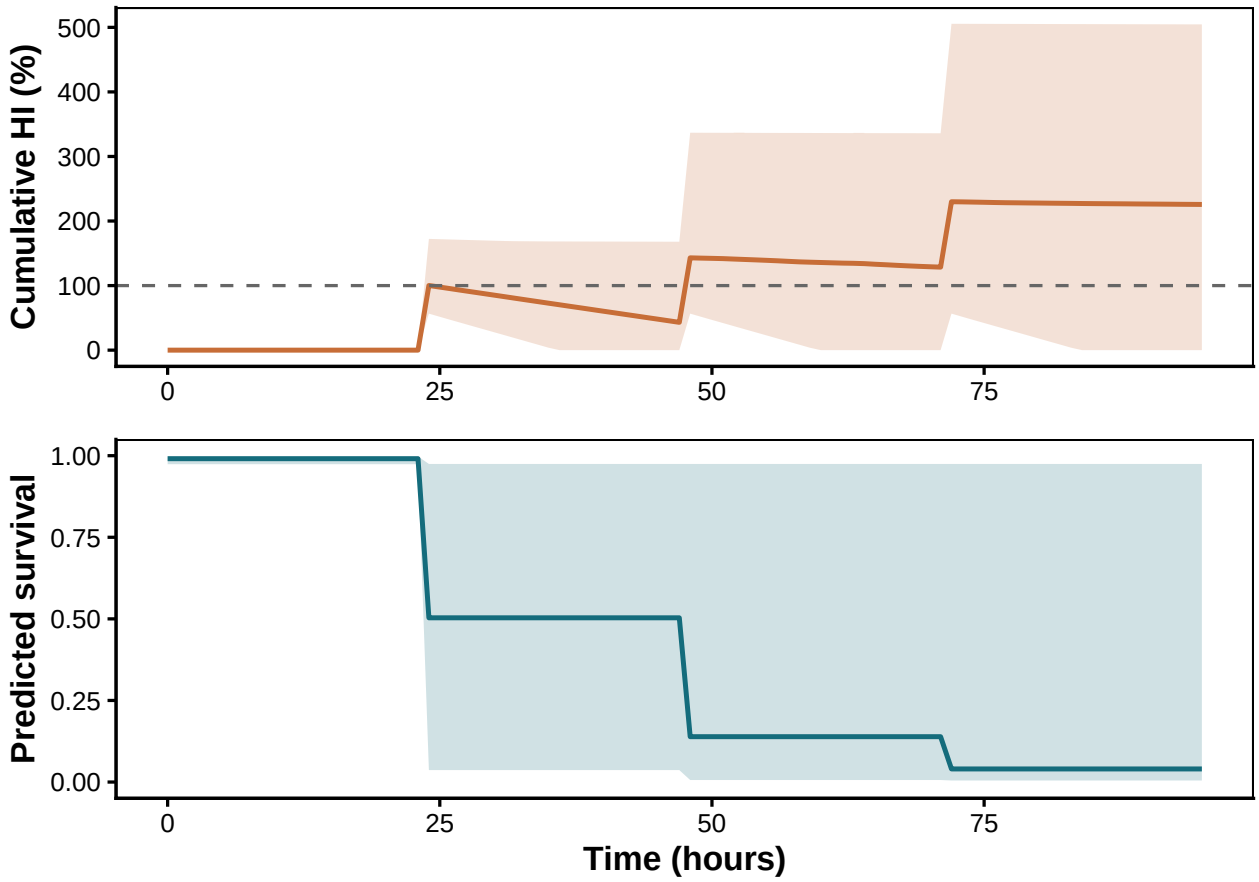


Figure S26: Same multi-spike trace, **with Sharpe-Schoolfield repair active**. Repair clears most of the dose between spikes, stopping the runaway accumulation seen without repair.

The diurnal scenario (Figure S27, Figure S28) is closer to a real coastal record. Day 1 stays below T_{crit} and accrues no injury; days 2–4 each contribute an injury bout that scales with the daily peak. Without repair, cumulative HI is well past 100 % by the end of day 4 and predicted survival has dropped to ~20 %. With repair on, off-peak hours and the cool first day act as recovery phases; the final trajectory is set by the net balance between daily accrual at sub-$CT_{max_{1hr}}$ peaks and overnight/cool-day repair.

```
plot_heat_injury(shrimp_hi$diurnal)
```

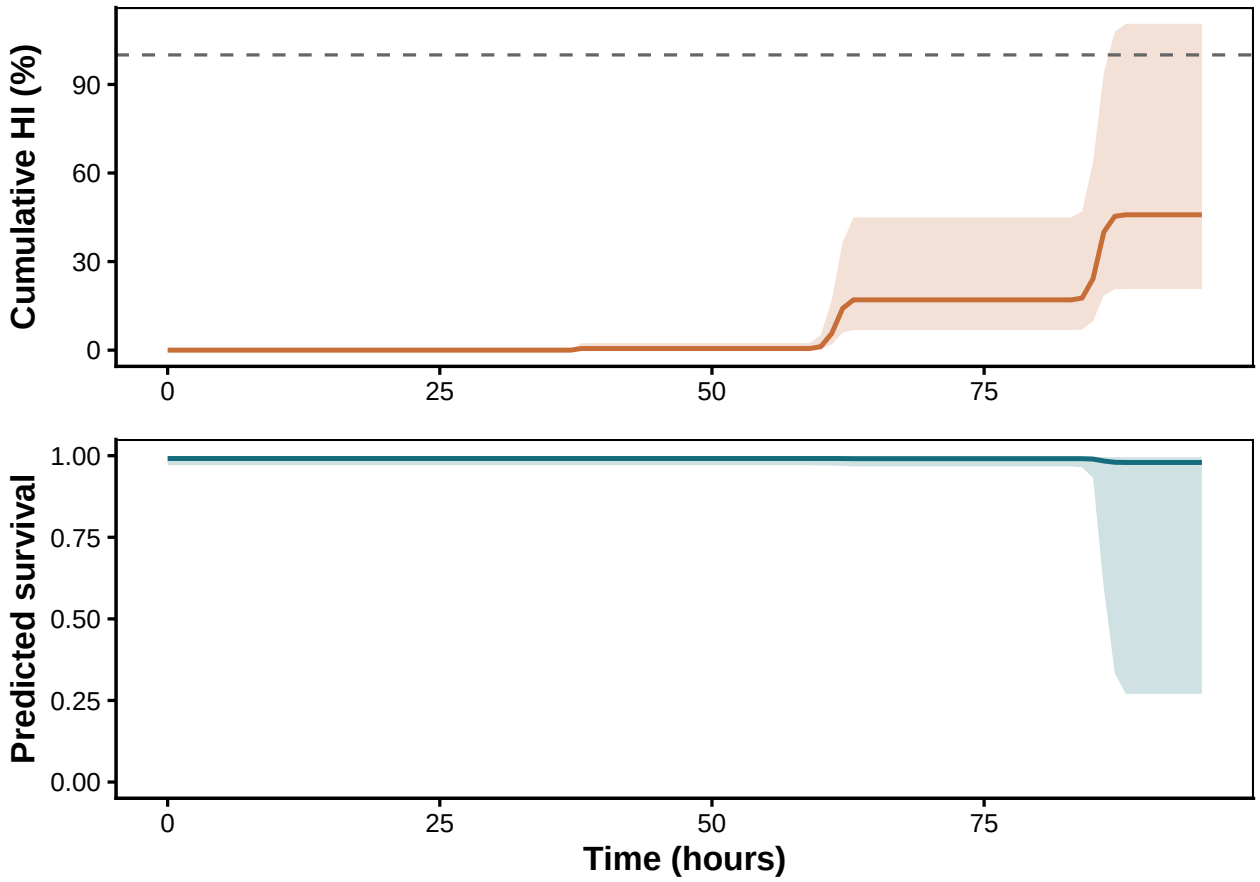


Figure S27: **4-day diurnal heatwave** trace, **no repair**. The cool first day flat-lines below T_{crit}; days 2–4 each accrue an injury bout scaling with the daily peak.

```
plot_heat_injury(shrimp_hi_diurnal_rep)
```

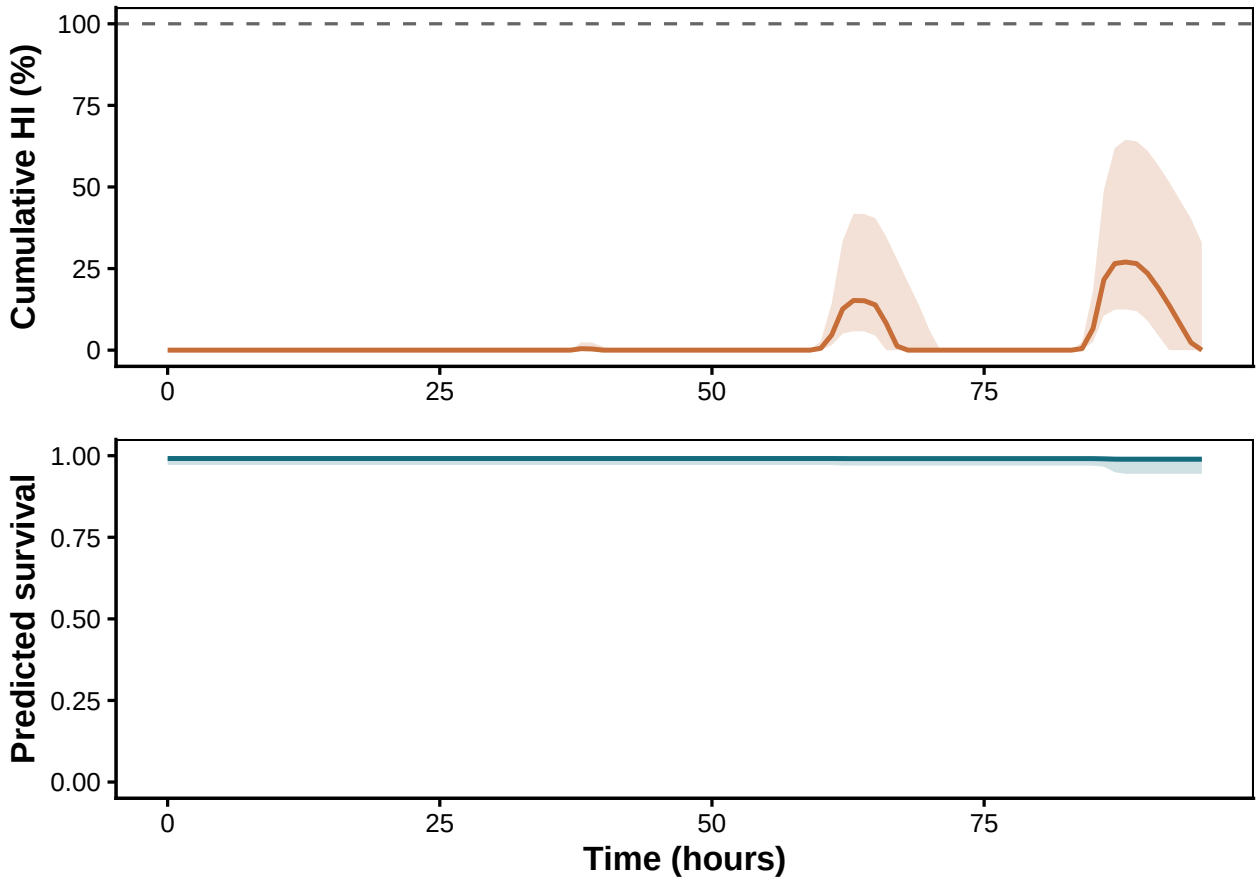


Figure S28: Same 4-day heatwave trace, **with Sharpe-Schoolfield repair active**. The cool first day and overnight troughs now act as recovery phases.

```
final_row <- function(hi) {
  s <- tail(hi$summary, 1)
  tibble::tibble(
    `HI median (%)` = round(s$hi_median, 1),
    `HI lower`      = round(s$hi_lower,  1),
    `HI upper`      = round(s$hi_upper,  1),
    `Survival median`  = round(s$surv_median, 3),
    `Survival lower`   = round(s$surv_lower,  3),
    `Survival upper`   = round(s$surv_upper,  3)
  )
}

shrimp_hi_table <- dplyr::bind_rows(
  final_row(shrimp_hi$flat)         |> dplyr::mutate(Scenario = "Flat",
                                                     Repair = "None"),
  final_row(shrimp_hi$single_spike) |> dplyr::mutate(Scenario = "Single spike",
                                                     Repair = "None"),
  final_row(shrimp_hi$multi_spike)  |> dplyr::mutate(Scenario = "Multi spike",
                                                     Repair = "None"),
```

Table S15: Final cumulative HI and predicted survival under each scenario for brown shrimp, posterior medians with 95% credible intervals.

Scenario	Repair	HI median (%)	HI lower	HI upper	Survival median	Survival lowe
Flat	None	0.0	0.0	0.0	0.992	0.969
Single spike	None	101.9	60.5	161.0	0.477	0.057
Multi spike	None	300.7	184.8	505.5	0.017	0.004
Multi spike	Sharpe-Schoolfield	225.7	0.0	504.6	0.040	0.005
Diurnal (4 days)	None	45.8	20.7	110.5	0.979	0.270
Diurnal (4 days)	Sharpe-Schoolfield	0.0	0.0	32.9	0.989	0.944

```
  final_row(shrimp_hi_multi_rep)     |> dplyr::mutate(Scenario = "Multi spike",
                                                     Repair = "Sharpe-Schoolfield"),
  final_row(shrimp_hi$diurnal)       |> dplyr::mutate(Scenario = "Diurnal (4 days)",
                                                     Repair = "None"),
  final_row(shrimp_hi_diurnal_rep)   |> dplyr::mutate(Scenario = "Diurnal (4 days)",
                                                     Repair = "Sharpe-Schoolfield")
) |>
  dplyr::select(Scenario, Repair, dplyr::everything())

tinytable::tt(shrimp_hi_table, escape = TRUE)
```

The shrimp case study covers the complete loop: raw counts → fitted joint 4PL → classical TDT quantities with calibrated uncertainty → heat-injury predictions under fluctuating field traces, with and without repair. Every quantity inherits the posterior of the same single fit. The same machinery applies to any TDT-style proportion dataset by changing only the column-name arguments passed to `standardize_data()`.

4.10 Sublethal time-to-knockdown data

The shrimp data also include a **sublethal** endpoint, time to loss of response to touch, where each cup contributes one observation of when its individuals were knocked down rather than a binary survival count at a pre-specified duration. The classical analysis fits a log-linear regression of $\log_{10}$(time to event) on temperature and reads $z = -1/\beta_1$ and $CT_{max_{1hr}} = \bar{T} + (\log_{10} 60 - \beta_0)/\beta_1$ off the coefficients (Jørgensen *et al.* 2021; Ørsted *et al.* 2022). Below we reproduce that classical analysis as a hierarchical Bayesian model, then re-cast the same observations as proportion-knocked-down counts and fit the joint 4PL on them (Ørsted *et al.* 2024), and finally compare the two pipelines' TDT lines and posterior summaries.

4.10.1 Data preparation

The bundled `shrimp_sublethal` dataset provides one row per cup with the elapsed time (minutes) to loss of response to touch — the raw clock-time strings have already been parsed and excluded

Table S16: Brown-shrimp sublethal time-to-knockdown panel: number of cups, median exposure time to knockdown, and 95% range, per assay temperature.

Temperature (°C)	N cups	Median (min)	Lower 95-pct (min)	Upper 95-pct (min)
31.0	60	158.99	12.69	348.03
31.5	59	63.82	3.55	197.16
32.0	60	60.15	1.51	176.85
32.5	60	6.41	1.17	77.03
33.0	60	1.24	0.31	19.77

rows dropped (see `?shrimp_sublethal`). We add the $\log_{10}$ time and mean-centred temperature columns the models use, keeping the grouping factors for date, tank, and cup.

```
# Bundled with the package (see ?shrimp_sublethal): clock-time strings already
# parsed to elapsed minutes-to-knockdown and excluded rows dropped (raw CSV in
# inst/extdata/). We add the log-time and centred-temperature columns used below.
tdt_sub <- shrimp_sublethal |>
  dplyr::mutate(
    log10_time = log10(time_to_event),
    temp_mean  = mean(assay_temp),
    temp_c     = assay_temp - temp_mean
  )
```

Table S16 summarises the sublethal panel.

```
tdt_sub |>
  dplyr::group_by(assay_temp) |>
  dplyr::summarise(
    `N cups`      = dplyr::n(),
    `Median (min)` = round(stats::median(time_to_event), 2),
    `Lower 95-pct (min)`  = round(stats::quantile(time_to_event, 0.025), 2),
    `Upper 95-pct (min)`  = round(stats::quantile(time_to_event, 0.975), 2),
    .groups       = "drop"
  ) |>
  dplyr::rename(`Temperature (°C)` = assay_temp) |>
  tinytable::tt(escape = TRUE)
```

4.10.2 Linear Bayesian time-to-event model

The classical sublethal TDT analysis fits a hierarchical log-linear regression: $\log_{10}(\text{time to event}) = \beta_0 + \beta_1(T - \bar{T}) + \text{random effects} + \varepsilon$, with random intercepts on the three experimental grouping factors (`date_experiment`, `tank_ID`, `cup_ID`). The two scalar TDT quantities follow as transformations of the posterior of (β_0, β_1):

$$z = -\frac{1}{\beta_1}, \qquad CT_{max_{1hr}} = \bar{T} + \frac{\log_{10}(60) - \beta_0}{\beta_1}.$$

```
priors_sub_lin <- c(
  brms::prior(normal(2, 1.5),    class = "Intercept"),
  brms::prior(normal(-1, 1),     class = "b"),
  brms::prior(exponential(2),    class = "sd"),
  brms::prior(exponential(2),    class = "sigma")
)

fit_sub_lin <- brms::brm(
  brms::bf(log10_time ~ temp_c +
           (1 | date_experiment) + (1 | tank_ID) + (1 | cup_ID)),
  data       = tdt_sub,
  prior      = priors_sub_lin,
  chains     = 4,
  cores      = 4,
  iter       = 8000,
  warmup     = 4000,
  init       = 0,
  control    = list(adapt_delta = 0.995, max_treedepth = 14),
  backend    = "cmdstanr",
  seed       = 123,
  silent     = 2,
  refresh    = 0,
  file       = file.path(models_dir, "fit_shrimp_sublethal_linear"),
  file_refit = "on_change"
)
```

Posterior summaries on the natural scale follow from the regression coefficients and the mean-centring temperature:

```
post_sub_lin <- posterior::as_draws_df(fit_sub_lin) |> as.data.frame()
temp_mean_sub <- mean(tdt_sub$assay_temp)

z_lin_draws       <- -1 / post_sub_lin$b_temp_c
CTmax_1hr_lin_draws <- temp_mean_sub +
  (log10(60) - post_sub_lin$b_Intercept) / post_sub_lin$b_temp_c

summarise_post <- function(x) {
  q <- stats::quantile(x, c(0.025, 0.5, 0.975), na.rm = TRUE, names = FALSE)
  tibble::tibble(median = q[2], lower = q[1], upper = q[3])
}
```

```
tibble::tibble(
  Quantity     = c("z (°C)", "CTmax at 1 hr (°C)"),
```

Table S17: Posterior summaries from the linear Bayesian sublethal time-to-event model: thermal sensitivity z and the critical temperature at the 1-hour reference duration $CT_{max_{1hr}}$, derived from the regression coefficients.

Quantity	Median	Lower	Upper
z (°C)	1.1	0.997	1.23
CTmax at 1 hr (°C)	31.5	31.207	31.75

```
  Median       = c(stats::median(z_lin_draws),
                   stats::median(CTmax_1hr_lin_draws)),
  Lower        = c(stats::quantile(z_lin_draws, 0.025, names = FALSE),
                   stats::quantile(CTmax_1hr_lin_draws, 0.025, names = FALSE)),
  Upper        = c(stats::quantile(z_lin_draws, 0.975, names = FALSE),
                   stats::quantile(CTmax_1hr_lin_draws, 0.975, names = FALSE))
) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

4.10.3 Joint 4PL on proportion knocked-down

Following Ørsted *et al.* (2024), the same time-to-knockdown observations can be re-cast as proportion-alive counts: at each assay temperature, the fraction of cups not yet knocked down by time t is a cumulative survival curve, the canonical 4PL input. For each (date × tank × temperature) trial we count cups still responding at a grid of evaluation times, then feed the resulting $(n_{\text{total}}, n_{\text{alive}})$ panel through `standardize_data()` and `fit_4pl()`.

```
ttd_range <- range(tdt_sub$time_to_event, na.rm = TRUE)
eval_times_min <- 10 ^ seq(log10(max(0.2, ttd_range[1] * 0.5)),
                           log10(ttd_range[2] * 1.5),
                           length.out = 18)

sub_4pl_panel <- tdt_sub |>
  dplyr::group_by(date_experiment, tank_ID, assay_temp) |>
  dplyr::summarise(n_total = dplyr::n(),
                   ttds    = list(time_to_event),
                   .groups = "drop") |>
  purrr::pmap_dfr(function(date_experiment, tank_ID, assay_temp,
                           n_total, ttds) {
    tibble::tibble(
      date_experiment = date_experiment,
      tank_ID         = tank_ID,
      assay_temp      = assay_temp,
      duration_min    = eval_times_min,
      n_total         = n_total,
      n_alive         = vapply(eval_times_min,
```

```
                                function(t) sum(ttds > t), integer(1))
    )
  })

std_sub_4pl <- standardize_data(
  sub_4pl_panel,
  temp           = "assay_temp",
  duration       = "duration_min",
  n_total        = "n_total",
  n_surv         = "n_alive",
  random_effects = c("date_experiment", "tank_ID"),
  duration_unit  = "minutes"
)
```

```
wf_sub_4pl <- fit_4pl(
  std_sub_4pl,
  random_effects = c("date_experiment", "tank_ID"),
  chains  = 4, iter = 8000, warmup = 4000, cores = 4, seed = 123,
  silent  = 2, refresh = 0,
  file    = file.path(models_dir, "fit_shrimp_sublethal_4pl"),
  file_refit = "on_change"
)
```

The fitted survival curves below are on the same axes as the lethal analysis — proportion still alive vs exposure duration — with the aggregated per-trial proportions overlaid.

```
psc_sub_4pl <- predict_survival_curves(
  wf_sub_4pl,
  temps  = sort(unique(std_sub_4pl$temp)),
  ndraws = 500
)
plot_survival_curves(psc_sub_4pl, observed = std_sub_4pl) +
  ggplot2::labs(x = "Exposure duration (minutes)",
                y = "Proportion of live individuals") +
  ggplot2::coord_cartesian(xlim = c(0, 600))
```

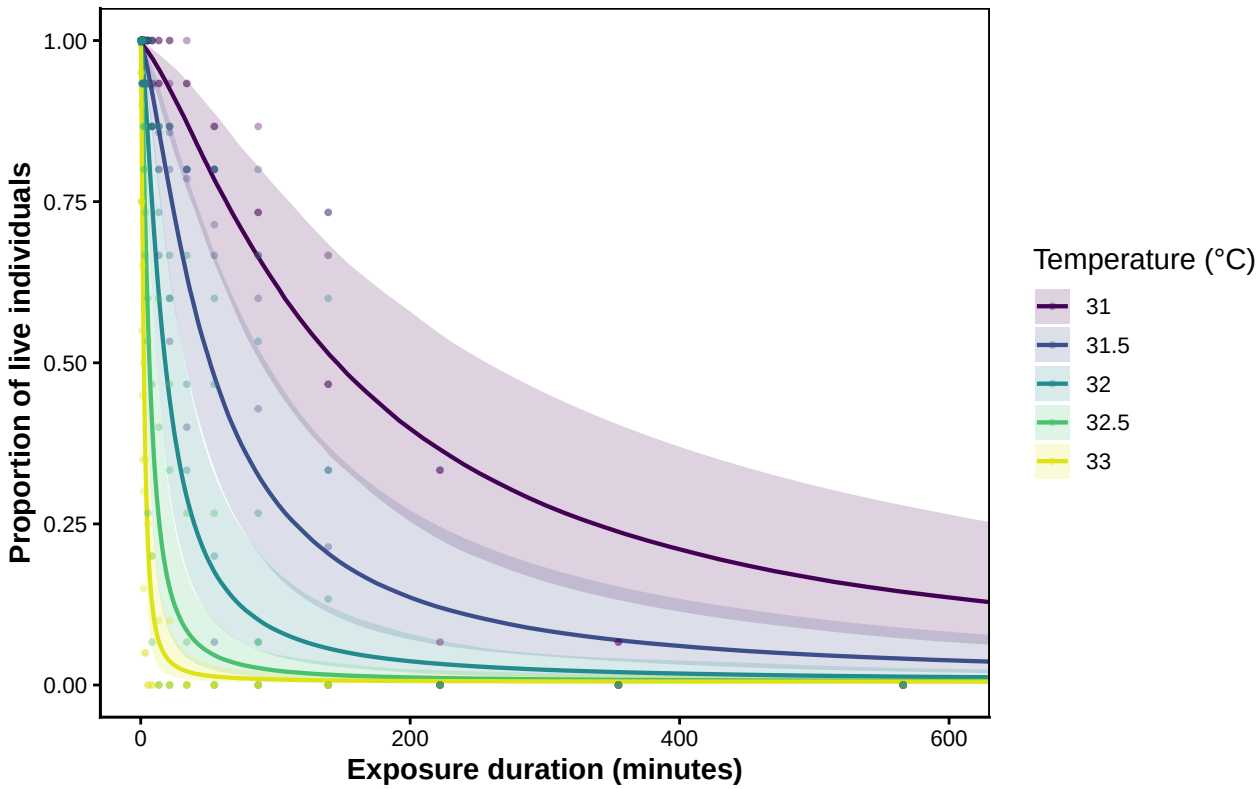


Figure S29: Posterior 4PL survival ('proportion still alive') curves for the brown-shrimp sublethal knockdown data at each assay temperature, with 95 % credible bands. Points are the aggregated proportion still alive per (date × tank × temperature) trial at each evaluation time.

The 4PL aggregation pools cups within each (date × tank × temperature) trial, so `cup_ID` is no longer needed as a grouping variable — the cup-level dispersion is absorbed by the beta-binomial precision parameter ϕ. The 4PL therefore uses two random intercepts (`date_experiment`, `tank_ID`) — one fewer than the cup-level linear fit above.

We then call `extract_tdt()` to summarise z and $CT_{max_{1hr}}$. We leave `lethal = FALSE` because the knockdown endpoint is sublethal: the fitted z measures the slope of performance reduction rather than damage accumulation, and the rate-multiplier T_{crit} formula is not interchangeable between the two (see the callout in Section 2.0.4).

```
et_sub_4pl <- extract_tdt(
  wf_sub_4pl,
  t_ref           = 60,
  time_multiplier = 1,
  ndraws          = 1000
)
```

```
tibble::tribble(
  ~Quantity,            ~Median,                              ~Lower,
  "z (°C)",             et_sub_4pl$z$summary$z_median,        et_sub_4pl$z$summary$z_lower,
```

Table S18: Posterior summaries from the joint Bayesian 4PL on proportion-knocked-down data: z and $CT_{max_{1hr}}$. T_{crit} is not reported for sublethal endpoints (see Section 2.0.4).

Quantity	Median	Lower	Upper
z (°C)	1.11	1.01	1.21
CTmax at 1 hr (°C)	31.43	31.18	31.67

```
  "CTmax at 1 hr (°C)",  et_sub_4pl$CTmax$summary$temp_median, et_sub_4pl$CTmax$summary$temp_lo
) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

4.10.4 Comparing the two sublethal pipelines

The two pipelines analyse the same knockdown observations through different statistical structures: the linear model regresses individual log-times on temperature, while the 4PL fits a sigmoidal cumulative-knockdown curve at each temperature and reads $\log_{10}$ LT50 from each posterior draw. Their posteriors of z and $CT_{max_{1hr}}$ should agree to within the data's information content.

```
temp_grid_sub <- seq(min(tdt_sub$assay_temp) - 0.25,
                     max(tdt_sub$assay_temp) + 0.25, by = 0.05)

lin_grid <- tibble::tibble(temp_c = temp_grid_sub - temp_mean_sub)
lin_lp   <- brms::posterior_linpred(fit_sub_lin, newdata = lin_grid,
                                    re_formula = NA)
lin_pred <- tibble::tibble(
  assay_temp     = temp_grid_sub,
  log10_time_med = apply(lin_lp, 2, stats::median),
  log10_time_lo  = apply(lin_lp, 2, stats::quantile, 0.025, names = FALSE),
  log10_time_hi  = apply(lin_lp, 2, stats::quantile, 0.975, names = FALSE),
  method         = "Linear Bayesian"
)

ltx_sub <- derive_tdt_curve(
  wf_sub_4pl,
  temp_grid       = temp_grid_sub,
  ndraws          = 1000,
  time_multiplier = 1
)

pl_4pl_pred <- ltx_sub$summary |>
  dplyr::transmute(
    assay_temp     = temp,
    log10_time_med = log10(duration_median),
    log10_time_lo  = log10(duration_lower),
```

```
    log10_time_hi  = log10(duration_upper),
    method         = "Joint 4PL (proportion knocked-down)"
  )

tdt_lines_sub <- dplyr::bind_rows(lin_pred, pl_4pl_pred) |>
  dplyr::mutate(method = factor(method,
                                levels = c("Linear Bayesian",
                                           "Joint 4PL (proportion knocked-down)")))
```

```
ggplot2::ggplot(tdt_lines_sub,
                ggplot2::aes(x = assay_temp, y = log10_time_med,
                             colour = method, fill = method)) +
  ggplot2::geom_ribbon(ggplot2::aes(ymin = log10_time_lo,
                                    ymax = log10_time_hi),
                       alpha = 0.18, colour = NA) +
  ggplot2::geom_line(linewidth = 1) +
  ggplot2::geom_jitter(
    data = tdt_sub,
    ggplot2::aes(x = assay_temp, y = log10_time),
    inherit.aes = FALSE,
    width = 0.05, height = 0,
    alpha = 0.35, size = 1.2, colour = "grey30"
  ) +
  ggplot2::scale_colour_manual(values = c("#2c7fb8", "#d95f02")) +
  ggplot2::scale_fill_manual(values   = c("#2c7fb8", "#d95f02")) +
  ggplot2::labs(x = "Assay temperature (°C)",
                y = expression(log[10]~"(time to 50% knockdown, min)"),
                colour = NULL, fill = NULL) +
  ggplot2::theme_classic() +
  ggplot2::theme(legend.position = "top")
```

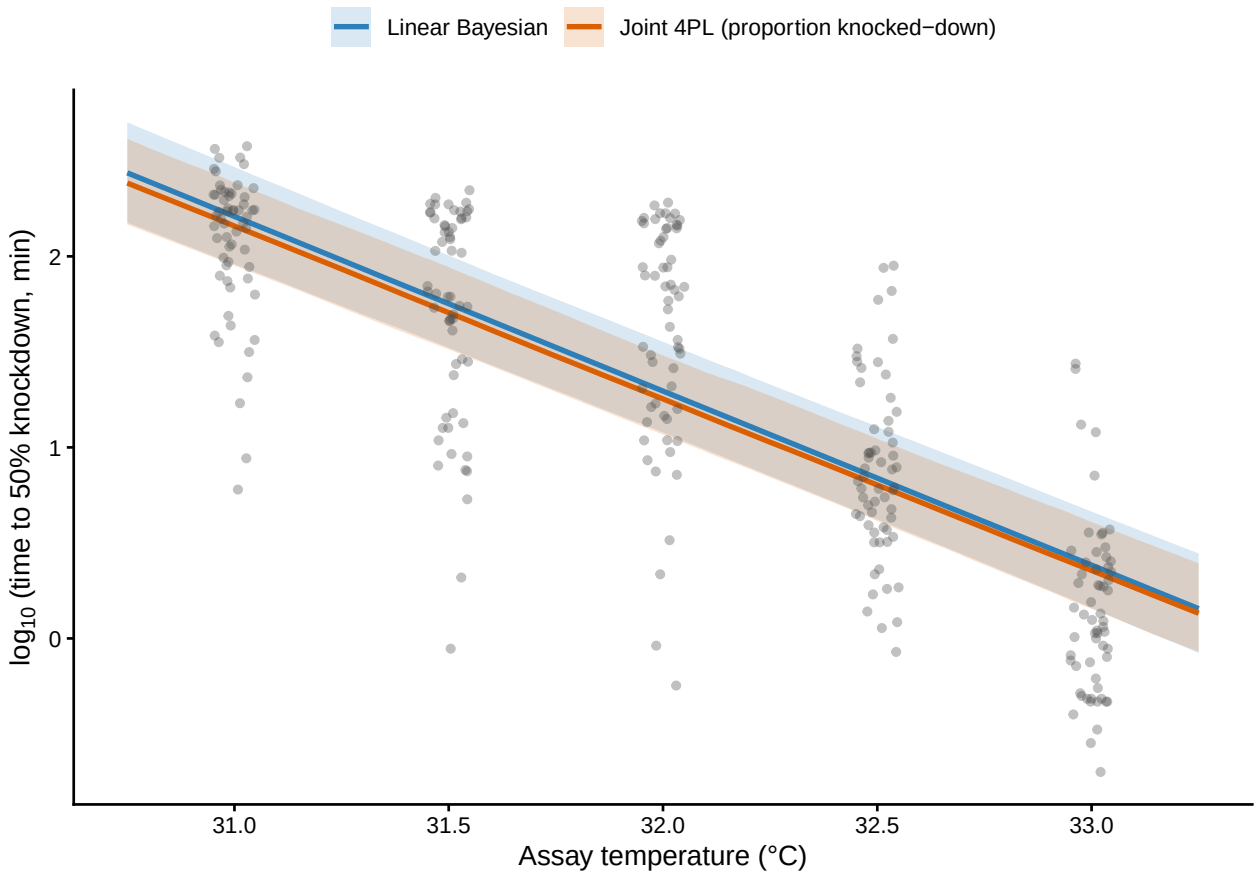


Figure S30: Posterior $\log_{10}$(time to 50% knockdown) versus assay temperature, from the linear Bayesian model (blue) and the joint 4PL on proportion-knocked-down data (orange). Shaded ribbons are 95% credible intervals; grey points are the individual observed knockdown times. The two pipelines analyse the *same* data and produce TDT lines that overlap across the assay range.84

Table S19: Side-by-side posterior summaries of z and $CT_{max_{1hr}}$ from the linear Bayesian time-to-event model and the joint 4PL on proportion-knocked-down data. Both fits use the same sublethal observations and the same hierarchical grouping structure (date and tank; the linear fit additionally carries a per-cup random intercept that is absorbed into `n_total` in the 4PL aggregation).

Quantity	Linear median	Linear lower	Linear upper	4PL median	4PL lower	4PL upper
z (°C)	1.1	0.997	1.23	1.11	1.01	1.21
CTmax at 1 hr (°C)	31.5	31.207	31.75	31.43	31.18	31.67

```
compare_sub <- tibble::tibble(
  Quantity = c("z (°C)", "CTmax at 1 hr (°C)"),
  `Linear median`  = c(stats::median(z_lin_draws),
                       stats::median(CTmax_1hr_lin_draws)),
  `Linear lower`   = c(stats::quantile(z_lin_draws, 0.025, names = FALSE),
                       stats::quantile(CTmax_1hr_lin_draws, 0.025, names = FALSE)),
  `Linear upper`   = c(stats::quantile(z_lin_draws, 0.975, names = FALSE),
                       stats::quantile(CTmax_1hr_lin_draws, 0.975, names = FALSE)),
  `4PL median`     = c(et_sub_4pl$z$summary$z_median,
                       et_sub_4pl$CTmax$summary$temp_median),
  `4PL lower`      = c(et_sub_4pl$z$summary$z_lower,
                       et_sub_4pl$CTmax$summary$temp_lower),
  `4PL upper`      = c(et_sub_4pl$z$summary$z_upper,
                       et_sub_4pl$CTmax$summary$temp_upper)
)

tinytable::tt(compare_sub, digits = 3, escape = TRUE)
```

The two sublethal pipelines yield consistent estimates of z and $CT_{max_{1hr}}$ on this dataset (medians within their mutual credible intervals; see Table S19). The 4PL has the practical advantage that the same `extract_tdt()` workflow used on the lethal counts also produces a posterior TDT curve and $CT_{max_{1hr}}$ — quantities the linear time-to-event model does not deliver directly. T_{crit}, however, should be read from the lethal fit, not the sublethal one (Section 2.0.4).

4.10.5 A caution on cross-endpoint comparison of T_{crit}

If Equation Equation S3 is nevertheless applied to both shrimp endpoints, the sublethal value lies ~3.5 °C above the lethal value and their 95% credible intervals do not overlap. This difference is not a biological threshold shift: loss of response to touch precedes death. Rather, $CT_{max_{1hr}}$ is similar between endpoints (~31.4–31.6 °C), while sublethal z (≈ 1.1 °C) is less than half lethal z (≈ 2.6 °C), yielding a gap of approximately $2.5 \times (2.6 - 1.1)$ °C in the derived T_{crit}. Outside the assay window, this extrapolation makes the sublethal TDT line predict knockdown later than death, which is biologically impossible. Thus, rate-multiplier T_{crit} is an endpoint-conditional operational summary, not a universal physiological threshold (Faber *et al.* 2026; Jørgensen *et al.* 2021).

Table S20: Zebrafish lethal-TDT panel summary, by life stage. Each row reports the number of trials, temperature and duration ranges, total individuals assayed, and number of experimental dates contributing to that stage.

life_stage	n trials	Temperature range (°C)	Duration range (hr)	Total individuals	n Dates
young_embryos	118	38.1–42.2	0.02–6	2944	3
old_embryos	106	38.1–42.2	0.08–10	2632	3
larvae	99	38.4–42.3	0.08–6	988	2

5 Case Study 2: Zebrafish lethal-TDT across life stages

We apply the same TLS/TDT workflow to lethal-TDT data from zebrafish (*Danio rerio*) at three early life stages: `young_embryos`, `old_embryos`, and `larvae`. Mortality observations across post-exposure days are collapsed to a single `n_dead` count per trial and analysed as proportion-survival data. We fit both three independent life-stage models and one joint model in which `life_stage` affects every 4PL sub-parameter, allowing direct posterior contrasts between stages.

5.1 Data preparation

The raw spreadsheet has a wide format: one row per replicate trial, with mortality and hatching counts spread across morning/afternoon columns for each day of observation. We sum those into total `n_dead`, derive `n_surv`, filter excluded rows, and centre temperature on the grand mean so all three life-stage fits use the same `temp_c`.

```
# Bundled with the package (see ?zebrafish_lethal): the per-day morning/afternoon
# mortality columns have already been summed into n_total / n_surv / n_dead and
# excluded rows dropped (raw CSV in inst/extdata/). One row per assay trial.
zf <- zebrafish_lethal

zf_temp_mean <- mean(zf$assay_temp, na.rm = TRUE)
```

```
zf |>
  dplyr::group_by(life_stage) |>
  dplyr::summarise(
    `n trials`              = dplyr::n(),
    `Temperature range (°C)` = paste(range(assay_temp), collapse = "–"),
    `Duration range (hr)`    = paste(round(range(duration_h), 2), collapse = "–"),
    `Total individuals`      = sum(n_total),
    `n Dates`                = length(unique(Date_experiment)),
    .groups = "drop"
  ) |>
  tinytable::tt(escape = TRUE)
```

We build one standardised dataset per life stage, sharing the same `temp_mean` (the grand mean across the full panel) so that mean-centred temperature is on a common scale across stages. This makes the three separate-fit intercepts comparable and aligns them with the joint model below.

```
zf_stages <- levels(zf$life_stage)

zf_std_by_stage <- lapply(zf_stages, function(s) {
  standardize_data(
    data           = dplyr::filter(zf, life_stage == s),
    temp           = "assay_temp",
    duration       = "duration_h",
    n_total        = "n_total",
    n_surv         = "n_surv",
    random_effects = "Date_experiment",
    duration_unit  = "hours",
    temp_mean      = zf_temp_mean     # shared centring across stages
  )
})
names(zf_std_by_stage) <- zf_stages
```

5.2 Classical two-stage TDT fits by life stage

As for the shrimp case study, we first fit the classical two-stage TDT pipeline before moving to the 4PL approaches. Stage 1 is fit separately at each temperature within each life stage, using either a binomial or beta-binomial logistic curve. Stage 2 then regresses the Stage-1 $\log_{10}$ LT50 estimates on assay temperature within each life stage.

```
zf_stage1_twostep <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  dplyr::bind_rows(
    fit_stage1_binomial_case(zf_std_by_stage[[s]]),
    fit_stage1_betabinom_case(zf_std_by_stage[[s]])
  ) |>
    dplyr::mutate(life_stage = factor(s, levels = zf_stages),
                  .before = 1)
}))

zf_stage2_twostep <- lapply(zf_stages, function(s) {
  d_s <- dplyr::filter(zf_stage1_twostep, life_stage == s)
  out <- lapply(
    split(d_s, d_s$method),
    fit_stage2_case,
    time_multiplier = 60,
    t_ref = 60,
    TC_rate_range = c(0.1, 1)
  )
  names(out) <- paste(s, names(out), sep = "||")
```

```
  out
}) |>
  unlist(recursive = FALSE)

zf_twostep_tdt <- dplyr::bind_rows(lapply(names(zf_stage2_twostep), function(nm) {
  parts <- strsplit(nm, "\\|\\|")[[1]]
  zf_stage2_twostep[[nm]]$summary |>
    dplyr::mutate(life_stage = factor(parts[1], levels = zf_stages),
                  .before = 1)
}))

zf_temp_grid <- seq(min(zf$assay_temp) - 0.25,
                    max(zf$assay_temp) + 0.25, by = 0.05)

zf_twostep_unc <- lapply(seq_along(zf_stage2_twostep), function(i) {
  propagate_twostep_case(
    zf_stage2_twostep[[i]]$fit,
    temp_grid = zf_temp_grid,
    n_sim = 1000, time_multiplier = 60, t_ref = 60,
    TC_rate_range = c(0.1, 1), seed = 200L + i
  )
})
names(zf_twostep_unc) <- names(zf_stage2_twostep)
```

```
zf_twostep_tdt_ci <- dplyr::bind_rows(lapply(
  names(zf_twostep_unc),
  function(nm) {
    parts <- strsplit(nm, "\\|\\|")[[1]]
    pt <- dplyr::filter(zf_twostep_tdt,
                        life_stage == parts[1], method == parts[2])
    ci <- zf_twostep_unc[[nm]]$summary_ci
    tibble::tibble(
      `Life stage`         = parts[1],
      `Stage-1 likelihood` = parts[2],
      `z (C)`         = format_interval(pt$z,         ci$z_lower,     ci$z_upper,     digits =
      `CTmax_1hr (C)` = format_interval(pt$CTmax_1hr, ci$CTmax_lower, ci$CTmax_upper, digits =
      `T_crit (C)`    = format_interval(pt$T_crit,    ci$Tcrit_lower, ci$Tcrit_upper, digits =
      `Stage-2 R2`                 = round(pt$r_squared, 3),
      `Stage-1 estimates used`     = pt$n_stage1,
      `Stage-1 estimates excluded` = pt$n_excluded
    )
  }
))

tinytable::tt(zf_twostep_tdt_ci, escape = TRUE)
```

Table S21: Classical two-stage TDT quantities for zebrafish, by life stage. Point estimates are from the Stage-2 regression. The 95% CIs for z, $CT_{max_{1hr}}$ and T_{crit} all come from 1000 parametric simulations from $\text{MVN}(\hat{\theta}, \widehat{\text{Cov}}(\hat{\theta}))$ of the Stage-2 coefficients. Each Stage-1 $\log_{10}$ LT50 is treated as a known point, as is standard practice. T_{crit} uses the geometric midpoint of the same $r^* \in [0.1, 1]$ % HI per hour range used by the 4PL extraction.

Life stage	Stage-1 likelihood	z (C)	CTmax_1hr (C)	T_crit (C)	Stage-
young_embryos	Two-step beta-binomial	2.22 [1.78, 2.96]	39.61 [39.32, 39.88]	34.05 [32.22, 35.19]	0.821
young_embryos	Two-step binomial	2.22 [1.78, 2.95]	39.62 [39.34, 39.88]	34.07 [32.4, 35.19]	0.819
old_embryos	Two-step beta-binomial	2.36 [1.78, 3.52]	41.4 [40.99, 42.07]	35.5 [33.85, 36.48]	0.841
old_embryos	Two-step binomial	2.48 [1.8, 3.96]	41.57 [41.06, 42.44]	35.38 [33.08, 36.5]	0.803
larvae	Two-step beta-binomial	2.16 [1.79, 2.73]	39.82 [39.61, 40.04]	34.41 [33.14, 35.28]	0.885
larvae	Two-step binomial	2.27 [1.97, 2.66]	39.77 [39.62, 39.91]	34.1 [33.28, 34.77]	0.937

```
zf_twostep_line_min <- dplyr::bind_rows(lapply(names(zf_stage2_twostep), function(nm) {
  parts <- strsplit(nm, "\\|\\|")[[1]]
  make_twostep_curve_case(
    zf_stage2_twostep[[nm]],
    temp_grid = zf_temp_grid,
    time_multiplier = 60,
    output_time_unit = "min"
  ) |>
    dplyr::mutate(life_stage = factor(parts[1], levels = zf_stages))
}))

zf_twostep_band_min <- dplyr::bind_rows(lapply(names(zf_twostep_unc), function(nm) {
  parts <- strsplit(nm, "\\|\\|")[[1]]
  zf_twostep_unc[[nm]]$curve_ci |>
    dplyr::mutate(life_stage = factor(parts[1], levels = zf_stages),
                  method     = parts[2])
}))

ggplot2::ggplot() +
  ggplot2::geom_ribbon(
    data = zf_twostep_band_min,
    ggplot2::aes(x = temp,
                 ymin = log10(duration_lower),
                 ymax = log10(duration_upper),
                 fill = life_stage,
                 group = interaction(life_stage, method)),
    alpha = 0.15, colour = NA
  ) +
  ggplot2::geom_point(
    data = dplyr::filter(zf_stage1_twostep, stage1_ok),
```

```
  ggplot2::aes(x = temp, y = log10_lt50 + log10(60),
               colour = life_stage, shape = method),
  size = 2
) +
ggplot2::geom_line(
  data = zf_twostep_line_min,
  ggplot2::aes(x = temp, y = log10(duration_median),
               colour = life_stage, linetype = method,
               group = interaction(life_stage, method)),
  linewidth = 0.9
) +
ggplot2::labs(
  x = "Assay temperature (C)",
  y = expression(log[10]~LT[50]~"(min)"),
  colour = "Life stage",
  fill = "Life stage",
  shape = "Stage-1 likelihood",
  linetype = "Stage-1 likelihood"
) +
ggplot2::theme_classic()
```

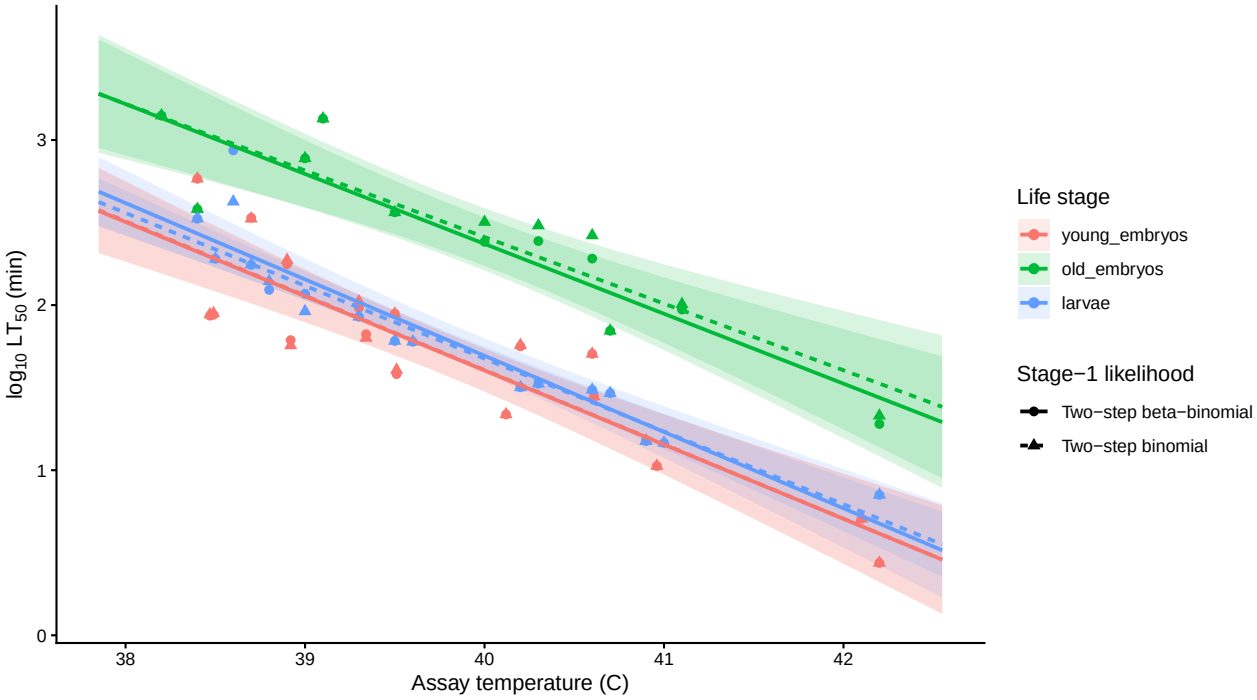


Figure S31: Classical two-stage TDT lines for zebrafish, by life stage. Points are Stage-1 $\log_{10}$ LT50 estimates; lines are the Stage-2 regressions; shaded bands are pointwise 95% confidence intervals from `predict.lm()` on the Stage-2 regression (Stage-1 LT50s treated as known points, as in standard practice). Solid and dashed lines distinguish binomial and beta-binomial Stage-1 likelihoods.

5.3 Approach 1: A separate 4PL model per life stage

The simplest option is to repeat the shrimp pipeline three times — one `fit_4pl()` per life stage with a random intercept on `mid` for `Date_experiment`, then one `extract_tdt()` per fit. The three fits are independent: each stage's posterior carries no information about the others.

```
models_dir <- here::here("output", "models")

zf_fits_sep <- lapply(zf_stages, function(s) {
  fit_4pl(
    zf_std_by_stage[[s]],
    random_effects = "Date_experiment",
    chains  = 4, iter = 8000, warmup = 4000, cores = 4, seed = 123,
    control = list(adapt_delta = 0.99, max_treedepth = 15),
    silent  = 2, refresh = 0,
    file    = file.path(models_dir,
                        paste0("fit_zf_separate_", s)),
    file_refit = "on_change"
  )
})
names(zf_fits_sep) <- zf_stages

# Workflow header for one stage as a discoverability cue; the other two have
# the same structure with different data shape.
print(zf_fits_sep[[zf_stages[1]]])
```

```
<bayes_tls>
  Data:     118 rows; 25 temperatures; 9 durations
  T_bar:    39.67
  Bounds:   response in (0.000, 1.000); low in (0.001, 0.499); up in (0.501, 0.999)
  RE:       Date_experiment
  Status:   fitted (16000 draws)
```

5.3.1 Sampling diagnostics for the separate fits

```
zf_sep_diag <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  diagnose_tdt_fit(zf_fits_sep[[s]]) |>
    dplyr::mutate(`Life stage` = s, .before = 1)
}))
tinytable::tt(zf_sep_diag, digits = 3, escape = TRUE)
```

5.3.2 Survival curves per life stage

Table S22: Sampling diagnostics for the three separate zebrafish 4PL fits, one row per life stage. Healthy targets: Rhat < 1.01, ESS > 400, zero divergences.

Life stage	rhat_max	ess_bulk_min	ess_tail_min	divergences	treedepth_hits	bfmi_min	rhat
young_embryos	1	2867	5351	0	6536	0.799	TRU
old_embryos	1	2752	4669	0	5	0.789	TRU
larvae	1	3206	3278	0	2	0.834	TRU

```
zf_sep_survplots <- lapply(zf_stages, function(s) {
  psc <- predict_survival_curves(zf_fits_sep[[s]],
                                 temps = sort(unique(zf_std_by_stage[[s]]$temp)),
                                 ndraws = 500)
  plot_survival_curves(psc, observed = zf_std_by_stage[[s]]) +
    ggplot2::coord_cartesian(xlim = c(0, 10)) +
    ggplot2::labs(title = s,
                  x = "Exposure duration (hours)") +
    ggplot2::theme(legend.position = "none")
})

patchwork::wrap_plots(zf_sep_survplots, ncol = 3)
```

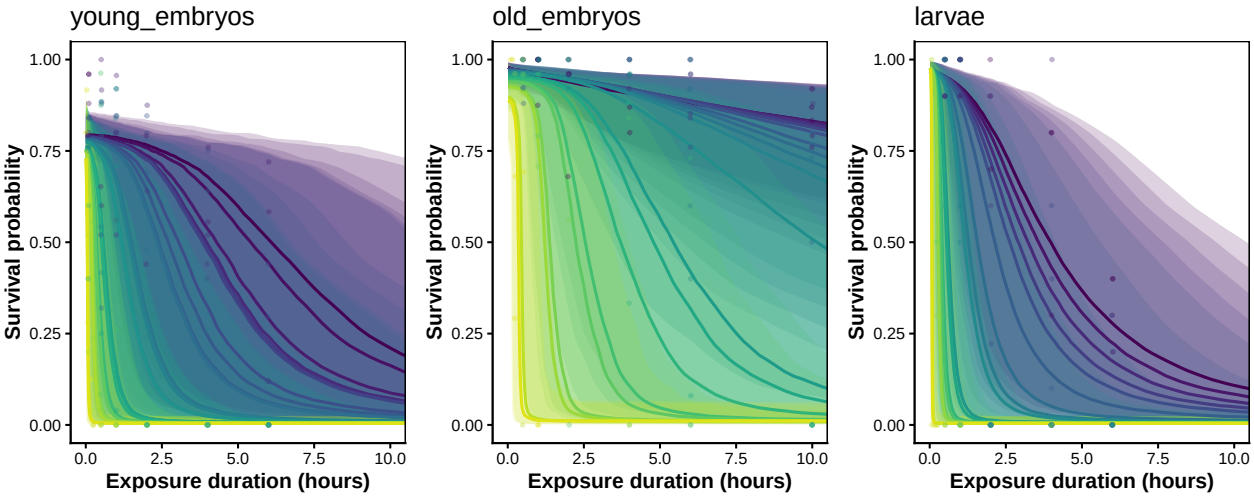


Figure S32: Posterior survival curves for each zebrafish life stage from the three independent 4PL fits, with 95% credible bands. One panel per stage; coloured points are observed per-replicate survival proportions.

5.3.3 TDT landscapes per life stage

```r
zf_temp_range <- range(zf$assay_temp, na.rm = TRUE)
zf_dur_range  <- range(zf$duration_h, na.rm = TRUE)
zf_shared_temp_grid <- seq(zf_temp_range[1], zf_temp_range[2],
                           length.out = 120)
zf_shared_dur_grid  <- seq(zf_dur_range[1],  zf_dur_range[2],
                           length.out = 120)

zf_sep_landplots <- lapply(zf_stages, function(s) {
  lsp <- derive_tdt_landscape(zf_fits_sep[[s]],
                              temp_grid     = zf_shared_temp_grid,
                              duration_grid = zf_shared_dur_grid,
                              ndraws        = 500)
  plot_tdt_landscape(lsp, observed = zf_std_by_stage[[s]]) +
    ggplot2::labs(title = s, y = "Exposure duration (hours)")
})

patchwork::wrap_plots(zf_sep_landplots, ncol = 3) +
  patchwork::plot_layout(guides = "collect") &
  ggplot2::theme(legend.position = "bottom")
```

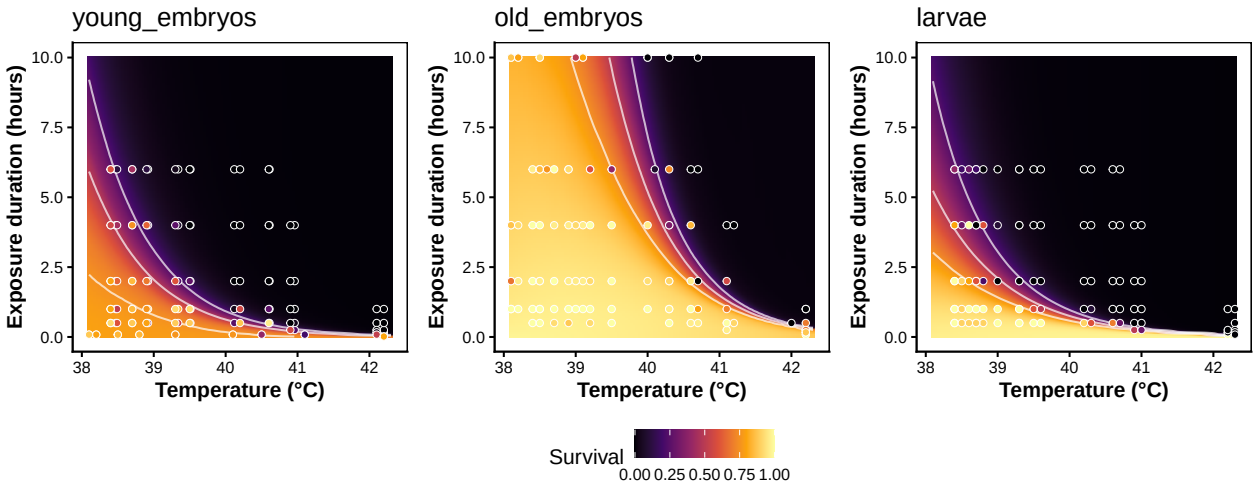


Figure S33: TDT landscape for each zebrafish life stage from the three independent 4PL fits. Heatmap shows posterior median predicted survival across temperature × duration; white contours mark 25, 50, and 75 % survival; points are observed proportions. All three panels share the same axis ranges so the surfaces can be compared directly.

5.3.4 TDT lines from the separate fits

`derive_tdt_curve()` returns the predicted time to the survival threshold at each temperature; computed for each life stage on a common temperature grid, the three lines can be overlaid on the same axes for direct comparison. We use the package-default threshold (`mid`), which coincides with the conventional absolute LT50 when the asymptotes are anchored near 0 and 1 and remains

meaningful when intrinsic background mortality pulls the upper asymptote below 1 — as it does for one of the zebrafish life stages (see Section 5.5 for the absolute-0.5 comparison).

```
zf_temp_grid <- seq(min(zf$assay_temp) - 0.25,
                    max(zf$assay_temp) + 0.25, by = 0.05)

zf_sep_ltx <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  ltx <- derive_tdt_curve(
    zf_fits_sep[[s]],
    temp_grid       = zf_temp_grid,
    ndraws          = 500,
    time_multiplier = 60,
    output_time_unit = "min"
  )
  ltx$summary |>
    dplyr::transmute(
      life_stage   = factor(s, levels = zf_stages),
      assay_temp   = temp,
      time_med_min = duration_median,
      time_lo_min  = duration_lower,
      time_hi_min  = duration_upper
    )
}))
```

```
# Colour-blind-safe triple, distinct in luminance from each other and from
# the panel background; replaces the unreadable viridis yellow for larvae.
zf_palette <- c(young_embryos = "#440154",   # dark purple
                old_embryos   = "#21918c",   # teal
                larvae        = "#f98e09")   # orange (inferno ~0.75)

# Y-axis cap = 48 hours, expressed in the same unit as time_med_min (minutes).
y_cap_min <- 48 * 60

p_lin_sep <- ggplot2::ggplot(zf_sep_ltx,
                             ggplot2::aes(x = assay_temp, y = time_med_min,
                                          colour = life_stage,
                                          fill   = life_stage)) +
  ggplot2::geom_ribbon(ggplot2::aes(ymin = time_lo_min, ymax = time_hi_min),
                       alpha = 0.18, colour = NA) +
  ggplot2::geom_line(linewidth = 1) +
  ggplot2::scale_colour_manual(values = zf_palette) +
  ggplot2::scale_fill_manual(values   = zf_palette) +
  ggplot2::coord_cartesian(ylim = c(0, y_cap_min)) +
  ggplot2::labs(x = "Assay temperature (°C)",
                y = "Time to 50% survival (min)",
                colour = "Life stage", fill = "Life stage") +
  ggplot2::theme_classic() +
  ggplot2::theme(panel.border = ggplot2::element_rect(colour = "black",
```

```
                                                    fill = NA,
                                                    linewidth = 0.6))

p_log_sep <- ggplot2::ggplot(zf_sep_ltx,
                             ggplot2::aes(x = assay_temp, y = time_med_min,
                                          colour = life_stage,
                                          fill   = life_stage)) +
  ggplot2::geom_ribbon(ggplot2::aes(ymin = time_lo_min, ymax = time_hi_min),
                       alpha = 0.18, colour = NA) +
  ggplot2::geom_line(linewidth = 1) +
  ggplot2::scale_colour_manual(values = zf_palette) +
  ggplot2::scale_fill_manual(values   = zf_palette) +
  ggplot2::scale_y_log10() +
  ggplot2::coord_cartesian(ylim = c(NA, y_cap_min)) +
  ggplot2::labs(x = "Assay temperature (°C)",
                y = expression(log[10] *
                               " time to 50% survival (min)")) +
  ggplot2::theme_classic() +
  ggplot2::theme(legend.position = "none",
                 panel.border = ggplot2::element_rect(colour = "black",
                                                      fill = NA,
                                                      linewidth = 0.6))

patchwork::wrap_plots(p_lin_sep, p_log_sep, ncol = 2) +
  patchwork::plot_layout(guides = "collect") &
  ggplot2::theme(legend.position = "bottom")
```

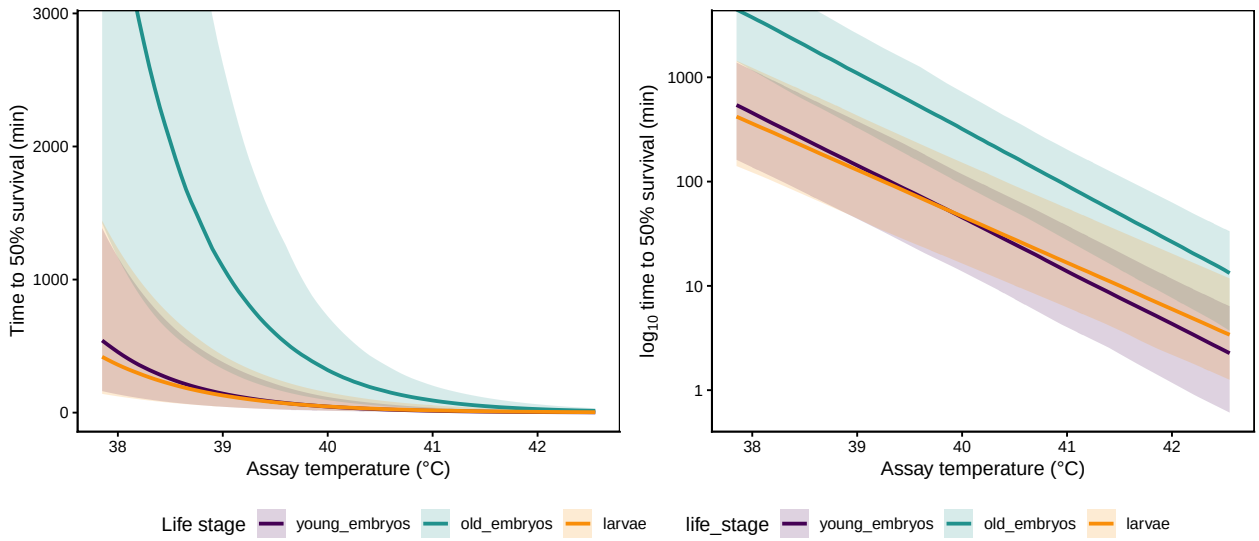


Figure S34: TDT lines for the three zebrafish life stages from the separate 4PL fits, using the package-default threshold. Linear-time panel on the left, $\log_{10}$-time panel on the right (slope is $-1/z$). Lines are posterior medians; shaded bands are 95 % credible intervals.

Table S23: Posterior summaries of z, $CT_{max_{1hr}}$ and T_{crit} from the three separate zebrafish 4PL fits, under the package-default relative threshold (`(low + up)/2` per draw). Absolute-threshold values are tabulated in Section 5.5. Medians with 95 % credible intervals. T_{crit} is integrated over $r^* \in [0.1, 1]$ % HI per hour, as in Case Study 1.

Life stage	z (°C)	CTmax_1hr (°C)	T_crit (°C)
young_embryos	1.97 [1.77, 2.17]	39.78 [38.49, 40.74]	34.77 [33.01, 36.24]
old_embryos	1.87 [1.6, 2.13]	41.38 [40.37, 42.13]	36.71 [35.16, 37.9]
larvae	2.25 [1.97, 2.44]	39.75 [38.62, 40.67]	34.26 [32.49, 35.77]

5.3.5 TDT-quantity posteriors per life stage

`extract_tdt()` summarises z and $CT_{max_{1hr}}$ from each separate fit. We set `lethal = TRUE` because the zebrafish endpoint is mortality, which adds T_{crit} to the output (the absolute-threshold sensitivity check in Section 5.5 reruns the same call with `target_surv = "absolute"`).

```
zf_et_sep <- lapply(zf_stages, function(s) {
  extract_tdt(zf_fits_sep[[s]],
              t_ref         = 60,
              TC_rate_range = c(0.1, 1),
              ndraws        = 500,
              lethal        = TRUE)
})
names(zf_et_sep) <- zf_stages
```

```
zf_sep_tdt_table <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  et <- zf_et_sep[[s]]
  tibble::tibble(
    `Life stage`        = s,
    `z (°C)`            = format_interval(et$z$summary$z_median,
                                          et$z$summary$z_lower,
                                          et$z$summary$z_upper, digits = 2),
    `CTmax_1hr (°C)`    = format_interval(et$CTmax$summary$temp_median,
                                          et$CTmax$summary$temp_lower,
                                          et$CTmax$summary$temp_upper, digits = 2),
    `T_crit (°C)`       = format_interval(et$T_crit$summary$temp_median,
                                          et$T_crit$summary$temp_lower,
                                          et$T_crit$summary$temp_upper, digits = 2)
  )
}))
tinytable::tt(zf_sep_tdt_table, escape = TRUE)
```

5.3.6 Pairwise life-stage contrasts (separate fits)

Because the three fits are independent, pairwise contrasts $\Delta_{ij} = q^{(i)} - q^{(j)}$ can be computed at the draw level: subtracting matched draws gives a per-draw difference distribution that carries the joint uncertainty in both fits.

```
make_contrast <- function(a, b, var) {
  # Pair posterior draws by .draw so the contrast is computed within
  # draw - this is the standard accessor-based contrast pattern shown
  # in the get_z_draws() docstring example.
  getter <- switch(var, z = get_z_draws, CTmax = get_ctmax_draws)
  value_col <- switch(var, z = "z", CTmax = "CTmax")
  paired <- dplyr::inner_join(getter(zf_et_sep[[a]]),
                              getter(zf_et_sep[[b]]),
                              by = ".draw", suffix = c("_a", "_b"))
  delta  <- paired[[paste0(value_col, "_a")]] -
            paired[[paste0(value_col, "_b")]]
  q <- stats::quantile(delta, c(0.025, 0.5, 0.975), names = FALSE)
  tibble::tibble(
    Contrast = paste(a, "-", b),
    Quantity = switch(var, z = "z (°C)", CTmax = "CTmax_1hr (°C)"),
    Median   = round(q[2], 3),
    Lower    = round(q[1], 3),
    Upper    = round(q[3], 3)
  )
}

pairs <- list(c("larvae", "old_embryos"),
              c("larvae", "young_embryos"),
              c("old_embryos", "young_embryos"))

zf_sep_contrasts <- dplyr::bind_rows(lapply(pairs, function(p) {
  dplyr::bind_rows(
    make_contrast(p[1], p[2], "z"),
    make_contrast(p[1], p[2], "CTmax")
  )
}))

tinytable::tt(zf_sep_contrasts, escape = TRUE)
```

5.4 Approach 2: A joint 4PL with `life_stage` as a covariate

A **joint model** fits all three life stages together and lets every 4PL sub-parameter depend on `life_stage`, with the dispersion ϕ and the date random-intercept variance σ_{date} pooled across stages. The advantages over separate fits are:

Table S24: Pairwise differences in z and $CT_{max_{1hr}}$ across zebrafish life stages from the three separate 4PL fits. Positive values mean the first-listed stage has the larger value.

Contrast	Quantity	Median	Lower	Upper
larvae − old_embryos	z (°C)	0.369	-0.029	0.715
larvae − old_embryos	CTmax_1hr (°C)	-1.638	-2.884	-0.165
larvae − young_embryos	z (°C)	0.276	-0.086	0.560
larvae − young_embryos	CTmax_1hr (°C)	-0.013	-1.443	1.486
old_embryos − young_embryos	z (°C)	-0.108	-0.456	0.235
old_embryos − young_embryos	CTmax_1hr (°C)	1.598	0.294	2.918

- σ_{date} is estimated from all eight experimental dates rather than from 2–3 dates per stage, giving more information about the random-effect scale.
- The priors on the per-stage asymptote and slope coefficients are shared, so each stage's posterior is gently regularised by the others where data are sparse.
- Pairwise life-stage contrasts are linear combinations of draws from a single posterior, with no cross-fit alignment to worry about.

We write the formula by hand to keep the same disjoint-bounds reparameterisation used by `make_4pl_formula()`. The life-stage covariate enters via **cell-means coding** (`~ 0 + life_stage + temp_c:life_stage`) rather than treatment-contrast coding. Cell-means estimates each stage's intercept and slope directly, with the same prior width on every stage's coefficient. Treatment contrasts (`temp_c * life_stage`) would place asymmetric priors on the non-reference stages — the effective slope for a non-reference stage is the sum of two `normal(0, 0.5)` coefficients, with implicit SD $\sqrt{2} \cdot 0.5 \approx 0.71$, $\sqrt{2}\times$ wider than the reference stage. Zebrafish life stages are biologically distinct enough that we do not want one of them designated as a "baseline".

All four sub-parameters (`lowraw`, `upraw`, `logk`, `mid`) receive this cell-means structure. No observation-level random effect is included: the beta-binomial family already absorbs the extra-binomial variance via ϕ.

```
zf_bounds <- getFromNamespace("compute_4pl_bounds", "bayesTLS")(lower = 0, upper = 1)
low_expr  <- sprintf("(%.6f + inv_logit(lowraw) * %.6f)",
                     zf_bounds$low_min, zf_bounds$low_w)
up_expr   <- sprintf("(%.6f + inv_logit(upraw)  * %.6f)",
                     zf_bounds$up_min,  zf_bounds$up_w)
main_rhs  <- sprintf("%s + (%s - %s) / (1 + exp(exp(logk) * (logd - mid)))",
                     low_expr, up_expr, low_expr)

f_zf_joint <- brms::bf(
  stats::as.formula(sprintf("n_surv | trials(n_total) ~ %s", main_rhs)),
  stats::as.formula("lowraw ~ 0 + life_stage + temp_c:life_stage"),
  stats::as.formula("upraw  ~ 0 + life_stage + temp_c:life_stage"),
  stats::as.formula("logk   ~ 0 + life_stage + temp_c:life_stage"),
  stats::as.formula("mid    ~ 0 + life_stage + temp_c:life_stage + (1 | Date_experiment)"),
  nl     = TRUE,
```

```
  family = brms::beta_binomial(link = "identity")
)
```

We assemble the standardised per-stage datasets back into a single tibble for the joint fit, keeping the shared mean-centring.

```
zf_joint_data <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  d <- zf_std_by_stage[[s]]
  d$life_stage <- factor(s, levels = zf_stages)
  d
}))
```

Priors mirror those used by `make_4pl_priors()`. Cell-means coding suppresses the global intercept, so each per-stage intercept (`life_stage<stage>`) receives its own prior centred on a plausible value for the corresponding sub-parameter; per-stage slopes pick up the general `class = "b"` priors.

```
low_target  <- 0 + 0.02 * 1
up_target   <- 0 + 0.98 * 1
lowraw_loc  <- stats::qlogis((low_target - zf_bounds$low_min) / zf_bounds$low_w)
upraw_loc   <- stats::qlogis((up_target  - zf_bounds$up_min)  / zf_bounds$up_w)
logk_loc    <- log(2)
mid_loc     <- stats::median(zf_joint_data$logd, na.rm = TRUE)

per_stage_intercept_priors <- function(centre, sigma, nlpar) {
  do.call(c, lapply(zf_stages, function(s) {
    brms::set_prior(
      sprintf("normal(%.6f, %.6f)", centre, sigma),
      class = "b", nlpar = nlpar,
      coef  = paste0("life_stage", s)
    )
  }))
}

zf_joint_priors <- c(
  per_stage_intercept_priors(lowraw_loc, 1.0, "lowraw"),
  per_stage_intercept_priors(upraw_loc,  1.0, "upraw"),
  per_stage_intercept_priors(logk_loc,   1.0, "logk"),
  per_stage_intercept_priors(mid_loc,    1.5, "mid"),
  brms::set_prior("normal(0, 0.5)", class = "b", nlpar = "lowraw"),
  brms::set_prior("normal(0, 0.5)", class = "b", nlpar = "upraw"),
  brms::set_prior("normal(0, 0.3)", class = "b", nlpar = "logk"),
  brms::set_prior("normal(0, 0.6)", class = "b", nlpar = "mid"),
  brms::set_prior("exponential(2)", class = "sd", nlpar = "mid",
                  group = "Date_experiment"),
  brms::set_prior("gamma(2, 0.1)",  class = "phi")
)
```

Table S25: Sampling diagnostics for the joint zebrafish 4PL fit (one model, all three life stages). Healthy targets: Rhat < 1.01, ESS > 400, zero divergences.

rhat_max	ess_bulk_min	ess_tail_min	divergences	treedepth_hits	rhat_pass	ess_pass	divergence_
1	2729	4354	0	10	TRUE	TRUE	TRUE

```
fit_zf_joint <- brms::brm(
  formula    = f_zf_joint,
  data       = zf_joint_data,
  prior      = zf_joint_priors,
  chains     = 4, iter = 8000, warmup = 4000, cores = 4, seed = 123,
  control    = list(adapt_delta = 0.995, max_treedepth = 15),
  init       = 0,
  backend    = "cmdstanr",
  silent     = 2, refresh = 0,
  file       = file.path(models_dir, "fit_zf_joint_4pl"),
  file_refit = "on_change"
)
```

5.4.1 Sampling diagnostics for the joint fit

```
np_zj      <- brms::nuts_params(fit_zf_joint)
divs_zj    <- sum(subset(np_zj, Parameter == "divergent__")$Value)
td_max_zj  <- max(np_zj$Value[np_zj$Parameter == "treedepth__"], na.rm = TRUE)
td_hits_zj <- sum(subset(np_zj, Parameter == "treedepth__")$Value >= td_max_zj)
rh_zj      <- brms::rhat(fit_zf_joint)
ess_zj     <- posterior::summarise_draws(posterior::as_draws(fit_zf_joint),
                                         "ess_bulk", "ess_tail")
tibble::tibble(
  rhat_max        = round(max(rh_zj,        na.rm = TRUE), 4),
  ess_bulk_min    = round(min(ess_zj$ess_bulk, na.rm = TRUE)),
  ess_tail_min    = round(min(ess_zj$ess_tail, na.rm = TRUE)),
  divergences     = divs_zj,
  treedepth_hits  = td_hits_zj,
  rhat_pass       = max(rh_zj, na.rm = TRUE) < 1.01,
  ess_pass        = min(ess_zj$ess_bulk, ess_zj$ess_tail, na.rm = TRUE) > 400,
  divergence_pass = divs_zj == 0
) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

5.4.2 TDT lines: independent vs joint fit

To compare the two approaches we build a per-stage TDT curve from the joint fit's posterior. The joint model is a hand-coded `brms::bf()` rather than a `bayes_tls` workflow, so we apply the same shortcut `derive_tdt_curve()` uses internally for the default (mid-based) threshold: extract the per-stage `mid` posterior and exponentiate. We also store the asymptote and slope posteriors here because the absolute-threshold sensitivity check in Section 5.5 consumes them.

```
ndraws_joint <- 500

zf_joint_pred_grid <- tidyr::expand_grid(
  assay_temp = zf_temp_grid,
  life_stage = factor(zf_stages, levels = zf_stages)
) |>
  dplyr::mutate(
    temp_c  = assay_temp - zf_temp_mean,
    logd    = 0,
    n_total = 1L
  )

pp_low <- brms::posterior_linpred(fit_zf_joint, newdata = zf_joint_pred_grid,
                                  nlpar = "lowraw", re_formula = NA,
                                  ndraws = ndraws_joint)
pp_up  <- brms::posterior_linpred(fit_zf_joint, newdata = zf_joint_pred_grid,
                                  nlpar = "upraw",  re_formula = NA,
                                  ndraws = ndraws_joint)
pp_lk  <- brms::posterior_linpred(fit_zf_joint, newdata = zf_joint_pred_grid,
                                  nlpar = "logk",   re_formula = NA,
                                  ndraws = ndraws_joint)
pp_mid <- brms::posterior_linpred(fit_zf_joint, newdata = zf_joint_pred_grid,
                                  nlpar = "mid",    re_formula = NA,
                                  ndraws = ndraws_joint)

low_mat <- zf_bounds$low_min + plogis(pp_low) * zf_bounds$low_w
up_mat  <- zf_bounds$up_min  + plogis(pp_up)  * zf_bounds$up_w
k_mat   <- exp(pp_lk)
mid_mat <- pp_mid

# At the mid-based threshold the exponential term vanishes, so
# log10(t_mid) = mid(T) directly. Convert hours -> minutes for plotting.
t_min <- 60 * 10 ^ mid_mat

zf_joint_summary <- zf_joint_pred_grid |>
  dplyr::mutate(
    time_med_min = apply(t_min, 2, stats::median, na.rm = TRUE),
    time_lo_min  = apply(t_min, 2, stats::quantile, 0.025,
                         na.rm = TRUE, names = FALSE),
    time_hi_min  = apply(t_min, 2, stats::quantile, 0.975,
```

```
                          na.rm = TRUE, names = FALSE)
  ) |>
  dplyr::filter(is.finite(time_med_min)) |>
  dplyr::select(assay_temp, life_stage,
                time_med_min, time_lo_min, time_hi_min)
```

```
zf_overlay_df <- dplyr::bind_rows(
  dplyr::mutate(zf_sep_ltx,        method = "Separate fits"),
  dplyr::mutate(zf_joint_summary,  method = "Joint fit")
) |>
  dplyr::mutate(method = factor(method,
                                levels = c("Separate fits", "Joint fit")))

overlay_alphas    <- c("Separate fits" = 0.12, "Joint fit" = 0.32)
overlay_linetypes <- c("Separate fits" = "dashed", "Joint fit" = "solid")

build_overlay <- function(y_log = FALSE) {
  p <- ggplot2::ggplot(zf_overlay_df,
                       ggplot2::aes(x = assay_temp, y = time_med_min,
                                    colour = life_stage,
                                    fill   = life_stage,
                                    linetype = method,
                                    group = interaction(life_stage, method))) +
    ggplot2::geom_ribbon(ggplot2::aes(ymin  = time_lo_min,
                                      ymax  = time_hi_min,
                                      alpha = method),
                         colour = NA) +
    ggplot2::geom_line(linewidth = 1) +
    ggplot2::scale_colour_manual(values = zf_palette) +
    ggplot2::scale_fill_manual(values   = zf_palette) +
    ggplot2::scale_alpha_manual(values  = overlay_alphas,
                                name    = "Method") +
    ggplot2::scale_linetype_manual(values = overlay_linetypes,
                                   name   = "Method") +
    ggplot2::labs(x = "Assay temperature (°C)",
                  colour = "Life stage", fill = "Life stage") +
    ggplot2::theme_classic() +
    ggplot2::theme(panel.border = ggplot2::element_rect(colour = "black",
                                                        fill = NA,
                                                        linewidth = 0.6))
  if (y_log) {
    p + ggplot2::scale_y_log10() +
      ggplot2::coord_cartesian(ylim = c(NA, y_cap_min)) +
      ggplot2::labs(y = expression(log[10] *
                                   " time to 50% survival (min)"))
  } else {
    p + ggplot2::coord_cartesian(ylim = c(0, y_cap_min)) +
```

```
      ggplot2::labs(y = "Time to 50% survival (min)")
  }
}

patchwork::wrap_plots(build_overlay(FALSE), build_overlay(TRUE), ncol = 2) +
  patchwork::plot_layout(guides = "collect") &
  ggplot2::theme(legend.position = "bottom")
```

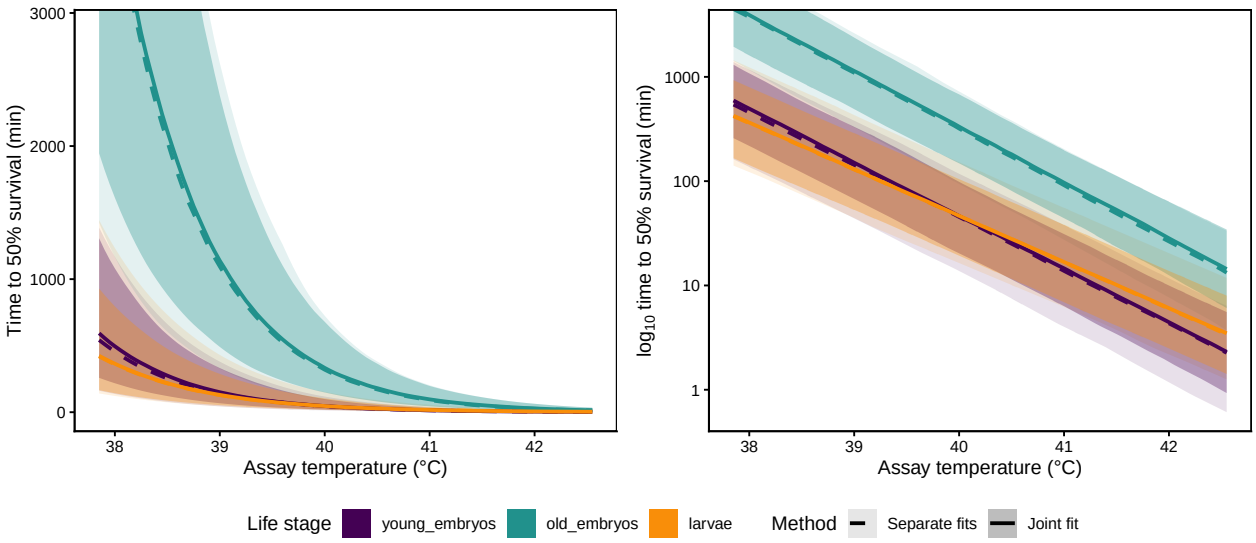


Figure S35: TDT lines for zebrafish comparing the three independent per-life-stage fits (dashed, lighter ribbons) with the joint 4PL fit (solid, darker ribbons); 95 % credible bands shown for both. Linear-time panel on the left, $\log_{10}$-time panel on the right. Same hue per life stage across methods. The joint-fit bands are typically narrower thanks to partial pooling of the priors and the shared ϕ. See Section 5.5 for the same overlay under the absolute 0.5 threshold.

5.4.3 TDT quantities from the joint fit

We compute z and $CT_{max_{1hr}}$ per life stage directly from each stage's relative-LT line built from the *joint* fit's posterior, with all life-stage covariation correctly propagated. Under the relative threshold (`(low + up)/2`), $\log_{10}\mathrm{LT}(T) = \mathrm{mid}(T)$ is linear in T, so $z = -1/\text{slope}$ and $CT_{max_{1hr}}$ (where the line crosses $\log_{10} t_{\mathrm{ref}}$) are read straight from the per-draw line — no regression, mirroring `derive_z()` / `derive_temperature_for_duration()` applied to the joint fit. Section Section 5.5 reproduces the same quantities under the *absolute* 0.5 threshold for comparison.

```
TC_rate_range_zf <- c(0.1, 1)
log10_low_zf     <- log10(TC_rate_range_zf[1] / 100)
log10_high_zf    <- log10(TC_rate_range_zf[2] / 100)

zf_joint_tdt <- lapply(zf_stages, function(s) {
  idx <- zf_joint_pred_grid$life_stage == s
```

```r
draws <- as.data.frame(t(t_min[, idx, drop = FALSE]))
colnames(draws) <- paste0("V", seq_len(ncol(draws)))
draws$temp        <- zf_joint_pred_grid$assay_temp[idx]
draws$target_surv <- "(low+up)/2"
long <- tidyr::pivot_longer(draws,
                            cols = -c(temp, target_surv),
                            names_to = ".draw_id",
                            values_to = "duration_out") |>
  dplyr::mutate(.draw = as.integer(sub("V", "", .draw_id)),
                duration_model = duration_out / 60) |>
  dplyr::filter(is.finite(duration_out), duration_out > 0) |>
  dplyr::select(.draw, temp, target_surv, duration_model, duration_out)

# z and CTmax per draw read directly from this stage's relative-LT line.
# Under the relative threshold log10 LT(T) = mid(T) is linear in T, so the
# slope (hence z = -1/slope) and the temperature at which the line crosses
# log10(t_ref) (CTmax) are read straight from the per-draw line - no
# regression. This mirrors derive_z() / derive_temperature_for_duration()
# applied to the joint fit, per life stage.
zct_draws <- long |>
  dplyr::mutate(log10_dur = log10(duration_out)) |>
  dplyr::group_by(.draw) |>
  dplyr::summarise(
    slope     = (log10_dur[which.max(temp)] - log10_dur[which.min(temp)]) /
                (max(temp) - min(temp)),
    base_logd = log10_dur[which.min(temp)],
    base_temp = min(temp),
    .groups   = "drop") |>
  dplyr::transmute(.draw,
                   z     = -1 / slope,
                   CTmax = base_temp + (log10(60) - base_logd) / slope) |>
  dplyr::filter(is.finite(z), is.finite(CTmax))
zct <- list(
  draws   = zct_draws,
  summary = tibble::tibble(
    z_median     = stats::median(zct_draws$z),
    z_lower      = stats::quantile(zct_draws$z, 0.025, names = FALSE),
    z_upper      = stats::quantile(zct_draws$z, 0.975, names = FALSE),
    CTmax_median = stats::median(zct_draws$CTmax),
    CTmax_lower  = stats::quantile(zct_draws$CTmax, 0.025, names = FALSE),
    CTmax_upper  = stats::quantile(zct_draws$CTmax, 0.975, names = FALSE)))

per_draw <- zct$draws |>
  dplyr::select(.draw, z, CTmax) |>
  dplyr::filter(is.finite(z), is.finite(CTmax)) |>
  dplyr::mutate(log10_rate = stats::runif(dplyr::n(),
                                          min = log10_low_zf,
```

```
                                              max = log10_high_zf),
                  T_crit     = CTmax + z * log10_rate)
  q_tc <- stats::quantile(per_draw$T_crit,
                          c(0.025, 0.5, 0.975), names = FALSE, na.rm = TRUE)
  T_crit_block <- list(
    draws   = tibble::tibble(.draw = per_draw$.draw,
                             temp  = per_draw$T_crit,
                             log10_rate = per_draw$log10_rate),
    summary = tibble::tibble(TC_rate_low  = TC_rate_range_zf[1],
                             TC_rate_high = TC_rate_range_zf[2],
                             temp_lower   = q_tc[1],
                             temp_median  = q_tc[2],
                             temp_upper   = q_tc[3])
  )

  list(z = zct, summary = zct$summary, T_crit = T_crit_block)
})
names(zf_joint_tdt) <- zf_stages
```

```
zf_joint_table <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  z_summary  <- zf_joint_tdt[[s]]$z$summary
  tc_summary <- zf_joint_tdt[[s]]$T_crit$summary
  tibble::tibble(
    `Life stage`     = s,
    `z (°C)`         = format_interval(z_summary$z_median,
                                       z_summary$z_lower,
                                       z_summary$z_upper, digits = 2),
    `CTmax_1hr (°C)` = format_interval(z_summary$CTmax_median,
                                       z_summary$CTmax_lower,
                                       z_summary$CTmax_upper, digits = 2),
    `T_crit (°C)`    = format_interval(tc_summary$temp_median,
                                       tc_summary$temp_lower,
                                       tc_summary$temp_upper, digits = 2)
  )
}))
tinytable::tt(zf_joint_table, escape = TRUE)
```

5.4.4 Comparison of the two approaches

```
zf_compare <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  et   <- zf_et_sep[[s]]
  jt   <- zf_joint_tdt[[s]]$z$summary
  tibble::tibble(
    `Life stage`            = s,
```

Table S26: Posterior summaries of z, $CT_{max_{1hr}}$, and T_{crit} from the joint 4PL fit, per life stage. Medians with 95 % credible intervals. T_{crit} uses the same rate-multiplier integration ($r^* \in [0.1, 1]$ % HI per hour) as the shrimp case study. See Section 5.5 for the same quantities under the absolute 0.5 threshold.

Life stage	z (°C)	CTmax_1hr (°C)	T_crit (°C)
young_embryos	1.96 [1.72, 2.2]	39.78 [39.1, 40.42]	34.88 [33.51, 36.11]
old_embryos	1.87 [1.61, 2.16]	41.4 [40.74, 42.05]	36.61 [35.38, 37.91]
larvae	2.25 [2.1, 2.42]	39.76 [38.88, 40.55]	34.15 [32.75, 35.45]

Table S27: Side-by-side comparison of z and $CT_{max_{1hr}}$ from the three separate fits (Approach 1) and the joint 4PL (Approach 2). The joint fit's credible intervals are typically narrower thanks to partial pooling, while medians are similar — the two approaches agree on the direction of life-stage effects.

Life stage	Separate z (°C)	Joint z (°C)	Separate CTmax (°C)	Joint CTmax (°C)
young_embryos	1.97 [1.77, 2.17]	1.96 [1.72, 2.2]	39.78 [38.49, 40.74]	39.78 [39.1, 40.42]
old_embryos	1.87 [1.6, 2.13]	1.87 [1.61, 2.16]	41.38 [40.37, 42.13]	41.4 [40.74, 42.05]
larvae	2.25 [1.97, 2.44]	2.25 [2.1, 2.42]	39.75 [38.62, 40.67]	39.76 [38.88, 40.55]

```
    `Separate z (°C)`       = format_interval(et$z$summary$z_median,
                                              et$z$summary$z_lower,
                                              et$z$summary$z_upper, digits = 2),
    `Joint z (°C)`          = format_interval(jt$z_median,
                                              jt$z_lower,
                                              jt$z_upper, digits = 2),
    `Separate CTmax (°C)`   = format_interval(et$CTmax$summary$temp_median,
                                              et$CTmax$summary$temp_lower,
                                              et$CTmax$summary$temp_upper,
                                              digits = 2),
    `Joint CTmax (°C)`      = format_interval(jt$CTmax_median,
                                              jt$CTmax_lower,
                                              jt$CTmax_upper, digits = 2)
  )
}))
tinytable::tt(zf_compare, escape = TRUE)
```

Figure S36 visualises the same comparison at the posterior level: one row per life stage, one column per TDT quantity, with separate-fit and joint-fit posterior densities overlaid. The joint posteriors are typically narrower than the separate ones, while medians agree closely — partial pooling makes the joint fit more precise without changing the location of each quantity.

```
zf_post_long <- dplyr::bind_rows(
  lapply(zf_stages, function(s) {
```

```
    # zf_et_sep is a standard extract_tdt() result → accessors apply.
    sep_long <- dplyr::bind_rows(
      get_z_draws(zf_et_sep[[s]])     |>
        dplyr::transmute(quantity = "z",         value = z),
      get_ctmax_draws(zf_et_sep[[s]]) |>
        dplyr::transmute(quantity = "CTmax_1hr", value = CTmax),
      get_tcrit_draws(zf_et_sep[[s]]) |>
        dplyr::transmute(quantity = "T_crit",    value = T_crit)
    ) |>
      dplyr::mutate(life_stage = s, method = "Separate")
    # zf_joint_tdt[[s]] is a custom object: $z$draws carries BOTH z and CTmax
    # columns (computed directly from the per-stage relative-LT line above), so
    # we keep its bespoke access pattern here.
    joint_long <- dplyr::bind_rows(
      tibble::tibble(life_stage = s, quantity = "z",         method = "Joint",
                     value = zf_joint_tdt[[s]]$z$draws$z),
      tibble::tibble(life_stage = s, quantity = "CTmax_1hr", method = "Joint",
                     value = zf_joint_tdt[[s]]$z$draws$CTmax),
      tibble::tibble(life_stage = s, quantity = "T_crit",    method = "Joint",
                     value = zf_joint_tdt[[s]]$T_crit$draws$temp)
    )
    dplyr::bind_rows(sep_long, joint_long)
  })
) |>
  dplyr::filter(is.finite(value)) |>
  dplyr::mutate(
    life_stage = factor(life_stage, levels = zf_stages),
    quantity   = factor(quantity,   levels = c("z", "CTmax_1hr", "T_crit")),
    method     = factor(method,     levels = c("Separate", "Joint"))
  )

zf_post_summary <- zf_post_long |>
  dplyr::group_by(life_stage, quantity, method) |>
  dplyr::summarise(
    med = stats::median(value, na.rm = TRUE),
    lo  = stats::quantile(value, 0.025, names = FALSE, na.rm = TRUE),
    hi  = stats::quantile(value, 0.975, names = FALSE, na.rm = TRUE),
    .groups = "drop"
  )

method_colours <- c("Separate" = "#2c7fb8", "Joint" = "#d95f02")

quantity_labels <- ggplot2::as_labeller(
  c(z         = "italic(z)~\"(°C per 10-fold change in time)\"",
    CTmax_1hr = "CT[max[1*hr]]~(degree*C)",
    T_crit    = "T[crit]~(degree*C)"),
  default = ggplot2::label_parsed
```

```
)

ggplot2::ggplot(zf_post_long,
                ggplot2::aes(x = value, fill = method, colour = method)) +
  ggplot2::geom_density(alpha = 0.35, linewidth = 0.4) +
  ggplot2::geom_vline(data = zf_post_summary,
                      ggplot2::aes(xintercept = med, colour = method),
                      linewidth = 0.7, show.legend = FALSE) +
  ggplot2::geom_vline(data = zf_post_summary,
                      ggplot2::aes(xintercept = lo, colour = method),
                      linetype = "dashed", linewidth = 0.4, alpha = 0.8,
                      show.legend = FALSE) +
  ggplot2::geom_vline(data = zf_post_summary,
                      ggplot2::aes(xintercept = hi, colour = method),
                      linetype = "dashed", linewidth = 0.4, alpha = 0.8,
                      show.legend = FALSE) +
  ggplot2::scale_colour_manual(values = method_colours, name = "Method") +
  ggplot2::scale_fill_manual(values   = method_colours, name = "Method") +
  ggplot2::facet_grid(life_stage ~ quantity, scales = "free",
                      labeller = ggplot2::labeller(
                        quantity   = quantity_labels,
                        life_stage = ggplot2::label_value
                      )) +
  ggplot2::labs(x = NULL, y = "Posterior density") +
  ggplot2::theme_classic(base_size = 11) +
  ggplot2::theme(
    legend.position  = "top",
    strip.background = ggplot2::element_blank(),
    panel.border     = ggplot2::element_rect(colour = "black",
                                             fill = NA, linewidth = 0.5)
  )
```

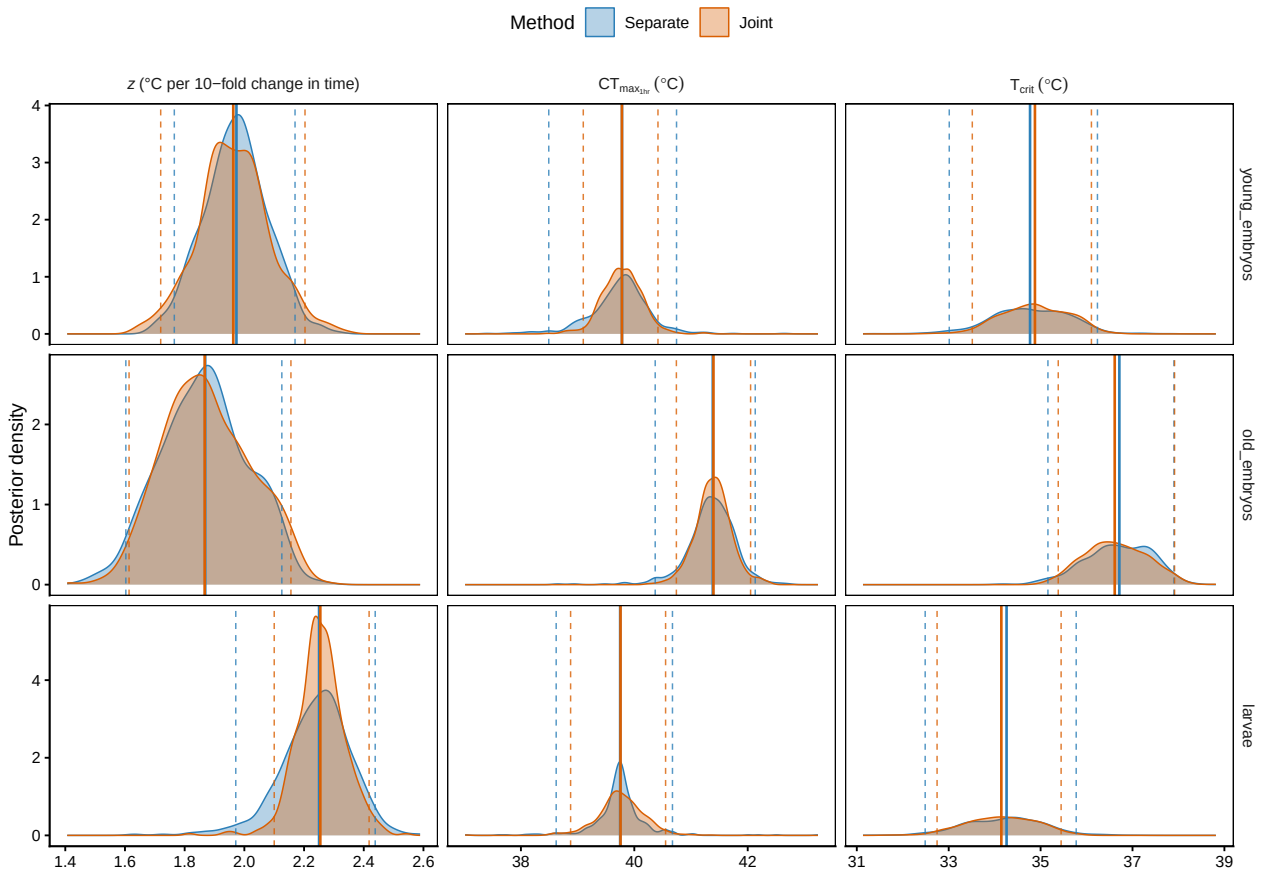


Figure S36: Posterior densities of z (left), $CT_{max_{1hr}}$ (middle), and T_{crit} (right) for each zebrafish life stage, separate-fit (blue) vs joint-fit (orange). Solid vertical lines mark the posterior median; dashed lines mark the 95 % credible interval.

The joint approach is the recommended default when the dataset spans an identifiable grouping factor (life stage here, but it could equally be population, sex, or species). It is the natural extension of the workflow in §[Joint Bayesian 4PL] from a single 4PL surface to a *family* of 4PL surfaces tied together by shared priors and (where the design allows) shared random-effect grouping.

5.4.5 Comparison with the classical two-stage fits

```
zf_joint_compare_rows <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  z_summary  <- zf_joint_tdt[[s]]$z$summary
  tc_summary <- zf_joint_tdt[[s]]$T_crit$summary
  tibble::tibble(
    life_stage = factor(s, levels = zf_stages),
    Approach = "Joint Bayesian 4PL",
    `z (C)` = format_interval(z_summary$z_median,
                              z_summary$z_lower,
                              z_summary$z_upper,
                              digits = 2),
```

```
    `CTmax_1hr (C)` = format_interval(z_summary$CTmax_median,
                                      z_summary$CTmax_lower,
                                      z_summary$CTmax_upper,
                                      digits = 2),
    `T_crit (C)` = format_interval(tc_summary$temp_median,
                                   tc_summary$temp_lower,
                                   tc_summary$temp_upper,
                                   digits = 2)
  )
}))

zf_twostep_compare_rows <- dplyr::bind_rows(lapply(
  names(zf_twostep_unc),
  function(nm) {
    parts <- strsplit(nm, "\\|\\|")[[1]]
    pt <- dplyr::filter(zf_twostep_tdt,
                        life_stage == parts[1], method == parts[2])
    ci <- zf_twostep_unc[[nm]]$summary_ci
    tibble::tibble(
      life_stage      = factor(parts[1], levels = zf_stages),
      Approach        = parts[2],
      `z (C)`         = format_interval(pt$z,         ci$z_lower,     ci$z_upper,     digits =
      `CTmax_1hr (C)` = format_interval(pt$CTmax_1hr, ci$CTmax_lower, ci$CTmax_upper, digits =
      `T_crit (C)`    = format_interval(pt$T_crit,    ci$Tcrit_lower, ci$Tcrit_upper, digits =
    )
  }
))

zf_tdt_compare_all <- dplyr::bind_rows(
  zf_twostep_compare_rows,
  zf_joint_compare_rows
) |>
  dplyr::arrange(life_stage, Approach) |>
  dplyr::rename(`Life stage` = life_stage)

tinytable::tt(zf_tdt_compare_all, escape = TRUE)
```

```
zf_twostep_compare_lines <- dplyr::bind_rows(lapply(
  names(zf_twostep_unc),
  function(nm) {
    parts <- strsplit(nm, "\\|\\|")[[1]]
    line <- zf_twostep_line_min |>
      dplyr::filter(life_stage == parts[1], method == parts[2]) |>
      dplyr::transmute(assay_temp = temp,
                       life_stage,
                       time_med_min = duration_median)
```

Table S28: Zebrafish TDT quantities from the classical two-stage pipeline and the joint Bayesian 4PL. Two-stage rows report point estimates with 95% intervals derived from the Stage-2 linear model only — z, $CT_{max_{1hr}}$ and T_{crit} from 1000 parametric simulations of the Stage-2 MVN coefficient distribution. Stage-1 LT50 uncertainty is not propagated, mirroring how classical two-step TDT estimates are reported in the literature. The 4PL rows report posterior medians with 95% credible intervals.

Life stage	Approach	z (C)	CTmax_1hr (C)	T_crit (C)
young_embryos	Joint Bayesian 4PL	1.96 [1.72, 2.2]	39.78 [39.1, 40.42]	34.88 [33.51, 36.11]
young_embryos	Two-step beta-binomial	2.22 [1.78, 2.96]	39.61 [39.32, 39.88]	34.05 [32.22, 35.19]
young_embryos	Two-step binomial	2.22 [1.78, 2.95]	39.62 [39.34, 39.88]	34.07 [32.4, 35.19]
old_embryos	Joint Bayesian 4PL	1.87 [1.61, 2.16]	41.4 [40.74, 42.05]	36.61 [35.38, 37.91]
old_embryos	Two-step beta-binomial	2.36 [1.78, 3.52]	41.4 [40.99, 42.07]	35.5 [33.85, 36.48]
old_embryos	Two-step binomial	2.48 [1.8, 3.96]	41.57 [41.06, 42.44]	35.38 [33.08, 36.5]
larvae	Joint Bayesian 4PL	2.25 [2.1, 2.42]	39.76 [38.88, 40.55]	34.15 [32.75, 35.45]
larvae	Two-step beta-binomial	2.16 [1.79, 2.73]	39.82 [39.61, 40.04]	34.41 [33.14, 35.28]
larvae	Two-step binomial	2.27 [1.97, 2.66]	39.77 [39.62, 39.91]	34.1 [33.28, 34.77]

```
    band <- zf_twostep_unc[[nm]]$curve_ci |>
      dplyr::transmute(assay_temp     = temp,
                       time_lo_min    = duration_lower,
                       time_hi_min    = duration_upper)
    dplyr::left_join(line, band, by = "assay_temp") |>
      dplyr::mutate(approach = parts[2])
  }
))

zf_twostep_joint_overlay <- dplyr::bind_rows(
  zf_joint_summary |>
    dplyr::mutate(approach = "Joint Bayesian 4PL"),
  zf_twostep_compare_lines
) |>
  dplyr::mutate(
    approach = factor(approach,
                      levels = c("Joint Bayesian 4PL",
                                 "Two-step binomial",
                                 "Two-step beta-binomial"))
  )

zf_approach_linetypes <- c(
  "Joint Bayesian 4PL" = "solid",
  "Two-step binomial" = "dashed",
  "Two-step beta-binomial" = "dotdash"
)
```

```
zf_approach_alphas <- c(
  "Joint Bayesian 4PL" = 0.28,
  "Two-step binomial" = 0.14,
  "Two-step beta-binomial" = 0.14
)

build_zf_twostep_overlay <- function(y_log = FALSE) {
  p <- ggplot2::ggplot(
    zf_twostep_joint_overlay,
    ggplot2::aes(x = assay_temp, y = time_med_min,
                 colour = life_stage,
                 fill = life_stage,
                 linetype = approach,
                 group = interaction(life_stage, approach))
  ) +
    ggplot2::geom_ribbon(
      ggplot2::aes(ymin = time_lo_min,
                   ymax = time_hi_min,
                   alpha = approach),
      colour = NA
    ) +
    ggplot2::geom_line(linewidth = 0.9) +
    ggplot2::scale_colour_manual(values = zf_palette) +
    ggplot2::scale_fill_manual(values = zf_palette) +
    ggplot2::scale_linetype_manual(values = zf_approach_linetypes,
                                   name = "Approach") +
    ggplot2::scale_alpha_manual(values = zf_approach_alphas,
                                guide = "none") +
    ggplot2::labs(x = "Assay temperature (C)",
                  colour = "Life stage", fill = "Life stage") +
    ggplot2::theme_classic() +
    ggplot2::theme(panel.border = ggplot2::element_rect(colour = "black",
                                                        fill = NA,
                                                        linewidth = 0.6))
  if (y_log) {
    p + ggplot2::scale_y_log10() +
      ggplot2::coord_cartesian(ylim = c(NA, y_cap_min)) +
      ggplot2::labs(y = expression(log[10] *
                                   " time to 50% survival (min)"))
  } else {
    p + ggplot2::coord_cartesian(ylim = c(0, y_cap_min)) +
      ggplot2::labs(y = "Time to 50% survival (min)")
  }
}

patchwork::wrap_plots(build_zf_twostep_overlay(FALSE),
                      build_zf_twostep_overlay(TRUE), ncol = 2) +
```

```
  patchwork::plot_layout(guides = "collect") &
  ggplot2::theme(legend.position = "bottom")
```

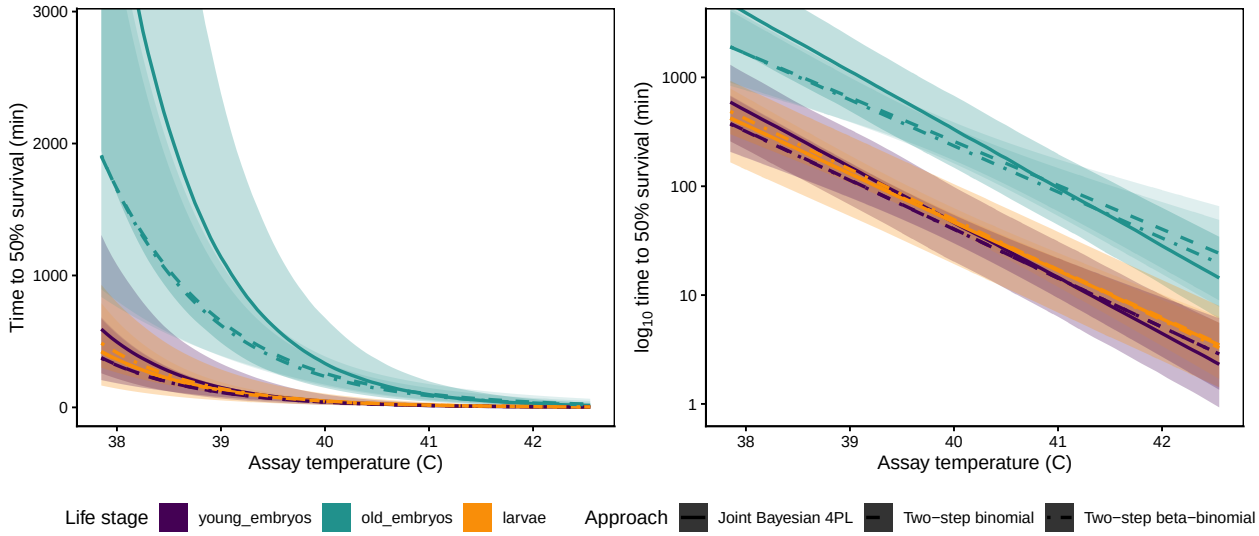


Figure S37: Zebrafish TDT-line comparison between the joint 4PL and the classical two-stage fits. The joint 4PL posterior median and 95% credible band are repeated from Figure S35; two-stage lines are point-estimate regressions from binomial and beta-binomial Stage-1 fits, with pointwise 95% confidence bands from `predict.lm()` on the Stage-2 regression (Stage-1 LT50s treated as known points).

5.5 Sensitivity check: an absolute survival threshold

All TDT curves and $CT_{max_{1hr}}$ posteriors above use the package-default threshold (the per-draw `mid` parameter, the midpoint between the fitted asymptotes). This coincides with the classical LT50 whenever the asymptotes are anchored near 0 and 1; it diverges from absolute LT50 when intrinsic background mortality pulls the upper asymptote below 1. For operational questions framed as "no more than half the cohort survives", an absolute 50 % survival threshold is the right anchor. Pass `target_surv = "absolute"` to `extract_tdt()` or `derive_tdt_curve()` (or any numeric in $(0, 1)$ for other percentiles) to swap the threshold without refitting. This section reruns the zebrafish analyses at the absolute threshold and compares the result to the default.

The joint fit makes the contrast concrete: the three life stages have very different upper asymptotes.

```
zf_joint_post <- brms::as_draws_df(fit_zf_joint)

zf_up_intercepts <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  raw <- zf_joint_post[[paste0("b_upraw_life_stage", s)]]
  asym <- zf_bounds$up_min + plogis(raw) * zf_bounds$up_w
  tibble::tibble(
    `Life stage`             = s,
    `Upper asymptote (raw)`        = format_interval(stats::median(raw),
```

Table S29: Posterior summary of the 4PL upper asymptote α_{up} at the grand-mean temperature, per life stage, from the joint fit. Medians with 95 % credible intervals on the natural survival scale. Values markedly below 1 indicate that a stage cannot reach saturating survival even at near-zero exposure, which biases absolute-LT50 comparisons across stages.

Life stage	Upper asymptote (raw)	Upper asymptote (probability)
young_embryos	0.1 [-0.53, 0.86]	0.763 [0.686, 0.851]
old_embryos	2.48 [1.87, 3.34]	0.961 [0.932, 0.982]
larvae	2.94 [1.75, 4.58]	0.974 [0.925, 0.994]

```
                                                    stats::quantile(raw, 0.025,
                                                                    names = FALSE),
                                                    stats::quantile(raw, 0.975,
                                                                    names = FALSE),
                                                    digits = 2),
    `Upper asymptote (probability)` =
      format_interval(stats::median(asym),
                      stats::quantile(asym, 0.025, names = FALSE),
                      stats::quantile(asym, 0.975, names = FALSE),
                      digits = 3)
  )
}))
tinytable::tt(zf_up_intercepts, escape = TRUE)
```

The young-embryo upper asymptote, ~0.76 at the grand-mean temperature, does not overlap the near-saturating values for the other two stages. Biologically, this means substantial intrinsic mortality unrelated to heat stress in that life stage; statistically, it means absolute 0.5 sits proportionally lower on the young-embryo survival curve than on the others. The mapping below makes the difference visible.

5.5.1 Recomputing the per-stage quantities with `target_surv = "absolute"`

```
zf_et_sep_abs <- lapply(zf_stages, function(s) {
  extract_tdt(zf_fits_sep[[s]],
              target_surv   = "absolute",   # = 0.5
              t_ref         = 60,
              TC_rate_range = c(0.1, 1),
              ndraws        = 500,
              lethal        = TRUE)
})
names(zf_et_sep_abs) <- zf_stages
```

Table S30: Posterior summaries of z, $CT_{max_{1hr}}$ and T_{crit} from the three separate zebrafish 4PL fits, under the **absolute 50 % survival** threshold (`target_surv = "absolute"`). Medians with 95 % credible intervals. Compare row-by-row with Table S23 (relative threshold).

Life stage	z (°C)	CTmax_1hr (°C)	T_crit (°C)
young_embryos	1.96 [1.73, 2.22]	39.62 [38.67, 40.61]	34.73 [33.12, 36.22]
old_embryos	1.83 [1.58, 2.13]	41.32 [40.41, 42.13]	36.74 [35.37, 38]
larvae	2.25 [2.03, 2.44]	39.74 [38.79, 40.77]	34.15 [32.51, 35.55]

```
zf_sep_tdt_table_abs <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  et <- zf_et_sep_abs[[s]]
  tibble::tibble(
    `Life stage`        = s,
    `z (°C)`            = format_interval(et$z$summary$z_median,
                                          et$z$summary$z_lower,
                                          et$z$summary$z_upper, digits = 2),
    `CTmax_1hr (°C)`    = format_interval(et$CTmax$summary$temp_median,
                                          et$CTmax$summary$temp_lower,
                                          et$CTmax$summary$temp_upper, digits = 2),
    `T_crit (°C)`       = format_interval(et$T_crit$summary$temp_median,
                                          et$T_crit$summary$temp_lower,
                                          et$T_crit$summary$temp_upper, digits = 2)
  )
}))
tinytable::tt(zf_sep_tdt_table_abs, escape = TRUE)
```

5.5.2 Joint-fit absolute-threshold posterior

The joint fit was hand-coded from `brms::bf()` and lives outside the `bayes_tls` workflow class, so we cannot point `extract_tdt()` at it directly. We reuse the posterior matrices `up_mat`, `low_mat`, `k_mat`, `mid_mat` built in Section 5.4.2 and apply the analytical inverse of the 4PL at survival = 0.5 per draw:

$$\log_{10} t_{0.5}(T) \;=\; \mathrm{mid}(T) \;+\; \tfrac{1}{k(T)} \log \tfrac{\mathrm{up}(T)-0.5}{0.5-\mathrm{low}(T)}.$$

```
arg_abs <- ifelse(up_mat > 0.5 & low_mat < 0.5,
                  (up_mat - 0.5) / (0.5 - low_mat), NA_real_)
log10_t_h_abs <- mid_mat + log(arg_abs) / k_mat
t_min_abs     <- 60 * 10 ^ log10_t_h_abs

zf_joint_summary_abs <- zf_joint_pred_grid |>
  dplyr::mutate(
    time_med_min = apply(t_min_abs, 2, stats::median, na.rm = TRUE),
```

```
    time_lo_min  = apply(t_min_abs, 2, stats::quantile, 0.025,
                         na.rm = TRUE, names = FALSE),
    time_hi_min  = apply(t_min_abs, 2, stats::quantile, 0.975,
                         na.rm = TRUE, names = FALSE)
  ) |>
  dplyr::filter(is.finite(time_med_min)) |>
  dplyr::select(assay_temp, life_stage,
                time_med_min, time_lo_min, time_hi_min)
```

```
zf_threshold_overlay <- dplyr::bind_rows(
  dplyr::mutate(zf_joint_summary,     threshold = "Relative (default)"),
  dplyr::mutate(zf_joint_summary_abs, threshold = "Absolute 50 %")
) |>
  dplyr::mutate(threshold = factor(threshold,
                                   levels = c("Relative (default)",
                                              "Absolute 50 %")))

thr_alphas    <- c("Relative (default)" = 0.32, "Absolute 50 %" = 0.12)
thr_linetypes <- c("Relative (default)" = "solid", "Absolute 50 %" = "dashed")

build_thr_overlay <- function(y_log = FALSE) {
  p <- ggplot2::ggplot(zf_threshold_overlay,
                       ggplot2::aes(x = assay_temp, y = time_med_min,
                                    colour = life_stage,
                                    fill   = life_stage,
                                    linetype = threshold,
                                    group = interaction(life_stage, threshold))) +
    ggplot2::geom_ribbon(ggplot2::aes(ymin  = time_lo_min,
                                      ymax  = time_hi_min,
                                      alpha = threshold),
                         colour = NA) +
    ggplot2::geom_line(linewidth = 1) +
    ggplot2::scale_colour_manual(values = zf_palette) +
    ggplot2::scale_fill_manual(values   = zf_palette) +
    ggplot2::scale_alpha_manual(values  = thr_alphas,    name = "Threshold") +
    ggplot2::scale_linetype_manual(values = thr_linetypes, name = "Threshold") +
    ggplot2::labs(x = "Assay temperature (°C)",
                  colour = "Life stage", fill = "Life stage") +
    ggplot2::theme_classic() +
    ggplot2::theme(panel.border = ggplot2::element_rect(colour = "black",
                                                        fill = NA,
                                                        linewidth = 0.6))
  if (y_log) {
    p + ggplot2::scale_y_log10() +
      ggplot2::coord_cartesian(ylim = c(NA, y_cap_min)) +
      ggplot2::labs(y = expression(log[10] * " time to threshold (min)"))
  } else {
```

```
    p + ggplot2::coord_cartesian(ylim = c(0, y_cap_min)) +
      ggplot2::labs(y = "Time to threshold (min)")
  }
}

patchwork::wrap_plots(build_thr_overlay(FALSE),
                      build_thr_overlay(TRUE), ncol = 2) +
  patchwork::plot_layout(guides = "collect") &
  ggplot2::theme(legend.position = "bottom")
```

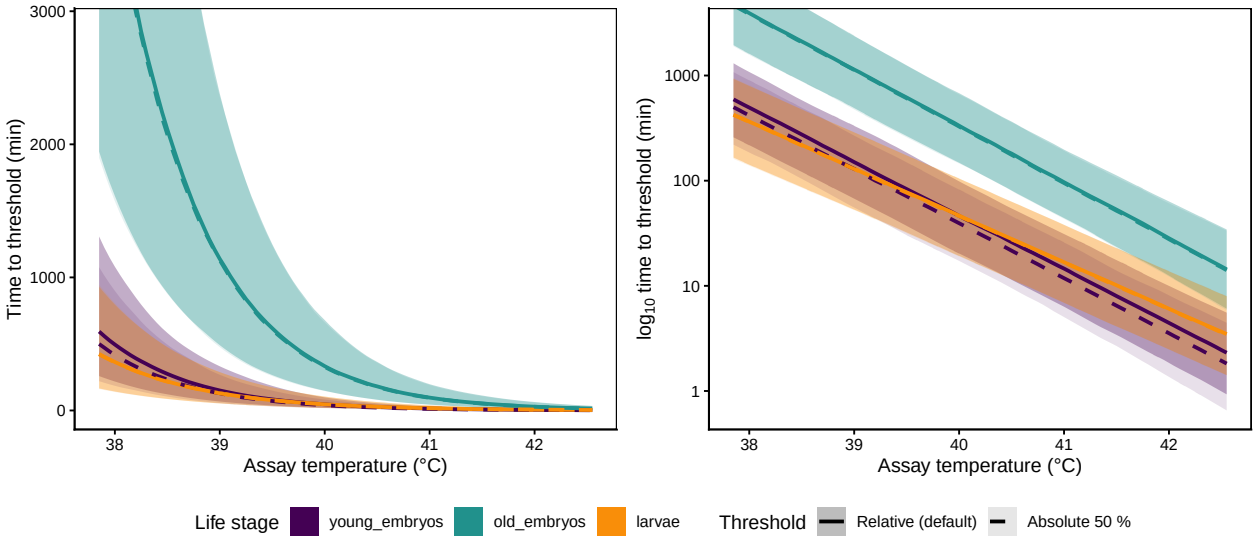


Figure S38: Zebrafish TDT lines from the joint 4PL fit under the two threshold definitions. **Solid, darker ribbons:** package-default mid-based threshold, repeated from Figure S35. **Dashed, lighter ribbons:** absolute 50 % survival. The two definitions are nearly indistinguishable for `old_embryos` and `larvae` (asymptotes close to 1) and diverge for `young_embryos` (asymptote ~0.76, see Table S29), where the absolute target is reached earlier in time.

5.5.3 Side-by-side comparison: relative vs absolute

For the separate fits we can put the two `extract_tdt()` outputs next to each other.

```
zf_threshold_compare <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  rel_s <- zf_et_sep[[s]]
  abs_s <- zf_et_sep_abs[[s]]
  tibble::tibble(
    `Life stage`           = s,
    `z (rel, °C)`          = format_interval(rel_s$z$summary$z_median,
                                             rel_s$z$summary$z_lower,
                                             rel_s$z$summary$z_upper,
                                             digits = 2),
```

Table S31: Side-by-side comparison of z and $CT_{max_{1hr}}$ from the three separate zebrafish 4PL fits under the two threshold definitions: package-default mid-based and absolute 50 % survival. Absolute-50 % shifts $CT_{max_{1hr}}$ downward for `young_embryos` (whose upper asymptote sits well below 1) but leaves the other two stages essentially unchanged. z is essentially identical under either definition for these data.

Life stage	z (rel, °C)	z (abs, °C)	CTmax_1hr (rel, °C)	CTmax_1hr (abs, °C)
young_embryos	1.97 [1.77, 2.17]	1.96 [1.73, 2.22]	39.78 [38.49, 40.74]	39.62 [38.67, 40.61]
old_embryos	1.87 [1.6, 2.13]	1.83 [1.58, 2.13]	41.38 [40.37, 42.13]	41.32 [40.41, 42.13]
larvae	2.25 [1.97, 2.44]	2.25 [2.03, 2.44]	39.75 [38.62, 40.67]	39.74 [38.79, 40.77]

```
    `z (abs, °C)`          = format_interval(abs_s$z$summary$z_median,
                                             abs_s$z$summary$z_lower,
                                             abs_s$z$summary$z_upper,
                                             digits = 2),
    `CTmax_1hr (rel, °C)`  = format_interval(rel_s$CTmax$summary$temp_median,
                                             rel_s$CTmax$summary$temp_lower,
                                             rel_s$CTmax$summary$temp_upper,
                                             digits = 2),
    `CTmax_1hr (abs, °C)`  = format_interval(abs_s$CTmax$summary$temp_median,
                                             abs_s$CTmax$summary$temp_lower,
                                             abs_s$CTmax$summary$temp_upper,
                                             digits = 2)
  )
}))
tinytable::tt(zf_threshold_compare, escape = TRUE)
```

The pattern in Table S31 matches the upper-asymptote summary in Table S29: stages whose asymptote is near 1 (`old_embryos`, `larvae`) give essentially identical z and $CT_{max_{1hr}}$ under either threshold, while the stage with a substantially sub-unit asymptote (`young_embryos`) sees $CT_{max_{1hr}}$ shift by a few tenths of a °C when the threshold is switched. The mid-based default is the right choice for cross-group comparisons because it isolates the dose-response shape from the group-level ceiling; the absolute threshold is the right choice for operational questions anchored to a fixed mortality fraction.

6 Case Study 3: Leaf PSII data

We apply `standardize_data()` → `fit_4pl()` → `extract_tdt()` to photosystem II (PSII) heat-response data from snow gum leaves (*Eucalyptus pauciflora* var. *pauciflora*). Chlorophyll fluorescence measurements of maximum potential quantum efficiency (F_v/F_m) were obtained from excised leaf segments of 11 glasshouse-grown trees before heat exposure and again after exposure. The response is retained PSII function, defined as post-exposure F_v/F_m divided by its pre-exposure value and bounded between 0 and 1.

The trials spanned 28–48 °C and 5 min to 2 h of exposure. Replicate plants originated from two glasshouse rooms and were measured over two days; `Day` and `G_Room` therefore enter the model as random intercepts on `mid`.

6.1 Data and standardisation

```
# Bundled with the package (see ?snowgum_psii): fvfm_prop (= final/initial
# Fv/Fm) is already computed (raw CSV in inst/extdata/).
leaf_raw <- snowgum_psii
str(leaf_raw)
```

```
'data.frame':   319 obs. of  8 variables:
 $ Temp        : num  34 34 34 34 34 34 39 39 39 39 ...
 $ Time        : num  15 15 15 15 15 15 5 5 5 5 ...
 $ initial_fvfm: num  0.785 0.762 0.724 0.756 0.794 ...
 $ final_fvfm  : num  0.714 0.6 0.57 0.625 0.621 ...
 $ fvfm_prop   : num  0.91 0.786 0.788 0.827 0.783 ...
 $ Unique_ID   : chr  "P-3-22" "P-1-24" "P-10-04" "P-3-34" ...
 $ G_Room      : chr  "A12" "A12" "A15" "A15" ...
 $ Day         : num  1 1 1 1 1 1 1 1 1 1 ...
```

```
# Standardise as a continuous-proportion (Beta) response. `standardize_data()`
# clamps the proportion into the open interval (0.001, 0.999) required by the
# Beta likelihood and records `response_type = "proportion"`, so the rest of the
# pipeline knows there is no `n_total` denominator. NB ~19% of `fvfm_prop` values
# are exact zeros (complete PSII shutdown / dead tissue); these are clamped to
# 0.001 - a deliberate simplification discussed in the limitation note below.
leaf_std <- standardize_data(
  leaf_raw,
  temp           = "Temp",
  duration       = "Time",
  proportion     = "fvfm_prop",
  random_effects = c("Day", "G_Room"),
  duration_unit  = "minutes"
)
temp_mean <- attr(leaf_std, "tdt_meta")$temp_mean

# Aliases for the two-stage comparison and summary chunks below, which were
# written against the hand-built frame: `fvfm_beta` is the clamped Beta response
# (represented by the library's `survival` column), `assay_temp` == temp.
leaf_std$fvfm_beta  <- leaf_std$survival
leaf_std$assay_temp <- leaf_std$temp

str(leaf_std)
```

```
tibble [319 x 15] (S3: tbl_df/tbl/data.frame)
 $ Temp        : num [1:319] 34 34 34 34 34 34 39 39 39 39 ...
 $ Time        : num [1:319] 15 15 15 15 15 15 5 5 5 5 ...
 $ initial_fvfm: num [1:319] 0.785 0.762 0.724 0.756 0.794 ...
 $ final_fvfm  : num [1:319] 0.714 0.6 0.57 0.625 0.621 ...
 $ fvfm_prop   : num [1:319] 0.91 0.786 0.788 0.827 0.783 ...
 $ Unique_ID   : chr [1:319] "P-3-22" "P-1-24" "P-10-04" "P-3-34" ...
 $ G_Room      : Factor w/ 2 levels "A12","A15": 1 1 2 2 1 2 2 1 2 2 ...
 $ Day         : Factor w/ 2 levels "1","2": 1 1 1 1 1 1 1 1 1 1 ...
 $ temp        : num [1:319] 34 34 34 34 34 34 39 39 39 39 ...
 $ duration    : num [1:319] 15 15 15 15 15 15 5 5 5 5 ...
 $ logd        : num [1:319] 1.18 1.18 1.18 1.18 1.18 ...
 $ survival    : num [1:319] 0.91 0.786 0.788 0.827 0.783 ...
 $ temp_c      : num [1:319] -5.72 -5.72 -5.72 -5.72 -5.72 ...
 $ fvfm_beta   : num [1:319] 0.91 0.786 0.788 0.827 0.783 ...
 $ assay_temp  : num [1:319] 34 34 34 34 34 34 39 39 39 39 ...
 - attr(*, "tdt_meta")=List of 5
  ..$ temp_mean     : num 39.7
  ..$ duration_unit : chr "minutes"
  ..$ random_effects: chr [1:2] "Day" "G_Room"
  ..$ response_type : chr "proportion"
  ..$ response_var  : chr "survival"
```

Response model and zeros. Unlike the count-based case studies, the leaf response is a continuous proportion with no denominator, so we use a Beta likelihood. The Beta density is undefined at exactly 0 or 1, but 60 of 319 observations (19%) are zeros, indicating complete loss of measurable PSII function. We clamp these values to 0.001 to retain a single Beta model. A zero-inflated Beta model that separately estimates complete shutdown and retained function among non-zero tissues would represent these structural zeros more faithfully, at the cost of an additional dose-dependent component.

```
leaf_summary <- tibble::tibble(
  `n individuals`              = length(unique(leaf_std$Unique_ID)),
  `Temperature range (°C)`     = paste(range(leaf_std$assay_temp), collapse = "-"),
  `Duration range (min)`       = paste(round(range(leaf_std$duration), 2), collapse = "-"),
  `Temperature centre (°C)`    = attr(leaf_std, "tdt_meta")$temp_mean,
  `n Day`                      = length(unique(leaf_std$Day)),
  `n glasshouse room`          = length(unique(leaf_std$G_Room))
)
tinytable::tt(leaf_summary, digits = 2, escape = TRUE)
```

6.2 Classical two-stage TDT fits

Before fitting the joint 4PL, we run the classical two-stage pipeline on retained PSII function. Stage 1 fits a logistic curve at each assay temperature to the continuous F_v/F_m proportion in $(0, 1)$;

Table S32: Summary of the leaf PSII TDT dataset after standardisation. Temperature centre is the grand mean used to mean-centre the temperature covariate.

n individuals	Temperature range (°C)	Duration range (min)	Temperature centre (°C)	n Day	n glasshou
11	28–48	5–120	40	2	2

Stage 2 regresses the resulting $\log_{10}$ LT50 estimates on assay temperature. We compare a quasi-binomial logistic fit with maximum-likelihood beta regression having estimated precision ϕ. Here LT50 denotes the duration at which retained function crosses 0.5.

```
fit_stage1_leaf_quasibin_case <- function(dat) {
  validate_stage1_case <- function(log10_lt50, slope, d) {
    is.finite(log10_lt50) &&
      is.finite(slope) &&
      slope < 0 &&
      abs(slope) > 1e-4 &&
      log10_lt50 >= min(d$logd, na.rm = TRUE) - 0.5 &&
      log10_lt50 <= max(d$logd, na.rm = TRUE) + 0.5
  }

  dat |>
    dplyr::group_by(temp) |>
    dplyr::group_modify(function(d, key) {
      if (dplyr::n_distinct(d$logd) < 3) {
        return(tibble::tibble(
          method       = "Two-step quasi-binomial",
          log10_lt50   = NA_real_,
          stage1_slope = NA_real_,
          phi          = NA_real_,
          stage1_ok    = FALSE
        ))
      }
      fit <- suppressWarnings(
        stats::glm(fvfm_beta ~ logd, data = d,
                   family = stats::quasibinomial(link = "logit"))
      )
      co <- stats::coef(fit)
      lt <- as.numeric(-co[1] / co[2])
      sl <- as.numeric(co[2])
      tibble::tibble(
        method       = "Two-step quasi-binomial",
        log10_lt50   = lt,
        stage1_slope = sl,
        phi          = NA_real_,
        stage1_ok    = validate_stage1_case(lt, sl, d)
      )
    }) |>
```

```
    dplyr::ungroup()
}

fit_stage1_leaf_beta_case <- function(dat) {
  validate_stage1_case <- function(log10_lt50, slope, d) {
    is.finite(log10_lt50) &&
      is.finite(slope) &&
      slope < 0 &&
      abs(slope) > 1e-4 &&
      log10_lt50 >= min(d$logd, na.rm = TRUE) - 0.5 &&
      log10_lt50 <= max(d$logd, na.rm = TRUE) + 0.5
  }

  beta_nll <- function(par, y, x) {
    mu  <- stats::plogis(par[1] + par[2] * x)
    mu  <- pmin(pmax(mu, 1e-8), 1 - 1e-8)
    phi <- exp(par[3])
    a   <- mu * phi
    b   <- (1 - mu) * phi
    -sum(stats::dbeta(y, a, b, log = TRUE))
  }

  dat |>
    dplyr::group_by(temp) |>
    dplyr::group_modify(function(d, key) {
      if (dplyr::n_distinct(d$logd) < 3) {
        return(tibble::tibble(
          method       = "Two-step beta",
          log10_lt50   = NA_real_,
          stage1_slope = NA_real_,
          phi          = NA_real_,
          stage1_ok    = FALSE
        ))
      }
      fit_glm <- suppressWarnings(
        stats::glm(fvfm_beta ~ logd, data = d,
                   family = stats::quasibinomial(link = "logit"))
      )
      start <- c(stats::coef(fit_glm), log(20))
      if (any(!is.finite(start))) {
        p0 <- pmin(pmax(mean(d$fvfm_beta, na.rm = TRUE), 1e-4), 1 - 1e-4)
        start <- c(stats::qlogis(p0), -1, log(20))
      }
      fit <- try(stats::optim(
        par     = start,
        fn      = beta_nll,
        y       = d$fvfm_beta,
```

```
        x       = d$logd,
        method  = "BFGS",
        control = list(maxit = 1000)
      ), silent = TRUE)
      if (inherits(fit, "try-error")) {
        return(tibble::tibble(
          method       = "Two-step beta",
          log10_lt50   = NA_real_,
          stage1_slope = NA_real_,
          phi          = NA_real_,
          stage1_ok    = FALSE
        ))
      }
      co <- fit$par
      lt <- as.numeric(-co[1] / co[2])
      sl <- as.numeric(co[2])
      tibble::tibble(
        method       = "Two-step beta",
        log10_lt50   = lt,
        stage1_slope = sl,
        phi          = exp(co[3]),
        stage1_ok    = validate_stage1_case(lt, sl, d)
      )
    }) |>
    dplyr::ungroup()
}
```

```
leaf_stage1_twostep <- dplyr::bind_rows(
  fit_stage1_leaf_quasibin_case(leaf_std),
  fit_stage1_leaf_beta_case(leaf_std)
)

leaf_stage2_twostep <- lapply(
  split(leaf_stage1_twostep, leaf_stage1_twostep$method),
  fit_stage2_case,
  time_multiplier = 1,
  t_ref           = 60,
  TC_rate_range   = c(0.1, 1)
)

leaf_twostep_tdt <- dplyr::bind_rows(
  lapply(leaf_stage2_twostep, `[[`, "summary")
)

leaf_twostep_grid <- seq(min(leaf_std$temp) - 1.5,
                         max(leaf_std$temp) + 1.5, by = 0.05)
```

```r
leaf_twostep_unc <- list(
  "Two-step quasi-binomial" = propagate_twostep_case(
    leaf_stage2_twostep[["Two-step quasi-binomial"]]$fit,
    temp_grid = leaf_twostep_grid,
    n_sim = 1000, time_multiplier = 1, t_ref = 60,
    TC_rate_range = c(0.1, 1), seed = 301
  ),
  "Two-step beta" = propagate_twostep_case(
    leaf_stage2_twostep[["Two-step beta"]]$fit,
    temp_grid = leaf_twostep_grid,
    n_sim = 1000, time_multiplier = 1, t_ref = 60,
    TC_rate_range = c(0.1, 1), seed = 302
  )
)

# The two-stage TDT line/CI are only defined over temperatures that have a valid
# Stage-1 LT50. Beyond that span the 3-4-point Stage-2 regression extrapolates a
# bow-tie CI that blows up to ~1e8 min and swamps the plots, so the figures clip
# each method's line and band to its own Stage-1 temperature range.
leaf_stage1_range <- leaf_stage1_twostep |>
  dplyr::filter(stage1_ok) |>
  dplyr::group_by(method) |>
  dplyr::summarise(tmin = min(temp), tmax = max(temp), .groups = "drop")

clip_to_stage1 <- function(df) {
  df |>
    dplyr::left_join(leaf_stage1_range, by = "method") |>
    dplyr::filter(temp >= tmin, temp <= tmax) |>
    dplyr::select(-tmin, -tmax)
}
```

```r
leaf_twostep_tdt_ci <- dplyr::bind_rows(lapply(
  names(leaf_twostep_unc),
  function(m) {
    pt <- dplyr::filter(leaf_twostep_tdt, method == m)
    ci <- leaf_twostep_unc[[m]]$summary_ci
    tibble::tibble(
      `Stage-1 likelihood` = m,
      `z (C)`         = format_interval(pt$z,         ci$z_lower,     ci$z_upper,     digits =
      `CTmax_1hr (C)` = format_interval(pt$CTmax_1hr, ci$CTmax_lower, ci$CTmax_upper, digits =
      `Stage-2 R2`                 = round(pt$r_squared, 3),
      `Stage-1 estimates used`     = pt$n_stage1,
      `Stage-1 estimates excluded` = pt$n_excluded
    )
  }
))
```

Table S33: Classical two-stage TDT quantities for snow gum leaf PSII. Point estimates are from the Stage-2 regression. The 95% CIs for z and $CT_{max_{1hr}}$ both come from 1000 parametric simulations from $\text{MVN}(\hat{\theta}, \widehat{\text{Cov}}(\hat{\theta}))$ of the Stage-2 coefficients. Each Stage-1 $\log_{10}$ LT50 is treated as a known point, as is standard practice. T_{crit} is not reported because loss of PSII function is a sublethal endpoint and the rate-multiplier T_{crit} is only meaningful for lethal endpoints.

Stage-1 likelihood	z (C)	CTmax_1hr (C)	Stage-2 R2	Stage-1 estimates used	Stage-
Two-step quasi-binomial	5.59 [3.28, 18.98]	39.08 [36.73, 40.54]	0.949	4	2
Two-step beta	5.12 [NA, NA]	38.75 [37.71, 39.5]	0.988	3	3

```
tinytable::tt(leaf_twostep_tdt_ci, escape = TRUE)
```

```
leaf_twostep_line_min <- dplyr::bind_rows(lapply(
  leaf_stage2_twostep,
  make_twostep_curve_case,
  temp_grid = leaf_twostep_grid,
  time_multiplier = 1,
  output_time_unit = "min"
))

leaf_twostep_band_min <- dplyr::bind_rows(lapply(
  names(leaf_twostep_unc),
  function(m) {
    leaf_twostep_unc[[m]]$curve_ci |>
      dplyr::mutate(method = m)
  }
))

# Clip the line and band to each method's Stage-1 temperature span (no wild
# extrapolation of the 3-4-point Stage-2 regression).
leaf_twostep_line_min <- clip_to_stage1(leaf_twostep_line_min)
leaf_twostep_band_min <- clip_to_stage1(leaf_twostep_band_min)

ggplot2::ggplot() +
  ggplot2::geom_ribbon(
    data = leaf_twostep_band_min,
    ggplot2::aes(x = temp,
                 ymin = log10(duration_lower),
                 ymax = log10(duration_upper),
                 fill = method),
    alpha = 0.18, colour = NA
  ) +
  ggplot2::geom_point(
    data = dplyr::filter(leaf_stage1_twostep, stage1_ok),
```

```
  ggplot2::aes(x = temp,
               y = log10_lt50,
               colour = method),
  size = 2.5
) +
ggplot2::geom_line(
  data = leaf_twostep_line_min,
  ggplot2::aes(x = temp, y = log10(duration_median),
               colour = method),
  linewidth = 1
) +
ggplot2::labs(
  x = "Assay temperature (C)",
  y = expression(log[10]~LT[50]~"(min)"),
  colour = "Two-stage fit",
  fill   = "Two-stage fit"
) +
ggplot2::theme_classic()
```

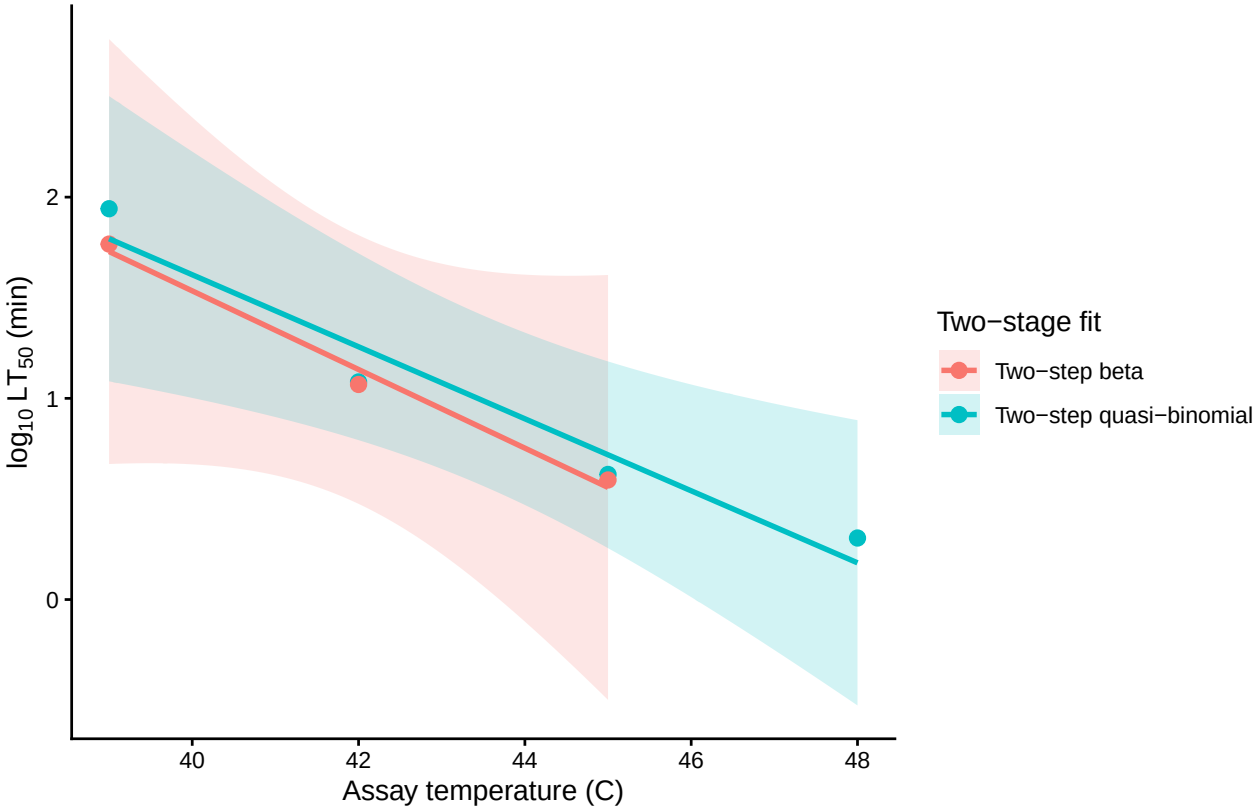


Figure S39: Classical two-stage TDT lines for snow gum leaf PSII. Points are Stage-1 $\log_{10}$ LT50 estimates; lines are the Stage-2 regressions; shaded bands are pointwise 95% confidence intervals from `predict.lm()` on the Stage-2 regression (Stage-1 LT50s treated as known points, as in standard practice). Each line/band is shown only over temperatures with a valid Stage-1 LT50: only 3-4 assay temperatures reach 50% retained function within 120 min. The beta Stage-1 fit allows extra dispersion within each assay temperature, but both approaches reduce to a single point estimate per temperature for Stage 2.

6.3 Fitting the joint Bayesian 4PL

`fit_4pl()` is the package's high-level fitting wrapper. It takes the standardised data and configures the joint 4PL for proportional response data: a beta distribution, the disjoint-bounds reparameterisation that keeps the asymptotes identifiable, and a `temp_c` slope on each 4PL sub-parameter. We add random intercepts on `mid` for `Day` and `G_Room` to absorb the day-to-day and room-to-room variation that would otherwise contaminate the dose-response surface.

```
# `fit_4pl()` configures the Beta (continuous-proportion) 4PL directly from the
# standardised data: because `standardize_data()` recorded
# `response_type = "proportion"`, the default family is `brms::Beta()`. We use
# the identity link (validated equivalent to the logit form in
# notes/2026-05-21-beta-family-support.qmd) and add random intercepts on `mid`
# for `Day` and `G_Room`.
wf_leaf <- fit_4pl(
  leaf_std,
```

Table S34: Sampling diagnostics for the leaf 4PL fit. Healthy targets: Rhat < 1.01, high ESS, near-zero divergences.

rhat_max	ess_bulk_min	ess_tail_min	divergences	treedepth_hits	bfmi_min	rhat_pass	ess_pass
1	4474	4234	7	1	0.877	TRUE	TRUE

```
  family         = brms::Beta(link = "identity"),
  random_effects = c("Day", "G_Room"),
  chains  = 4, iter = 8000, warmup = 4000, cores = 4, seed = 123,
  init    = 0,
  backend = "cmdstanr",
  control = list(adapt_delta = 0.99, max_treedepth = 20),
  file_refit = "on_change",
  file    = file.path(models_dir, "fit_leaf_function_4pl")
)

# Workflow header: data shape, T_bar, asymptote bounds, random effects, draws.
print(wf_leaf)
```

```
<bayes_tls>
  Data:     319 rows; 6 temperatures; 5 durations
  T_bar:    39.72
  Bounds:   response in (0.000, 1.000); low in (0.001, 0.499); up in (0.501, 0.999)
  RE:       Day, G_Room
  Status:   fitted (16000 draws)
```

6.3.1 Sampling diagnostics

```
tinytable::tt(diagnose_tdt_fit(wf_leaf), digits = 3, escape = TRUE)
```

The diagnostics table is a numerical pass/fail summary. The canonical visual check is the chain trace ("fuzzy caterpillar"): healthy chains overlap and explore the same posterior support; chains that drift, diverge, or hang in different regions signal a problem with the fit.

```
brms::mcmc_plot(
  get_brmsfit(wf_leaf),
  type     = "trace",
  variable = c("b_lowraw_Intercept", "b_upraw_Intercept",
               "b_logk_Intercept",   "b_mid_Intercept",
               "b_mid_temp_c",       "phi")
) +
  ggplot2::theme_classic(base_size = 11) +
  ggplot2::theme(legend.position = "bottom")
```

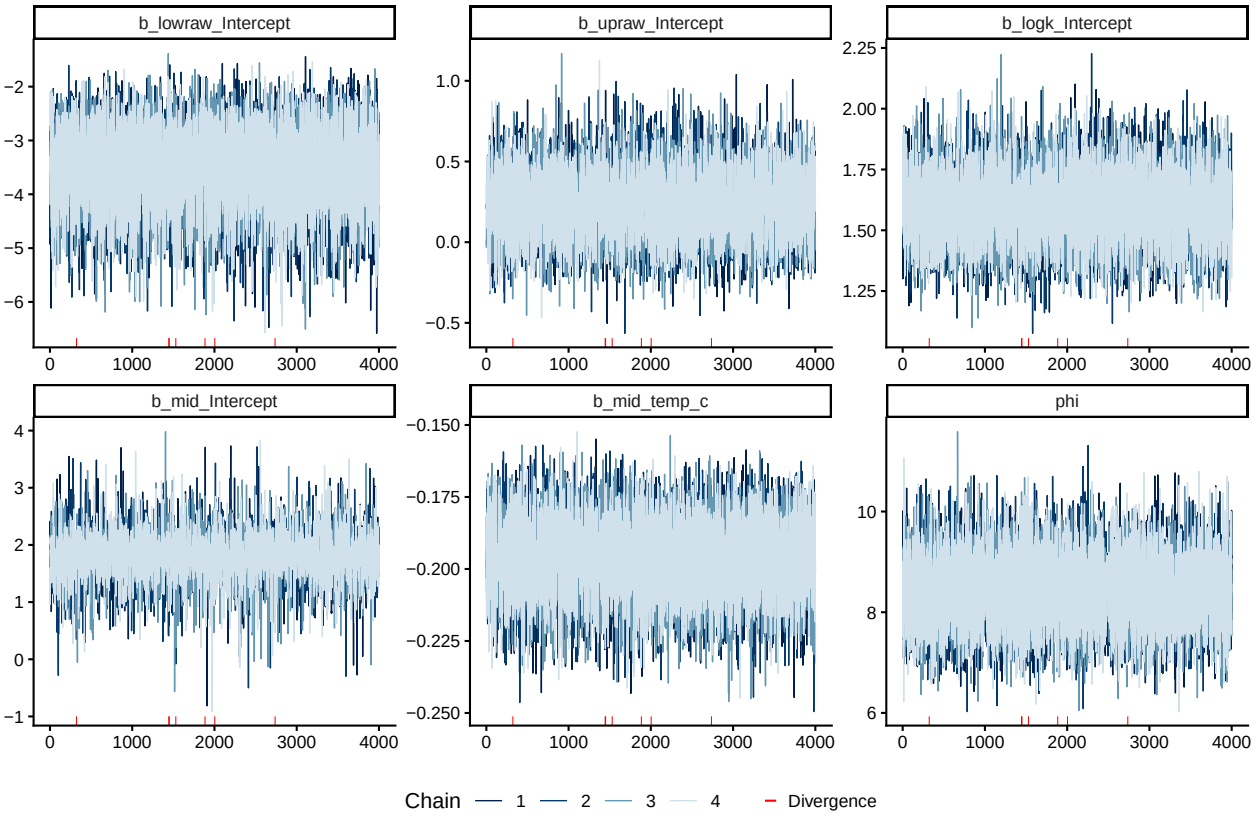


Figure S40: Posterior chain traces for the leaf 4PL fixed-effect parameters and the beta dispersion ϕ. Each panel overlays four chains; well-mixed chains overlap throughout.

6.3.2 Posterior parameters on the natural scale

```
tinytable::tt(tdt_parameter_table(wf_leaf), digits = 3, escape = TRUE)
```

6.4 Retained PSII function curves with observed overlay

Here the library's `survival` interface represents retained PSII function. `predict_survival_curves()` evaluates the fitted 4PL on a grid of assay temperatures and durations and returns posterior retained-function proportions. Overlaying observed proportions checks how closely the fitted curves track the data.

```
psc_leaf <- predict_survival_curves(
  wf_leaf,
  temps     = sort(unique(leaf_std$temp)),
  ndraws    = 500
)

# Add the library-facing variable for plotting: `survival` = retained function.
```

Table S35: Posterior medians and 95% credible intervals for the leaf 4PL fit, transformed back to the natural scale. `low` and `up` are the bottom and top asymptotes of retained PSII function at the grand-mean temperature; `k` is the slope on $\log_{10}$ time; `mid` intercept is $\log_{10}$ threshold time in minutes at the grand-mean temperature; `mid temp_c slope` is β_1, from which $z = -1/\beta_1$.

parameter	median	lower	upper
low (lower asymptote)	0.0159	0.00413	0.0477
up (upper asymptote)	0.7783	0.73267	0.8263
k (slope)	4.9191	3.7961	6.4792
mid intercept (at T_bar)	1.8083	0.96052	2.5602
mid temp_c slope	-0.1956	-0.22257	-0.1724
z (°C)	5.1117	4.49291	5.8014

```
leaf_std$survival <- leaf_std$fvfm_prop

plot_survival_curves(psc_leaf, observed = leaf_std) +
  ggplot2::labs(x = "Exposure duration (minutes)") +
  ggplot2::coord_cartesian(xlim = c(0, 120))
```

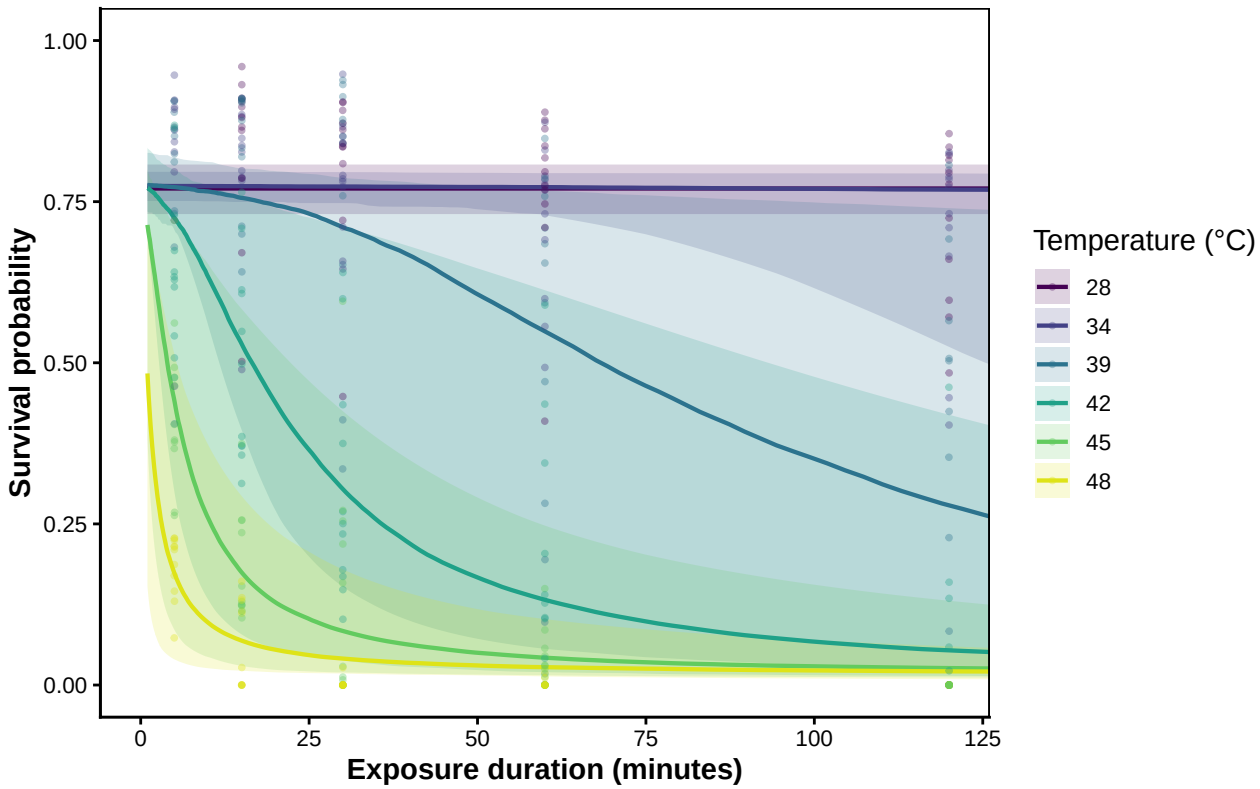


Figure S41: Posterior curves for retained PSII function in snow gum leaves at each assayed temperature, with 95% credible bands. Coloured points are observed per-replicate retained-function proportions.

6.5 TDT landscape

`derive_tdt_landscape()` evaluates the fitted 4PL on a dense temperature-by-duration grid, and `plot_tdt_landscape()` renders it as a heatmap with contours at 10%, 50%, and 90% retained PSII function. This surface is summarised by the TDT line below.

```
leaf_dur_grid <- seq(min(leaf_std$duration), max(leaf_std$duration),
                     length.out = 120)
lsp_leaf <- derive_tdt_landscape(wf_leaf,
                                 duration_grid = leaf_dur_grid,
                                 ndraws        = 500)
plot_tdt_landscape(lsp_leaf, observed = leaf_std)
```

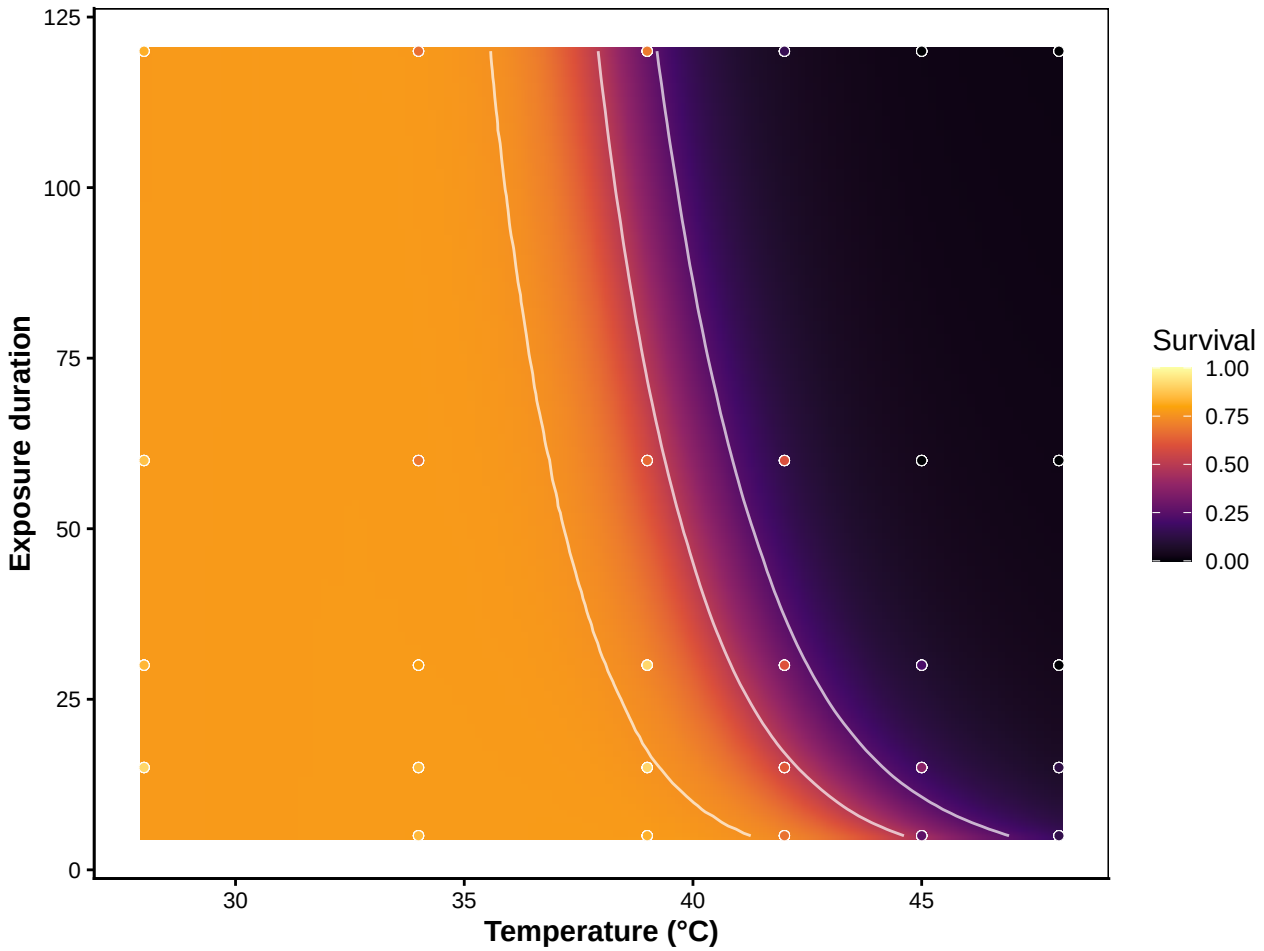


Figure S42: TDT landscape for snow gum PSII function: posterior median retained function across a 120 × 120 grid of assay temperatures × durations. White contours mark 10%, 50%, and 90% retained-function isolines. Overlaid points are observed proportions.

6.6 TDT line: time to the threshold vs temperature

The one-dimensional **TDT line** gives the predicted time to the retained-function threshold at each temperature. `derive_tdt_curve()` constructs it from the joint fit, and `plot_tdt_curve()` shows linear and $\log_{10}$ time axes side by side. The default threshold is the midpoint between the fitted asymptotes; pass `target_surv = "absolute"` for an absolute $F_v/F_m = 0.5$ threshold.

```
ltx_leaf <- derive_tdt_curve(
  wf_leaf,
  temp_grid        = seq(min(leaf_std$temp) - 1.5,
                         max(leaf_std$temp) + 1.5, by = 0.05),
  ndraws           = 500,
  time_multiplier  = 1,
  output_time_unit = "minutes"
)
```

```
plot_tdt_curve(ltx_leaf, panels = "both", colour = "#b2182b")
```

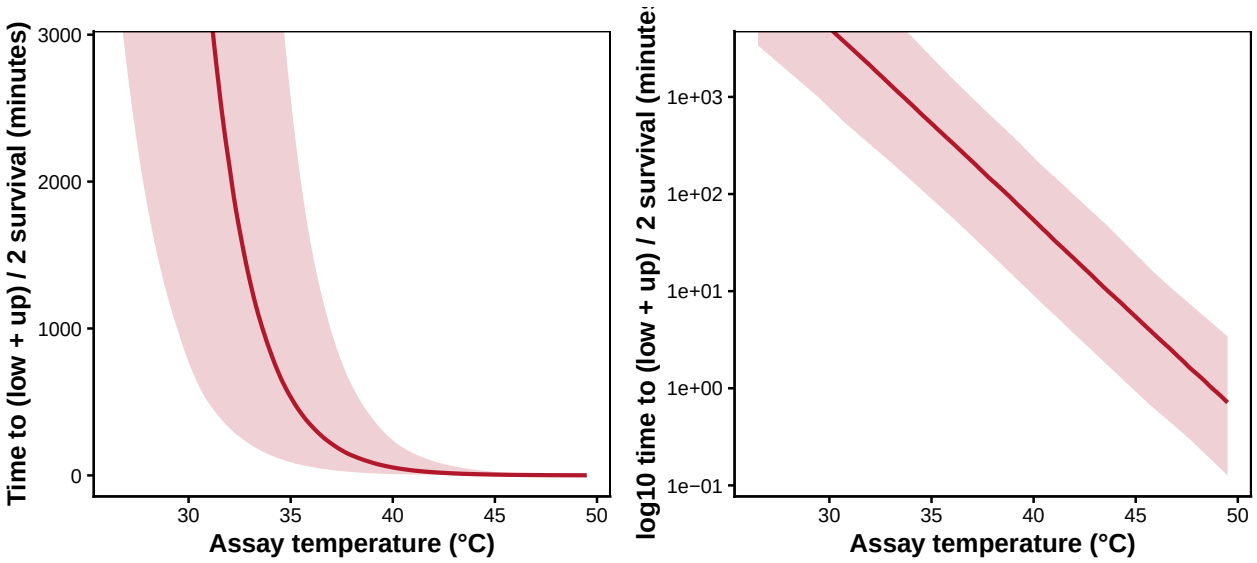


Figure S43: Snow gum leaf PSII function posterior TDT line: predicted time to threshold vs assay temperature, with 95 % credible band. Linear-time panel on the left, $\log_{10}$-time panel on the right (slope is $-1/z$). Both panels share the same posterior.

6.7 Classical TDT quantities: z, $CT_{max_{1hr}}$

`extract_tdt()` returns z (read directly from the posterior — at the relative threshold $z = -1/\beta_1$ per draw, the slope of mid(T)) and $CT_{max_{1hr}}$ (the temperature at which the retained-function threshold is reached after 1 h, obtained by per-draw 4PL inversion).

```
et_leaf <- extract_tdt(
  wf_leaf,
  t_ref           = 60,
  time_multiplier = 1,   # leaf model time is MINUTES; t_ref = 60 => CTmax at 1 h.
                         # extract_tdt() now auto-derives this from duration_unit,
                         # but we keep it explicit for clarity.
  TC_rate_range   = c(0.1, 1),
  ndraws          = 500,
  lethal          = FALSE
)
```

```
leaf_extract <- tibble::tribble(
  ~Quantity,                          ~Median, ~Lower, ~Upper,
  "z (°C)",                               et_leaf$z$summary$z_median,
                                      et_leaf$z$summary$z_lower,
                                      et_leaf$z$summary$z_upper,
  "CTmax_1hr (°C)",                   et_leaf$CTmax$summary$temp_median,
```

Table S36: TDT summaries for snow gum leaf PSII function, with full posterior uncertainty. z is read directly from the posterior (the temperature slope on mid; $z = -1/\beta_1$ at the relative threshold). $CT_{max_{1hr}}$ is the temperature at which the retained-function threshold is reached after 1 h (4PL inversion per draw).

Quantity	Median	Lower	Upper
z (°C)	5.1	4.5	5.9
CTmax_1hr (°C)	40	35.5	43.9

```
                                    et_leaf$CTmax$summary$temp_lower,
                                    et_leaf$CTmax$summary$temp_upper
)
tinytable::tt(leaf_extract, digits = 2, escape = TRUE)
```

```
z_draws_leaf   <- get_z_draws(et_leaf) |>
  dplyr::transmute(temp = z)
z_q <- stats::quantile(z_draws_leaf$temp,
                       c(0.025, 0.5, 0.975), names = FALSE, na.rm = TRUE)
z_post_leaf <- list(
  draws   = z_draws_leaf,
  summary = tibble::tibble(temp_lower  = z_q[1],
                           temp_median = z_q[2],
                           temp_upper  = z_q[3])
)

patchwork::wrap_plots(
  plot_temperature_density(z_post_leaf,
                           x_label = expression(italic(z)~"(°C per 10-fold change in time)")),
  plot_temperature_density(et_leaf$CTmax,
                           x_label = expression(CT[max[1*hr]]~("°C"))),
  ncol = 2
)
```

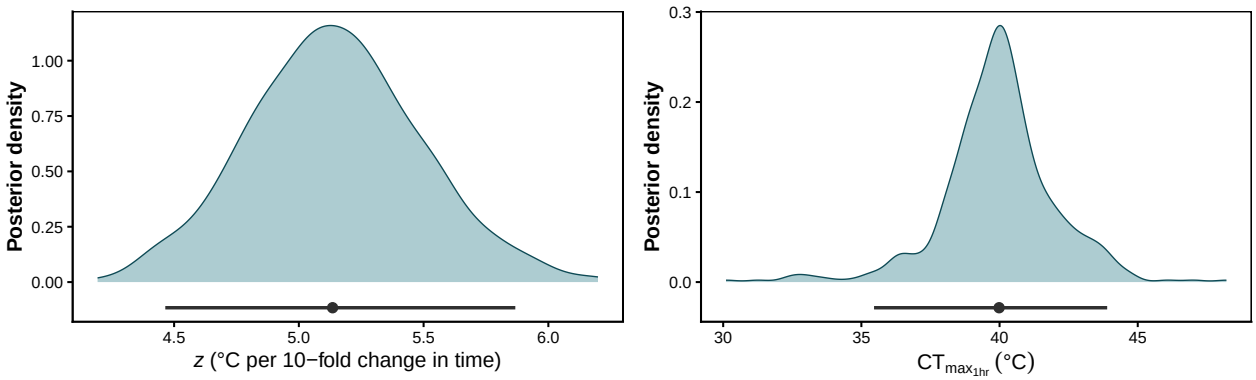


Figure S44: Posterior densities of z (left) and $CT_{max_{1hr}}$ (right) for snow gum leaf PSII function. The horizontal bar below each density is the 95% credible interval; the point marks the posterior median. T_{crit} is not shown because loss of PSII function is a sublethal endpoint.

6.8 Comparison between approaches

The classical two-stage pipeline fits a 0–1 logistic at each temperature, so its LT50 is necessarily the **absolute** $F_v/F_m = 0.5$ crossing; it cannot represent the fitted PSII plateau ($u \approx 0.78$) or target a relative midpoint. For a like-with-like comparison, this section extracts the joint 4PL at the same absolute threshold (`target_surv = "absolute"`). The main leaf analysis (Section 6.7, Section 6.6) instead uses the package-default relative midpoint, representing 50% of the fitted functional range. The absolute $F_v/F_m = 0.5$ crossing occurs within the assayed 5–120 min window only at intermediate temperatures (~39–45 °C).

```
# Compare like-with-like: the forced-0/1 two-stage can only read an absolute 0.5
# crossing, so extract the joint 4PL at the same absolute threshold here.
et_leaf_abs <- extract_tdt(
  wf_leaf, t_ref = 60, time_multiplier = 1, target_surv = "absolute",
  ndraws = 500, lethal = FALSE
)

leaf_compare_approaches <- dplyr::bind_rows(
  dplyr::bind_rows(lapply(
    names(leaf_twostep_unc),
    function(m) {
      pt <- dplyr::filter(leaf_twostep_tdt, method == m)
      ci <- leaf_twostep_unc[[m]]$summary_ci
      tibble::tibble(
        Approach        = m,
        `z (C)`         = format_interval(pt$z,         ci$z_lower,     ci$z_upper,     digits
        `CTmax_1hr (C)` = format_interval(pt$CTmax_1hr, ci$CTmax_lower, ci$CTmax_upper, digits
      )
    }
  )),
  tibble::tibble(
```

Table S37: Snow gum leaf PSII TDT quantities from the classical two-stage pipeline and the joint Bayesian 4PL, compared at a **common absolute threshold** ($F_v/F_m = 0.5$). The two-stage pipeline forces a 0–1 logistic per temperature and so can only target an absolute 0.5 crossing; for a like-with-like comparison the joint 4PL row is therefore extracted with `target_surv = "absolute"` rather than the package-default relative midpoint (the main leaf result in Table S36 uses the relative midpoint, representing 50% of the fitted functional range given the plateau $u \approx 0.78$). Two-stage rows report point estimates with 95% intervals derived from the Stage-2 linear model only; Stage-1 LT50 uncertainty is not propagated. The 4PL row reports posterior medians with 95% credible intervals.

Approach	z (C)	CTmax_1hr (C)
Two-step quasi-binomial	5.59 [3.28, 18.98]	39.08 [36.73, 40.54]
Two-step beta	5.12 [NA, NA]	38.75 [37.71, 39.5]
Joint Bayesian 4PL	4.86 [4.23, 5.57]	39.48 [35.76, 43.19]

```
    Approach = "Joint Bayesian 4PL",
    `z (C)` = format_interval(et_leaf_abs$z$summary$z_median,
                              et_leaf_abs$z$summary$z_lower,
                              et_leaf_abs$z$summary$z_upper,
                              digits = 2),
    `CTmax_1hr (C)` = format_interval(et_leaf_abs$CTmax$summary$temp_median,
                                      et_leaf_abs$CTmax$summary$temp_lower,
                                      et_leaf_abs$CTmax$summary$temp_upper,
                                      digits = 2)
  )
)

tinytable::tt(leaf_compare_approaches, escape = TRUE)
```

```
leaf_twostep_band_cmp <- dplyr::bind_rows(lapply(
  names(leaf_twostep_unc),
  function(m) {
    leaf_twostep_unc[[m]]$curve_ci |>
      dplyr::transmute(
        temp,
        duration_lower,
        duration_upper,
        method = m
      )
  }
))

# Clip the two-stage line + band to each method's Stage-1 temperature span so the
# extrapolated bow-tie CI (up to ~1e8 min) does not swamp the panel; the joint
# 4PL is well-defined across the assayed range, so clip it to the assay window.
```

```
leaf_twostep_band_cmp <- clip_to_stage1(leaf_twostep_band_cmp)
leaf_twostep_line_cmp <- clip_to_stage1(leaf_twostep_line_min)

# Joint 4PL line at the SAME absolute 0.5 threshold as the two-stage, on the
# two-stage temperature grid, so the comparison is like-with-like.
ltx_leaf_abs <- derive_tdt_curve(
  wf_leaf, temp_grid = leaf_twostep_grid, ndraws = 500,
  time_multiplier = 1, output_time_unit = "minutes", target_surv = "absolute"
)
leaf_4pl_line <- ltx_leaf_abs$summary |>
  dplyr::transmute(temp,
                   duration_median,
                   duration_lower,
                   duration_upper,
                   method = "Joint Bayesian 4PL") |>
  dplyr::filter(temp >= min(leaf_std$temp), temp <= max(leaf_std$temp))

leaf_method_cols <- c(
  "Joint Bayesian 4PL"       = "#b2182b",
  "Two-step quasi-binomial"  = "#2166ac",
  "Two-step beta"            = "#1b9e77"
)

p_leaf_cmp_lin <- ggplot2::ggplot() +
  ggplot2::geom_ribbon(
    data = leaf_4pl_line,
    ggplot2::aes(x = temp, ymin = duration_lower, ymax = duration_upper,
                 fill = method),
    alpha = 0.20, colour = NA
  ) +
  ggplot2::geom_ribbon(
    data = leaf_twostep_band_cmp,
    ggplot2::aes(x = temp, ymin = duration_lower, ymax = duration_upper,
                 fill = method),
    alpha = 0.15, colour = NA
  ) +
  ggplot2::geom_line(
    data = leaf_4pl_line,
    ggplot2::aes(x = temp, y = duration_median, colour = method),
    linewidth = 1
  ) +
  ggplot2::geom_line(
    data = leaf_twostep_line_cmp,
    ggplot2::aes(x = temp, y = duration_median, colour = method),
    linewidth = 0.9
  ) +
  ggplot2::scale_colour_manual(values = leaf_method_cols) +
```

```
  ggplot2::scale_fill_manual(values   = leaf_method_cols) +
  ggplot2::labs(x = "Assay temperature (C)",
                y = expression("Time to "*italic(F)[v]/italic(F)[m]*" = 0.5 (min)"),
                colour = "Approach", fill = "Approach") +
  ggplot2::theme_classic()

p_leaf_cmp_log <- p_leaf_cmp_lin +
  ggplot2::scale_y_log10() +
  ggplot2::labs(y = expression(log[10]~"time to "*italic(F)[v]/italic(F)[m]*" = 0.5 (min)")) +
  ggplot2::theme(legend.position = "none")

patchwork::wrap_plots(p_leaf_cmp_lin, p_leaf_cmp_log, ncol = 2) +
  patchwork::plot_layout(guides = "collect") &
  ggplot2::theme(legend.position = "bottom")
```

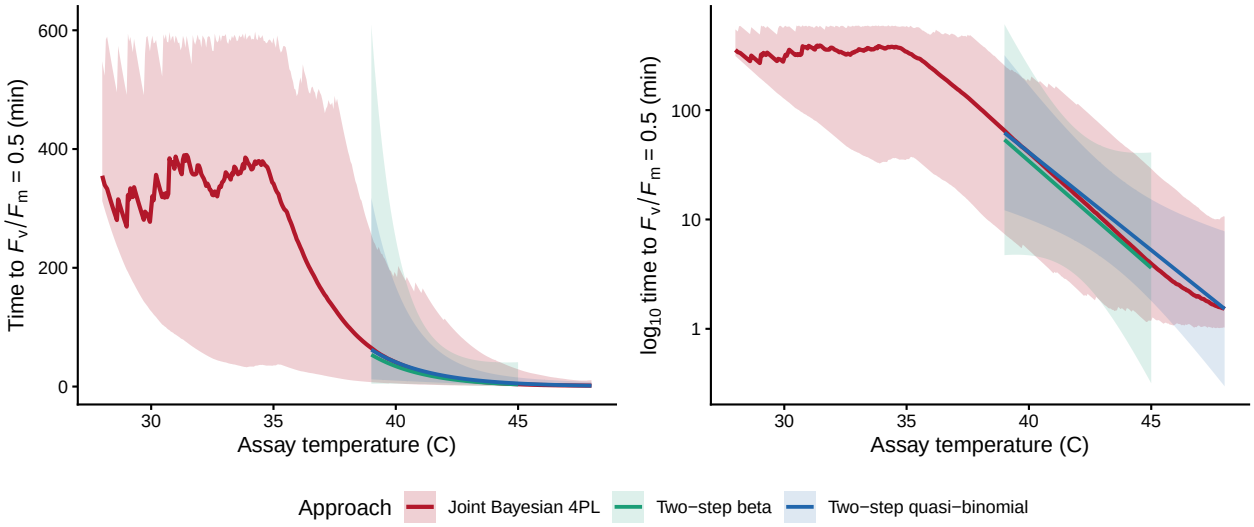


Figure S45: Snow gum leaf PSII TDT-line comparison at a common absolute threshold ($F_v/F_m = 0.5$). The joint 4PL posterior median and 95% credible band (time to $F_v/F_m = 0.5$, `target_surv = "absolute"` — the only threshold the forced-0/1 two-stage can target) are shown across the assayed 28–48 °C range; the two classical two-stage lines and their pointwise 95% `predict.lm()` confidence bands are shown only over each method's Stage-1 temperature span (39–48 °C quasi-binomial, 39–45 °C beta), since only those assay temperatures yield a valid Stage-1 LT50. Where they overlap the three methods agree closely; the joint 4PL additionally constrains the cooler temperatures the two-stage cannot reach.

6.9 Heat injury under reference traces

For this sublethal endpoint, `predict_heat_injury()` is interpreted as projecting cumulative functional injury and retained PSII function through a temperature trace. We specify T_c as an illustrative no-accumulation threshold rather than deriving T_{crit}, which is reserved for lethal endpoints.

We illustrate the function on four reference traces built by `make_temperature_scenarios()`: a flat baseline, a single one-hour spike, three one-hour spikes, and a four-day diurnal cycle that mimics a typical summer heatwave. The single-spike trace is calibrated as an analytical sanity check — its spike temperature is set to the posterior median of $CT_{max_{1hr}}$, so by definition the trace delivers approximately one LT_{50} dose by the end of the spike hour. These values are plausible leaf temperatures on sunny, still conditions.

```
leaf_T_c       <- 32.5
leaf_CTmax_med <- et_leaf$CTmax$summary$temp_median

leaf_scens <- make_temperature_scenarios(
  baseline   = 30,
  spike_temp = leaf_CTmax_med,
  n_hours    = 96,
  spike_times_single = 24,
  spike_times_multi  = c(24, 48, 72),
  diurnal_n_days     = 4,
  diurnal_day_peaks  = c(38.0, 41.0, 44.0, 43.5),
  diurnal_night_temp = c(20.0, 15.5, 17.5, 19.0),
  diurnal_peak_fwhm  = 5,
  diurnal_noise_sd   = 0.5,
  diurnal_seed       = 1L
)

leaf_hi <- purrr::map(leaf_scens, function(sc) {
  predict_heat_injury(trace = sc, workflow = wf_leaf,
                      T_c = leaf_T_c, ndraws = 500)
})
```

Setting `repair = TRUE` adds a temperature-dependent Sharpe-Schoolfield repair kernel so injury accumulated at hot temperatures can be partially cleared during cooler intervals. Here we re-run only the two scenarios that accrue meaningful HI without repair — the multi-spike and the diurnal traces.

```
leaf_repair_pars <- list(
  TA = 14065, TAL = 50000, TAH = 120000,
  TL = 10.5 + 273.15, TH = 32.5 + 273.15, TREF = 30 + 273.15,
  r_ref = 0.15
)

leaf_hi_multi_rep <- predict_heat_injury(
  leaf_scens$multi_spike, wf_leaf, T_c = leaf_T_c, ndraws = 500,
  repair = TRUE, repair_pars = leaf_repair_pars
)
leaf_hi_diurnal_rep <- predict_heat_injury(
  leaf_scens$diurnal, wf_leaf, T_c = leaf_T_c, ndraws = 500,
  repair = TRUE, repair_pars = leaf_repair_pars
)
```

```
plot_temperature_scenarios(leaf_scens, T_c = leaf_T_c)
```

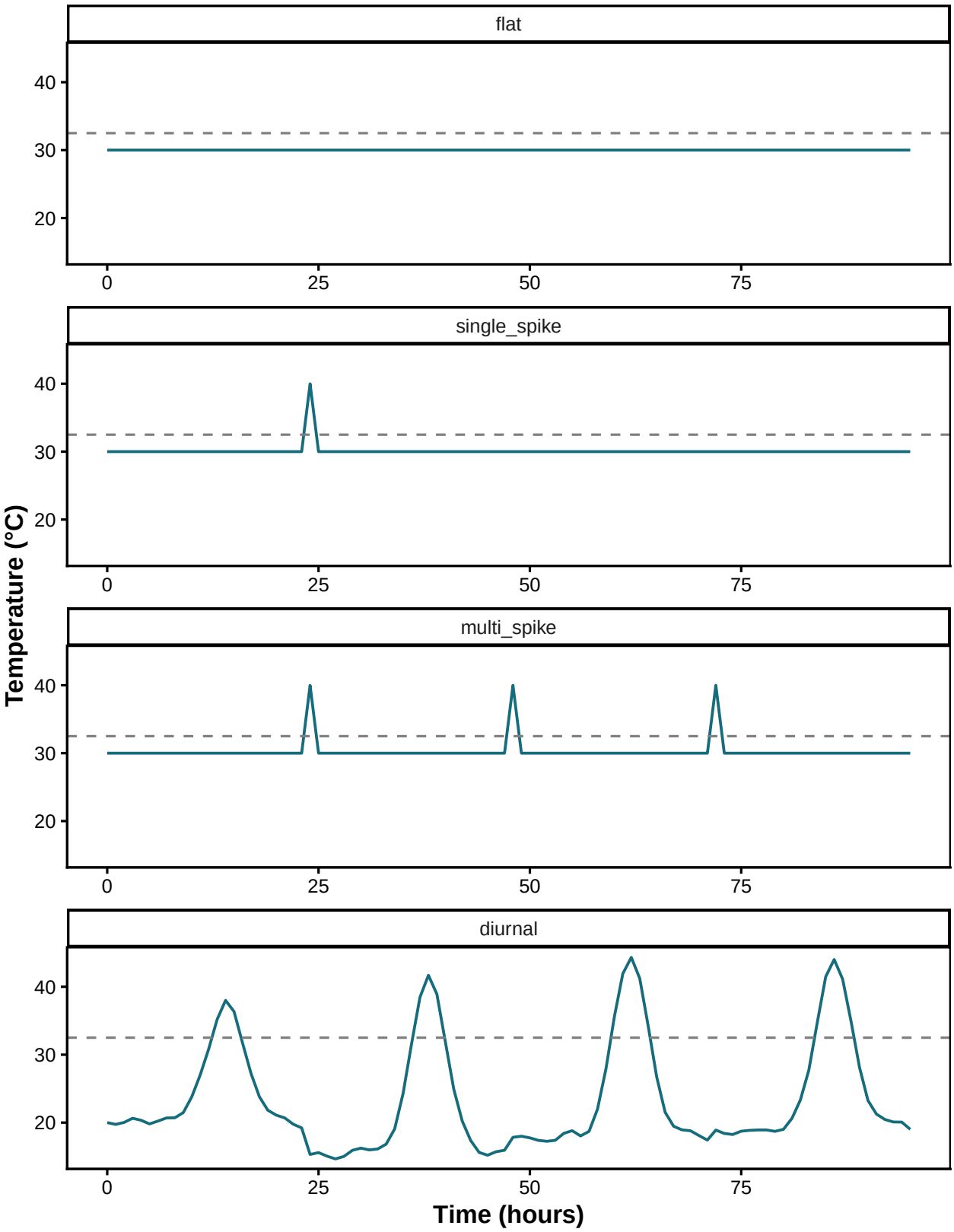


Figure S46: The four reference temperature traces used for the leaf functional-injury prediction. Baseline temperature is 30 °C; 1-hour spikes are set to the posterior median $CT_{max_{1hr}}$ so each spike delivers approximately one threshold dose. The diurnal scenario is a clustered 4-day heatwave: a cool day below T_c followed by three hotter days. The dashed line marks the chosen threshold $T_c = 32.5$ °C.

```
plot_heat_injury(leaf_hi$single_spike)
```

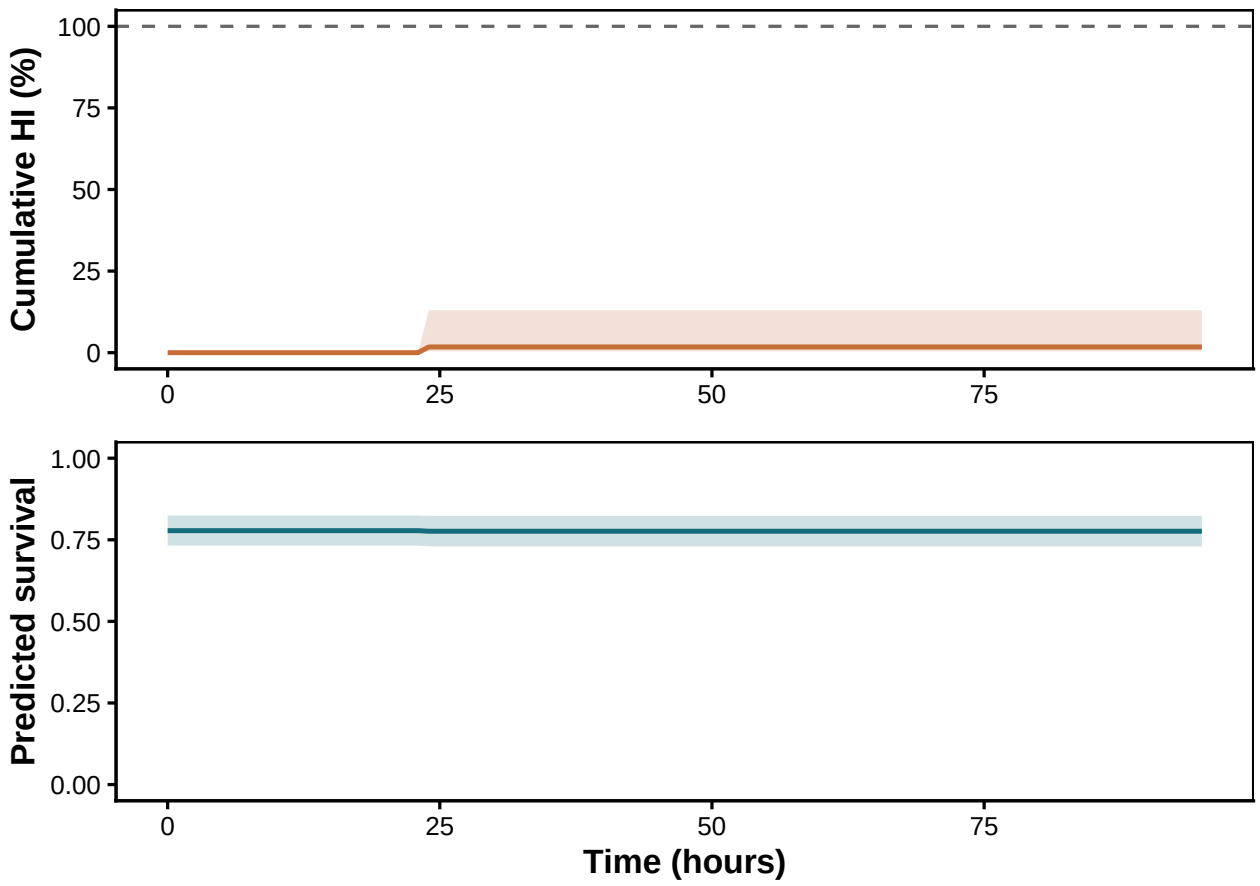


Figure S47: Cumulative functional injury (top) and predicted retained PSII function (bottom; plotted through the library's survival interface) under the **single-spike** trace, **no repair**. Because the spike sits at the posterior median $CT_{max_{1hr}}$, approximately one threshold dose accrues by the end of the spike hour.

```
plot_heat_injury(leaf_hi$multi_spike)
```

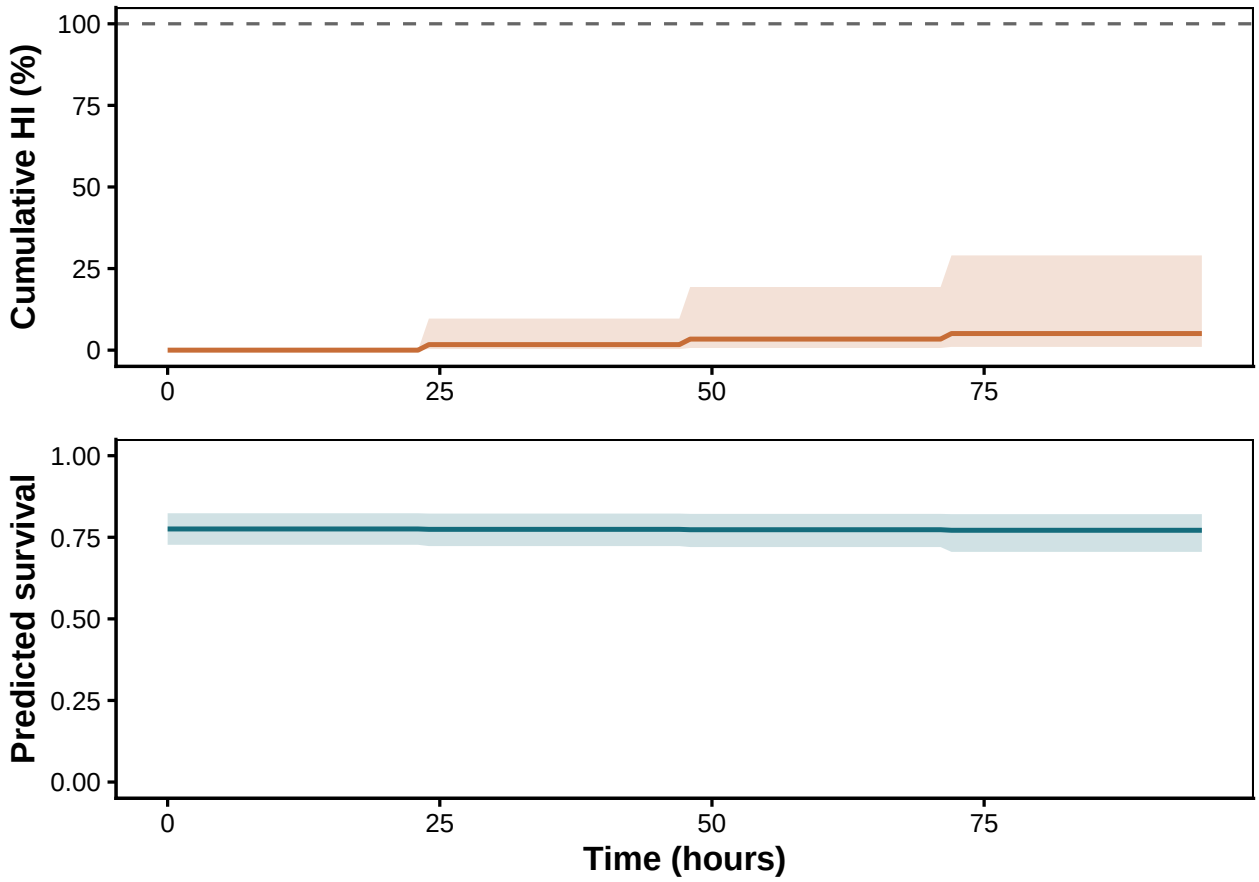


Figure S48: **Multi-spike** trace, **no repair**. Three 1-hour spikes at $CT_{max_{1hr}}$ deliver approximately three threshold doses, driving cumulative functional injury above 100% and retained function toward zero.

```
plot_heat_injury(leaf_hi_multi_rep)
```

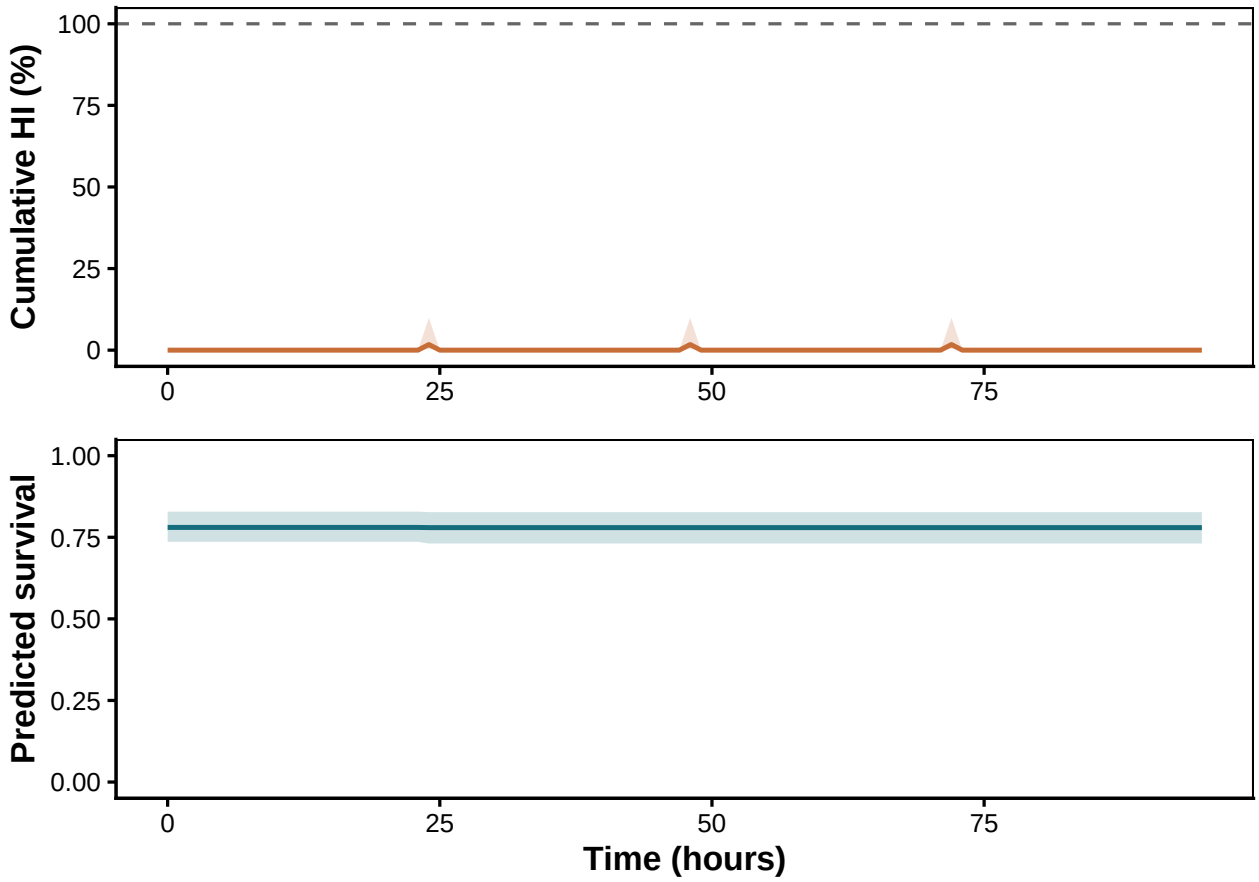


Figure S49: Same multi-spike trace, **with Sharpe-Schoolfield repair active**. Repair clears most of the dose between spikes, stopping the runaway accumulation seen without repair.

The diurnal scenario (Figure S50, Figure S51) approximates a field heatwave. Day 1 remains below T_c and accumulates no injury; days 2–4 each add injury as daily peaks rise. Without repair, cumulative functional injury exceeds 100% by day 4 and predicted retained function declines strongly. With repair, cool periods partly offset injury accumulated near daily peaks.

```
plot_heat_injury(leaf_hi$diurnal)
```

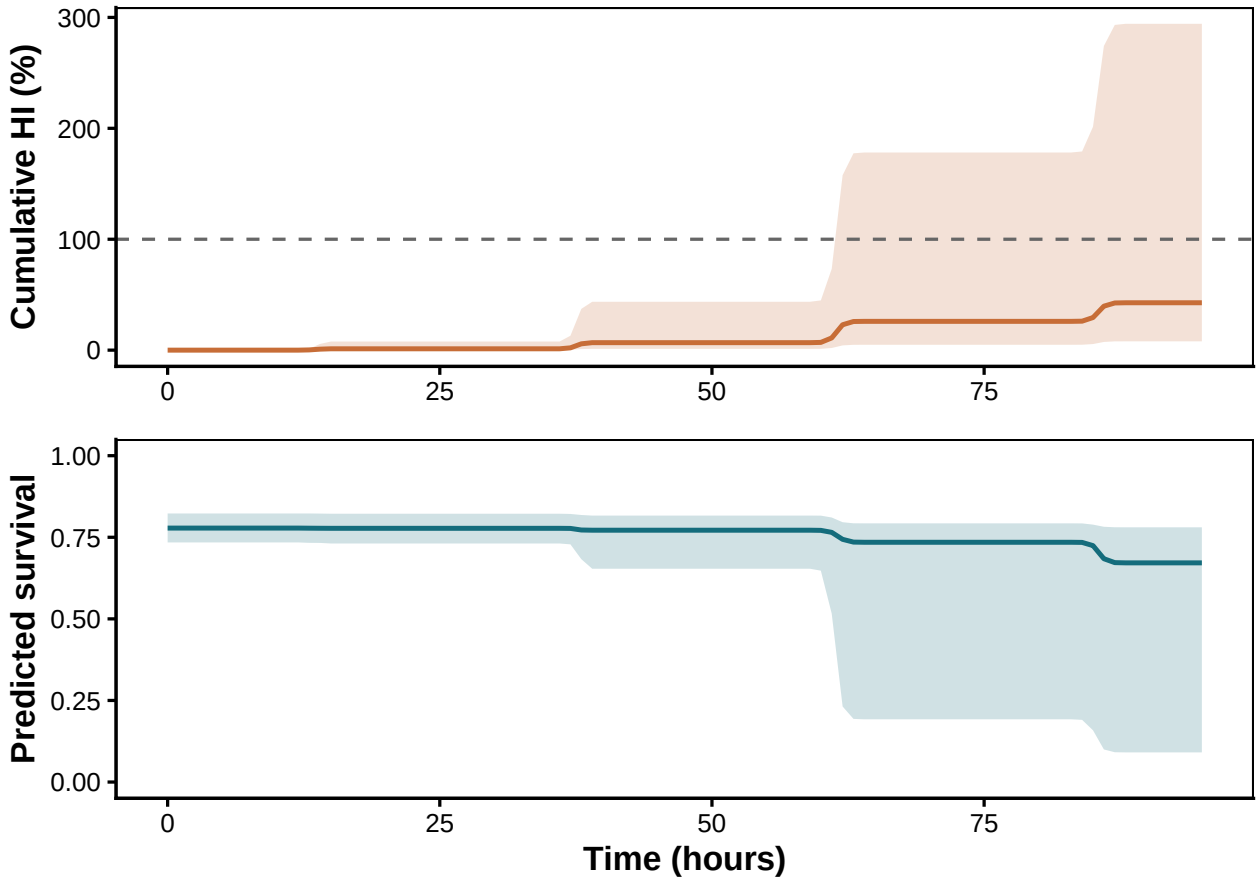


Figure S50: **4-day diurnal heatwave** trace, **no repair**. The cool first day remains below T_c; days 2–4 each accrue functional injury as daily peaks rise.

```
plot_heat_injury(leaf_hi_diurnal_rep)
```

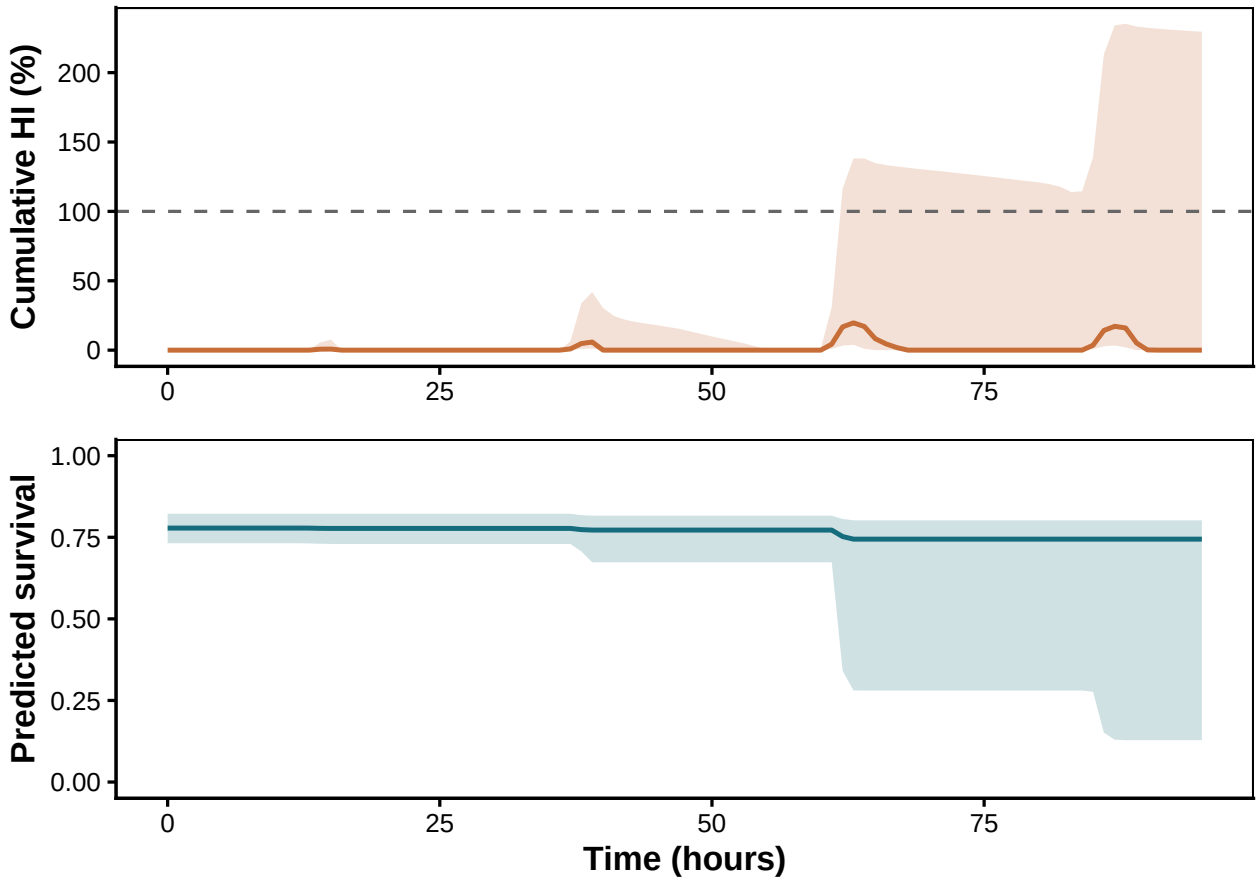


Figure S51: Same 4-day heatwave trace, **with Sharpe-Schoolfield repair active**. The cool first day and overnight troughs now act as recovery phases.

```
final_row <- function(hi) {
  s <- tail(hi$summary, 1)
  tibble::tibble(
    `HI median (%)` = round(s$hi_median, 1),
    `HI lower`      = round(s$hi_lower,  1),
    `HI upper`      = round(s$hi_upper,  1),
    `Retained function median` = round(s$surv_median, 3),
    `Retained function lower`  = round(s$surv_lower,  3),
    `Retained function upper`  = round(s$surv_upper,  3)
  )
}

leaf_hi_table <- dplyr::bind_rows(
  final_row(leaf_hi$flat)         |> dplyr::mutate(Scenario = "Flat",
                                                   Repair = "None"),
  final_row(leaf_hi$single_spike) |> dplyr::mutate(Scenario = "Single spike",
                                                   Repair = "None"),
  final_row(leaf_hi$multi_spike)  |> dplyr::mutate(Scenario = "Multi spike",
                                                   Repair = "None"),
```

Table S38: Final cumulative functional injury and predicted retained PSII function under each scenario for snow gum leaves, posterior medians with 95% credible intervals.

Scenario	Repair	HI median (%)	HI lower	HI upper	Retained function median	Ret
Flat	None	0.0	0.0	0.0	0.778	0.73
Single spike	None	1.8	0.4	13.0	0.776	0.73
Multi spike	None	5.1	1.0	29.0	0.771	0.70
Multi spike	Sharpe-Schoolfield	0.0	0.0	0.0	0.780	0.73
Diurnal (4 days)	None	42.7	7.9	294.3	0.672	0.09
Diurnal (4 days)	Sharpe-Schoolfield	0.0	0.0	229.5	0.744	0.12

```
  final_row(leaf_hi_multi_rep)    |> dplyr::mutate(Scenario = "Multi spike",
                                                  Repair = "Sharpe-Schoolfield"),
  final_row(leaf_hi$diurnal)      |> dplyr::mutate(Scenario = "Diurnal (4 days)",
                                                  Repair = "None"),
  final_row(leaf_hi_diurnal_rep)  |> dplyr::mutate(Scenario = "Diurnal (4 days)",
                                                  Repair = "Sharpe-Schoolfield")
) |>
  dplyr::select(Scenario, Repair, dplyr::everything())

tinytable::tt(leaf_hi_table, escape = TRUE)
```

The snow gum leaf case study takes continuous retained-function proportions through a fitted joint 4PL to posterior TDT summaries and illustrative functional-injury projections. Every derived quantity inherits the posterior of the same fit.

7 Case Study 4: Seed data

We apply `standardize_data()` → `fit_4pl()` → `extract_tdt()` to heat-response data from imbibed seeds of mountain hickory (*Acacia penninervis* var. *penninervis*), using post-exposure respiration as a viability indicator. Seeds were surface-sterilised and imbibed on 1% agar at 22 °C for 24 h before temperature-by-duration heat treatments. Each seed was then placed in a 2 mL tube containing paraffin wax and agar, leaving 0.3 mL of headspace to increase the signal-to-noise ratio for closed-system respirometry. Oxygen consumption per seed was measured as $\dot{V}O_2$ (mL seed^{-1} h^{-1}), and the proportion of five replicate seeds respiring was calculated for each treatment. Because seed heat tolerance is high relative to animal or soft leaf tissues, treatments spanned 22–80 °C and 15 min to 2 h.

7.1 Data and standardisation

Table S39: Summary of the seed lethal-TDT dataset after standardisation. Temperature centre is the grand mean used to mean-centre the temperature covariate.

n trials	Temperature range (°C)	Duration range (min)	Median individuals/trial	Temperature centre (°C)
20	22–80	15–120	5	56

```
# Bundled with the package (see ?acacia_seeds); raw CSV in inst/extdata/.
seed_raw <- acacia_seeds

seed_std <- standardize_data(
  data           = seed_raw,
  temp           = "Temperature",
  duration       = "Duration",
  n_total        = "Seeds_Total",
  n_surv         = "Seeds_Resp",
  random_effects = NULL,
  duration_unit  = "minutes"
)
```

```
seed_summary <- tibble::tibble(
  `n trials`                 = nrow(seed_std),
  `Temperature range (°C)`   = paste(range(seed_std$temp), collapse = "-"),
  `Duration range (min)`     = paste(round(range(seed_std$duration), 2),
                                     collapse = "-"),
  `Median individuals/trial` = stats::median(seed_std$n_total),
  `Temperature centre (°C)`  = attr(seed_std, "tdt_meta")$temp_mean
)
tinytable::tt(seed_summary, digits = 2, escape = TRUE)
```

7.2 Fitting the joint Bayesian 4PL

This design is sparse: 5 assay temperatures × 4 durations form 20 treatment cells, each containing 5 seeds. The package default places a `temp_c` slope on all four 4PL sub-parameters (8 fixed effects), which is too flexible for this dataset. We therefore fit the **constant-shape** variant with `temp_effects = "mid"`: temperature affects only the midpoint ($\log_{10}$ LT50), while asymptotes and steepness are shared across temperatures. This classical TDT parameterisation estimates the midpoint temperature slope, $\beta_1 = -1/z$, while sharing information about curve shape. We retain the default `beta_binomial(link = "identity")` likelihood.

```
wf_seed <- fit_4pl(
  seed_std,
  temp_effects = "mid",
  random_effects = NULL,
```

Table S40: Sampling diagnostics for the seed constant-shape 4PL fit. Healthy targets: Rhat < 1.01, high ESS, zero divergences. Treedepth hits (a flagged but non-fatal efficiency warning) are expected here: the sparse design weakly constrains the curve steepness, producing long posterior tails the sampler must traverse.

rhat_max	ess_bulk_min	ess_tail_min	divergences	treedepth_hits	bfmi_min	rhat_pass	ess_pass
1	3924	5643	0	2041	0.81	TRUE	TRUE

```
  chains  = 4, iter = 8000, warmup = 4000, cores = 4, seed = 123,
  control = list(adapt_delta = 0.99, max_treedepth = 20),
  file    = file.path(models_dir, "fit_seed_lethal_4pl")
)

# Workflow header: data shape, T_bar, asymptote bounds, random effects, draws.
print(wf_seed)
```

```
<bayes_tls>
  Data:     20 rows; 5 temperatures; 4 durations
  T_bar:    56.4
  Bounds:   response in (0.000, 1.000); low in (0.001, 0.499); up in (0.501, 0.999)
  Status:   fitted (16000 draws)
```

7.2.1 Sampling diagnostics

```
tinytable::tt(diagnose_tdt_fit(wf_seed), digits = 3, escape = TRUE)
```

The diagnostics table is a numerical pass/fail summary. The canonical visual check is the chain trace ("fuzzy caterpillar"): healthy chains overlap and explore the same posterior support; chains that drift, diverge, or hang in different regions signal a problem with the fit.

```
brms::mcmc_plot(
  get_brmsfit(wf_seed),
  type     = "trace",
  variable = c("b_lowraw_Intercept", "b_upraw_Intercept",
               "b_logk_Intercept",   "b_mid_Intercept",
               "b_mid_temp_c")
) +
  ggplot2::theme_classic(base_size = 11) +
  ggplot2::theme(legend.position = "bottom")
```

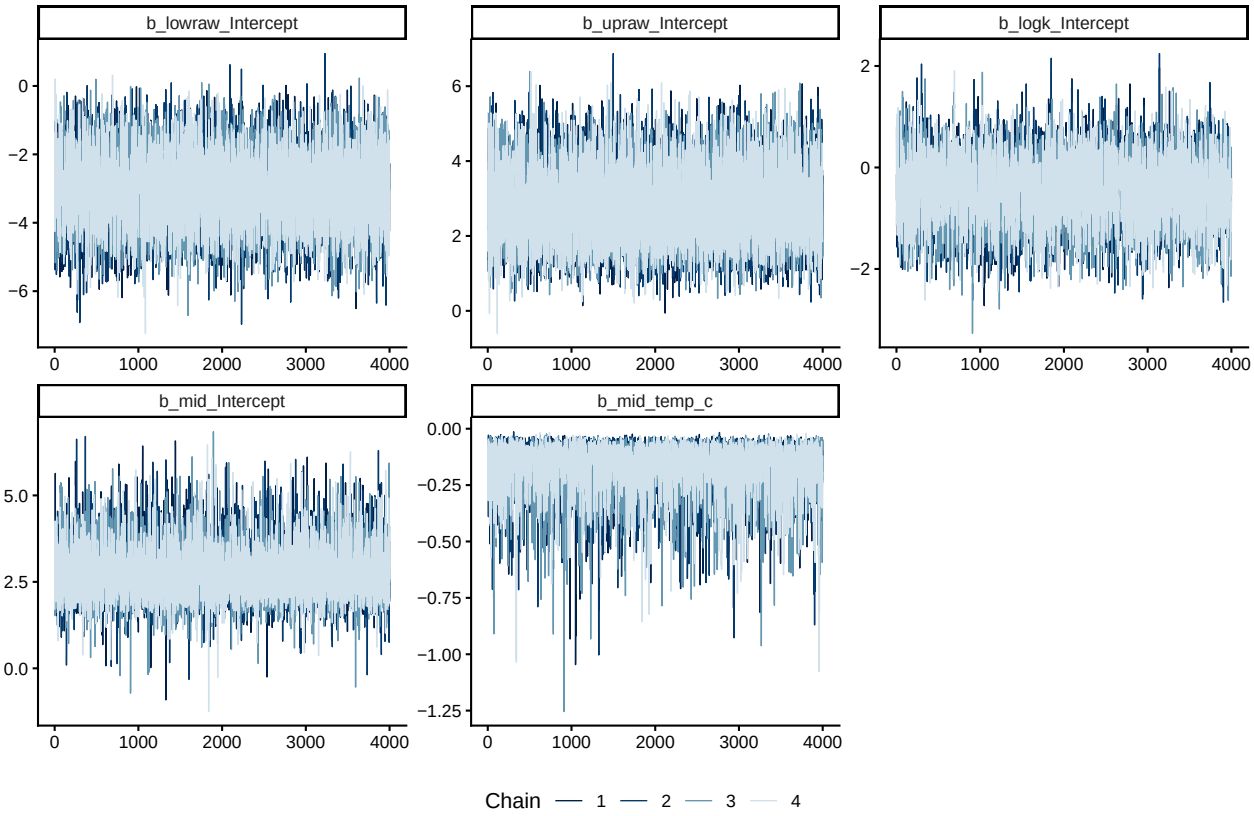


Figure S52: Posterior chain traces for the seed 4PL fixed-effect parameters. Each panel overlays four chains; well-mixed chains overlap throughout.

7.2.2 Posterior parameters on the natural scale

```
tinytable::tt(tdt_parameter_table(wf_seed), digits = 3, escape = TRUE)
```

7.3 Survival curves with observed overlay

`predict_survival_curves()` evaluates the fitted 4PL on a grid of assay temperatures and exposure durations and returns posterior survival probabilities. Overlaying the per-replicate observed proportions gives a visual check that the fitted curves track the data at every temperature.

```
psc_seed <- predict_survival_curves(
  wf_seed,
  temps     = sort(unique(seed_std$temp)),
  ndraws    = 1000
)
plot_survival_curves(psc_seed, observed = seed_std) +
  ggplot2::labs(x = "Exposure duration (minutes)")
```

Table S41: Posterior medians and 95% credible intervals for the seed 4PL fit, transformed back to the natural scale. `low` and `up` are the bottom and top asymptotes of the survival curve at the grand-mean temperature; `k` is the slope on $\log_{10}$ time; `mid` intercept is $\log_{10}$ LT50 in minutes at the grand-mean temperature; `mid temp_c slope` is β_1, from which $z = -1/\beta_1$.

parameter	median	lower	upper
low (lower asymptote)	0.0233	0.00409	0.1393
up (upper asymptote)	0.9695	0.87431	0.9953
k (slope)	0.6414	0.16889	2.1108
mid intercept (at T_bar)	2.643	1.53038	4.6295
mid temp_c slope	-0.1535	-0.48488	-0.0487
z (°C)	6.5143	2.06238	20.5296

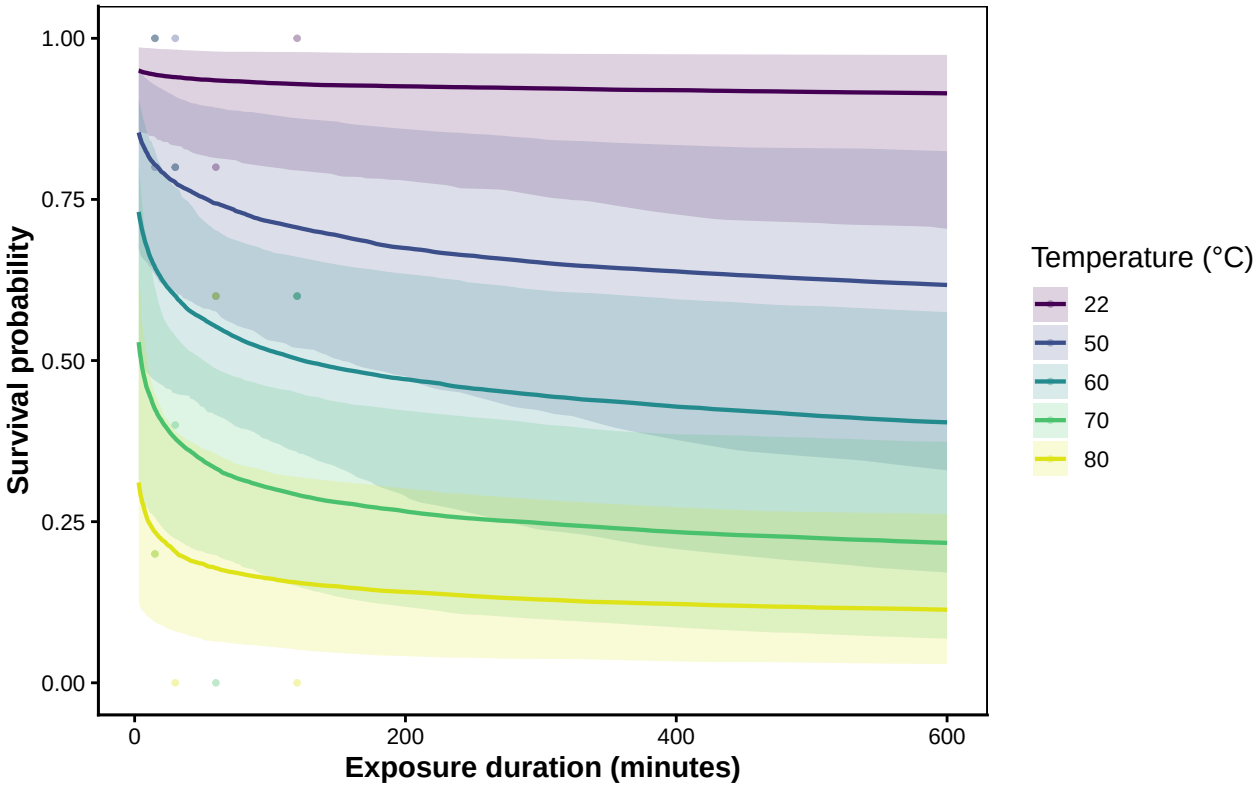


Figure S53: Posterior survival curves for seeds at each assayed temperature, with 95% credible bands. Coloured points are observed per-replicate survival proportions.

7.4 TDT landscape

`derive_tdt_landscape()` evaluates the fitted 4PL on a dense (temperature, duration) grid and `plot_tdt_landscape()` renders it as a heatmap with overlaid contours at 25, 50, and 75% survival.

This is the two-dimensional view of the dose-response surface that the TDT line summarises with one number per temperature.

```
seed_dur_grid <- seq(min(seed_std$duration), max(seed_std$duration),
                     length.out = 120)
lsp_seed <- derive_tdt_landscape(wf_seed,
                                 duration_grid = seed_dur_grid,
                                 ndraws        = 1000)
plot_tdt_landscape(lsp_seed, observed = seed_std)
```

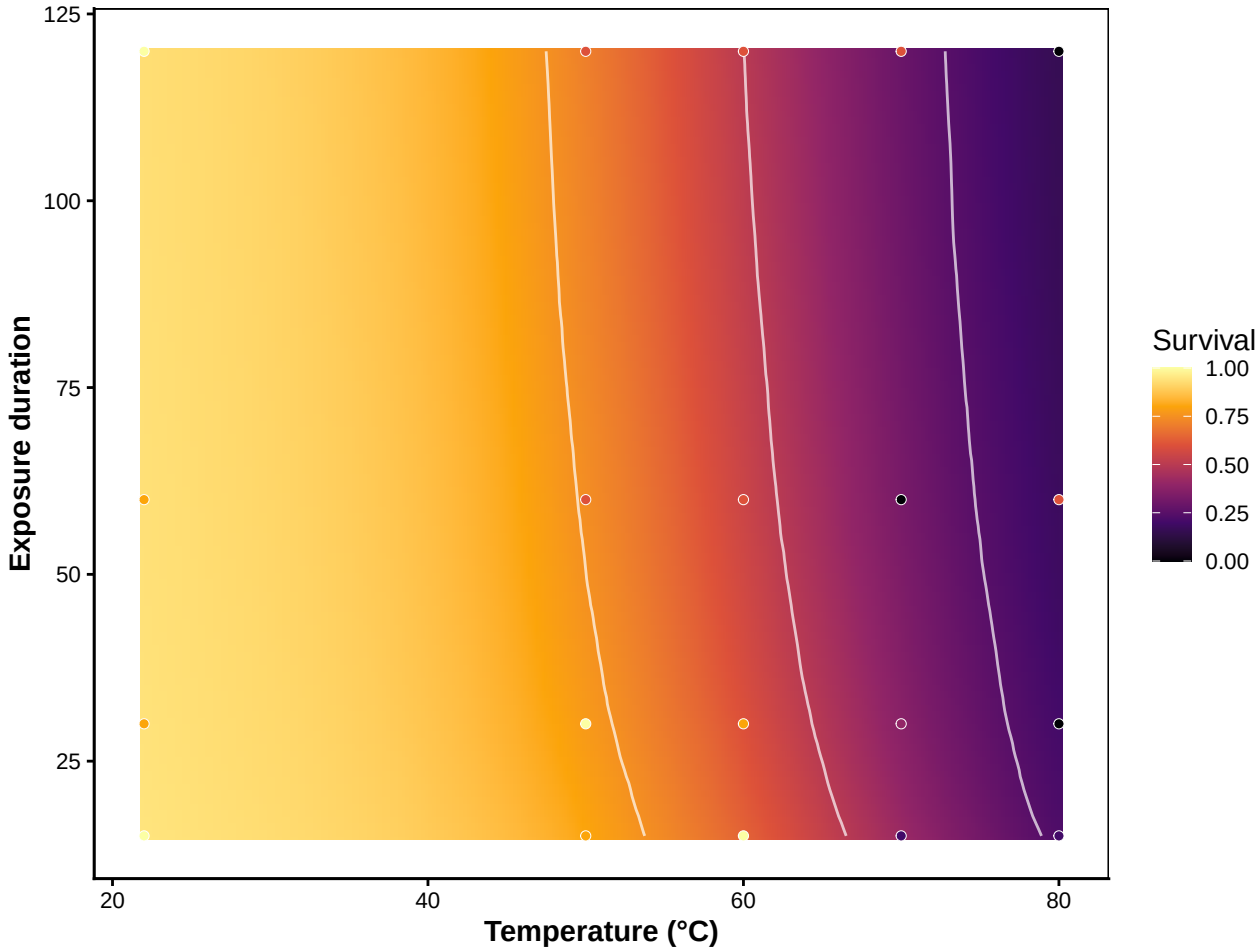


Figure S54: TDT landscape for seeds: posterior median predicted survival across a 120 × 120 grid of assay temperatures × durations. White contours mark 25%, 50%, and 75% survival isolines. Overlaid points are raw observed proportions.

7.5 TDT line: time to the threshold vs temperature

The one-dimensional **TDT line** summarises the dose-response surface as the predicted time to the survival threshold at each temperature. `derive_tdt_curve()` builds it from the joint fit, and `plot_tdt_curve()` shows it on linear and $\log_{10}$ time axes side-by-side: the linear panel reads as

"minutes of exposure half the seeds tolerate" for ecological interpretation; the $\log_{10}$ panel linearises the relationship into the canonical TDT straight line, whose slope is $-1/z$. The default threshold is mid (the midpoint between the fitted asymptotes); pass `target_surv = "absolute"` for the classical absolute-t_{50} definition.

```
ltx_seed <- derive_tdt_curve(
  wf_seed,
  temp_grid        = seq(min(seed_std$temp) - 1.5,
                         max(seed_std$temp) + 1.5, by = 0.05),
  ndraws           = 500,
  time_multiplier  = 1,
  output_time_unit = "minutes"
)
```

```
plot_tdt_curve(ltx_seed, panels = "both", colour = "#b2182b")
```

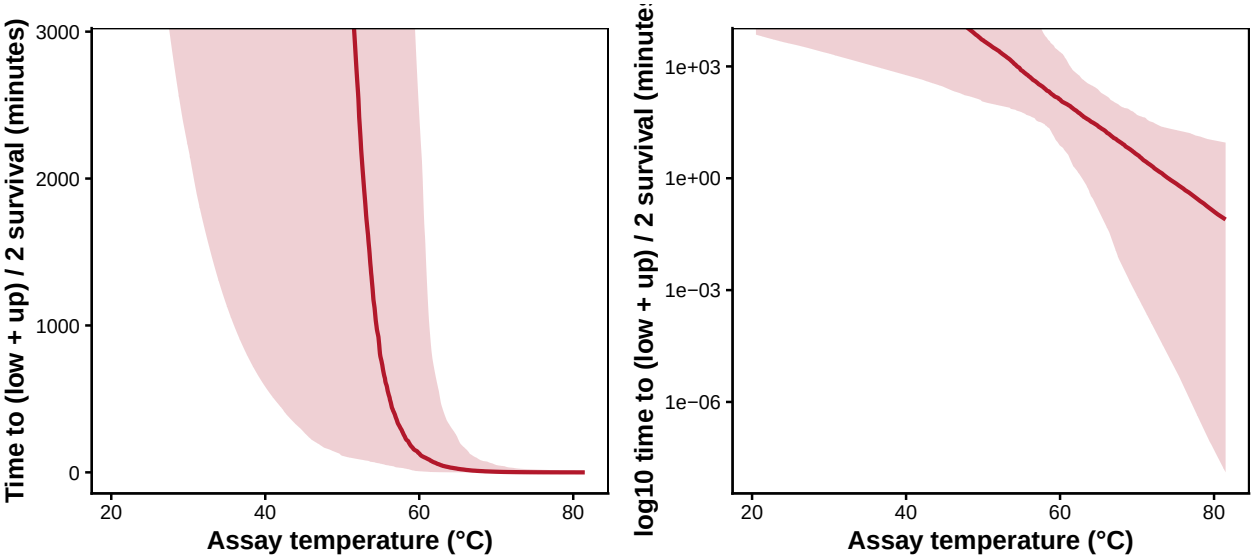


Figure S55: Seed posterior TDT line: predicted time to threshold vs assay temperature, with 95 % credible band. Linear-time panel on the left, $\log_{10}$-time panel on the right (slope is $-1/z$). Both panels share the same posterior.

7.6 Classical TDT quantities: z, $CT_{max_{1hr}}$, T_{crit}

`extract_tdt()` always returns z (the slope of $\log_{10}$ LT on temperature) and $CT_{max_{1hr}}$ (the temperature at which the threshold is reached after 1 h, obtained by per-draw 4PL inversion). Passing `lethal = TRUE` additionally returns T_{crit} under the rate-multiplier definition. Here we treat absence of respiration after exposure as loss of viability and therefore opt in to this lethal-endpoint summary.

Table S42: Classical TDT summaries for seeds, with full posterior uncertainty. z is read directly from the posterior (the temperature slope on mid; $z = -1/\beta_1$ at the relative threshold). $CT_{max_{1hr}}$ is the temperature at 50% survival after 1-hour exposure (4PL inversion per draw). T_{crit} is the temperature at which the TDT damage rate falls below r^* % HI per hour, with r^* sampled uniformly on $\log_{10}$ across $[0.1, 1]$.

Quantity	Median	Lower	Upper
z (°C)	6.7	2.2	21
CTmax_1hr (°C)	62.3	55.2	70
T_crit (°C, r* in [0.1, 1] per hour)	45.1	5.6	59

```
et_seed <- extract_tdt(
  wf_seed,
  t_ref         = 60,
  TC_rate_range = c(0.1, 1),
  ndraws        = 500,
  lethal        = TRUE
)
```

```
seed_extract <- tibble::tribble(
  ~Quantity,                          ~Median, ~Lower, ~Upper,
  "z (°C)",                               et_seed$z$summary$z_median,
                                      et_seed$z$summary$z_lower,
                                      et_seed$z$summary$z_upper,
  "CTmax_1hr (°C)",                   et_seed$CTmax$summary$temp_median,
                                      et_seed$CTmax$summary$temp_lower,
                                      et_seed$CTmax$summary$temp_upper,
  "T_crit (°C, r* in [0.1, 1] per hour)",
                                      et_seed$T_crit$summary$temp_median,
                                      et_seed$T_crit$summary$temp_lower,
                                      et_seed$T_crit$summary$temp_upper
)
tinytable::tt(seed_extract, digits = 2, escape = TRUE)
```

A note on what this design can support. Exposure durations (15–120 min) do not bracket LT50 at most temperatures: at 22–60 °C the proportion respiring never falls below ~0.6 within 120 min, whereas at 70–80 °C most seeds have ceased respiring by the shortest exposure. With only five seeds per cell, the observations also include non-monotone sampling noise. The constant-shape model shares curve shape across temperatures, making z and the response surface estimable with a well-behaved fit, but z is identified through that shared-shape assumption rather than directly observed LT50 crossings at each temperature. Its credible interval is accordingly wide (upper bound ~21 °C per 10-fold change in time), and $CT_{max_{1hr}}$ (~62.3 °C) is interpolated between the 60 and 70 °C treatments. Durations that bracket LT50 at each temperature would sharpen these estimates.

```
z_draws_seed <- get_z_draws(et_seed) |>
  dplyr::transmute(temp = z)
z_q <- stats::quantile(z_draws_seed$temp,
                       c(0.025, 0.5, 0.975), names = FALSE, na.rm = TRUE)
z_post_seed <- list(
  draws   = z_draws_seed,
  summary = tibble::tibble(temp_lower  = z_q[1],
                           temp_median = z_q[2],
                           temp_upper  = z_q[3])
)

patchwork::wrap_plots(
  plot_temperature_density(z_post_seed,
                           x_label = expression(italic(z)~"(°C per 10-fold change in time)")),
  plot_temperature_density(et_seed$CTmax,
                           x_label = expression(CT[max[1*hr]]~("°C"))),
  plot_temperature_density(et_seed$T_crit,
                           x_label = expression(T[crit]~("°C"))),
  ncol = 3
)
```

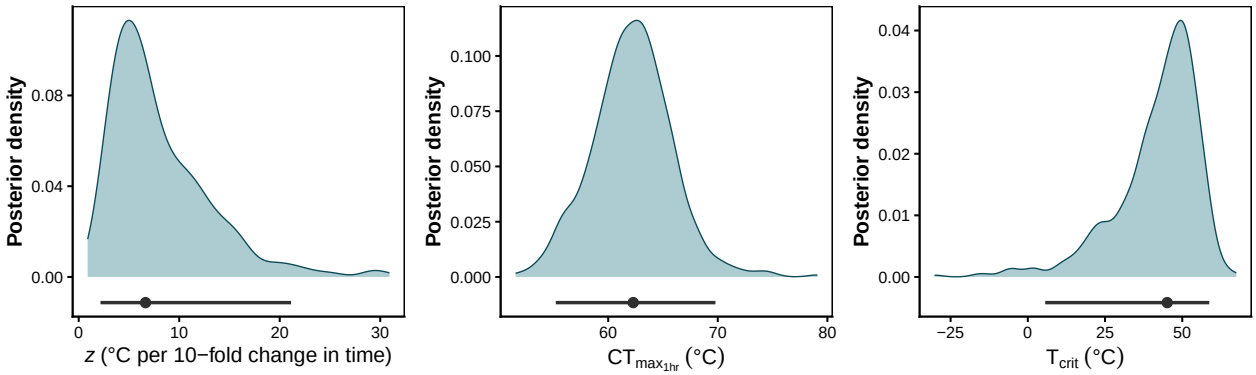


Figure S56: Posterior densities of z (left), $CT_{max_{1hr}}$ (middle), and T_{crit} (right) for seeds. The horizontal bar below each density is the 95% credible interval; the point marks the posterior median. The wide z posterior reflects the assay design (durations that do not bracket the LT50 at most temperatures), not a sampling problem.

The seed case study carries respiration-count data through a constant-shape joint 4PL to posterior TDT quantities, illustrating how diagnostics and credible intervals reveal the limits of an under-bracketed assay design.

8 Manuscript Figure 4: cross-case-study summary

This figure compiles the headline TDT quantities (z and $CT_{max_{1hr}}$) from the four case studies into a single multi-panel display, intended for the main manuscript. Each panel overlays the joint Bayesian

4PL posterior (green ridge + green median/95% CrI) with the classical two-stage point estimate (orange median/95% CI) at the matched Stage-1 likelihood — beta-binomial for the count-based case studies (shrimp, zebrafish, seed), beta for the proportional case study (snow gum leaf PSII). The zebrafish row splits into three ridges, one per life stage. The *Acacia* seed row shows only the joint 4PL: its sparse design yields a valid Stage-1 LT50 at too few temperatures to fit the classical two-stage at all, so there is no two-stage estimate to overlay — itself a useful contrast.

```
# 4PL posterior draws (one row per draw, per case study × group × quantity).
fig4_draws <- dplyr::bind_rows(
  tibble::tibble(
    case_study = "Brown shrimp", group = "Brown shrimp",
    quantity   = "z",         value = get_z_draws(et_shrimp)$z
  ),
  tibble::tibble(
    case_study = "Brown shrimp", group = "Brown shrimp",
    quantity   = "CTmax_1hr", value = get_ctmax_draws(et_shrimp)$CTmax
  ),
  dplyr::bind_rows(lapply(zf_stages, function(s) {
    dr <- zf_joint_tdt[[s]]$z$draws
    dplyr::bind_rows(
      tibble::tibble(case_study = "Zebrafish", group = s,
                     quantity = "z",         value = dr$z),
      tibble::tibble(case_study = "Zebrafish", group = s,
                     quantity = "CTmax_1hr", value = dr$CTmax)
    )
  })),
  tibble::tibble(
    case_study = "Snow gum leaf PSII", group = "Snow gum leaf PSII",
    quantity   = "z",         value = get_z_draws(et_leaf)$z
  ),
  tibble::tibble(
    case_study = "Snow gum leaf PSII", group = "Snow gum leaf PSII",
    quantity   = "CTmax_1hr", value = get_ctmax_draws(et_leaf)$CTmax
  ),
  tibble::tibble(
    case_study = "Acacia seed", group = "Acacia seed",
    quantity   = "z",         value = get_z_draws(et_seed)$z
  ),
  tibble::tibble(
    case_study = "Acacia seed", group = "Acacia seed",
    quantity   = "CTmax_1hr", value = get_ctmax_draws(et_seed)$CTmax
  )
) |>
  dplyr::filter(is.finite(value))

# Median + 95% interval rows for both approaches, one row per case × group ×
# quantity × approach. 4PL = posterior; two-step = analytic / MVN CI.
mk_row <- function(cs, grp, q, approach, med, lo, hi) {
```

```
  tibble::tibble(case_study = cs, group = grp, quantity = q,
                 approach = approach, median = med, lower = lo, upper = hi)
}

shrimp_bb_pt <- dplyr::filter(shrimp_twostep_tdt, method == "Two-step beta-binomial")
shrimp_bb_ci <- shrimp_twostep_unc[["Two-step beta-binomial"]]$summary_ci
leaf_bb_pt   <- dplyr::filter(leaf_twostep_tdt,   method == "Two-step beta")
leaf_bb_ci   <- leaf_twostep_unc[["Two-step beta"]]$summary_ci

zf_rows <- dplyr::bind_rows(lapply(zf_stages, function(s) {
  z_sum <- zf_joint_tdt[[s]]$z$summary
  tw_pt <- dplyr::filter(zf_twostep_tdt, life_stage == s,
                         method == "Two-step beta-binomial")
  tw_ci <- zf_twostep_unc[[paste0(s, "||Two-step beta-binomial")]]$summary_ci
  dplyr::bind_rows(
    mk_row("Zebrafish", s, "z",         "Joint Bayesian 4PL",
           z_sum$z_median,     z_sum$z_lower,     z_sum$z_upper),
    mk_row("Zebrafish", s, "CTmax_1hr", "Joint Bayesian 4PL",
           z_sum$CTmax_median, z_sum$CTmax_lower, z_sum$CTmax_upper),
    mk_row("Zebrafish", s, "z",         "Two-step",
           tw_pt$z,            tw_ci$z_lower,     tw_ci$z_upper),
    mk_row("Zebrafish", s, "CTmax_1hr", "Two-step",
           tw_pt$CTmax_1hr,    tw_ci$CTmax_lower, tw_ci$CTmax_upper)
  )
}))

fig4_summary <- dplyr::bind_rows(
  mk_row("Brown shrimp", "Brown shrimp", "z", "Joint Bayesian 4PL",
         et_shrimp$z$summary$z_median,
         et_shrimp$z$summary$z_lower,
         et_shrimp$z$summary$z_upper),
  mk_row("Brown shrimp", "Brown shrimp", "CTmax_1hr", "Joint Bayesian 4PL",
         et_shrimp$CTmax$summary$temp_median,
         et_shrimp$CTmax$summary$temp_lower,
         et_shrimp$CTmax$summary$temp_upper),
  mk_row("Brown shrimp", "Brown shrimp", "z", "Two-step",
         shrimp_bb_pt$z, shrimp_bb_ci$z_lower, shrimp_bb_ci$z_upper),
  mk_row("Brown shrimp", "Brown shrimp", "CTmax_1hr", "Two-step",
         shrimp_bb_pt$CTmax_1hr,
         shrimp_bb_ci$CTmax_lower, shrimp_bb_ci$CTmax_upper),
  zf_rows,
  mk_row("Snow gum leaf PSII", "Snow gum leaf PSII", "z", "Joint Bayesian 4PL",
         et_leaf$z$summary$z_median,
         et_leaf$z$summary$z_lower,
         et_leaf$z$summary$z_upper),
  mk_row("Snow gum leaf PSII", "Snow gum leaf PSII", "CTmax_1hr", "Joint Bayesian 4PL",
         et_leaf$CTmax$summary$temp_median,
```

```
        et_leaf$CTmax$summary$temp_lower,
        et_leaf$CTmax$summary$temp_upper),
  mk_row("Snow gum leaf PSII", "Snow gum leaf PSII", "z", "Two-step",
        leaf_bb_pt$z, leaf_bb_ci$z_lower, leaf_bb_ci$z_upper),
  mk_row("Snow gum leaf PSII", "Snow gum leaf PSII", "CTmax_1hr", "Two-step",
        leaf_bb_pt$CTmax_1hr,
        leaf_bb_ci$CTmax_lower, leaf_bb_ci$CTmax_upper),
  # Acacia seed: joint 4PL only. The classical two-stage cannot be fit here -
  # only 2 of 5 assay temperatures yield a valid Stage-1 LT50 (fewer than the 3
  # needed for the Stage-2 regression), so there is no two-step estimate to plot.
  mk_row("Acacia seed", "Acacia seed", "z", "Joint Bayesian 4PL",
        et_seed$z$summary$z_median,
        et_seed$z$summary$z_lower,
        et_seed$z$summary$z_upper),
  mk_row("Acacia seed", "Acacia seed", "CTmax_1hr", "Joint Bayesian 4PL",
        et_seed$CTmax$summary$temp_median,
        et_seed$CTmax$summary$temp_lower,
        et_seed$CTmax$summary$temp_upper)
)

# Factor ordering: rows are the three case studies; within zebrafish the three
# life stages are ordered young → old → larvae (top to bottom of the ridge
# stack, so we reverse for the y-axis levels).
case_levels  <- c("Brown shrimp", "Zebrafish", "Snow gum leaf PSII",
                  "Acacia seed")
group_levels <- c("Brown shrimp",
                  rev(zf_stages),
                  "Snow gum leaf PSII",
                  "Acacia seed")
quant_levels <- c("z", "CTmax_1hr")

fig4_draws <- fig4_draws |>
  dplyr::mutate(
    case_study = factor(case_study, levels = case_levels),
    group      = factor(group,      levels = group_levels),
    quantity   = factor(quantity,   levels = quant_levels)
  )
fig4_summary <- fig4_summary |>
  dplyr::mutate(
    case_study = factor(case_study, levels = case_levels),
    group      = factor(group,      levels = group_levels),
    quantity   = factor(quantity,   levels = quant_levels),
    approach   = factor(approach,
                        levels = c("Joint Bayesian 4PL", "Two-step"))
  )

quantity_labeller <- ggplot2::as_labeller(
```

```
  c(z         = "italic(z)~\"(°C per 10-fold change in time)\"",
    CTmax_1hr = "CT[max[1*hr]]~(degree*C)"),
  default = ggplot2::label_parsed
)

approach_cols <- c("Joint Bayesian 4PL" = "#006d2c",
                   "Two-step"           = "#a63603")

# Per-case-study panel builder. We use patchwork rather than facet_grid so
# every case study gets its own independent x-axis range in each column -
# facet_grid forces a shared x within columns, which compresses the
# narrow-CI rows. The 4PL pointrange is drawn at the density baseline
# (no nudge), and the two-step pointrange is nudged down so it sits just
# beneath the 4PL marker rather than overlapping it.
build_fig4_panel <- function(case) {
  d <- dplyr::filter(fig4_draws,   case_study == case) |>
    dplyr::mutate(group = droplevels(group))
  s <- dplyr::filter(fig4_summary, case_study == case) |>
    dplyr::mutate(group = droplevels(group))
  s_4pl <- dplyr::filter(s, approach == "Joint Bayesian 4PL")
  s_2s  <- dplyr::filter(s, approach == "Two-step",
                         is.finite(median))

  p <- ggplot2::ggplot(d, ggplot2::aes(x = value, y = group)) +
    ggridges::geom_density_ridges(
      fill = "#006d2c", colour = NA, alpha = 0.45, scale = 0.85,
      rel_min_height = 0.005
    ) +
    ggplot2::geom_vline(
      data = s_4pl,
      ggplot2::aes(xintercept = median),
      colour = "#006d2c", linetype = "dashed",
      linewidth = 0.35, alpha = 0.6
    ) +
    ggplot2::geom_pointrange(
      data = s_4pl,
      ggplot2::aes(x = median, xmin = lower, xmax = upper, y = group,
                   colour = "Joint Bayesian 4PL"),
      size = 0.35, fatten = 2.2, linewidth = 0.6
    )

  if (nrow(s_2s) > 0) {
    p <- p +
      ggplot2::geom_pointrange(
        data = s_2s,
        ggplot2::aes(x = median, xmin = lower, xmax = upper, y = group,
                     colour = "Two-step"),
```

```
        size = 0.35, fatten = 2.2, linewidth = 0.6,
        position = ggplot2::position_nudge(y = -0.18)
      )
  }

  p +
    ggplot2::scale_colour_manual(values = approach_cols, name = NULL,
                                 breaks = c("Joint Bayesian 4PL",
                                            "Two-step")) +
    ggplot2::facet_wrap(
      ~ quantity, scales = "free_x", nrow = 1,
      labeller = ggplot2::labeller(quantity = quantity_labeller)
    ) +
    ggplot2::labs(x = NULL, y = NULL, title = case) +
    ggplot2::theme_classic(base_size = 11) +
    ggplot2::theme(
      legend.position  = "bottom",
      axis.text.y      = ggplot2::element_blank(),
      axis.ticks.y     = ggplot2::element_blank(),
      strip.background = ggplot2::element_blank(),
      strip.text       = ggplot2::element_text(face = "bold"),
      panel.border     = ggplot2::element_rect(colour = "black",
                                               fill = NA, linewidth = 0.5),
      plot.title       = ggplot2::element_text(face = "bold", size = 11,
                                               hjust = 0)
    )
}

fig4_cases <- c("Brown shrimp", "Zebrafish",
                "Snow gum leaf PSII", "Acacia seed")
fig4_panels <- lapply(fig4_cases, build_fig4_panel)
# Row heights proportional to the number of groups in each case study.
fig4_heights <- vapply(fig4_cases, function(cs) {
  length(unique(dplyr::filter(fig4_draws, case_study == cs)$group))
}, integer(1))

patchwork::wrap_plots(fig4_panels, ncol = 1) +
  patchwork::plot_layout(heights = fig4_heights, guides = "collect") &
  ggplot2::theme(legend.position = "bottom")
```

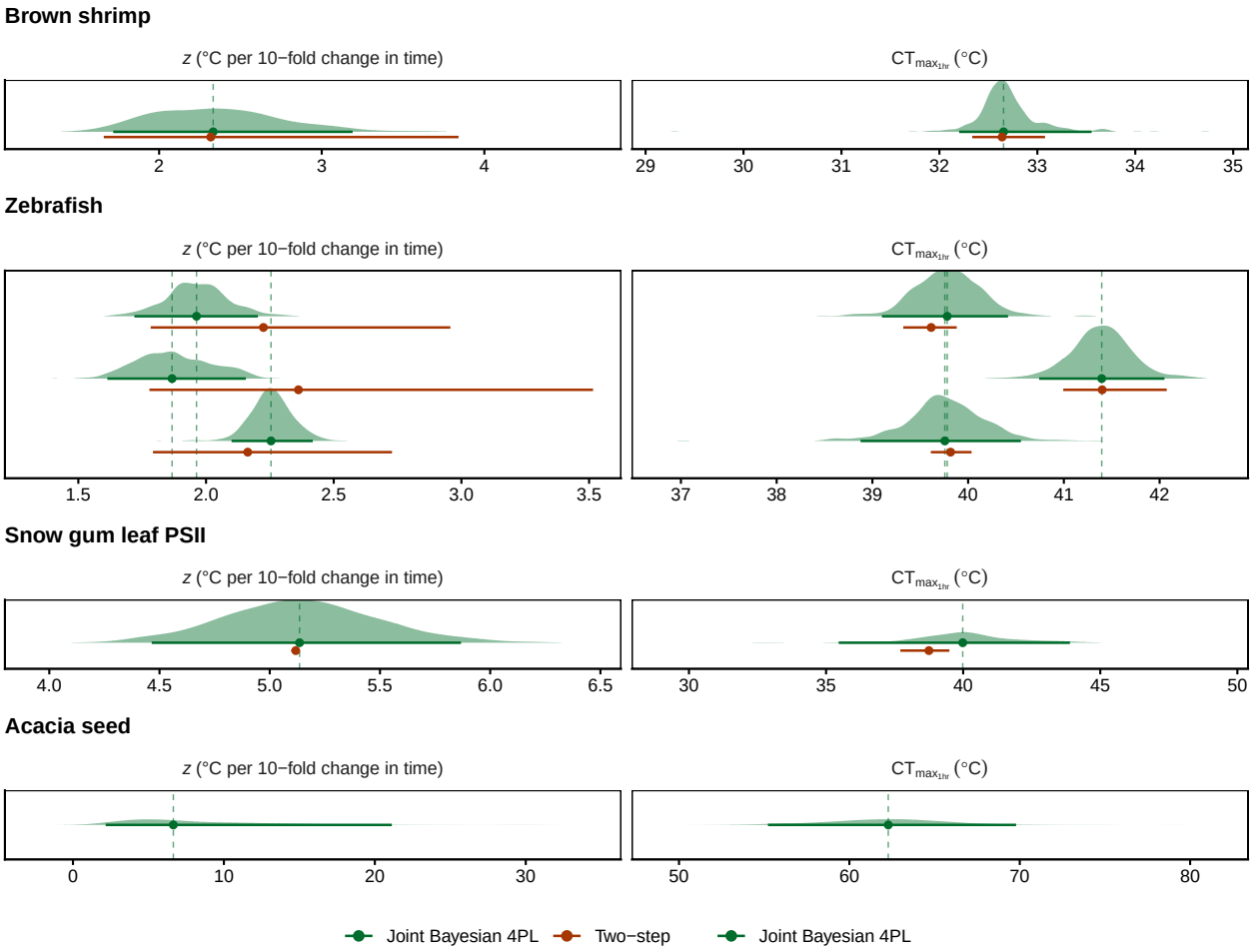


Figure S57: Manuscript Figure 4. Posterior densities of z (left column) and $CT_{max_{1hr}}$ (right column) from the joint Bayesian 4PL fits (green ridges, #006d2c) for the four case studies (rows). The 4PL posterior median and 95% credible interval are overlaid as a green pointrange sitting at the density baseline; the two-step classical estimate (median + 95% CI, orange #a63603) is shown directly below as an orange pointrange — beta-binomial Stage 1 for the brown shrimp and zebrafish counts, beta Stage 1 for the snow gum leaf PSII proportions. The *Acacia* seed has no two-step estimate because its sparse design yields a valid Stage-1 LT50 at too few temperatures. Two-step z intervals are obtained by inverting the Stage-2 slope's 95% CI; intervals are NA (no orange bar) when the slope CI crosses zero, signalling an unidentified Stage-2 slope. Dashed vertical lines mark the 4PL posterior median for each entity. The zebrafish row splits into three ridges, one per life stage. Each case study has independent x-axes in both columns. z is in °C (it is the temperature change required for a 10-fold change in time to the threshold).

9 References

Arnold, P.A., Noble, D.W.A., Nicotra, A.B., Kearney, M.R., Rezende, E.L., Andrew, S.C., *et al.* (2025). A framework for modelling thermal load sensitivity across life. *Global Change Biology*, 31, e70315.

Faber, A.H., Møller, F.D., Ørsted, M., Ehlers, B.K. & Overgaard, J. (2026). Separating good from bad – a methodological assessment of the critical temperature that separates stressful and permissive temperatures in ectotherms.

Jørgensen, L.B., Malte, H. & Overgaard, J. (2021). A unifying model to estimate thermal tolerance limits in ectotherms across static, dynamic and fluctuating exposures to thermal stress. *Scientific Reports*, 11, 12840.

Noble, D.W.A., Mayfield, M.M., Hoffmann, A.A., Chen, Z.-H., Lade, S.J., Bai, X., *et al.* (2026). A systems modelling approach to predict biological responses to extreme heat. *Trends in Ecology & Evolution*, 41, 451–464.

Ørsted, M., Jørgensen, L.B. & Overgaard, J. (2022). Finding the right thermal limit: A framework to reconcile ecological, physiological and methodological aspects of CTmax in ectotherms. *Journal of Experimental Biology*, 225, jeb244514.

Ørsted, M., Willot, Q., Olsen, A.K., Kongsgaard, V. & Overgaard, J. (2024). Thermal limits of survival and reproduction depend on stress duration: A case study of Drosophila suzukii. *Ecology Letters*, 27, e14421.

Rezende, E.L., Bozinovic, F., Szilágyi, A. & Santos, M. (2020). Predicting temperature mortality and selection in natural Drosophila populations. *Science*, 369, 1242–1245.

Rezende, E.L., Castañeda, L.E. & Santos, M. (2014). Tolerance landscapes in thermal ecology. *Functional Ecology*, 28, 799–809.

10 Session Information

```
pander::pander(sessionInfo())
```

R version 4.5.3 (2026-03-11)

Platform: aarch64-apple-darwin20

locale: C.UTF-8||C.UTF-8||C.UTF-8||C||C.UTF-8||C.UTF-8

attached base packages: *stats*, *graphics*, *grDevices*, *utils*, *datasets*, *methods* and *base*

other attached packages: *bayesTLS(v.0.1.0)*, *readxl(v.1.4.5)*, *pander(v.0.6.6)*, *patchwork(v.1.3.2)*, *tinytable(v.0.16.0)*, *tidybayes(v.3.0.7)*, *posterior(v.1.7.0)*, *brms(v.2.23.0)*, *Rcpp(v.1.1.1-1.1)*, *ggplot2(v.4.0.3)*, *purrr(v.1.2.2)*, *tibble(v.3.3.1)*, *tidyr(v.1.3.2)*, *dplyr(v.1.2.1)* and *here(v.1.0.1)*

loaded via a namespace (and not attached): *svUnit(v.1.0.8)*, *tidyselect(v.1.2.1)*, *viridisLite(v.0.4.3)*, *farver(v.2.1.2)*, *loo(v.2.9.0.9000)*, *S7(v.0.2.2)*, *fastmap(v.1.2.0)*, *TH.data(v.1.1-3)*, *tensorA(v.0.36.2.1)*, *pacman(v.0.5.1)*, *digest(v.0.6.39)*, *estimability(v.1.5.1)*, *lifecycle(v.1.0.5)*, *StanHeaders(v.2.36.0.9000)*, *survival(v.3.8-6)*, *processx(v.3.9.0)*, *magrittr(v.2.0.5)*, *compiler(v.4.5.3)*, *rlang(v.1.2.0)*, *tools(v.4.5.3)*, *yaml(v.2.3.12)*, *knitr(v.1.51)*, *labeling(v.0.4.3)*, *bridgesampling(v.1.2-1)*, *curl(v.7.1.0)*, *pkgbuild(v.1.4.8)*, *plyr(v.1.8.9)*, *RColorBrewer(v.1.1-3)*, *cmdstanr(v.0.9.0)*, *multcomp(v.1.4-28)*, *abind(v.1.4-8)*, *withr(v.3.0.2)*, *grid(v.4.5.3)*, *stats4(v.4.5.3)*, *xtable(v.1.8-8)*, *inline(v.0.3.21)*, *ggridges(v.0.5.7)*, *emmeans(v.2.0.3)*, *scales(v.1.4.0)*, *MASS(v.7.3-65)*, *isoband(v.0.3.0)*, *tinytex(v.0.59)*, *cli(v.3.6.6)*, *mvtnorm(v.1.3-7)*, *rmarkdown(v.2.31)*, *generics(v.0.1.4)*, *otel(v.0.2.0)*, *RcppParallel(v.5.1.11-2)*, *reshape2(v.1.4.5)*, *rstan(v.2.36.0.9000)*, *stringr(v.1.6.0)*, *splines(v.4.5.3)*, *bayesplot(v.1.15.0.9000)*, *parallel(v.4.5.3)*, *cellranger(v.1.1.0)*,

matrixStats(v.1.5.0), *vctrs(v.0.7.3)*, *V8(v.6.0.4)*, *Matrix(v.1.7-4)*, *sandwich(v.3.1-1)*, *jsonlite(v.2.0.0)*, *litedown(v.0.7)*, *arrayhelpers(v.1.1-0)*, *ggdist(v.3.3.3)*, *glue(v.1.8.1)*, *codetools(v.0.2-20)*, *distributional(v.0.7.0)*, *stringi(v.1.8.7)*, *gtable(v.0.3.6)*, *QuickJSR(v.1.9.2)*, *pillar(v.1.11.1)*, *htmltools(v.0.5.9)*, *Brobdingnag(v.1.2-9)*, *R6(v.2.6.1)*, *rprojroot(v.2.1.1)*, *evaluate(v.1.0.5)*, *lattice(v.0.22-9)*, *backports(v.1.5.1)*, *rstantools(v.2.6.0.9000)*, *coda(v.0.19-4.1)*, *gridExtra(v.2.3)*, *nlme(v.3.1-168)*, *checkmate(v.2.3.4)*, *xfun(v.0.57)*, *zoo(v.1.8-14)* and *pkgconfig(v.2.0.3)*